FICS and FACS: Fast IOPPs and Accumulation via Code-Switching

Anubhav Baweja
abaweja@upenn.edu
UPenn

Pratyush Mishra
prat@upenn.edu
UPenn

Tushar Mopuri
tmopuri@upenn.edu
UPenn

Matan Shtepel
mshtepel@andrew.cmu.edu
CMU

May 20, 2026

Abstract

Recent work on hash-based succinct arguments has focused on using linear-time encodable codes to improve prover efficiency. We present two new protocols that improve the efficiency of such succinct arguments:

- FICS, the first IOPP (for multilinear polynomial evaluation) for linear-time encodable codes of block length n that achieves $O(n)$ prover time and $O(\lambda \log\log n + \log n)$ verifier oracle query complexity.
- FACS, the first accumulation scheme for NP that achieves linear prover time and $O(\lambda)$ oracle queries per step of the accumulation.

We construct both protocols in a modular manner by defining a sequence of simple intermediate relations, designing efficient Interactive Oracle Reductions (IORs) between them, and composing them. We obtain our final protocols by instantiating the foregoing generic construction with a novel "codeswitching" IOR for tensor codes (Ron-Zewi and Rothblum [JACM 2024]) that achieves much better efficiency than prior IORs for such codes.

To ease IOR development and analysis, we introduce a new notion of knowledge soundness called *round-by-round tree extractibility* (RBRTE), and show that it enjoys several attractive properties: (1) IORs achieving RBRTE are closed under composition; (2) applying the BCS transform to such IORs yields a secure non-interactive reduction in the random oracle model; and (3) RBRTE captures a larger class of protocols than prior work. In particular, while it is not known how to show that IORs which prove claims about codes without efficient decoders satisfy prior notion of round-by-round knowledge soundness, we show that such IORs do achieve RBRTE.

Contents

1 Introduction **1**
- 1.1 Contributions . . . 2
- 1.2 Related work . . . 4

2 Technical overview **9**
- 2.1 Interactive oracle reductions . . . 9
- 2.2 Sumcheck as an IOR . . . 9
- 2.3 IOR toolkit . . . 11
- 2.4 FICS: our new IOPP for multilinear evaluation . . . 12
- 2.5 Halving IOR . . . 14
- 2.6 Generalizing FICS to obtain a new blueprint for constructing IOPPs . . . 15
- 2.7 FACS: linear-time accumulation for NP . . . 15
- 2.8 Codeswitch for tensor codes . . . 16

3 Interactive oracle reductions (IORs) **21**
- 3.1 Formal definition . . . 21

4 Round-by-round tree-extractability **23**
- 4.1 Background: knowledge soundness . . . 23
- 4.2 Constrained transcript trees . . . 24
- 4.3 Round-by-round tree-extractability . . . 26
- 4.4 Main results about RBRTE IORs . . . 27

5 Preliminaries **29**
- 5.1 Notation . . . 29
- 5.2 Coding theory . . . 29

6 IOR toolkit **32**
- 6.1 The sumcheck protocol as an IOR . . . 32
- 6.2 Homogenization IOR . . . 36
- 6.3 Batching IOR . . . 38
- 6.4 Codeswitch IOR . . . 41

7 Codeswitch for tensor codes **43**
- 7.1 Consistency checking IOR . . . 43
- 7.2 Inner product IOR . . . 45
- 7.3 Final tensor codeswitch IOR . . . 50

8 Compiler for constructing an IOR to MEP given an MLIOP **51**

9 IOPP for multilinear evaluation with linear prover time and fast verification **54**
- 9.1 Construction . . . 54
- 9.2 Halve IOR . . . 55

10 Accumulation scheme with linear-time prover and small query complexity **57**
- 10.1 Sufficient ingredients for accumulation schemes . . . 57
- 10.2 The cast IOR for accumulation . . . 58
- 10.3 The fold IOR for accumulation . . . 59

A Proof of Lemma 4.5 (Part 1) **60**
- A.1 Definition of rewinding state-restoration knowledge soundness . . . 60
- A.2 RBRTE implies SRKS using CTreeFinder . . . 62
- A.3 Proof of Lemma B.6 . . . 64

B Proof of Lemma 4.5 (Part 2) **70**
B.1 Prerequisites . . . 70
B.2 Non-interactive reductions . . . 71
B.3 Compilation with BCS transform gives the main theorem . . . 73
B.4 Replacing oracles with Merkle commitments . . . 73
B.5 Fiat–Shamir transformation . . . 74

C Proof of Lemma 4.7 **77**

D Proof of Lemma 4.8 **80**

E Interleaved multilinear-evaluation proximity **82**

F Extended related work **84**
F.1 Other notions of special soundness . . . 84
F.2 Other notions of accumulation . . . 85
F.3 IVC/PCD from SNARKs . . . 85

Acknowledgements **86**

References **86**

1 Introduction

Recent years have seen great progress in the development of succinct arguments. Amongst these, the systems obtained by applying (a variant of) the BCS transformation [BCS16] to information-theoretic *Interactive Oracle Proofs* (IOPs) are of particular interest due to their numerous attractive properties, including avoidance of public-key cryptography, plausible post-quantum security, transparent setups, and (relatively) fast prover time.

The key aim in designing such protocols is to simultaneously minimize prover time, verifier time, and proof size. We recap the state of existing efforts in this direction, focusing on two popular kinds of systems: monolithic arguments, which prove a single computation in one go, and incrementally-verifiable computation (IVC) systems [Val08; BCCT13], which are able to prove the correctness of incremental computations as they unfold.

Monolithic succinct arguments. Recent years have seen much progress in designing IOPs which achieve the optimal *linear* prover time [BCG20; BCL22; RR25; GLSTW23; XZS22; RR24; BCFRRZ25; NST24]. The key component of these IOPs are linear-time encodable codes [Spi96] and Interactive Oracle Proofs *of Proximity* (IOPPs) for these: the IOP prover sends oracles to the verifier that are purportedly codewords, and the IOPP demonstrates that the oracles are indeed close to the code (and sometimes that the underlying message satisfies some additional condition). Unfortunately, state-of-the-art IOPPs for linear-time encodable codes that achieve negligible soundness error [BCFRRZ25; NST24] compromise on verifier query complexity. Namely, for codes of block length n, they have $O(\lambda \log n)$ verifier query complexity, where λ is the security parameter. In contrast, the state-of-the-art IOPP for quasilinear-time encodable codes [ACFY25] achieves verifier query complexity $O(\lambda \log \log n)$. Since a reduction in query complexity of an IOPP directly correlates to reduced proof size of the corresponding succinct argument[1], this gap suggests the following open question:

> **Q1:** Can we design IOPPs for linear-time-encodable codes which achieve linear prover time, state-of-the-art $O(\lambda \log \log n)$ verification, and $\mathrm{negl}(\lambda)$ soundness error?

Incrementally-verifiable computation. Almost all constructions of IVC schemes rely on some form of "recursive composition" of arguments, and the overhead of this step plays a key part in the efficiency of the overall IVC scheme. The state-of-the-art in reducing this overhead relies on accumulation [BCMS20; BCLMS21] or folding [KST22] schemes. While most constructions of accumulation schemes [BGH19; BCMS20; BCLMS21; BDFG21; KST22; KS23; BC23; EG23; KS24] are not based on IOPs and instead rely on quantum-insecure public-key assumptions, there has been some recent work on IOP-based accumulation schemes [BMNW25b; BMNW25a].

These works minimize the verifier work, and hence reduce the overhead incurred due to accumulation in the IVC. Unfortunately, they suffer from some key drawbacks: they either only support a limited accumulation depth [BMNW25a] (which in turn limits how many steps the IVC before soundness breaks down), or incur quasi-linear prover time [BMNW25b]. This motivates our next open question:

> **Q2:** Can we design IOP-based accumulation (and hence IVC) schemes that achieve linear prover time and support unbounded accumulation depth?

In this work, we answer both questions affirmatively.

[1] For example, switching from FRI [BBHR18] (which achieves query complexity $O(\lambda \log n)$) to WHIR [ACFY25] (with $O(\lambda \log \log n)$ query complexity) yields between 2-2.5$\times$ improvement in the proof size of the corresponding succinct arguments.

1.1 Contributions

In this work, we present:

- FICS, an IOPP for a large class of linear-time encodable codes that achieves linear prover time, $O(\lambda \log\log n + \log n)$ query complexity, and $O(\lambda \log n \log\log n)$ verifier time.[2]
- FACS, an accumulation scheme for NP that achieves linear prover time and requires proof size $O(\lambda \log n)$ hashes (corresponding to a query complexity of $O(\lambda)$ per input for the underlying information-theoretic oracle proof).[3]

Both of the foregoing results are obtained by designing and composing tailored *Interactive Oracle Reductions* (IORs) [BCGGRS19; BMNW25b], which are generalizations of both reductions of knowledge [KP23] and IOPs. In particular, we develop a new framework consisting of robust relations that capture sumcheck and multilinear polynomial evaluation claims over the messages underlying purported codewords, and a toolkit of new IORs that operate over these relations. At the core of this framework are IORs for "codeswitching" [RR25; RR24] that reduce claims about proximity to a code $\mathcal{C}$ to claims about proximity to a (potentially) different code $\mathcal{C}'$.[4]

We develop a novel codeswitching IOR for tensor codes of arbitrary constant dimension. We propose a new notion of knowledge soundness for IORs, and show how to use these to prove that (a variant of) the BCS transformation can be used to obtain *non-interactive* reductions.

We detail each of these contributions next.

IOP of proximity for multilinear evaluation. At a high level, all IOPs for NP with linear-time provers proceed in two steps: encode the NP witness with a linear-time encodable code, and then (via by-now standard techniques), use the sumcheck protocol [LFKN92] to reduce checks about membership in the NP language to an *interactive oracle proof of proximity (IOPP) for multilinear evaluation* with respect to a code $\mathcal{C}$. Briefly, an IOPP for multilinear evaluation is an IOP where for public $\boldsymbol{w} \in \mathbb{F}^n$, $\boldsymbol{z} \in \mathbb{F}^{\log k}$ and $y \in \mathbb{F}$, the prover convinces the verifier that it knows $\boldsymbol{m} \in \mathbb{F}^k$ such that:

$$\delta(\boldsymbol{w}, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m})) < \varepsilon \quad \text{and} \quad \hat{m}(\boldsymbol{z}) = y \ ,$$

where $\hat{m}(\boldsymbol{X})$ is the multilinear extension of $\boldsymbol{m}$. Our first contribution is FICS, an IOPP for multilinear evaluation with respect to a large class of linear-time encodable codes.

Theorem 1.1 (informal, Theorem 2.6)**.** *Let $\mathcal{C}$ be a linear-time encodable code with block length n that has an efficient codeswitch IOR. There exists an IOPP for multilinear evaluation for $\mathcal{C}$ with linear prover time, $O(\lambda \log\log n + \log n)$ query complexity, $O(\lambda \log n \log\log n)$ verifier time, and soundness error is* $\mathrm{negl}(\lambda)$.

FICS improves upon the query complexity of the state-of-the-art prior work on IOPPs for linear-time encodable codes [BCFRRZ25; NST24] from $O(\lambda \log n)$ to $O(\lambda \log\log n + \log n)$, while maintaining linear prover time. We use FICS to construct the first IOP for NP with linear prover time and $O(\lambda \log\log n + \log n)$ query complexity.

Accumulation and IVC from linear-time encodable codes. As noted above, almost all modern constructions of IVC (and its generalization, Proof-Carrying Data [CT10]) are built from accumulation schemes. Informally,

[2] Subsequent work [BMMS25] has used our framework to construct an IOPP for linear-time encodable codes with linear prover time and $O(\lambda + \log n)$ query complexity. See Section 1.2 for more details.

[3] Concurrent work [BCFW25] presents WARP, an accumulation scheme that asymptotically matches FACS's prover and verifier complexity. See Section 1.2 for more details.

[4] Throughout this paper, unless specified otherwise, all codes will be assumed to have constant distance and constant rate.

an accumulation scheme for NP allows an *accumulation prover* to aggregate proofs for several NP claims of size $O(n)$ into a single *accumulator*, which is a proof for an NP claim of the same size. Then, an *accumulation verifier* can cheaply check that the new accumulator correctly aggregates the proofs, and finally a *decider* can then later on check that the final accumulator for the combined claim is valid at a cost equal to that of verifying a single original claim.

In this work, we present FACS, an accumulation scheme for NP with the following attractive properties:

Theorem 1.2 (informal, Corollary 10.3)**.** *There exists an accumulation scheme for NP in the random oracle model and the following efficiency when accumulating k NP claims of size n:*
1. *Accumulation prover run time of $O(kn)$.*
2. *Accumulation verifier run time of $O(k\lambda \log n)$.*
3. *Accumulation decider run time of $O(n)$.*

Additionally, because our scheme only leverages the ROM, it is plausibly post-quantum-secure.

Applying this scheme to construct IVC gives an IVC scheme with truly linear-time prover: while prior constructions that relied on IOP-like techniques all incurred prover time that scaled as $\Omega(tn \log n)$, where t is the number of computation steps, by constructing an IVC scheme from FACS we shave off the logarithmic factor to obtain a time $O(tn)$ per step.

A new framework for constructing IOPPs and accumulation schemes. As noted above, we obtained both the aforementioned results by designing and composing new *Interactive Oracle Reductions* (IORs) [BCGGRS19; BMNW25b]. Informally, an IOR from a (pair) relation $\mathcal{R}$ to another relation $\mathcal{R}'$ is an interactive protocol that allows a prover to convince a verifier that an instance $\mathbb{x}$ is in the language $\mathcal{L}(\mathcal{R})$ if and only if a different instance $\mathbb{x}'$ is in $\mathcal{L}(\mathcal{R}')$, where the prover can send the verifier oracles to query.

We develop a new framework consisting of such relations and IORs that operate over them. In particular, these relations consist of purported codewords and claims about the underlying messages, including evaluation claims and sumcheck-based claims over the multilinear extension of the messages. This framework allows us to clearly express ideas such as a prover sending over a new oracle that is a purported codeword, and making a complex sumcheck-based claim about the underlying message (that is often used to establish connections between this new oracle and other oracles the verifier already has access to).

IORs for codeswitching. A key component of FICS and FACS are our codeswitch IORs, which reduce the task of testing proximity of $\boldsymbol{w}$ to a code $\mathcal{C}$ to testing proximity of a related word $\boldsymbol{w}'$ to a code $\mathcal{C}'$ with $O(\lambda)$ verifier query complexity. We construct such codeswitch IORs for $\mathcal{C}$ that is a tensor code of arbitrary dimension and arbitrary $\mathcal{C}'$. Our novel codeswitch IOR for tensor codes is the first that achieves query complexity that is independent of n, but we note that it requires super-constant field size as opposed to prior work [RR24; RR25] that construct codeswitch IORs for tensor codes with $O(1)$ soundness error over fields of size $O(1)$.

New knowledge-soundness notion for IORs. We develop new definitions and tools for analyzing and sequentially composing IORs. In particular, a key contribution of this work is a new notion of knowledge-soundness for IORs that we call Round-By-Round Tree-Extractability, or RBRTE for short. RBRTE generalizes prior notions including round-by-round knowledge soundness [CY24] and tree-extractability [KP23]. We show that IORs that satisfy this notion can be securely compiled to non-interactive reductions in the random oracle model.

We introduce RBRTE because our IORs rely on linear-time encodable codes, which are not known to be polynomial-time decodable. Existing notions of knowledge soundness for IORs are either (a) insufficient for IORs that rely on such codes (round-by-round knowledge soundness), or (b) difficult to work with (rewinding

state restoration knowledge soundness). RBRTE on the other hand is both easy to use (in our opinion) and suffices for IORs that rely on linear-time encodable codes. We provide more details in Section 4.

1.2 Related work

Since our work (in particular its many sub-components) intersects with several 'lines' of prior work, we provide a detailed review of these in order to position our contributions within this context.

1.2.1 Soundness notions for IORs

We recap prior notions of soundness of IORs and reductions of knowledge [KP23].

Interactive oracle reductions (IORs) were first introduced in [BCGGRS19] and further developed in [BMNW25b]. Another work [KP23] introduced reductions of knowledge, which are essentially a special kind of IOR, where the prover can only send messages and not oracles.

We extend the existing formalization of IORs by providing a new notion of knowledge-soundness for them, namely RBRTE. We show that this notion captures protocols where extraction utilizes rewinding, which seems to include all protocols involving codes that are not known to be efficiently decodable. This is important as this set includes all known linear-time encodable codes. We provide an overview of the soundness guarantees of prior work below.

SRKS and RBRKS: A well known sufficient condition for compiling an interactive protocol into a knowledge sound non-interactive protocol is state-restoration knowledge soundness (SRKS), defined for IOPs in [CY24, Definition 23.2.5] and for IORs in [BMNW25b]. However, SRKS is an unwieldy definition and it is often difficult to directly prove that a protocol satisfies it. For this reason, prior work has defined the more natural notion of round-by-round knowledge soundness for IOPs [CY24, Chapter 31] as well as for IORs [BMNW25b], both of which have been shown to imply their respective notions of SRKS. However, the definition of RBRKS is implicitly "straight-line", making it difficult to prove for protocols involving codes that are not efficiently decodable. This is not true for RBRTE, as we show in detail in Section 4.

Tree-extraction: Tree-extraction has also been used in prior work [KP23] to guarantee knowledge soundness for interactive reductions where extraction utilizes rewinding, but this notion does not guarantee compilation into knowledge sound non-interactive versions. Additionally, this notion only guaranteed the security of sequentially composed reductions up to a constant number of sequential compositions.

RBRTE: Our new notion of RBRTE combines and extends the ideas of RBRKS and tree-extraction, to simultaneously support protocols where extraction utilizes rewinding and also guarantee compilation into a knowledge sound non-interactive protocol. Additionally, with respect to composition, show a stronger result than [KP23]: in many natural settings, the composition of even a super-constant number of RBRTE IORs yields an RBRTE IOR. We provide a more detailed comparison of RBRTE with prior work in Section 4.1. Prior work has also defined various notions of special soundness, which can be thought of as tree-extraction in the setting of interactive proofs (IPs). Since extending these to the setting of IORs is plausible, we discuss them in Section F.1.

1.2.2 IOPPs

We recap state-of-the-art IOPPs that achieve polylogarithmic verifier time and query complexity. Our efficiency comparisons are summarized in Table 1.

FRI and BaseFold: Ben-Sasson et al [BBHR18] initiated the study of concretely-efficient IOPPs by designing FRI, an IOPP for Reed–Solomon (RS) codes that achieves linear prover time, $O(\lambda \log n)$ verifier

time, and $O(\lambda \log n)$ query complexity. BaseFold [ZCF24] generalized FRI to work with a larger class of codes called *foldable* codes, and showed how to construct an IOPP for multilinear evaluation on top of the base protocol. We note that while the prover time of both FRI and BaseFold is linear, the codes they support are *not* known to be linear-time encodable.

STIR and WHIR: STIR [ACFY24] is a recent IOPP for Reed–Solomon codes that improves upon FRI by reducing the verifier's query complexity to $O(\lambda \log\log n + \log n)$ query complexity, but compromises on IOPP prover time, which is now $O(n \log n)$. (In practice, this relaxation impacts prover time only minimally.) WHIR further improves upon STIR in two ways: it incorporates support for sumcheck-like constraints on the encoded polynomial, which includes multilinear evaluation queries, and it reduces the IOPP verifier time by eliminating an additive $O(\lambda^2)$ term in the verifier time.

Ligero++: Bhadauria et al. [BFHVXZ20] designed Ligero++, an IOPP for Reed–Solomon codes that achieves efficiency similar to that of FRI, but improves run time concretely by relying on code interleaving. Essentially, instead of directly encoding a message of size k into a single codeword of size n, the prover splits the message into into $O(\log n)$ chunks, each of size $k/\log n$, and encodes each chunk into a codeword of size $O(n/\log n)$. Then, to test proximity, Ligero++ proposes taking a random linear combination of the codewords and applying FRI to this linear combination. While this does not improve asymptotic efficiency, Bhadauria et al. show that it does improve concrete efficiency.

Blaze: The recent work of Brehm et al. [BCFRRZ25] develops Blaze, which is the first IOPP for any linear-time encodable code that simultaneously achieves polylogarithmic verifier complexity and plausibly concrete efficiency. Roughly, Blaze follows the blueprint of Ligero++, but makes two changes. First, it replaces Reed–Solomon codes with Repeat–Accumulate–Accumulate (RAA) codes, which are linear-time encodable codes. Second, instead of relying on FRI, Blaze develops a new IOPP for RAA codes that relies on "codeswitching" techniques [RR24]. Roughly, Blaze's IOPP switches a proximity claim about RAA codes of size $n/\log n$ to a proximity claim about RS codes (also of size $n/\log n$) and then applies BaseFold (instantiated with RS codes) to check this proximity claim. Compared to Blaze, FICS can be seen as performing a sequence of smaller codeswitches. Compared to FICS, Blaze's query complexity is worse, and the authors also do not prove any knowledge-soundness results for it.

Tensor IOPPs: In contrast to our work which is in the negligible soundness regime, previous work that has constructed IOPs for NP in the constant soundness regime has also (implicitly) constructed IOPPs for tensor codes with constant soundness error. In particular, [BCG20] constructs a linear-time prover t-round IOPP for t-wise tensor codes such that the verifier time and query complexity are $\Omega(n^{1/t})$ (see [BCG20, Lemma 10.1]). The subsequent work of [BCL22] improved the query complexity to $O(\log(n))$ (using proof composition), but the verifier time remains $\Omega(n^{1/t})$.

Another line of work by Ron-Zewi and Rothblum [RR24] also (implicitly) constructs constant query IOPPs for tensor codes, but they achieve neither polylogarithmic verifier time nor linear prover time. However, the focus of this work was minimizing the length of the oracles sent, and not the foregoing efficiency parameters. In subsequent work, [RR25] focus on prover time, and present an IOPP for tensor codes with linear prover time and $\mathrm{polylog}(n)$ query complexity. This line of work introduced and makes use of codeswitching, and we compare their codeswitching techniques with ours in Section 1.2.4.

The techniques used in the constant soundness regime differ significantly from ours; they do not make use of the generator matrix of their codes, and often employ proof composition.

Remark 1.3 (verifier time)**.** We note that the IOPP verifier time for FICS is worse than that of prior IOPPs. However, the verifier time of the non-interactive arguments obtained by applying (a variant of) the BCS transform to the IOPPs is dominated by the time required to check q Merkle tree paths, where q is the IOPP verifier query complexity. Plugging in the cost of checking a Merkle tree path ($O(\log n)$ hashes) into the

IOPP	code	prover time	oracle queries	verifier time
FRI [BBHR18]	RS	$O(n)$	$O(\lambda \log n)$	$O(\lambda \log n)$
BaseFold [ZCF24]	Foldable	$O(n)$	$O(\lambda \log n)$	$O(\lambda \log n)$
Brakedown [GLSTW23]	Brakedown	$O(n)$	$O(\lambda\sqrt{n})$	$O(\lambda\sqrt{n})$
Ligero++ [BFHVXZ20]	RS	$O(n)$	$O(\lambda \log n)$	$O(\lambda \log n)$
Blaze [BCFRRZ25]	RAA	$O(n)$	$O(\lambda \log n)$	$O(\lambda \log n)$
BrakingBase [NST24]	Brakedown	$O(n)$	$O(\lambda \log n)$	$O(\lambda \log n)$
STIR [ACFY24]	RS	$O(n \log n)$	$O(\lambda \log \log n + \log n)$	$O(\lambda \log n + \lambda^2)$
WHIR [ACFY25]	RS	$O(n \log n)$	$O(\lambda \log \log n + \log n)$	$O(\lambda \log n)$
FICS	*	$O(n)$	$O(\lambda \log \log n + \log n)$	$O(\lambda \log n \log \log n)$

Table 1: Comparison of IOPPs. The '*' indicates that FICS works with any linear-time encodable code that has an efficient codeswitch IOR

.

corresponding verifier time results in (the non-interactive variant of) FICS's verifier time being better than Blaze, STIR, and FRI/BaseFold, and the same as WHIR.

Concurrent work. In concurrent work, [MZ25] construct an IOPP for trivariate Reed–Muller codes with $O(n)$ prover time, $O\left(\lambda + \frac{\lambda^2 \log\log n}{\log^2 n}\right)$ query complexity and $O(\lambda \log n)$ verifier time. This IOPP is based on the line-versus-point tests in the low-soundness regime and only requires $O(\log \log n)$ rounds of interaction. However, such codes are not known to be linear-time encodable, and thus this work does not yield IOPs with linear prover time.

Subsequent work. In subsequent work, [BMMS25] present the first IOPP for linear-time encodable codes that achieves both linear prover time and $O(\lambda)$ verifier query complexity (which they prove is optimal). Their IOPP is obtained by using our framework of relations and IORs to construct a 'lossy batching IOR' that reduces the query complexity of a ubiquitous subproblem that arises in IOPP constructions: checking that a batch of claims hold.

In another subsequent work, [BFRW25] present an Interactive Oracle Polynomial Commitment Scheme (IOPCS) with linear prover time and $O(\lambda)$ query complexity. Such an IOPCS, much like an IOPP with the same parameters, yields IOPs with linear prover time and $O(\lambda)$ query complexity, which can subsequently be used to construct SNARKs in the ROM.

1.2.3 Accumulation schemes

The construction of accumulation/folding schemes has a rich literature. We divide it into three categories: accumulation schemes that rely on the hardness of discrete logarithms, those that rely on lattice assumptions, and finally those that rely on IOP-style techniques. We discuss the first two categories in Section F.2, and the last one below.

The work of Bünz et al [BMNW25a] initiated the study of hash-based accumulation schemes by constructing an accumulation scheme for NP that works with *any* (constant-distance) linear code, including arbitrary linear-time encodable codes [Spi96; DI14; GLSTW23]. They can thus achieve linear prover time, but this comes at the expense of a severe limitation: their scheme can only support a bounded number of consecutive accumulation steps, and in fact there is a concrete attack that breaks soundness when attempting to accumulate further. IVC and PCD schemes based on their accumulation scheme inherit this limitation.

[BMNW25b; KNS24; Sze24] are follow-up works that removes this limitation and constructs unbounded depth accumulation from Reed–Solomon codes. While the asymptotic cost of the accumulation verifier in

accumulation scheme	code	prover time	verifier time	depth
[BMNW25a]	any	$O(kn)$	$O(k\lambda \log n)$	bounded
[BMNW25b; KNS24; Sze24]	RS	$O(kn \log n)$	$O(k\lambda \log n)$	unbounded
FACS	*	$O(kn)$	$O(k\lambda \log n)$	unbounded

Table 2: Comparison of efficiency of hash-based accumulation schemes when accumulating k NP claims of size n. The '*' indicates that FACS works with any linear-time encodable code that has an efficient codeswitch IOR. The decider for all the schemes runs in $O(n)$ time.

these works is the same as that of FACS, their prover cost is quasi-linear. Amongst these works, [BMNW25b] additionally leverages list-decoding-based bounds for improved concrete efficiency.

In comparison, our work is the first hash-based accumulation scheme that simultaneously supports unbounded depth accumulation and linear prover time, logarithmic verification time, and linear decider time. However, compared to [BMNW25a], we support a more restricted class of codes (codes that support an efficient codeswitch IOR), and unlike [BMNW25b], we did not try to leverage list-decoding-based bounds; this is an interesting direction for future work. We summarize the properties of the aforementioned hash-based accumulation schemes in Table 2.

Concurrent work. In concurrent work, [BCFW25] present WARP, an accumulation scheme that asymptotically matches FACS's prover and verifier complexity. Additionally, WARP can be instantiated with any linear code that has mutual correlated agreement, whereas FACS can be instantiated with codes that are efficiently codeswitchable. The cost of this added generality is that techniques of WARP do not support succinct verification and thus do not yield an IOPP. Furthermore, for the end-goal of linear-time accumulation, only supporting codeswitchable codes is not a limitation, as all known linear-time encodable codes have efficient codeswitch IORs. The key difference between WARP and FACS is that instead of running a codeswitch IOR at every accumulation step, the WARP prover accumulates the codeswitch claims into the accumulator, potentially leading to better concrete efficiency.

Another contribution of [BCFW25] is a new notion of round-by-round knowledge soundness (RBRKS) for IORs, that like the existing notion, implies straightline non-interactive extraction, whereas RBRTE implies rewinding extraction. We note that RBRKS and RBRTE are not known to be comparable, and in fact two subsequent works [BFRW25; BMMS25] are shown to (or believed to) satisfy RBRTE, but are both unsure how to show that they satisfy RBRKS.

1.2.4 Codeswitching

Informally, codeswitching is an interactive oracle protocol that reduces a claim about the proximity of a purported codeword $\boldsymbol{w}$ to a code $\mathcal{C}$ to a claim about the proximity of a different (but related) purported codeword $\boldsymbol{w}'$ to a code $\mathcal{C}'$. In this work, we introduce a new codeswitch from $\mathcal{C}$ that is a t-wise tensor of linear-time encodable codes to an arbitrary code $\mathcal{C}'$. Our protocol leverages the structure inherent in the generator matrices of tensor codes to design efficient protocols that prove that $\boldsymbol{w}$ is close to some vector obtained by multiplying this structured generator matrix by a message vector. If n is the block length of the code $\mathcal{C}$ and λ is a security parameter, both these codeswitches require $O(n)$ prover time, $O(\lambda \log n)$ verifier time, $O(\lambda)$ query complexity, and $2^{-\lambda}$ soundness error.

Another line of work by Ron-Zewi and Rothblum [RR24] uses codeswitches from tensor codes to codes with efficient IOPPs, but they achieve neither polylogarithmic verifier time nor linear prover time. In followup work [RR25] focuses on reducing prover time down to linear, and they construct a codeswitch from a tensor

code to another (smaller) tensor code that achieves linear prover time and polylogarithmic verifier time. We note that both these works focus on constructing codeswitches with constant soundness error over constant sized fields, while our work uses large fields and achieves soundness error $2^{-\lambda}$. Furthermore, our codeswitch also allows switching from a tensor code to an arbitrary code, by using different techniques that leverage the generator matrix of the tensor code.

Blaze [BCFRRZ25] and BrakingBase [NST24] describe codeswitches from (interleaved) RAA codes [DJM98] and Brakedown's code [GLSTW23] respectively, but do not use a "reduction framework". When instantiated with the same interleaving parameter, these codeswitches match the efficiency metrics of our protocols. However, while their protocols switch from their respective linear-time encodable codes $\mathcal{C}$ to particular codes $\mathcal{C}'$ that have efficient IOPPs, our protocols allow switching to arbitrary codes and even support different proximity parameters for both codewords. This flexibility enables us to construct cheaper IOPPs and accumulation schemes for $\mathcal{C}$. For the first result, we set $\mathcal{C}'$ to be an interleaving of a slightly smaller linear-time encodable code, while for the second we set $\mathcal{C}'$ to be $\mathcal{C}$ itself, but "boost" the proximity parameter.

The aforementioned flexibility of our codeswitches becomes apparent when we formalize them as IORs. This viewpoint yields additional benefits, such as modular descriptions of our protocols and the ability to cleanly compose them with other IORs. In contrast, all prior works are described "monolithically", which precludes composability and easy extensibility.

2 Technical overview

2.1 Interactive oracle reductions

Our paper uses the notion of *interactive oracle reductions (IORs)*, first introduced in [BCGGRS19] and further developed in [BMNW25b] At a high level IORs allow $\mathbf{P}$ to reduce the task of convincing $\mathbf{V}$ that an instance is in a language $\mathcal{L}(\mathcal{R})$ to convincing $\mathbf{V}$ that a related instance is in $\mathcal{L}(\mathcal{R}')$[5]. Before we describe IORs, we first describe oracle relations[6], which are a set of tuples of the form $(\mathbb{x}, \vec{\mathbb{y}}, \mathbb{w})$, where $\mathbb{x}$ is a public instance, $\vec{\mathbb{y}}$ consists of oracles that $\mathbf{P}$ has full access to and $\mathbf{V}$ has oracle access to, and $\mathbb{w}$ is a private witness that only $\mathbf{P}$ has access to. An IOR from relation $\mathcal{R} := \{(\mathbb{x}, \vec{\mathbb{y}}, \mathbb{w})\}$ to $\mathcal{R}' := \{(\mathbb{x}', \vec{\mathbb{y}}', \mathbb{w}')\}$ is a k round interactive protocol between $\mathbf{P}$ and $\mathbf{V}$. In each round $i \in [\mathsf{k}]$ $\mathbf{P}$ sends a plain-text message msg_i and an oracle Π_i to $\mathbf{V}$, and $\mathbf{V}$ replies with public-coin randomness ρ_i. In the last round, $\mathbf{V}$ queries the oracles $\vec{\mathbb{y}}$ and $[\Pi_i]_{i\in[\mathsf{k}]}$ at certain points, and chooses to either accept or reject. If $\mathbf{V}$ accepts, $\mathbf{P}$ outputs a new witness $\mathbb{w}'$ and $\mathbf{V}$ outputs a new instance $(\mathbb{x}', \mathbb{y}')$, where the contents of $\mathbb{y}'$ are selected from the input oracle strings $\vec{\mathbb{y}}$ and the oracles sent by $\mathbf{P}$ during the interaction $[\Pi_i]_{i\in[\mathsf{k}]}$. At a high level, an IOR guarantees the following to $\mathbf{V}$: if there exists a $\mathbb{w}'$ such that $(\mathbb{x}', \mathbb{y}', \mathbb{w}')$ belongs to $\mathcal{R}'$, then $\mathbf{P}$ must know of a corresponding witness $\mathbb{w}$ such that $(\mathbb{x}, \mathbb{y}, \mathbb{w})$ belongs to $\mathcal{R}$. IORs are formally defined in Section 3.

Remark 2.1 (IOPs)**.** Note that an interactive oracle proof (IOP) [BCS16] for a relation $\mathcal{R}$ can be seen as an IOR from $\mathcal{R}$ to the trivial relation $\mathcal{R}_{\mathsf{true}} := \{(\mathbb{x} := \mathsf{accept}, \vec{\mathbb{y}} := \bot, \mathbb{w} := \bot)\}$, consisting of only one element with an empty witness. That is, an IOP is a special case of an IOR where the new tuple $(\mathbb{x}', \mathbb{y}', \mathbb{w}')$ trivially belongs to $\mathcal{R}'$, and thus $\mathbf{P}$ knows $\mathbb{w}$ such that $(\mathbb{x}, \mathbb{y}, \mathbb{w}) \in \mathcal{R}$ if and only if $\mathbf{V}$ accepts.

Composition of IORs. One can compose an IOR IOR_1 from $\mathcal{R}_1$ to $\mathcal{R}_2$ and an IOR IOR_2 from $\mathcal{R}_2$ to $\mathcal{R}_3$ to obtain an IOR IOR' from $\mathcal{R}_1$ to $\mathcal{R}_3$. Intuitively, this can be thought of as IOR' first using IOR_1 to reduce the task of checking if $(\mathbb{x}_1, \vec{\mathbb{y}}_1) \in \mathcal{L}(\mathcal{R}_1)$ to checking if $(\mathbb{x}_2, \vec{\mathbb{y}}_2) \in \mathcal{L}(\mathcal{R}_2)$, and then using IOR_2 to reduce this check to checking if $(\mathbb{x}_3, \vec{\mathbb{y}}_3) \in \mathcal{L}(\mathcal{R}_3)$. We show in Section C that RBRTE IORs are closed under composition as long as the input IORs satisfy some mild technical conditions; most natural IORs, including all the IORs considered in this work, satisfy these conditions.

2.2 Sumcheck as an IOR

Prior work has interpreted the sumcheck protocol as a code-based reduction about code-based relations. This reduction is used to obtain efficient IOPPs in recent work such as BaseFold [ZCF24], WHIR [ACFY25], and Blaze [BCFRRZ25], and will be vital in our constructions as well. In this section we show how to capture these ideas in the IOR framework. We will start by recalling the sumcheck protocol in the context of polynomials, and then show how to interpret it as a code-based IOR.

The seminal sumcheck protocol [LFKN92] is a fundamental building block for constructing succinct arguments. It allows $\mathbf{P}$ to convince $\mathbf{V}$ with oracle access to a polynomial $p(\boldsymbol{X}) \in \mathbb{F}^{\leq d}[X_1, \ldots, X_\mu]$ that its evaluations over the boolean hypercube sum up to a claimed value σ, using just one oracle query as opposed to the naive 2^μ queries. This is done via a μ-round interactive protocol that reduces the claim "$\sum_{\boldsymbol{x}\in\{0,1\}^\mu} p(\boldsymbol{x}) = \sigma$" to a single polynomial evaluation claim "$p(\boldsymbol{z}) = y$", where $\boldsymbol{z} \in \{0,1\}^\mu$ is a random

[5]The language $\mathcal{L}(\mathcal{R})$ consists of all tuples $(\mathbb{x}, \vec{\mathbb{y}})$ for which there exists a $\mathbb{w}$ such that $(\mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \in \mathcal{R}$.

[6]We actually use *indexed oracle relations*, which consist of tuples of the form $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w})$. However for the technical overview we only deal with oracle relations for clarity, since for a fixed $\mathbb{i}$ and an indexed oracle relation $\mathcal{R}$, all tuples $(\mathbb{x}, \vec{\mathbb{y}}, \mathbb{w})$ such that $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \in \mathcal{R}$ form an oracle relation, which is denoted $\mathcal{R}^{\mathbb{i}}$.

point chosen by $\mathbf{V}$. It is easy to see that we have just interpreted the sumcheck protocol as an IOR, but one where the relations involve polynomial oracles.

Code-based relations. Prior work has leveraged linear error-correcting codes to avoid using polynomial oracles and instead deal entirely with vectors. They consider the simple case of multilinear polynomials: instead of having polynomial oracle access to a multilinear polynomial $p(\boldsymbol{X}) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_\mu]$, they require oracle access to $\boldsymbol{w} \in \mathbb{F}^n$ obtained by encoding $\boldsymbol{p} \in \mathbb{F}^k$ under a code $\mathcal{C}$ with parameters $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$ such that $k = 2^\mu$. This suffices because for certain codes $\mathcal{C}$ (e.g. Reed-Solomon codes [ACFY24; ACFY25], Repeat-Accumulate-Accumulate codes [BCFRRZ25]), $\mathbf{P}$ can commit to a multilinear polynomial $p(\boldsymbol{X})$ as $\boldsymbol{w} := \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{p})$ and then convince $\mathbf{V}$ with oracle access to $\boldsymbol{w}$ that $p(\boldsymbol{z}) = y$. One caveat is that all prior work can only guarantee that we can extract a $\boldsymbol{p}$ such that $\delta(\boldsymbol{w}, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{p})) \leq \delta_{\mathsf{ext}}$, for some δ_{ext} smaller than the unique decoding radius of $\mathcal{C}$. However, this still binds $\mathbf{P}$ that has committed to a vector $\boldsymbol{w}$ to a unique $\boldsymbol{p}$.

The SEP relation. We now use this insight to define a code-based version of the sumcheck relation. We first note that for our protocols, sumcheck claims of the form $\sum_{\boldsymbol{x} \in \{0,1\}^\mu} p(\boldsymbol{x}) = \sigma$ are not sufficiently expressive. Instead we need to talk about sumcheck claims about the sums of products of ℓ polynomials. We capture this in the following relation.

Definition 2.2 ($\mathsf{SEP}^{(\mathcal{C},\varepsilon,\ell)}$ relation (informal))**.** *The* sumprod-evaluation proximity (SEP) *relation* $\mathsf{SEP}^{(\mathcal{C},\varepsilon,\ell)}$ *consists of tuples of the form*

- $\mathbb{x}$*:* $(f(\boldsymbol{X}, \boldsymbol{Z}), \sigma)$*, where* $f(X_1, \ldots, X_{\log k}, Z_1, \ldots, Z_\ell) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_{\log k + \ell}]$ *and* $\sigma \in \mathbb{F}$*,*
- $\vec{\mathbb{y}}$*:* $[\boldsymbol{w}_i]_{i \in [\ell]}$*, where* $\boldsymbol{w}_i \in \mathbb{F}^n$ *for each* $i \in [\ell]$*,*
- $\mathbb{w}$*:* $[\boldsymbol{m}_i]_{i \in [\ell]}$*, where* $\boldsymbol{m}_i \in \mathbb{F}^k$ *for each* $i \in [\ell]$*,*

such that they satisfy the conditions

$$\begin{aligned} \forall i \in [\ell] \quad &: \quad \delta(\boldsymbol{w}_i, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_i)) \leq \varepsilon \ , \\ \sum_{\boldsymbol{b} \in \{0,1\}^{\log k}} & f(\boldsymbol{b}, \hat{m}_1(\boldsymbol{b}), \ldots, \hat{m}_\ell(\boldsymbol{b})) = \sigma \ . \end{aligned}$$

As a concrete example of how this is a more expressive relation, we note that prior work [ACFY25] has used claims of the form "$\sum_{\boldsymbol{b} \in \{0,1\}^{\log k}} f(\boldsymbol{b}, \hat{m}(\boldsymbol{b})) = \sigma$" to enforce that

$$\sum_{\boldsymbol{b} \in \{0,1\}^{\log k}} \hat{m}(\boldsymbol{b}) \cdot \mathrm{eq}(\boldsymbol{b}, \boldsymbol{z}) = y, \text{ which implies that } \hat{m}(\boldsymbol{z}) = y \ .$$

The MEP relation. Accordingly, we define our main oracle relation, the *multilinear-evaluation proximity* relation $\mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$, which makes ℓ evaluation claims about ℓ multilinear polynomials simultaneously.

Definition 2.3 ($\mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$ relation (informal))**.** *The relation* $\mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$ *consists of tuples of the form*

- $\mathbb{x}$*:* $[(\boldsymbol{z}_i, y_i)]_{i \in [\ell]}$*, where for each* $i \in [\ell]$ *:* $\boldsymbol{z}_i \in \mathbb{F}^{\log k}$ *and* $y_i \in \mathbb{F}$*,*
- $\vec{\mathbb{y}}$*:* $[\boldsymbol{w}_i]_{i \in [\ell]}$*, where* $\boldsymbol{w}_i \in \mathbb{F}^n$ *for each* $i \in [\ell]$*,*
- $\mathbb{w}$*:* $[\boldsymbol{m}_i]_{i \in [\ell]}$*, where* $\boldsymbol{m}_i \in \mathbb{F}^k$ *for each* $i \in [\ell]$*,*

such that they satisfy the conditions

$$\begin{aligned} \forall i \in [\ell] \quad &: \quad \delta(\boldsymbol{w}_i, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_i)) \leq \varepsilon \ , \\ \forall i \in [\ell] \quad &: \quad \hat{m}_i(\boldsymbol{z}_i) = y_i \ . \end{aligned}$$

Notice that the ℓ claims in $\mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$ involve distinct evaluation points $[\boldsymbol{z}_i]_{i\in[\ell]}$. We also define a related relation, the homogeneous multilinear-evaluation proximity relation $\mathsf{HEP}^{(\mathcal{C},\varepsilon,\ell)}$, identically to $\mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$ except that all ℓ claims involve the same evaluation point $\boldsymbol{z}$.
In Section 6.1.2, we present a sumcheck based IOR ECSumcheck[7] from $\mathsf{SEP}^{(\mathcal{C},\varepsilon,\ell)}$ to $\mathsf{HEP}^{(\mathcal{C},\varepsilon,\ell)}$. We can thus define expressive sumcheck claims and reduce them to evaluation claims, all while not using any polynomial oracles.

Remark 2.4 (notation for relations)**.** Oftentimes we will use $\mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$ with $\ell = 1$, in which case we will abuse notation and just write $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$ for convenience. We will also use MEP when talking about $\mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$ inline, if $(\mathcal{C},\varepsilon,\ell)$ is clear from context. We use analogous notation for HEP and SEP.

2.3 IOR toolkit

To obtain our desired IOPP and accumulation scheme, we construct a toolkit of IORs, many of which are formalizations of previously known protocols in the IOR framework.

1. **Batching IOR.** Given ℓ *homogeneous* polynomial evaluations claims "$\hat{m}_1(\boldsymbol{z}) = y_1, \ldots, \hat{m}_\ell(\boldsymbol{z}) = y_\ell$" and ℓ corresponding proximity claims "$\delta(\mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_i), \boldsymbol{w}_i) \leq \varepsilon$", one is often interested in reducing this to a single evaluation claim "$\hat{m}(\boldsymbol{z}) = y$" and a corresponding proximity claim "$\delta(\mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}), \boldsymbol{w}) \leq \varepsilon_{\text{new}}$". This is precisely what our Batch IOR from $\mathsf{HEP}^{(\mathcal{C},\varepsilon+\varepsilon_b,\ell)}$ to $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$ does. Here, ε_b is a fixed constant that represents the "batching distance loss" incurred while using the batching IOR, which we discuss in Remark 6.11.
   Batch is a formalization of a commonly used technique that exploits the fact that linear codes exhibit a property called *proximity gaps* [BKS18], which at a high level states that a random linear combination of codewords is far from the code if even one of the codewords is far from the code. We refer the reader to Section 6.3 for more details and a full description of Batch, but point out that Batch has linear prover time, $O(\lambda \log n)$ verifier time, and $O(\lambda)$ query complexity.
2. **Homogenization IOR.** Given that we can now batch ℓ *homogenous* multilinear evaluation claims, it would be helpful to be able to reduce ℓ multilinear evaluation claims with different evaluation points to ℓ claims over the *same* evaluation point. This is precisely what our IOR Homogenize from $\mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$ to $\mathsf{HEP}^{(\mathcal{C},\varepsilon,\ell)}$ does. Homogenize adapts the 'batch-opening' IOP in HyperPlonk [CBBZ23, Section 3.8]; it reduces the MEP instance to an SEP instance, and then invokes the ECSumcheck IOR to reduce this to an HEP instance. We provide more details in Section 6.2, but point out that Homogenize has linear prover time, $O(\lambda \log n)$ verifier time, and $O(\lambda)$ query complexity.
3. **Codeswitching IOR.** Codeswitching is a powerful tool that has been used in past influential works [RR25; RR24; BCFRRZ25] to build succinct arguments. We formalize codeswitching as an IOR CodeSwitch from $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$ to $\mathsf{MEP}^{(\mathcal{C}',\varepsilon')}$, where $\mathcal{C}'$ could be different from $\mathcal{C}$. That is, it reduces checking a claim "$\delta(\mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}), \boldsymbol{w}) \leq \varepsilon$ and $\hat{m}(\boldsymbol{z}) = y$" to checking a claim of the form "$\delta(\mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m}'), \boldsymbol{w}') \leq \varepsilon'$ and $\hat{m}'(\boldsymbol{z}') = y'$". The prover efficiency of such a reduction primarily depends on 2 factors: (i) the structure of $\mathcal{C}$, and (ii) $T_{\mathcal{C}'}$, the encoding time of $\mathcal{C}'$. If there exists a CodeSwitch IOR from $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$ to $\mathsf{MEP}^{(\mathcal{C}',\varepsilon')}$ such that the prover time is $O(n + T_{\mathcal{C}'})$, the number of oracle queries is $O(\lambda)$, and the verifier time is $O(\lambda \log n)$, then we call $\mathcal{C}$ an "efficiently codeswitchable" code. In Section 7, we show that tensor codes are efficiently codeswitchable. [NST24] show that systematic LDPC codes (which include linear-time encodable codes like Brakedown's code [GLSTW23]) are efficiently codeswitchable as well. We summarize our results in the following lemma.

[7] This IOR has linear prover time, $O(\lambda\ell \log n)$ verifier time, and $O(\lambda\ell)$ query complexity.

Lemma 2.5 (Codeswitch IOR (informal))**.** *There exists an IOR* CodeSwitch *from* $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$ *to* $\mathsf{MEP}^{(\mathcal{C}',\varepsilon')}$ *such that*

(a) $\mathcal{C}$ *is a tensor code with parameters* $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$ *and* $\mathcal{C}'$ *is an arbitrary code with parameters* $(\mathbb{F}, n', k, \delta_{\mathcal{C}'})$ *and encoding time* $T_{\mathcal{C}'}$*, such that both codes have constant rate and distance, and*

(b) $\varepsilon \in (\delta_{\mathcal{C}}/8, \delta_{\mathcal{C}}/3 - 2\varepsilon_b)$*, and* $\varepsilon' \in (\delta_{\mathcal{C}'}/8, \delta_{\mathcal{C}'}/3 - \varepsilon_b)$*, where* ε_b *is the loss in distance incurred due to an application of* Batch*,*

with the following efficiency properties:

prover time	*verifier time*	*verifier queries*
$O(n + T_{\mathcal{C}'})$	$O(\lambda \log n)$	$O(\lambda)$

Note that CodeSwitch does not need to be invoked with different codes. Indeed, we use CodeSwitch in our accumulation to "codeswitch" to the same code, relying on it only to regain distance lost due to the Batch IOR. That is, we invoke CodeSwitch with $\mathcal{C}' = \mathcal{C}$ and $\varepsilon' < \varepsilon$ to reduce a claim of the form "$\delta(\mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}), \boldsymbol{w}) \leq \varepsilon$ and $\hat{m}(\boldsymbol{z}) = y$" to a claim of the form "$\delta(\mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}'), \boldsymbol{w}') \leq \varepsilon'$ and $\hat{m}'(\boldsymbol{z}') = y'$". We believe that this is the first formalization of this idea in the literature, and suspect that it may be useful in other contexts as well. We provide an overview of the techniques of the codeswitch IOR for tensor codes to Section 2.8.

4. **Compiler from MLIOP to IOR.** An MLIOP is an interactive oracle proof where the prover's oracles are restricted to being multilinear polynomials, and the verifier's queries are restricted to be evaluation queries for these oracles. We described in Section 2.2 how to replace polynomial oracles in relations to obtain corresponding code-based oracle relations. We show in Section 8 that this can be extended to MLIOPs by presenting a compiler that given an MLIOP for a relation $\mathcal{R}$, outputs an IOR from $\mathcal{R}$ to an instance of MEP. At a high level, like in the relation case, the compiler first replaces all the polynomial oracles with their encodings under an error-correcting code, and then uses the MLIOP verifier's queries to these oracles to construct an instance of MEP. Details can be found in Section 8.

In the next two sections we focus on how to use our IOR toolbox to obtain our IOPPs and accumulation schemes.

2.4 FICS: our new IOPP for multilinear evaluation

We construct a new IOPP for multilinear evaluation with respect to linear-time encodable codes, called FICS. This is precisely an IOP for an instance of $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$, which is what we discuss in this section.

Theorem 2.6 (FICS)**.** *Let* $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$ *be an efficiently codeswitchable code family (Definition 2.7) that is linear-time encodable. Then* FICS *is an IOPP for multilinear evaluation relative to* $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$ *that is RBRTE and has the following efficiency parameters:*

prover time	*verifier time*	*verifier queries*
$O(n)$	$O(\lambda \log n \log\log n)$	$O(\lambda \log\log n + \log n)$

We provide a definition of efficiently codeswitchable code families to contextualize this theorem.

Definition 2.7 (Efficiently codeswitchable code family)**.** *An efficiently codeswitchable code family over a field* $\mathbb{F}$ *is a set of codes* $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$*, with corresponding parameter functions* $n(\cdot)$ *and* $\delta(\cdot)$*, such that:*

- *For each* k*,* $\mathcal{C}_k$ *is a linear code with parameters* $(\mathbb{F}, n(k), k, \delta(k))$ *that is efficiently codeswitchable (see previous section for an informal definition).*

- *There exist constants δ_0 and ρ_0, and some $k_0 \in \mathbb{N}$ such that for all $k \geq k_0$, we have $\delta(k) \geq \delta_0$ and $k/n(k) \geq \rho_0$.*

Additionally, we say that a code family is linear-time encodable if for all k, $\mathcal{C}_k$ is a linear-time encodable code.

Prior work [GLSTW23; BCFRRZ25] shows that there exist linear-time encodable code families for arbitrary fields and binary extension fields respectively. BrakingBase [NST24] showed Brakedown's code is efficiently codeswitchable by showing that it is a systematic LDPC code, and developed an efficient codeswitch for it (their codeswitch is presented in the context of Brakedown's code, but it works for any systematic LDPC code). In this work we show that tensor codes of linear-time encodable codes are also efficiently codeswitchable (see Section 2.8).

The halving IOR. A key part of the construction of FICS is our halving IOR Halve, from $\mathsf{MEP}^{(\mathcal{C}_k,\varepsilon)}$ to $\mathsf{MEP}^{(\mathcal{C}_{k/2},\varepsilon)}$, where $\mathcal{C}_k, \mathcal{C}_{k/2}$ are from an efficiently codeswitchable code family $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$. That is, it reduces the problem of convincing $\mathbf{V}$ that

$$\delta(\boldsymbol{w}, \mathsf{Enc}_{\mathcal{C}_k}(\boldsymbol{m})) \leq \varepsilon \quad \text{and} \quad \hat{m}(\boldsymbol{z}) = y \ ,$$

where $\boldsymbol{m} \in \mathbb{F}^k$ and $\boldsymbol{z} \in \mathbb{F}^{\log k}$, to the problem of convincing $\mathbf{V}$ that

$$\delta\Big(\boldsymbol{w}', \mathsf{Enc}_{\mathcal{C}_{k/2}}(\boldsymbol{m}')\Big) \leq \varepsilon \quad \text{and} \quad \hat{m}'(\boldsymbol{z}') = y' \ ,$$

where $\boldsymbol{m}' \in \mathbb{F}^{k/2}$ and $\boldsymbol{z} \in \mathbb{F}^{\log k - 1}$. $\mathbf{P}$ of Halve runs in linear time and $\mathbf{V}$ runs in time $O(\lambda \log n)$ and makes $O(\lambda)$ queries. We defer the details of the construction of Halve, which leverages codeswitching, to Section 2.5.

IOPP construction. For now we focus on how to use Halve to construct FICS. Let $\mathcal{C} := C_k$ be a code from an efficiently codeswitchable code family $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$. Section 2.5 shows that for each $i \in [\log\log k]$, there exists an IOR from $\mathsf{MEP}^{(\mathcal{C}_{k/2^{i-1}},\varepsilon)}$ to $\mathsf{MEP}^{(\mathcal{C}_{k/2^i},\varepsilon)}$, which we denote Halve_i. We define RS to be the Reed-Solomon code with message length $k/\log k$, constant distance δ_{RS}, and constant rate. Prior work, WHIR [ACFY25] presents an IOP for $\mathsf{MEP}^{(\mathsf{RS},\delta_{\mathsf{RS}}/4)}$ with quasi-linear prover time, $O(\lambda \log n)$ verifier time and $O(\lambda \log\log n + \log n)$ query complexity.

We obtain FICS, our IOP for $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$, by composing the following IORs:

1. For $i \in [\log\log k]$: Halve_i from $\mathsf{MEP}^{(\mathcal{C}_{k/2^{i-1}},\varepsilon)}$ to $\mathsf{MEP}^{(\mathcal{C}_{k/2^i},\varepsilon)}$,
2. CodeSwitch from $\mathsf{MEP}^{(\mathcal{C}_{k/\log k},\varepsilon)}$ to $\mathsf{MEP}^{(\mathsf{RS},\delta_{\mathsf{RS}}/4)}$, and
3. WHIR from $\mathsf{MEP}^{(\mathsf{RS},\delta_{\mathsf{RS}}/4)}$ to $\mathcal{R}_{\mathsf{true}}$.

Remark 2.8 (choice of $\mathbb{F}$ and $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$)**.** Notice that we must be able to define a Reed-Solomon code RS over the field $\mathbb{F}$. This implies that the field $\mathbb{F}$ must be FFT-friendly (to allow for encoding of RS in quasi-linear time). Thus the code family $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$ must also be defined over the same field $\mathbb{F}$, but this is not a concern since:

- We can set $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$ to be the code family from Brakedown [GLSTW23], which is field-agnostic, or a code family of tensor codes of any field-agnostic linear-time encodable code.
- We could alternatively set $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$ to be the code family from Blaze [BCFRRZ25]. While this code family is defined over binary extension fields, this is not a problem since [LAH16] shows that binary extension fields are FFT-friendly.

Efficiency analysis. We first analyze $\mathbf{P}$: $\mathbf{P}$ of each Halve_i runs in linear time, and the FICS prover runs $\mathsf{Halve}_1, \ldots, \mathsf{Halve}_{\log\log k}$ on inputs of size $k, k/2, k/4, \ldots, k/\log k$. Therefore this takes $O(k + k/2 + k/4 +$

$\ldots + k/\log k) = O(k) = O(n)$ time. $\mathbf{P}$ for CodeSwitch runs in linear time except for encoding the $\boldsymbol{m}'$ corresponding to $\mathsf{MEP}^{(\mathsf{RS},\delta_{\mathsf{RS}}/4)}$ under RS, which can also be done in $O(k)$ time since $|\boldsymbol{m}'| = k/\log k$. $\mathbf{P}$ of WHIR runs in quasi-linear time, but since it is run on an input of size $k/\log k$ this runs in linear time as well, so the total time taken by the FICS prover is linear.

We now analyze $\mathbf{V}$: since $\mathbf{V}$ of Halve runs in $O(\lambda \log n)$ time and makes $O(\lambda)$ queries, and there are $\log\log k$ iterations of Halve, the contribution to total time is $O(\lambda \log n \log\log n)$ and the contribution to total query complexity is $O(\lambda \log\log n)$. $\mathbf{V}$ of CodeSwitch runs in $O(\lambda \log n)$ time and has query complexity $O(\lambda)$, and $\mathbf{V}$ of WHIR runs in $O(\lambda \log n)$ time and makes $O(\lambda \log\log n + \log n)$ queries. Therefore, the total time taken by the FICS verifier is $O(\lambda \log n \log\log n)$ and the total number of queries made is $O(\lambda \log\log n + \log n)$.

2.5 Halving IOR

Halve is an IOR from $\mathsf{MEP}^{(\mathcal{C}_k,\varepsilon)}$ to $\mathsf{MEP}^{(\mathcal{C}_{k/2},\varepsilon)}$, where $\mathcal{C}_k, \mathcal{C}_{k/2}$ are from an efficiently codeswitchable code family $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$. For brevity we will denote $\mathcal{C}_k$ as $\mathcal{C}$ and $\mathcal{C}_{k/2}$ as $\mathcal{C}'$. Our construction is based on the following fact about multilinear polynomial evaluation: given $\boldsymbol{m} \in \mathbb{F}^k$ and an evaluation point $\boldsymbol{z} \in \mathbb{F}^{\log k}$, one can interpret $\boldsymbol{z} = (z_1 \,\|\, \boldsymbol{z}')$ and express $\hat{m}(\boldsymbol{z})$ as

$$\begin{aligned}
\hat{m}(\boldsymbol{z}) &= \sum_{\boldsymbol{b}\in\{0,1\}^{\log k}} \mathrm{eq}(\boldsymbol{z},\boldsymbol{b}) \cdot \hat{m}(\boldsymbol{b}) \\
&= (1-z_1) \cdot \sum_{\boldsymbol{b}'\in\{0,1\}^{\log k-1}} \mathrm{eq}(\boldsymbol{z}',\boldsymbol{b}') \cdot \hat{m}(0,\boldsymbol{b}') + z_1 \cdot \sum_{\boldsymbol{b}'\in\{0,1\}^{\log k-1}} \mathrm{eq}(\boldsymbol{z}',\boldsymbol{b}') \cdot \hat{m}(1,\boldsymbol{b}') \\
&= (1-z_1) \cdot \hat{m}_L(\boldsymbol{z}') + z_1 \cdot \hat{m}_R(\boldsymbol{z}') \ ,
\end{aligned}$$

where $\hat{m}_L, \hat{m}_R$ are multilinear extensions of $\boldsymbol{m}_L, \boldsymbol{m}_R \in \mathbb{F}^{k/2}$ respectively, and $\boldsymbol{m} = (\boldsymbol{m}_L \,\|\, \boldsymbol{m}_R)$. Before discussing the construction of Halve, we must first define the interleaved code $\mathsf{IC}[\mathcal{C}', 2]$, which has message length k.

Definition 2.9 (Interleaved code $\mathsf{IC}[\mathcal{B}, 2]$ (informal))**.** *Let $\mathcal{B}$ be a linear code with parameters $(\mathbb{F}, n, k, \delta_{\mathcal{B}})$. The* interleaved code $\mathcal{C} := \mathsf{IC}[\mathcal{B}, 2]$ *is a linear code with parameters $(\mathbb{F}, 2n, 2k, \delta_{\mathcal{B}}/2)$ such that*

$$\mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}) = \mathsf{Enc}_{\mathcal{B}}(\boldsymbol{m}_L) \,\|\, \mathsf{Enc}_{\mathcal{B}}(\boldsymbol{m}_R) \ .$$

The halving IOR starts off by running a codeswitch IOR from $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$ to $\mathsf{MEP}^{(\mathsf{IC}[\mathcal{C}',2],\varepsilon+\varepsilon_b)}$, where ε_b is the distance loss incurred due to the Batch IOR (ε_b is a small constant, and we defer the reader to Remark 6.11 for details). Let us parse the instance and witness of $\mathsf{MEP}^{(\mathsf{IC}[\mathcal{C}',2],\varepsilon+\varepsilon_b)}$ as $\big(\mathbb{x} := (\boldsymbol{z}, y), \quad \vec{\mathbb{y}} := \boldsymbol{w}, \quad \mathbb{w} := \boldsymbol{m}\big)$, then by the definition of the code $\mathsf{IC}[\mathcal{C}', 2]$, we have that

$$\boldsymbol{w} = \mathsf{Enc}_{\mathsf{IC}[\mathcal{C}',2]}(\boldsymbol{m}) = \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m}_L) \,\|\, \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m}_R) = \boldsymbol{w}_L \,\|\, \boldsymbol{w}_R \ .$$

Parse $\boldsymbol{z} = (z_1, \boldsymbol{z}')$. $\mathbf{V}$ needs to be convinced that $\hat{m}(\boldsymbol{z}) = y$ and $\delta\big(\boldsymbol{w}, \mathsf{Enc}_{\mathsf{IC}[\mathcal{C}',2]}(\boldsymbol{m})\big) \le \varepsilon + \varepsilon_b$. To prove the first condition, $\mathbf{P}$ sends y_L, that is purportedly $\hat{m}_L(\boldsymbol{z}')$, and y_R, that is purportedly $\hat{m}_R(\boldsymbol{z}')$, to $\mathbf{V}$, who checks that $y = (1-z_1)\cdot y_L + z_1 \cdot y_R$. Now, $\mathbf{P}$ needs to convince $\mathbf{V}$ that "$\hat{m}_L(\boldsymbol{z}') = y_L$ and $\hat{m}_R(\boldsymbol{z}') = y_R$" instead of "$\hat{m}(\boldsymbol{z}) = y$". The second condition is guaranteed to hold if $\delta(\boldsymbol{w}_L, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_L)) \le \varepsilon + \varepsilon_b$ and $\delta(\boldsymbol{w}_R, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_R)) \le \varepsilon + \varepsilon_b$. Thus, $\mathbf{P}$ is just left to convince $\mathbf{V}$ of the following claims:

- $\delta(\boldsymbol{w}_L, \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m}_L)) \le \varepsilon + \varepsilon_b$ and $\hat{m}_L(\boldsymbol{z}') = y_L$, and
- $\delta(\boldsymbol{w}_R, \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m}_R)) \le \varepsilon + \varepsilon_b$ and $\hat{m}_R(\boldsymbol{z}') = y_R$.

This is precisely an instance of $\mathsf{HEP}^{(\mathcal{C}',\varepsilon+\varepsilon_b,2)}$, so we can compose with the Batch IOR (Section 6.3) to reduce this to an instance of $\mathsf{MEP}^{(\mathcal{C}',\varepsilon)}$. The full construction of Halve can be found in Section 9.2.

2.6 Generalizing FICS to obtain a new blueprint for constructing IOPPs

We now generalize the ideas in FICS to present an IOR-based blueprint for constructing an IOP for $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$. Let $\mathcal{C}_0 := \mathcal{C}$ have parameters $(\mathbb{F}, k_0, n_0, \delta_{\mathcal{C}_0})$ and $\varepsilon_0 := \varepsilon$. Furthermore, let $\mathcal{C}_1, \ldots, \mathcal{C}_t$ be a sequence of codes such that for each $i \in [1, \ldots, t]$:

- $\mathcal{C}_i$ has parameters $(\mathbb{F}, k_i, n_i, \delta_{\mathcal{C}_i})$ such that $k_i < k_{i-1}$,
- $\varepsilon_i \in (0, \delta_{\mathcal{C}_i}/2)$ is the i-th proximity parameter, and
- Reduce_i is an IOR from $\mathsf{MEP}^{(\mathcal{C}_{i-1},\varepsilon_{i-1})}$ to $\mathsf{MEP}^{(\mathcal{C}_i,\varepsilon_i)}$ with prover time PT_i, verifier time VT_i and query complexity q_i.

Finally, let $\mathsf{IOP}_{\mathsf{fin}}$ be an IOP for $\mathsf{MEP}^{(\mathcal{C}_t,\varepsilon_t)}$ with prover time $\mathsf{PT}_{\mathsf{fin}}$, verifier time $\mathsf{VT}_{\mathsf{fin}}$ and query complexity q_{fin}. This could either be an existing IOP from prior work, or the naive IOP where $\mathbf{P}$ simply sends the entire witness message to $\mathbf{V}$ (since by this point it is quite small). We can construct an IOP for $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$ by composing the IORs $\mathsf{Reduce}_1, \ldots, \mathsf{Reduce}_t$ to obtain an IOR from $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$ to $\mathsf{MEP}^{(\mathcal{C}_t,\varepsilon_t)}$, which is then further composed with the IOP $\mathsf{IOP}_{\mathsf{fin}}$ for $\mathsf{MEP}^{(\mathcal{C}_t,\varepsilon_t)}$. $\mathbf{P}$ time of the resulting IOP is $\sum_{i\in[t]} \mathsf{PT}_i + \mathsf{PT}_{\mathsf{fin}}$, $\mathbf{V}$ time is $\sum_{i\in[t]} \mathsf{VT}_i + \mathsf{VT}_{\mathsf{fin}}$, and the query complexity is $\sum_{i\in[t]} q_i + q_{\mathsf{fin}}$.

2.7 FACS: linear-time accumulation for NP

FACS is an accumulation scheme for NP that achieves linear prover time and requires proof size $O(\lambda \log n)$ hashes per input. Recall that a non-interactive argument of knowledge ARG for an NP-complete relation $\mathcal{R}$ allows $\mathbf{P}$ to produce a proof π that convinces $\mathbf{V}$ that it knows a witness $\mathbb{w}$ such that $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$. At a high level, an accumulation scheme for ARG is used to check that $\mathbf{V}(\mathbb{x}_1, \pi_1), \ldots, \mathbf{V}(\mathbb{x}_n, \pi_n)$ all accept more efficiently than running $\mathbf{V}$ on each proof separately.

In more detail, the accumulation scheme ACC consists of $\mathcal{P}$, $\mathcal{V}$ and a decider $\mathcal{D}$. We initialize the empty accumulator $\mathsf{acc}_0 := \perp$. $\mathcal{P}$, given a tuple $(\mathbb{x}, \pi)$ "accumulates" it into the accumulator acc_0 to obtain acc_1 and also produces a proof pf_1 that the accumulation step was performed correctly. We denote this by $(\mathsf{acc}_1, \mathsf{pf}_1) \leftarrow \mathcal{P}(\mathbb{x}_1, \pi_1, \mathsf{acc}_0)$. For each $i \in \{2, \ldots, n\}$ we can continue to accumulate $(\mathsf{acc}_i, \mathsf{pf}_i) \leftarrow \mathcal{P}(\mathbb{x}_i, \pi_i, \mathsf{acc}_{i-1})$, to obtain acc_n and $[\mathsf{pf}_i]_{i\in[n]}$. An accumulation scheme guarantees that if the accumulation verifier $\mathcal{V}$ accepts the proofs that each accumulation step was performed correctly, i.e. for all $i \in [n] : \mathcal{V}(\mathbb{x}_i, \pi_i, \mathsf{acc}_{i-1}, \mathsf{acc}_i, \mathsf{pf}_i) = 1$, and the final accumulator is accepted by the accumulation decider $\mathcal{D}$, i.e. $\mathcal{D}(\mathsf{acc}_n) = 1$, then it must be the case that for all $i \in [n]$: $\mathbf{V}(\mathbb{x}_i, \pi_i) = 1$. Notice that the decider $\mathcal{D}$ is run only once at the end, but $\mathcal{V}$ is run n times, at every accumulation step. It is thus very important for the accumulation verifier to be efficient.

We construct an argument of knowledge ARG for the NP-complete R1CS relation $\mathcal{R}_{\mathsf{R1CS}}$ and a corresponding accumulation scheme ACC with a linear-time $\mathcal{P}$ and an $\mathcal{V}$ that checks $O(\lambda)$ Merkle paths. We use [BMNW25b, Theorem 4.3] for this construction, which we informally state here.

Lemma 2.10 (informal version of Theorem 4.3, [BMNW25b])**.** *There exists a polynomial-time transformation that given an NP relation $\mathcal{R}$ and the following ingredients:*

- $\mathsf{RDX}_{\mathsf{CAST}}$*, a non-interactive reduction (NIRK) from $\mathcal{R}$ to $\mathcal{R}_{\mathsf{ACC}}$.*
- $\mathsf{RDX}_{\mathsf{FOLD}}$*, a non-interactive reduction from $\mathcal{R}^2_{\mathsf{ACC}}$ to $\mathcal{R}_{\mathsf{ACC}}$.*

outputs $(\mathsf{ARG}, \mathsf{ACC})$*, where* ARG *is a non-interactive argument for $\mathcal{R}$ and* ACC *is an accumulation scheme for* ARG*, both in the random oracle model, that inherit the efficiency properties of the ingredients.*

Here, $\mathcal{R}^2_{\mathsf{ACC}}$ is the multi-instance relation corresponding to $\mathcal{R}_{\mathsf{ACC}}$, which consists of tuples of the form $((\mathbb{x}_1, \mathbb{x}_2), (\mathbb{w}_1, \mathbb{w}_2))$ such that $(\mathbb{x}_i, \mathbb{w}_i) \in \mathcal{R}_{\mathsf{ACC}}$ for all $i \in [2]$. We fix $\mathcal{R}$ to be $\mathcal{R}_{\mathsf{R1CS}}$ and $\mathcal{R}_{\mathsf{ACC}}$ to be

$\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$ for some $\varepsilon \in (\delta_{\mathcal{C}}/8, \delta_{\mathcal{C}}/3 - \varepsilon_b)$. It is easy to inspect that $\mathcal{R}^2_{\mathsf{ACC}}$ is then the relation $\mathsf{MEP}^{(\mathcal{C},\varepsilon,2)}$. We are now left to construct the required NIRKs $\mathsf{RDX}_{\mathsf{CAST}}$ and $\mathsf{RDX}_{\mathsf{FOLD}}$.

Remark 2.11 (many-to-one accumulation)**.** The full details of our accumulation scheme can be found in Section 10. There we deal with the full notion of accumulation, which differs from the one presented here in that it allows for many-to-one accumulation, i.e. at each accumulation step we would like to accumulate n old accumulators $[\mathsf{acc}_i]_{i\in[n]}$ and m new tuples $(\mathbb{x}_i, \pi_i)_{i\in[m]}$ into a single new accumulator acc'. The only modification this requires is that $\mathsf{RDX}_{\mathsf{FOLD}}$ must instead be a NIRK from $\mathcal{R}^*_{\mathsf{ACC}}$ to $\mathcal{R}_{\mathsf{ACC}}$, where $\mathcal{R}^*_{\mathsf{ACC}}$ contains an arbitrary m number of instance-witness pairs for ARG. This is a simple extension of the techniques presented in this section, and we refer the reader to Section 10 for further details.

$\mathsf{RDX}_{\mathsf{CAST}}$. To construct the NIRK $\mathsf{RDX}_{\mathsf{CAST}}$, it suffices to construct an IOR CAST from $\mathcal{R}_{\mathsf{R1CS}}$ to $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$ with a linear-time $\mathbf{P}$ and a $\mathbf{V}$ that makes $O(\lambda)$ queries, as it can then be compiled into a NIRK via (a variant of) the BCS transform.

We construct CAST by applying our compiler from MLIOPs to IORs (see Section 2.3) to existing MLIOPs for $\mathcal{R}_{\mathsf{R1CS}}$. Prior work [Set20; Hu+25] has presented such MLIOPs where $\mathbf{P}$ runs in linear-time and sends ℓ (a constant) polynomial oracles to $\mathbf{V}$, which can be compiled into an IOR from $\mathcal{R}_{\mathsf{R1CS}}$ to $\mathsf{MEP}^{(\mathcal{C},\varepsilon+\varepsilon_b,\ell)}$. We then further compose with $\mathsf{Homogenize}$, Batch, $\mathsf{CodeSwitch}$ to obtain the IOR CAST from $\mathcal{R}_{\mathsf{R1CS}}$ to $\mathsf{MEP}^{(\mathcal{C},\varepsilon+\varepsilon_b)}$.

$\mathsf{RDX}_{\mathsf{FOLD}}$. To construct the NIRK $\mathsf{RDX}_{\mathsf{FOLD}}$, it suffices to construct an IOR FOLD from $\mathsf{MEP}^{(\mathcal{C},\varepsilon+\varepsilon_b,2)}$ to $\mathsf{MEP}^{(\mathcal{C},\varepsilon+\varepsilon_b)}$ with a linear-time $\mathbf{P}$ and a $\mathbf{V}$ that makes $O(\lambda)$ queries, as it can then be compiled into an NIRK via (a variant of) the BCS transform. FOLD can be obtained by composing the following IORs:

(a) $\mathsf{Homogenize}$ from $\mathsf{MEP}^{(\mathcal{C},\varepsilon+\varepsilon_b,2)}$ to $\mathsf{HEP}^{(\mathcal{C},\varepsilon+\varepsilon_b,2)}$, and
(b) Batch from $\mathsf{HEP}^{(\mathcal{C},\varepsilon+\varepsilon_b,2)}$ to $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$.
(c) $\mathsf{CodeSwitch}$ from $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$ to $\mathsf{MEP}^{(\mathcal{C},\varepsilon+\varepsilon_b)}$ to "boost the proximity parameter",

It is easy to inspect that the constructed NIRKs satisfy the required efficiency properties. We refer the reader to Section 10 for further details and analysis.

2.8 Codeswitch for tensor codes

In this section we motivate the construction of our efficient $\mathsf{CodeSwitch}$ IOR for tensor codes.

Tensor codes. Given a linear code $\mathcal{B} \subseteq \mathbb{F}^n$ with message length k and distance $\delta_{\mathcal{B}}$, we define the tensor code $\mathcal{C} := \mathsf{TC}[\mathcal{B}, 2] \subseteq \mathbb{F}^{n^2}$ with message length k^2 and distance $\delta^2_{\mathcal{B}}$ as follows: to encode a message under $\mathcal{C}$, we arrange the given message into a matrix of dimension $k \times k$, encode the rows under $\mathcal{B}$ to obtain a matrix of dimension $n \times k$, and then encode the columns under $\mathcal{B}$ to obtain a matrix of dimension $n \times n$.[8] Thus, if $\mathcal{B}$ can be encoded in linear time then $\mathcal{C}$ can also be encoded in linear time (with respect to its block length).

Generator matrix. The encoding of $\boldsymbol{m}$ under a linear code is equal to the matrix-vector product of the *generator matrix* of the code with $\boldsymbol{m}$. The key reason we choose tensor codes for building a codeswitch is because the generator matrix $\boldsymbol{G}_{\mathcal{C}}$ of $\mathcal{C}$ is the Kronecker product of the generator matrix $\boldsymbol{G}_{\mathcal{B}}$ of $\mathcal{B}$ with itself. This structural property is exploited in our codeswitch to make the prover efficient.

Outline of the codeswitch IOR. We show how to switch from $\mathcal{C} = \mathsf{TC}[\mathcal{B}, 2]$ to any code $\mathcal{C}'$ with message length k and block length n'.[9] Recall that a $\mathsf{CodeSwitch}$ IOR is a reduction from an instance of $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$ to

[8] In Section 7, we generalize the "tensoring dimension" to an arbitrary constant $t \geq 2$, we assume $t = 2$ here for simplicity.

[9] We assume that k, n, n' are powers of 2. This is without loss of generality and does not lead to an asymptotic loss in any efficiency parameter of the IOR.

an instance of $\mathsf{MEP}^{(\mathcal{C}',\varepsilon')}$. Thus, given an instance of $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$, the prover wants to reduce the following claim to a claim about $\mathcal{C}'$: "there exists a message $\boldsymbol{m} \in \mathbb{F}^k$ such that the following two properties hold: (a) $\delta(\boldsymbol{G}_{\mathcal{C}} \cdot \boldsymbol{m}, \boldsymbol{w}) \leq \varepsilon$ (proximity claim), and (b) $\hat{m}(\boldsymbol{z}) = y$ (evaluation claim)." This reduction is performed using the following high-level outline:

1. The honest prover encodes $\boldsymbol{m}$ under $\mathcal{C}'$ and sends the resulting codeword $\boldsymbol{w}' := \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m})$ to the verifier. Then, it proves that $\boldsymbol{w}$ and $\boldsymbol{w}'$ encode the same message $\boldsymbol{m}$. If the verifier is convinced of this claim, the prover can then reduce the proximity claim to the following claim: "$\delta(\boldsymbol{w}', \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m})) \leq \varepsilon' + \varepsilon_b$".[10] This claim, together with the evaluation claim simply comprise an instance of $\mathsf{MEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b)}$. These steps are formalized via the "consistency checking" IOR later in this section.
2. To check that $\boldsymbol{w}$ and $\boldsymbol{w}'$ encode the same message, the consistency checking IOR produces several inner product claims, which are then reduced to sumcheck claims (instances of $\mathsf{SEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b)}$). This reduction is formalized via the "inner product" IOR later in this section.
3. Finally, the SEP instances produced by the inner product IOR are reduced to instances of HEP using ECSumcheck. The resulting MEP and HEP instances obtained from previous steps are then reduced to a single HEP instance using Homogenize. This HEP instance is then reduced to the final $\mathsf{MEP}^{(\mathcal{C}',\varepsilon')}$ instance using Batch.

This outline is presented visually in Fig. 1.

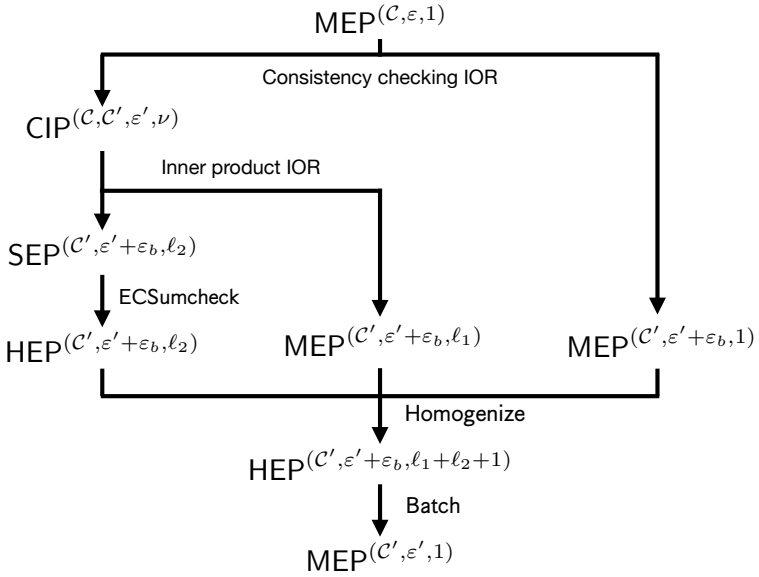


Figure 1: Structure of the CodeSwitch IOR from $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ to $\mathsf{MEP}^{(\mathcal{C}',\varepsilon',1)}$ where $\mathcal{C}$ is a tensor code. The steps labelled "consistency checking IOR" and "inner product IOR" only apply to the CodeSwitch IOR for tensor codes, but the subsequent reductions are generic and independent of $\mathcal{C}$.

Consistency checking IOR. Recall from the outline above, that in addition to producing an instance of $\mathsf{MEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b)}$, the consistency checking IOR checks that $\boldsymbol{w}$ and $\boldsymbol{w}'$ encode the same message $\boldsymbol{m}$. This is done via "spot checks": the verifier picks $\nu = O(\lambda)$ many random indices $\alpha_1, \ldots, \alpha_\nu \in \{0, \ldots, n-1\}$, and checks that for all $i \in [\nu]$, we have $\boldsymbol{G}_{\mathcal{C}}[\alpha_i] \cdot \boldsymbol{m} = \boldsymbol{w}[\alpha_i]$. If these ν equalities hold, the probability that $\delta(\mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}), \boldsymbol{w}) \leq \varepsilon$ is at least $1 - 2^{-\lambda}$. Additionally, if $\delta(\boldsymbol{w}', \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m})) \leq \varepsilon + \varepsilon_b$, then the closest

[10] Here $\varepsilon + \varepsilon_b$ is less than the unique decoding radius of $\mathcal{C}'$. Recall that ε_b is a small constant; see Remark 6.11 for details.

codeword to both $\boldsymbol{w}$ and $\boldsymbol{w}'$ is $\mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m})$, as desired. These checks are captured by the *code-inner-product relation* $\mathsf{CIP}^{(\mathcal{C},\mathcal{C}',\varepsilon,\nu)}$:

Definition 2.12 ($\mathsf{CIP}^{(\mathcal{C},\mathcal{C}',\varepsilon,\nu)}$ relation (informal))**.** *The relation* $\mathsf{CIP}^{(\mathcal{C},\mathcal{C}',\varepsilon,\nu)}$ *consists of tuples of the form* $(\mathbb{x} := (\alpha_1, \ldots, \alpha_\nu), \vec{\mathbb{y}} := (\boldsymbol{w}, \boldsymbol{w}'), \mathbb{w} := \boldsymbol{m})$ *such that* $\forall i \in [\nu]$*, we have* $\boldsymbol{G}_{\mathcal{C}}[\alpha_i] \cdot \boldsymbol{m} = \boldsymbol{w}[\alpha_i]$*, and* $\delta(\boldsymbol{w}', \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m})) \leq \varepsilon + \varepsilon_b$*.*

This reduction is an IOR from $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$ to $\mathsf{CIP}^{(\mathcal{C},\mathcal{C}',\varepsilon,\nu)} \times \mathsf{MEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b)}$, and it is discussed in detail in Section 7.1. The efficiency of this IOR does not rely on the fact that $\mathcal{C}$ is a tensor code; this will be exploited in the following reduction.

Inner Product IOR. Given an instance of $\mathsf{CIP}^{(\mathcal{C},\mathcal{C}',\varepsilon,\nu)}$, we show how to reduce it to instances of MEP and SEP using the structure of tensor codes. We describe our techniques in terms of polynomial oracles for readability, but when describing our final IORs in Section 7 we replace these with vector oracles to the encodings of the multilinear polynomials' evaluations over the boolean hypercube.

Attempt 1: A natural approach is to interpret every inner-product claim $\boldsymbol{G}_{\mathcal{C}}[\alpha_i] \cdot \boldsymbol{m} = \boldsymbol{w}[\alpha_i]$ as a sumcheck claim. Clearly, if the prover sends polynomial oracles to the multilinear extensions of each $\boldsymbol{G}_{\mathcal{C}}[\alpha_i]$, and the multilinear extension of $\boldsymbol{m}$, then the prover and the verifier can engage in ν many sumcheck protocols to verify all the inner-product claims. However, each polynomial oracle query translates to $\Omega(\lambda)$ many vector oracle queries (assuming state-of-the-art code-based polynomial commitment schemes such as STIR [ACFY24] and WHIR [ACFY25]), and therefore the total number of queries made by the verifier will be $\Omega(\lambda\nu) = \Omega(\lambda^2)$, which is larger than our verifier efficiency budget.

Attempt 2: Batching sumcheck claims. We can attempt to reduce the number of queries by batching the ν sumcheck claims together as follows. The verifier sends a random challenge $\beta \xleftarrow{\$} \mathbb{F}$, and the prover convinces the verifier that

$$\left(\sum_{i=1}^{\nu} \beta^{i-1} \cdot \boldsymbol{G}_{\mathcal{C}}[\alpha_i]\right) \cdot \boldsymbol{m} = \sum_{i=1}^{\nu} \beta^{i-1} \cdot \boldsymbol{w}[\alpha_i] \ . \tag{1}$$

If the prover convinces the verifier of this claim, then except with probability $\nu/|\mathbb{F}|$, the verifier is convinced that $\boldsymbol{G}_{\mathcal{C}}[\alpha_i] \cdot \boldsymbol{m} = \boldsymbol{w}[\alpha_i]$ for all $i \in [\nu]$.

This allows the prover to send a single polynomial oracle to the multilinear extension of $\boldsymbol{q} := \left(\sum_{i=1}^{\nu} \beta^{i-1} \cdot \boldsymbol{G}_{\mathcal{C}}[\alpha_i]\right)$, instead of the ν many polynomial oracles sent in the first approach. However, the prover now has to convince the verifier that the polynomial in the oracle is indeed the multilinear extension of $\sum_{i=1}^{\nu} \beta^{i-1} \cdot \boldsymbol{G}_{\mathcal{C}}[\alpha_i]$. This can be done by "spot checking" the oracle of $\boldsymbol{q}$ with respect to the ν oracles to $[\boldsymbol{G}_{\mathcal{C}}[\alpha_i]]_{i=1}^{\nu}$, but that still requires $\Omega(\lambda^2)$ many queries.

Our scheme: Interpret the entire generator matrix as a polynomial. The key problem in our approach so far is that we are initializing multiple polynomial oracles, and the verifier has to query at least ν many of them, which is prohibitively expensive. Therefore, instead of initializing a polynomial oracle for each row of the generator matrix, we consider a single polynomial oracle that represents the entire generator matrix. This motivates us to rephrase Eq. (1) in terms of the entire generator matrix $\boldsymbol{G}_{\mathcal{C}}$ using a selector vector $\boldsymbol{h} \in \mathbb{F}^n$ such that $\boldsymbol{h}[\alpha_i] = \beta^{i-1}$ for all $i \in [\nu]$, and 0 otherwise. This gives us the following vector-matrix-vector product claim:

$$\boldsymbol{h}^T \cdot \boldsymbol{G}_{\mathcal{C}} \cdot \boldsymbol{m} = \sum_{i=1}^{\nu} \beta^{i-1} \cdot \boldsymbol{w}[\alpha_i] \ . \tag{2}$$

We can define $\hat{c} : \mathbb{F}^{\log n + \log k} \to \mathbb{F}$ as the unique multilinear polynomial such that $\hat{c}(\langle x \rangle, \langle y \rangle) = \boldsymbol{G}_{\mathcal{C}}[x][y]$ for

all $x \in \{0, \ldots, n-1\}$ and $y \in \{0, \ldots, k-1\}$.[11] Similarly, we define the selector polynomial $\hat{h} : \mathbb{F}^{\log n} \to \mathbb{F}$ such that $\hat{h}(\langle x \rangle) = \boldsymbol{h}[x]$ and the message polynomial $\hat{m} : \mathbb{F}^{\log k} \to \mathbb{F}$ such that $\hat{m}(\langle y \rangle) = \boldsymbol{m}[y]$. We can now rephrase Eq. (2) as the following sumcheck claim:

$$\sum_{\substack{\boldsymbol{x} \in \{0,1\}^{\log n}, \\ \boldsymbol{y} \in \{0,1\}^{\log k}}} \hat{c}(\boldsymbol{x}, \boldsymbol{y}) \cdot \hat{m}(\boldsymbol{y}) \cdot \hat{h}(\boldsymbol{x}) = \sum_{i=1}^{\nu} \beta^{i-1} \cdot \boldsymbol{w}[\alpha_i] \ . \tag{3}$$

We can use sumcheck to reduce Eq. (3) to a polynomial evaluation claim for $\hat{c}$, $\hat{h}$, and $\hat{m}$ at a random point. The verifier can evaluate $\hat{h}$ at a random point on its own in $O(\lambda \log n)$ time[12], and requires at most $O(1)$ queries to the polynomial oracles of $\hat{c}$ and $\hat{m}$. Assuming an implementation of these oracles based on WHIR [ACFY25], this translates to $O(\lambda \log \log n + \log n)$ vector oracle queries. Thus, we have decreased the verifier overhead by a factor of λ by reducing the inner-product claims to a single sumcheck claim.

However, we now have a new problem: the above sumcheck claim involves a multilinear polynomial $\hat{c}$ that has size $O(nk)$, and therefore it takes $O(nk)$ time to produce a polynomial oracle for $\hat{c}$. Furthermore, even if we efficiently reduce the given $\mathsf{CIP}^{(\mathcal{C}, \mathcal{C}', \varepsilon, \nu)}$ to the sumcheck claim of Eq. (3), using the ECSumcheck IOR to reduce it further to an instance of MEP would require $O(nk)$ time on behalf of the prover (since the size of the domain is $2^{\log n} \cdot 2^{\log k} = nk$). But we still have *not used any properties of tensor codes*. Therefore, if we allow the prover to take quadratic time, we obtain the following result:

Lemma 2.13 (All codes are codeswitchable in quadratic prover time (informal))**.** *Consider linear codes $\mathcal{C}$ and $\mathcal{C}'$ with block lengths n and n' respectively, the same message length k, and constant rate and distance. There exists a* CodeSwitch *IOR from* $\mathsf{MEP}^{(\mathcal{C}, \varepsilon)}$ *to* $\mathsf{MEP}^{(\mathcal{C}', \varepsilon')}$ *with $O(nk)$ prover time, $O(\lambda)$ queries, $O(\lambda \log n)$ verifier time.*

Lemma 2.13 is interesting in its own right, but a quadratic prover time CodeSwitch IOR is not particularly useful. This is where we finally exploit the structure of tensor codes, and achieve the desired verifier time/query complexity without sacrificing prover time.

Reducing prover time using tensor codes. Consider the generator matrix $\boldsymbol{G}_{\mathcal{C}}$ of $\mathcal{C}$, and consider indices $i \in \{0, \ldots, n-1\}$ and $j \in \{0, \ldots, k-1\}$ such that $i = i_1 + \sqrt{n} \cdot i_2$ and $j = j_1 + \sqrt{k} \cdot j_2$, where $i_1, i_2 \in \{0, \ldots, \sqrt{n} - 1\}$ and $j_1, j_2 \in \{0, \ldots, \sqrt{k} - 1\}$. Since $\boldsymbol{G}_{\mathcal{C}} = \boldsymbol{G}_{\mathcal{B}} \otimes \boldsymbol{G}_{\mathcal{B}}$, we have $\boldsymbol{G}_{\mathcal{C}}[i][j] = \boldsymbol{G}_{\mathcal{B}}[i_1][j_1] \cdot \boldsymbol{G}_{\mathcal{B}}[i_2][j_2]$. Therefore, if we consider the multilinear extensions $\hat{b}$ and $\hat{c}$ of $\boldsymbol{G}_{\mathcal{B}}$ and $\boldsymbol{G}_{\mathcal{C}}$ respectively, we have $\hat{c}(\boldsymbol{x}, \boldsymbol{y}) = \hat{b}(\boldsymbol{x}_1, \boldsymbol{y}_1) \cdot \hat{b}(\boldsymbol{x}_2, \boldsymbol{y}_2)$, where $\boldsymbol{x} = (\boldsymbol{x}_1 \,\|\, \boldsymbol{x}_2)$ and $\boldsymbol{y} = (\boldsymbol{y}_1 \,\|\, \boldsymbol{y}_2)$, and $\boldsymbol{x}_1, \boldsymbol{x}_2 \in \{0,1\}^{\log n/2}$ and $\boldsymbol{y}_1, \boldsymbol{y}_2 \in \{0,1\}^{\log k/2}$. This allows us to further simplify the sumcheck claim in Eq. (3):

$$\sum_{\substack{\boldsymbol{x} \in \{0,1\}^{\log n}, \\ \boldsymbol{y} \in \{0,1\}^{\log k}}} \hat{b}(\boldsymbol{x}_1, \boldsymbol{y}_1) \cdot \hat{b}(\boldsymbol{x}_2, \boldsymbol{y}_2) \cdot \hat{m}(\boldsymbol{y}) \cdot \hat{h}(\boldsymbol{x}) = \sum_{i=1}^{\nu} \beta^{i-1} \cdot \boldsymbol{w}[\alpha_i] \ . \tag{4}$$

The advantage of this representation is that the verifier only needs oracle access to $\hat{b}$ (which has size $\sqrt{nk}$) as opposed to $\hat{c}$ (which has size nk). In order to realize this polynomial oracle, the prover only has to encode a vector of length $\sqrt{nk}$, as opposed to one of length nk. If $\mathcal{C}$ has constant-rate, this takes the same amount of time (asymptoticly) as encoding the witness of length k.

[11] Since $\hat{c}$ only depends on the code $\mathcal{C}$, and not on the instance or witness of $\mathsf{CIP}^{(\mathcal{C}, \mathcal{C}', \varepsilon, \nu)}$, this polynomial can be initialized by a preprocessing step implemented by the *indexer*, which is formalized in Section 3.

[12] Because each $\mathrm{eq}(\langle \alpha_i \rangle, \boldsymbol{r})$ can be computed in $O(\log n)$ time, and a linear combination of ν of these polynomials can be computed in $O(\nu \log n)$ time.

Reducing size of sumcheck claim. Even though the size of the polynomials is now linear, the sumcheck claim in Eq. (4) is still over a domain of size $2^{\log n + \log k} = nk$. While it may be possible to implement an honest prover for this particular sumcheck claim that requires $O(\sqrt{nk})$ time, by exploiting the tensor structure of $\mathcal{C}$, we show how to reduce this claim to sumcheck claims over smaller domains of size $O(n)$, and therefore show that we have linear prover time just by invoking the ECSumcheck IOR in a black-box manner.

The key observation is that all polynomials in Eq. (4) are over at most $\log n$ variables. Therefore, we can try to decompose the claim into simpler ones by sending some intermediate polynomial oracles. Let's first try to decouple the $\hat{h}(\boldsymbol{x})$ term from the rest of the claim, by sending a polynomial oracle to $\hat{w}$, the multilinear extension of $\boldsymbol{w}$. This enables us to decompose Eq. (4) into the following two claims:

$$\sum_{\boldsymbol{x} \in \{0,1\}^{\log n}} \hat{w}(\boldsymbol{x}) \cdot \hat{h}(\boldsymbol{x}) = \sum_{i=1}^{\nu} \beta^{i-1} \cdot \boldsymbol{w}[\alpha_i] \ , \tag{5}$$

$$\forall \boldsymbol{x} \in \{0,1\}^{\log n} : \sum_{\boldsymbol{y} \in \{0,1\}^{\log k}} \hat{b}(\boldsymbol{x}_1, \boldsymbol{y}_1) \cdot \hat{b}(\boldsymbol{x}_2, \boldsymbol{y}_2) \cdot \hat{m}(\boldsymbol{y}) = \hat{w}(\boldsymbol{x}) \ . \tag{6}$$

Clearly, Eq. (5) is a sumcheck claim over a domain of size n, and therefore can be reduced to a polynomial evaluation claim in $O(n)$ time using the ECSumcheck IOR. Furthermore, note that both sides of Eq. (6) are multilinear polynomials in $\boldsymbol{x}$. Therefore, in order to be equal over the entire boolean hypercube, they must be equal as polynomials. Therefore, we can evaluate both sides at a random point $\boldsymbol{r} \in \mathbb{F}^{\log n}$:

$$\sum_{\boldsymbol{y} \in \{0,1\}^{\log k}} \hat{b}(\boldsymbol{r}_1, \boldsymbol{y}_1) \cdot \hat{b}(\boldsymbol{r}_2, \boldsymbol{y}_2) \cdot \hat{m}(\boldsymbol{y}) = \hat{w}(\boldsymbol{r}) \ , \tag{7}$$

where $\boldsymbol{r} = (\boldsymbol{r}_1 \,\|\, \boldsymbol{r}_2)$ such that $\boldsymbol{r}_1, \boldsymbol{r}_2 \in \{0,1\}^{\log n/2}$. This reduction from Eq. (6) to Eq. (7) incurs a soundness error of at most $\log n/|\mathbb{F}|$. In order to prove the claim in Eq. (7), the prover can send a claimed sum σ to the verifier, and prove these two claims:

$$\hat{w}(\boldsymbol{r}) = \sigma \quad \text{and} \quad \sum_{\boldsymbol{y} \in \{0,1\}^{\log k}} \hat{b}(\boldsymbol{r}_1, \boldsymbol{y}_1) \cdot \hat{b}(\boldsymbol{r}_2, \boldsymbol{y}_2) \cdot \hat{m}(\boldsymbol{y}) = \sigma \ .$$

The first claim is simply the evaluation of the polynomial oracle of $\hat{w}$ at the random point $\boldsymbol{r}$, and therefore corresponds to an instance of the $\mathsf{MEP}^{(\mathcal{C}', \varepsilon' + \varepsilon_b)}$ relation. The second claim is a sumcheck claim over a domain of size k, and can be reduced to a polynomial evaluation claim in $O(k)$ time using the ECSumcheck IOR, which finally gives us our desired reduction to instances of MEP.

Simplifications. In this overview, we have glossed over some details that are discussed in Section 7. In particular,

1. The claim $\sum_{\boldsymbol{y} \in \{0,1\}^{\log k}} \hat{b}(\boldsymbol{r}_1, \boldsymbol{y}_1) \cdot \hat{b}(\boldsymbol{r}_2, \boldsymbol{y}_2) \cdot \hat{m}(\boldsymbol{y}) = \sigma$ cannot be described cleanly in terms of the simple SEP relation, and therefore needs to be broken down further. In particular, each component multilinear polynomial of the SEP instance (which would be $\hat{b}(\boldsymbol{r}_1, \boldsymbol{y}_1), \hat{b}(\boldsymbol{r}_2, \boldsymbol{y}_2), \hat{m}(\boldsymbol{y})$ in this instance) must be summed over $\boldsymbol{y} \in \{0,1\}^{\log k}$, so that the ECSumcheck IOR yields an instance of HEP as opposed to one of MEP. Therefore, the prover initializes polynomials $\hat{e}_i(\boldsymbol{y}) := \hat{b}(\boldsymbol{r}_i, \boldsymbol{y}_i)$ for $i \in [2]$, and runs sumcheck on the claim "$\sum_{\boldsymbol{y} \in \{0,1\}^{\log k}} \hat{e}_1(\boldsymbol{y}) \cdot \hat{e}_2(\boldsymbol{y}) \cdot \hat{m}(\boldsymbol{y}) = \sigma$" instead.
2. We need to realize polynomial oracles for $\hat{b}$ and $\hat{w}$ when $\boldsymbol{b}$ and $\boldsymbol{w}$ have length $\sqrt{nk}$ and n respectively. However, $\mathcal{C}'$ has a message length of k, so cannot directly be used to obtain encodings for $\boldsymbol{b}$ and $\boldsymbol{w}$. This is resolved by encoding $\boldsymbol{b}$ and $\boldsymbol{w}$ under an *interleaved code* of $\mathcal{C}'$, and is described in Remark 6.16.

3 Interactive oracle reductions (IORs)

We recap the formal definition of IORs from [BCGGRS19; BMNW25b].

Indexed relations. An *indexed relation* $\mathcal{R}$ is a set of triples $\{(\mathbb{i}, \mathbb{x}, \mathbb{w})\}$ where $\mathbb{i}$ is the index, $\mathbb{x}$ is the instance, and $\mathbb{w}$ is the witness; the corresponding *indexed language* $\mathcal{L}(\mathcal{R})$ is the set of pairs $(\mathbb{i}, \mathbb{x})$ for which there exists a witness $\mathbb{w}$ such that $(\mathbb{i}, \mathbb{x}, \mathbb{w}) \in \mathcal{R}$. We use $\mathcal{R}^{\mathbb{i}}$ to denote the set of all tuples $(\mathbb{x}, \mathbb{w})$ such that $(\mathbb{i}, \mathbb{x}, \mathbb{w}) \in \mathcal{R}$.

Pair relations. Some proof systems exhibit a gap between completeness and soundness. We formalize this using pair relations of the form $\mathcal{R} = (\mathcal{R}, \tilde{\mathcal{R}})$, where completeness holds for a relation $\mathcal{R}$, but soundness only guarantees membership in a *relaxed* relation $\tilde{\mathcal{R}} \supseteq \mathcal{R}$.

Indexed oracle pair relations. In our IORs, we consider *indexed oracle pair relations* where the instance is split into a *short instance* $\mathbb{x}$ and a tuple of *instance strings* $\vec{\mathbb{y}} = (\mathbb{y}_1, \ldots, \mathbb{y}_{\mathsf{n}})$ written over some alphabet Σ. For notational convenience, we write $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \in \mathcal{R}$ to denote membership in an indexed oracle relation. In the relaxed relation $\tilde{\mathcal{R}}$, we allow instance strings to written over an augmented alphabet $\Sigma \sqcup \{\perp\}$; this will be useful when compiling IORs into non-interactive reductions.

Relations relative to a random oracle. A *relation relative to a random oracle*, denoted $\mathcal{R}^{\mathcal{U}}$, is a set of relations $\{\mathcal{R}^{\rho} : \rho \in \mathrm{supp}(\mathcal{U})\}$, where $\mathrm{supp}(\mathcal{U})$ denotes $\bigcup_{\lambda \in \mathbb{N}} \mathrm{supp}(\mathcal{U}(\lambda))$.

3.1 Formal definition

Let $\mathcal{R}$ and $\mathcal{R}'$ be indexed oracle pair relations. A (public-coin, holographic) interactive oracle reduction from $\mathcal{R}$ to $\mathcal{R}'$ is a tuple of polynomial-time algorithms $\mathsf{IOR} = (\mathbf{I}_{\mathsf{IOR}}, \mathbf{P}_{\mathsf{IOR}}, \mathbf{V}_{\mathsf{IOR}})$ with the following syntax.

- The indexer $\mathbf{I}_{\mathsf{IOR}}$ is a deterministic algorithm which receives as input an index $\mathbb{i}$ (for $\mathcal{R}$). It outputs a *short index* ι, *index oracle* $\vec{\mathbb{I}}$, and a new index $\mathbb{i}'$ (for $\mathcal{R}'$).
- The prover $\mathbf{P}_{\mathsf{IOR}}$ is an interactive algorithm which receives as input the index $\mathbb{i}$, short instance $\mathbb{x}$, instance oracles $\vec{\mathbb{y}}$, and witness $\mathbb{w}$. It engages in k rounds of interaction with the verifier $\mathbf{V}_{\mathsf{IOR}}$. In the i-th round, $\mathbf{P}_{\mathsf{IOR}}$ sends a proof oracle Π_i and a non-oracle message msg_i, and then receives a challenge r_i from $\mathbf{V}_{\mathsf{IOR}}$.
- The verifier $\mathbf{V}_{\mathsf{IOR}}$ is an interactive algorithm which receives as input the short index ι, short instance $\mathbb{x}$, oracle access to index oracle $\vec{\mathbb{I}}$, and oracle access to instance oracles $\vec{\mathbb{y}}$. It engages in k rounds of interaction with $\mathbf{P}_{\mathsf{IOR}}$. In the i-th round, $\mathbf{V}_{\mathsf{IOR}}$ receives oracle access to a proof oracle Π_i and a non-oracle message msg_i, and then sends a uniformly random challenge r_i.

At the end of the interaction, $\mathbf{V}_{\mathsf{IOR}}$ outputs a decision $b \in \{0, 1\}$ and a new instance $(\mathbb{x}', \vec{\mathbb{y}}')$; the new instance oracles are chosen as a function of the old instance oracles $\vec{\mathbb{y}}$ and the transcript of the interaction with the prover. The prover outputs $(b, \mathbb{x}', \vec{\mathbb{y}}', \mathbb{w}')$. We denote the IOR interaction by

$$(b, \mathbb{x}', \vec{\mathbb{y}}', \mathbb{w}') \leftarrow \langle \mathbf{P}_{\mathsf{IOR}}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}), \mathbf{V}_{\mathsf{IOR}}^{\vec{\mathbb{I}}, \vec{\mathbb{y}}}(\iota, \mathbb{x}) \rangle$$

Without loss of generality, the prover always moves first and the verifier is split into two phases. In the *interaction phase*, it samples challenges and sends them to the prover. In the *query phase*, it queries the index, instance, and proof oracles. The verifier's output is a deterministic function of its input and the transcript, which we denote

$$(b, \mathbb{x}', \vec{\mathbb{y}}') \leftarrow \mathbf{V}_{\mathsf{IOR}}^{\vec{\mathbb{I}}, \vec{\mathbb{y}}, \vec{\Pi}}(\iota, \mathbb{x}, \vec{\mathsf{msg}}, \vec{r}).$$

Completeness. An IOR is said to be complete if for any $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \in \mathcal{R}$,

$$\Pr\left[b = 1 \wedge (\mathbb{i}', \mathbb{x}', \vec{\mathbb{y}}', \mathbb{w}') \in \mathcal{R}' \;\middle|\; \begin{array}{l} (\iota, \vec{\mathbb{I}}, \mathbb{i}') \leftarrow \mathbf{I}_{\mathsf{IOR}}(\mathbb{i}) \\ (b, \mathbb{x}', \vec{\mathbb{y}}', \mathbb{w}') \leftarrow \langle \mathbf{P}_{\mathsf{IOR}}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}), \mathbf{V}_{\mathsf{IOR}}^{\vec{\mathbb{I}}, \vec{\mathbb{y}}, \vec{\Pi}}(\iota, \mathbb{x}) \rangle \end{array} \right] = 1.$$

Soundness. An IOR is said to have soundness error ε if for any $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}) \notin \mathcal{L}(\mathcal{R})$ and adversary $\widetilde{\mathbf{P}}_{\mathsf{IOR}}$,

$$\Pr\left[b = 1 \wedge (\mathbb{i}', \mathbb{x}', \vec{\mathbb{y}}', \mathbb{w}') \in \mathcal{R}' \;\middle|\; \begin{array}{l} (\iota, \vec{\mathbb{l}}, \mathbb{i}') \leftarrow \mathbf{I}_{\mathsf{IOR}}(\mathbb{i}) \\ (b, \mathbb{x}', \vec{\mathbb{y}}', \mathbb{w}') \leftarrow \langle \widetilde{\mathbf{P}}_{\mathsf{IOR}}, \mathbf{V}_{\mathsf{IOR}}^{\vec{\mathbb{l}}, \vec{\mathbb{y}}, \vec{\Pi}}(\iota, \mathbb{x}) \rangle \end{array}\right] \leq \varepsilon(\mathbb{i}, \mathbb{x}).$$

Knowledge Soundness. There are several notions of knowledge soundness for IORs. In Section 4, we present round-by-round tree-extractability, a new notion of knowledge soundness for IORs, and explain how it relates to existing notions from prior work.

Efficiency measures. We consider the following efficiency measures for an IOR, which may be a function of the short index ι.

- *Alphabet:* Σ is the set of symbols used to write the index, instance, and proof strings. In this paper we only consider $\Sigma = \mathbb{F}$ for some finite field $\mathbb{F}$.
- *Round complexity:* k is the number of back-and-forth interactions in the protocol.
- *Oracle length:* $\mathsf{L}^{\mathtt{I}}$ is the length of the index oracle, $\mathsf{L}^{\mathtt{Y}_j}$ is the length of the j-th instance oracle, and $\mathsf{L}^{\mathtt{P}_i}$ is the length of the i-th proof oracle. Let $\mathsf{L}_{\mathtt{max}}$ denote the maximum of these oracle lengths.
- *Message length:* s is the sum of the lengths of the prover messages, $\sum_{i=1}^{\mathsf{k}} |\mathsf{msg}_i|$.
- *Query complexity:* the verifier reads $\mathsf{q}^{\mathtt{I}}$ locations from the index string, $\mathsf{q}^{\mathtt{Y}_j}$ locations from the j-th instance string, and $\mathsf{q}^{\mathtt{P}_i}$ locations from the i-th proof string. Let q denote the total number of queries.
- *Prover time:* $T_{\mathbf{P}}$ is a function of the instance length $|\vec{\mathbb{y}}|+|\mathbb{x}|$ which represents the prover's time.
- *Verifier time:* $T_{\mathbf{V}}$ is a function of the instance length $|\vec{\mathbb{y}}|+|\mathbb{x}|$ which represents the verifier's time, *not including the time required to query the instance oracles*.
- *Randomness:* r_i is the number of bits in the i-th challenge sent by the verifier. We use r to denote the total number of random bits in all challenges.

The key efficiency measures that we are interested in this paper are the prover time, the query complexity, and the verifier time.

Remark 3.1 (non-oracle messages)**.** The definition in [BMNW25b] did not contain non-oracle messages, but it was remarked that it would be straightforward to include them. We have modified the definition of an IOR to always include non-oracle messages.

Remark 3.2 (fixed indices)**.** We sometimes write that an IOR is from $\mathcal{R}_1^{\mathbb{i}_1}$ to $\mathcal{R}_2^{\mathbb{i}_2}$, if we are only concerned with the case when the index for the input from $\mathcal{R}_1$ is always $\mathbb{i}_1$, which implies that the index for $\mathcal{R}_2$ is also fixed to $\mathbb{i}_2$ (which must be the output of the indexer on input $\mathbb{i}_1$).

4 Round-by-round tree-extractability

In this section we define *round-by-round tree-extractability* (RBRTE), a new framework for proving that IORs can be compiled into non-interactive reductions that are knowledge sound in the ROM. RBRTE is a 'hybrid' of two existing notions of knowledge soundness, *round-by-round knowledge soundness* [CY24; BMNW25b] and *tree-extractability* [KP23] (also called *special soundness* [CY24] in the IOP setting). We start by recalling the aforementioned notions and motivating RBRTE in Section 4.1. Finally, in Sections 4.2 and 4.3 we present the definition of RBRTE. Finally, in Section 4.4 we provide an overview of our main results about RBRTE.

4.1 Background: knowledge soundness

Prior work [BCS16; BMNW25b] has shown that (a variant of) the BCS transform of an IOP or IOR yields a non-interactive argument or reduction that is knowledge sound if the underlying interactive protocol satisfies state-restoration knowledge-soundness (SRKS). However, SRKS is an unwieldy definition: it is often difficult to directly prove that a protocol satisfies it. Indeed, prior work instead uses the easier-to-prove notion of round-by-round knowledge soundness (RBRKS) for IOPs [CY24, Chapter 31] as well as for IORs [BMNW25b]. This suffices because RBRKS implies SRKS for both IOPs and IORs. For example, several IOPPs [ACFY24; ACFY25] and IORs [BMNW25b] have used RBRKS to prove that their schemes compile into knowledge sound non-interactive protocols via (a variant of) the BCS transform.

The challenge with using codes that are not efficiently decodable. All aforementioned work that uses RBRKS deals with Reed–Solomon (RS) codes, and crucially relies on their efficient decodability to prove RBRKS, as the extractors for these protocols invoke the RS-decoder as a subroutine. However, this extraction strategy no longer works when dealing with codes that are not known to be decodable in polynomial time (since the extractor must run in polynomial time). Unfortunately, this class of codes includes all known linear-time encodable codes for large fields.

To work around this, prior works that build IOPPs for such codes [GLSTW23; NST24; BCFRRZ25] either entirely forgo proving knowledge-soundness guarantees [BCFRRZ25], or prove standard interactive knowledge soundness by using a rewinding extractor [GLSTW23; NST24]. Unfortunately, neither notion is known to imply SRKS, and as a result the non-interactive versions of these IOPPs are not known to be knowledge sound in the random oracle model. One could hope to address this by using the aforementioned rewinding strategy to prove RBRKS, but unfortunately, RBRKS seems to be an inherently straight-line notion where the extractor receives just a single accepting transcript for the protocol and cannot rewind the prover.

Another approach: tree extraction. There do exist approaches which support extraction that requires rewinding: special soundness for IOPs [CY24, Chapter 30] and tree extraction for RoKs [KP23]. At a high level, in these approaches, one first rewinds the prover to construct a tree of accepting transcripts, and then uses a "tree-extractor" to deterministically extract a valid witness from this tree. Special soundness has been shown to imply SRKS for IOPs [CY24, Chapter 30], so an adaptation of tree-extractability (which already generalizes special soundness to the RoK setting) to the IOR setting seems like a plausible candidate for proving SRKS of our IORs.

Unfortunately, this by itself does not suffice for our purposes, because for some of the IORs described in Section 2.3 (Batch in particular), we cannot necessarily extract a valid witness from an arbitrary tree of accepting transcripts. In more detail, sometimes a transcript is accepting not because the prover knows a valid witness, but because the prover got "lucky" with the verifier's challenges somewhere along the way.

Our solution. We define the new notion of round-by-round tree-extractability for IORs, which overcomes the aforementioned problem by requiring that the tree construction algorithm be able to (efficiently) construct

transcript trees where in every transcript, the prover is constrained to not get "lucky". This enables us to construct deterministic tree-extractors that are guaranteed to extract a witness from such a "constrained" tree. As it turns out, the notion of RBRKS also enforces exactly such a guarantee, i.e. that the prover did not get lucky, but only for a single accepting transcript. In fact, in Section 4.4 we show not only that RBRTE implies SRKS for IORs, but also that every RBRKS IOR is RBRTE (we stress that the proof of the latter is still non-trivial).

4.2 Constrained transcript trees

Before we define RBRTE, we must formally define constrained transcript trees. At a high level, a constrained transcript tree T of an IOR with respect to the input $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$ is a 'tree of accepting transcripts' for the IOR run on the given input. In more detail, the nodes of the tree are labelled with prover messages (with the nodes in layer i containing prover messages for round i) and the edges of the tree are labelled with verifier challenges (with outgoing edges from layer i containing verifier challenges for round i), such that every path from the root to a leaf corresponds to an accepting view for the IOR verifier $\mathbf{V}$ with input corresponding to $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$. Additionally, we require that the tree satisfies certain constraints, which we capture as 'vertical' and 'challenge' predicates. We present a formal definition below.

Definition 4.1 (Constrained transcript tree)**.** *Let* $\mathsf{IOR} = (\mathbf{I}_{\mathsf{IOR}}, \mathbf{P}_{\mathsf{IOR}}, \mathbf{V}_{\mathsf{IOR}})$ *be a* k*-round IOR from an indexed oracle relation* $\mathcal{R}$ *to an index oracle relation* $\mathcal{R}'$*, and let*

- $\vec{a} := a_1, \ldots, a_{\mathsf{k}} \in \mathbb{N}$ *be arity parameters,*
- *for all* $i \in [\mathsf{k}]$, VD_i *and* PD_i *be the domain of possible verifier and prover messages in the* i*-th round of the interaction respectively.*
- ϕ *be a 'vertical' boolean-output predicate which takes as input an index-instance pair, an IOR transcript of the form* $(\alpha_1, \rho_1, \ldots, \alpha_{\mathsf{k}}, \rho_{\mathsf{k}})$ *where* $\alpha_i := (\Pi_i, \mathsf{msg}_i)$ *is the* i*-th prover message and* ρ_i *is the* i*-th verifier challenge, and*
- θ *be a 'challenge' boolean-output predicate which takes as input an ordered set of verifier challenges from* $\mathsf{VD}_1^{a_1} \times \mathsf{VD}_2^{a_1 a_2} \times \ldots \times \mathsf{VD}_{\mathsf{k}}^{\prod_{i \in [\mathsf{k}]} a_i}$.

We say that a $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$*-constrained transcript tree for* IOR *is a depth-*k *tree* T *that satisfies the following properties:*

1. *Each node in the* i*-th layer has* a_i*-many children.*[13]
2. *For every layer* $i \in [\mathsf{k}]$*: every node in layer* i *is labeled with an IOR prover message for round* i*.*
3. *For every layer* $i \in [\mathsf{k}]$*: the* a_i *outgoing edges are labeled with distinct verifier challenges.*
4. *Every path from the root to a leaf corresponds to an accepting view for the IOR verifier* $\mathbf{V}$ *with inputs corresponding to* $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$*, such that the verifier outputs* $(\mathbb{x}', \vec{\mathbb{y}}')$ *consistent with the value in the leaf.*
5. *The leaf nodes are labeled with tuples* $(\mathbb{x}', \vec{\mathbb{y}}', \mathbb{w}')$ *such that* $(\mathbb{i}', \mathbb{x}', \vec{\mathbb{y}}', \mathbb{w}') \in \tilde{\mathcal{R}}'$*, where* $\mathbb{i}'$ *is the output of* $\mathbf{I}_{\mathsf{IOR}}$ *on input* $\mathbb{i}$*.*
6. *The vertical predicate is satisfied: for every path* p *down the tree, the transcript* $(\alpha_1, \rho_1, \ldots, \alpha_{\mathsf{k}}, \rho_{\mathsf{k}})$ *corresponding to* p *satisfies*
$$\phi(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \alpha_1, \rho_1, \ldots, \alpha_{\mathsf{k}}, \rho_{\mathsf{k}}) = 1 \ .$$
7. *The challenge predicate is satisfied:*
$$\theta(\rho(\mathsf{T})) = 1 \ ,$$
where we use $\rho(\mathsf{T})$ *to denote the ordered set of all verifier challenges on the edges of* T*.*

[13] Such a tree has $\mathsf{k}+1$ layers, with layer 1 being the root layer and layer $\mathsf{k}+1$ being the leaf layer, and the tree has $\prod_{i \in [\mathsf{k}]} a_i$ leaves.

For brevity, we will sometimes use $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$-tree to denote $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$-constrained transcript tree.

We illustrate what a $((a_1, a_2), \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$-tree looks like in Fig. 2.

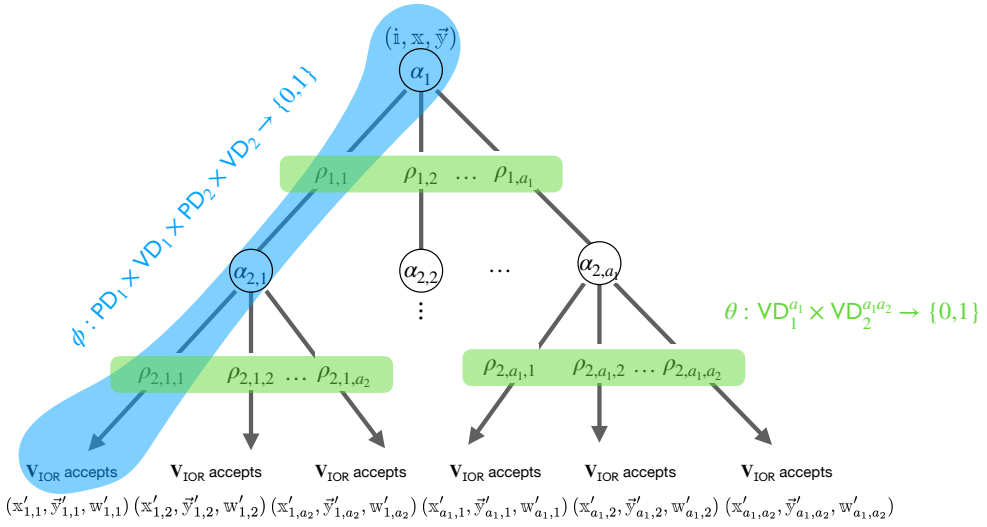


Figure 2: An $((a_1, a_2), \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$-constrained transcript tree for an IOR from $\mathcal{R}$ to $\mathcal{R}'$. Each path from root to leaf ends in $\mathbf{V}_{\mathsf{IOR}}$ accepting, and furthermore must satisfy the vertical predicate ϕ. In addition, each label on each leaf is labeled with $(\mathbb{i}', \mathbb{x}'_{i,j}, \vec{\mathbb{y}}'_{i,j}, \mathbb{w}'_{i,j}) \in \tilde{\mathcal{R}}'$, which is the index-instance-witness triple outputted at the end of the IOR interaction. Finally, the verifier challenges on the edges of the tree must satisfy the 'challenge predicate' $\theta : \mathsf{VD}_1^{a_1} \times \mathsf{VD}_2^{a_1 a_2} \to \{0,1\}$. We depict one application of the vertical predicate ϕ (blue) and the application of the challenge predicate θ (green) in the figure.

Intuition. The intuition behind the above definition is that an extractor $\mathbf{E}$ that, given rewinding access to a malicious prover that succeeds in convincing a verifier to accept on input $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$, can extract a valid witness as follows:

- $\mathbf{E}$ can run a *expected polynomial time* algorithm $\mathsf{CTreeFinder}$ that keeps rewinding the prover and giving it new challenges to construct a constrained transcript tree T corresponding to $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$.
- $\mathbf{E}$ can run a *deterministic polynomial time* algorithm $\mathbf{E}^{\mathrm{Tree}}$ that given T outputs a valid witness.

Accordingly, our new notion of constrained transcript trees imposes a number of conditions. Amongst these, Conditions 1 to 5 are standard structural properties about the shape and contents of the tree, and have been used in prior definitions of transcript trees in [CY24, Chapter 30.1.4] and [KP23, Section 4.3]. In some settings these conditions suffice to extract a valid witness, but in our setting just the fact that every path corresponds to an accepting transcript does not suffice. This is because sometimes the verifier is convinced by a transcript not because the prover sent material from which the extractor can derive the witness, but because the prover got "lucky" somewhere along the way.

Thus, we introduce Conditions 6 and 7 to establish which trees are valid and can be 'extracted from' by $\mathbf{E}^{\mathrm{Tree}}$. Condition 6 allows us to impose that in every path from the root to a leaf, the prover never got "lucky" and fools the verifier into accepting an unextractable transcript. This is analogous to imposing that the transcript along *every* root-to-leaf path did not go into a "doomed set", a notion used in round-by-round (knowledge) soundness [CY24, Chapter 31]. Condition 7 allows us to impose that the verifier challenges corresponding to the edges of the tree satisfy certain properties. This stems from the fact that sometimes $\mathbf{E}^{\mathrm{Tree}}$ requires that certain challenges are distinct, or linearly independent.

A balancing act. As noted, the extractor $\mathbf{E}^{\mathrm{Tree}}$ often requires the predicates ϕ and θ to be of a certain form. At the same time, these predicates must be carefully chosen to ensure that the tree-finding algorithm $\mathsf{CTreeFinder}$ succeeds with high probability. For example, if θ_2 always outputs 0, then $\mathsf{CTreeFinder}$ will never be able to find a valid tree. Thus, we must ensure that we balance both these requirements while choosing the predicates ϕ and θ.

4.3 Round-by-round tree-extractability

In this section, we present our new notion of round-by-round tree-extractability (RBRTE), which ensures that $\mathsf{CTreeFinder}$ can find a satisfying constrained transcript tree with high probability. It does so by enforcing that the parameters of the tree, $\vec{a}, \phi, \theta$, satisfy certain constraints that we detail below.

Firstly, we impose the constraint that $\prod_{i=1}^{\mathsf{k}} a_i = \mathrm{poly}(n)$ (i.e. the size of the tree is polynomial in the size of the instance). Next, in order to ensure that $\mathsf{CTreeFinder}$ succeeds with high probability, we impose that our predicates are 'round-by-round' predicates. At a high level, this constraint ensures that even a tree obtained in the naive manner, by rewinding the malicious prover and running it on new random challenges for each round, satisfies all the predicates with high probability.

We present the formal definition of such predicates below.

Definition 4.2 ($(\overrightarrow{\mathsf{VD}}, \vec{a})$-challenge-predicate)**.** *Consider a* k*-round IOR,* IOR*, from* $\mathcal{R}$ *to* $\mathcal{R}'$*. We say that* θ *is a* $(\overrightarrow{\mathsf{VD}}, \vec{a})$*-challenge-predicate if for* $\rho(\mathsf{T}) \xleftarrow{\$} \mathsf{VD}_1^{a_1} \times \mathsf{VD}_2^{a_1 a_2} \times \ldots \times \mathsf{VD}_{\mathsf{k}}^{\prod_{i\in[\mathsf{k}]} a_i}$*, i.e. each challenge in* $\rho(\mathsf{T})$ *is independently sampled from the corresponding* VD_i*, we have*

$$\Pr_{\rho(\mathsf{T}) \xleftarrow{\$} \mathsf{VD}_1^{a_1} \times \mathsf{VD}_2^{a_1 a_2} \times \ldots \times \mathsf{VD}_{\mathsf{k}}^{\prod_{i\in[\mathsf{k}]} a_i}} [\theta(\rho(\mathsf{T})) = 0] < \mathrm{negl}(\lambda) \ .$$

Definition 4.3 ($(\overrightarrow{\mathsf{PD}}, \overrightarrow{\mathsf{VD}}, \mathsf{k})$-vertical-predicate)**.** *Consider a* k*-round IOR,* IOR*, from* $\mathcal{R}$ *to* $\mathcal{R}'$*. We say that* $\phi : (\mathsf{PD}_1 \times \mathsf{VD}_1) \times \ldots \times (\mathsf{PD}_{\mathsf{k}} \times \mathsf{VD}_{\mathsf{k}}) \to \{0,1\}$[14] *is a* $(\overrightarrow{\mathsf{PD}}, \overrightarrow{\mathsf{VD}}, \mathsf{k})$*-vertical-predicate for* IOR *if the following properties hold:*

- *The function returns 1 on an empty transcript. That is,*

$$\phi(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \varnothing) = 1 \ .$$

- *The output value is unchanged by prover messages. That is, for all* $i \in [\mathsf{k}]$*:*

$$\phi(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \alpha_1, \rho_1, \ldots, \alpha_{i-1}, \rho_{i-1}, \alpha_i) = \phi(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \alpha_1, \rho_1, \ldots, \alpha_{i-1}, \rho_{i-1}) \ ,$$

  for any $\alpha_i \in \mathsf{PD}_i$*.*
- *The probability of the output value going from 1 to 0 by a verifier challenge is* $\mathrm{negl}(\lambda)$*. That is, for all* $i \in [\mathsf{k}]$ *and* $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \alpha_1, \rho_1, \ldots, \alpha_i)$ *such that* $\phi(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \alpha_1, \rho_1, \ldots, \alpha_i) = 1$*, the following holds:*

$$\Pr_{\rho_i \xleftarrow{\$} \mathsf{VD}_i} [\phi(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \alpha_1, \rho_1, \ldots, \alpha_i, \rho_i) = 0] \leq \mathrm{negl}(\lambda) \ .$$

Intuitively, the vertical predicate starts as 1, and the probability of it becoming 0 in any given round is negligible. Looking ahead, in our proofs, the vertical predicate being satisfied will guarantee that the prover never got "lucky" with the verifier's challenges.

Using these definitions of challenge and vertical predicates, we can now define round-by-round tree-extractability.

[14] For brevity, we abuse notation and avoid mentioning the domain for $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$.

Definition 4.4 (Round-by-round tree-extractable IORs)**.** *Consider* IOR*, an IOR from* $\mathcal{R}$ *to* $\mathcal{R}'$*, and* $\mathtt{i}$*, an index of* $\tilde{\mathcal{R}}$*.* IOR *is said to be* $(\vec{a}, \phi, \theta)$-round-by-round tree-extractable (RBRTE) *with respect to index* $\mathtt{i}$ *if the following conditions hold:*

- *for any instance* $(\mathbb{x}, \vec{\mathbb{y}})$*, we have* $\prod_{i=1}^{\mathsf{k}} a_i = \mathrm{poly}(|(\mathtt{i}, \mathbb{x}, \vec{\mathbb{y}})|)$*,*
- ϕ *is a* $(\overrightarrow{\mathsf{PD}}, \overrightarrow{\mathsf{VD}}, \mathsf{k})$*-vertical-predicate,*
- θ *is a* $(\overrightarrow{\mathsf{VD}}, \vec{a})$*-challenge-predicate, and*
- *there exists a deterministic polynomial-time extractor* $\mathbf{E}^{\mathrm{Tree}}$ *such that for any instance* $(\mathbb{x}, \vec{\mathbb{y}})$*,* $\mathbf{E}^{\mathrm{Tree}}$ *takes as input a* $(\vec{a}, \phi, \theta, \mathtt{i}, \mathbb{x}, \vec{\mathbb{y}})$*-constrained transcript tree* T *for* IOR*, and outputs a witness* $\mathbb{w}$ *such that* $(\mathtt{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \in \tilde{\mathcal{R}}$*.*

Furthermore, we say IOR *is RBRTE if for each index* $\mathtt{i}$*, there exists a tuple* $(\vec{a}, \phi, \theta)$ *such that* IOR *is* $(\vec{a}, \phi, \theta)$*-RBRTE with respect to* $\mathtt{i}$*.*

4.4 Main results about RBRTE IORs

4.4.1 Compilation to NIRKs

In Section A and Section B we prove that RBRTE IORs can be compiled to non-interactive reductions of knowledge (NIRKs) via (a variant of) the BCS transform [BCS16].

Lemma 4.5 (Compilation of RBRTE IORs to NIRK)**.** *Let* IOR *be an RBRTE IOR from* $\mathcal{R}$ *to* $\mathcal{R}'$*. Then the non-interactive reduction obtained by applying (a variant of) the BCS transform (from [BMNW25b, Appendix B]) to* IOR *is a NIRK with rewinding knowledge soundness.*

This lemma is the result of two sub-lemmata, which can be informally summarized as follows:

1. Lemma A.6, which shows that RBRTE IORs also satisfy a property known as rewinding state-restoration knowledge sound (SRKS) (see Definition A.2), and
2. Lemma B.11, which shows that the transformation from [BMNW25b] applied to a rewinding SRKS IOR yields a NIRK with rewinding knowledge soundness.

Remark 4.6. While we use the same transformation as [BMNW25b], Lemma 4.5 is still non-trivial, as [BMNW25b] only showed that the transformation outputs a straight-line knowledge sound NIRK, given a straight-line SRKS IOR, which does not suffice for us.

4.4.2 Composition of RBRTE IORs

In Section C we formally define sequential composition of IORs, and prove that the sequential composition of two RBRTE IORs yields a new RBRTE IOR. For brevity, in the rest of the paper composition of IORs always refers to sequential composition.

Lemma 4.7 (RBRTE IOR composition)**.** *Let* IOR *be an* $(\vec{a}, \phi, \theta)$*-RBRTE* IOR *from* $\mathcal{R}$ *to* $\mathcal{R}'$ *and* IOR' *be an* $(\vec{a}', \phi', \theta')$*-RBRTE* IOR *from* $\mathcal{R}'$ *to* $\mathcal{R}''$*. Then the sequential composition* $\mathsf{IOR}'' := \mathsf{IOR}' \circ \mathsf{IOR}$ *is a* $(\vec{a} \,\|\, \vec{a}', \phi \,\|\, \phi', \theta \wedge \theta')$*-RBRTE IOR from* $\mathcal{R}$ *to* $\mathcal{R}''$*.*[15]

Furthermore, the efficiency metrics of $\mathsf{IOR}' \circ \mathsf{IOR}$ *are the sum of the round complexities, oracle lengths, message lengths, query complexities, prover times, and verifier times.*

[15] The "concatenation" of vertical predicates $\phi \,\|\, \phi'$ is a predicate whose round-by-round representation is the concatenation of the round-by-round representation of ϕ and ϕ'.

Informally, this lemma states that if we have two RBRTE IORs that require constrained transcript trees of size $s = \prod_{i \in [\mathsf{k}]} a_i$ and $s' = \prod_{i \in [\mathsf{k}']} a'_i$ to extract a witness, then the composition of the two IORs only requires a constrained transcript tree of size $s \cdot s'$ to extract a witness. This means that we can compose IORs a non-constant number of times as long as the size of the constrained transcript tree required by the obtained IOR remains polynomial in the size of the instance (for example, this happens when some of the IORs involved in the composition only require constant-sized constrained transcript trees to extract a witness).

4.4.3 RBRKS implies RBRTE

In Section D we show that round-by-round-knowledge-sound IOPs and IORs (presented in [CY24, Definition 31.1.4] and [BMNW25b, Definition 5.5] respectively) are RBRTE. Concretely, this allows us to use IOPs and IORs which have been proven RBRKS (such as ones in [ACFY24; ACFY25; BMNW25b]) as building blocks in our protocols without requiring additional soundness proofs.

Lemma 4.8 (RBRKS implies RBRTE)**.** *If an IOR* IOR *is round-by-round-knowledge-sound (see Definition D.2), then it is round-by-round tree-extractable.*

We note that the arity parameters corresponding to such an RBRKS IOR for all $\mathtt{i}$ are always of the form $\vec{a} = 1^{\mathsf{k}}$, where k is the round complexity of the IOR. That is, the constrained transcript tree required is always just a 'straight-line'. Additionally, Lemma 4.8 and Lemma 4.7 together yield the following corollary, which to the best of our knowledge has not been formally stated or proven before.

Corollary 4.9. *The sequential composition of two round-by-round knowledge sound IORs is also round-by-round knowledge sound.*

5 Preliminaries

5.1 Notation

Vector notation. We use $\vec{a} := (a_0, \ldots, a_{n-1})$ to denote *ordered* sets or vectors, and denote the sets $(1, 2, \ldots, n)$ by $[n]$ and $(0, 1, \ldots, n-1)$ by $[n]_0$. We use a_j to denote the j-th element of $\vec{a}$.

For a finite field $\mathbb{F}$, we use lowercase letters $x \in \mathbb{F}$ to denote a field element and we use bold lowercase letters $\boldsymbol{x} \in \mathbb{F}^n$ to denote an n-dimensional vector of field elements[16]. Given a vector $\boldsymbol{x} \in \mathbb{F}^{2n}$ of even length, we use $\boldsymbol{x}_L$ and $\boldsymbol{x}_R$ to denote the left and right halves of $\boldsymbol{x}$, respectively. We use $\boldsymbol{x}[i:j]$ to denote the vector consisting of the elements $\boldsymbol{x}[i], \ldots, \boldsymbol{x}[j]$.

We use bold uppercase letters $\boldsymbol{M} \in \mathbb{F}^{n \times m}$ to denote matrices , with $\boldsymbol{M}[i]$ representing the i-th row of the matrix. We use $\boldsymbol{x}[i]$ to denote the i-th element of $\boldsymbol{x}$, and $\boldsymbol{M}[i, j]$ to denote the element in the i-th row and j-th column of $\boldsymbol{M}$.

Binary representation. Given integers i and n, $\langle i \rangle_n$ represents the n-bit binary representation of i in the most-significant-bit order. We drop the subscript n when it is clear from context. We also define the inverse operation $\mathsf{int}(\boldsymbol{x})$ which maps $\boldsymbol{x} = \langle v \rangle_n \in \{0,1\}^n$ to v.

Multilinear polynomials. We use $\mathbb{F}^{\leq d}[X_1, \ldots, X_\mu]$ to denote the set of all μ-variate polynomials over $\mathbb{F}$ such that maximum degree of each variable in every monomial is at most d. We denote a multivariate polynomial by $p(X_1, \ldots, X_\mu)$ or by $p(\boldsymbol{X})$ when μ is clear from context.

$\mathbb{F}^{\leq 1}[X_1, \ldots, X_\mu]$ is the set of all μ-variate *multilinear* polynomials, and a polynomial $p(\boldsymbol{X}) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_\mu]$ is uniquely defined by its evaluations $\boldsymbol{p} := [p(\langle i \rangle)]_{i=0}^{2^\mu - 1}$ over the μ-dimensional boolean hypercube $\{0,1\}^\mu \subset \mathbb{F}^\mu$.

Multilinear extension. The *multilinear extension* of a vector $\boldsymbol{v} \in \mathbb{F}^{2^\mu}$ is the unique multilinear polynomial $\hat{v}(\boldsymbol{X}) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_\mu]$ such that $\hat{v}(\boldsymbol{i}) = \boldsymbol{v}[\mathsf{int}(\boldsymbol{i})]$ for all $\boldsymbol{i} \in \{0,1\}^\mu$.

Lagrange basis polynomial. The *Lagrange basis polynomial* $\mathrm{eq}_\mu(\boldsymbol{X}, \boldsymbol{Y}) := \prod_{i=1}^{\mu}(X_i Y_i + (1-X_i)(1-Y_i))$ is a multilinear polynomial that checks that $\boldsymbol{X} = \boldsymbol{Y}$ for any $\boldsymbol{X}, \boldsymbol{Y} \in \{0,1\}^\mu$. We omit μ when it is clear from context. For any arbitrary $p(\boldsymbol{X}) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_\mu]$ and $\boldsymbol{z} \in \mathbb{F}^\mu$, one can express

$$p(\boldsymbol{z}) = \sum_{\boldsymbol{i} \in \{0,1\}^\mu} p(\boldsymbol{i}) \cdot \mathrm{eq}(\boldsymbol{i}, \boldsymbol{z}) \ .$$

Hamming distance. We use Δ to denote hamming distance, i.e for $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{F}^k$, $\Delta(\boldsymbol{x}, \boldsymbol{y}) := |\{i \in [k] : \boldsymbol{x}[i] \neq \boldsymbol{y}[i]\}|$, and δ to denote fractional hamming distance, i.e $\delta(\boldsymbol{x}, \boldsymbol{y}) := \Delta(\boldsymbol{x}, \boldsymbol{y})/k$. We sometimes abuse notation and use δ to denote both a constant and the function above. For a set $S \subset \mathbb{F}^k$ and an element $\boldsymbol{x} \in \mathbb{F}^k$ we define $\Delta(\boldsymbol{x}, S) := \min_{\boldsymbol{s} \in S} \Delta(\boldsymbol{x}, \boldsymbol{s})$, and we define $\delta(\boldsymbol{x}, S) := \Delta(\boldsymbol{x}, S)/k$.

5.2 Coding theory

In this section we describe the necessary coding theory background for our work.

Definition 5.1 (Linear codes)**.** *A linear code $\mathcal{C}$ of message length k, block length n, over an alphabet $\mathbb{F}$ is a k-dimensional subspace of $\mathbb{F}^n$. The* rate *of the code is defined to be k/n and the* distance *of a code $\mathcal{C}$ is defined to be*

$$\delta_{\mathcal{C}} := \min_{\boldsymbol{x} \neq \boldsymbol{y} \in \mathcal{C}} \delta(\boldsymbol{x}, \boldsymbol{y})$$

[16] Unless otherwise specified, we assume that vectors over $\mathbb{F}$ are zero-indexed.

We will often abbreviate the parameters of a code as $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$*. The* unique decoding distance *of a code is* $\delta_{\mathcal{C}}/2$*. This is the radius of the largest ball* B *around a codeword* $\boldsymbol{w} \in \mathbb{F}^n$*, such that for any* $\boldsymbol{b} \in B$*, we have* $\delta(\boldsymbol{w}', \boldsymbol{b}) > \delta_{\mathcal{C}}/2$ *for any* $\boldsymbol{w}' \in \mathcal{C}$ *such that* $\boldsymbol{w}' \neq \boldsymbol{w}$*. We say a word* $\boldsymbol{w} \in \mathbb{F}^n$ *is* δ*-far from (or* δ*-close to) a code* $\mathcal{C}$ *if* $\delta(\boldsymbol{w}, \mathcal{C}) > \delta$ *(or* $\delta(\boldsymbol{w}, \mathcal{C}) \leq \delta$*).*

Throughout this paper, we exclusively consider linear codes that have constant rate and constant distance, unless otherwise specified.

Encoding. An *encoding function* of a code $\mathcal{C}$ is a bijection $\mathsf{Enc}_{\mathcal{C}} : \mathbb{F}^k \to \mathcal{C}$. We define the message $\boldsymbol{m}$ of a codeword $\boldsymbol{c} \in \mathcal{C}$ relative to an encoding function $\mathsf{Enc}_{\mathcal{C}}$ as $\boldsymbol{m} := \mathsf{Enc}_{\mathcal{C}}{}^{-1}(\boldsymbol{c})$. If the code is linear, the encoding function can be written as $k \times n$ matrix $\boldsymbol{G}_{\mathcal{C}}$ known as the *generator matrix* such that $\mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}) = \boldsymbol{G}_{\mathcal{C}}\boldsymbol{m}$. The generator matrix is said to be *systematic* if

$$\boldsymbol{G}_{\mathcal{C}} = \begin{bmatrix} \boldsymbol{I}_k \\ \boldsymbol{C} \end{bmatrix}$$

where $\boldsymbol{I}_k$ is the identity matrix of dimension $k \times k$. We also use $T_{\mathcal{C}}$ to denote the time complexity of encoding a message under $\mathcal{C}$.

Definition 5.2 (Interleaved codes)**.** *Let* $\eta \in \mathbb{N}$ *be an interleaving parameter. Let* $\mathcal{B}$ *be a linear code with parameters* $(\mathbb{F}, n, k, \delta_{\mathcal{B}})$*. The* interleaved code $\mathcal{C} := \mathsf{IC}[\mathcal{B}, \eta]$ *is a linear code with parameters* $(\mathbb{F}, \eta n, \eta k, \delta_{\mathcal{B}}/\eta)$ *such that*

$$\mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}) = \mathsf{Enc}_{\mathcal{B}}(\boldsymbol{m}[0 : k-1]) \,\|\, \ldots \,\|\, \mathsf{Enc}_{\mathcal{B}}(\boldsymbol{m}[(\eta-1)k : \eta k - 1]) \ .$$

Definition 5.3 (Tensor codes)**.** *Let* $t \in \mathbb{N}$ *be a tensor parameter. Consider a linear code* $\mathcal{B}$ *with parameters* $(\mathbb{F}, n, k, \delta_{\mathcal{B}})$*, and its generator matrix* $\boldsymbol{G}_{\mathcal{B}} \in \mathbb{F}^{n \times k}$*. The* tensor code $\mathcal{C} = \mathsf{TC}[\mathcal{B}, t]$ *is a linear code with parameters* $(\mathbb{F}, n^t, k^t, \delta_{\mathcal{B}}^t)$ *and generator matrix* $\boldsymbol{G}_{\mathcal{C}} \in \mathbb{F}^{n^t \times k^t}$*, such that*

$$\boldsymbol{G}_{\mathcal{C}} = \bigotimes_{i=1}^{t} \boldsymbol{G}_{\mathcal{B}} \ ,$$

where $\bigotimes$ *denotes the Kronecker product.*

Remark 5.4 (encoding time of interleaved and tensor codes)**.** The encoding time of $\mathsf{IC}[\mathcal{B}, \eta]$ is $O(\eta \cdot T_{\mathcal{B}})$. The encoding time of $\mathsf{TC}[\mathcal{B}, t]$ is $O(T_{\mathcal{B}}^t)$. Therefore, for constant η and t, if the encoding time of $\mathcal{B}$ is linear in the block length, then the encoding time of $\mathsf{IC}[\mathcal{B}, \eta]$ and $\mathsf{TC}[\mathcal{B}, t]$ is also linear in their respective block lengths.

5.2.1 Proximity gaps of linear codes

Linear codes exhibit a property called *proximity gaps* used in previous work on polynomial commitment schemes [ACFY24; ACFY25] and accumulation schemes [BMNW25a; BMNW25b]. This property has been particularly well studied for Reed-Solomon codes [BCIKS23], but in this paper we will consider the weaker guarantees known for general linear codes [BKS18].

Lemma 5.5 (Proximity gap and correlated agreement [BKS18, Theorem 4.1])**.** *Consider a code* $\mathcal{C}$ *with parameters* $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$*. Consider a set of vectors* $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_\ell\} \subset \mathbb{F}^n$ *and* $\varepsilon \leq \delta_{\mathcal{C}}/3$ *such that*

$$\Pr_{a_1, \ldots, a_\ell \leftarrow \mathbb{F}} \left[\delta\left(\sum_{i \in [\ell]} a_i \boldsymbol{u}_i, \mathcal{C} \right) \leq \varepsilon - \kappa \right] > \frac{1}{\kappa |\mathbb{F}|}$$

then for all $i \in [\ell]$ we have $\delta(\boldsymbol{u}_i, \mathcal{C}) \leq \varepsilon$. Furthermore, there exists a set of indices $S \subseteq [n]_0$ and codewords $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_\ell \in \mathcal{C}$ such that $|S| \geq (1-\varepsilon)n$ and we have $\boldsymbol{u}_i[j] = \boldsymbol{v}_i[j]$ for all $i \in [\ell]$ and $j \in S$. The first and second properties are referred to as proximity gaps *and* correlated agreement *respectively.*

Proximity gaps have been used in previous work on polynomial commitment schemes [ACFY24; ACFY25] and accumulation schemes [BMNW25a; BMNW25b]. This property has been particularly well studied for Reed-Solomon codes [BCIKS23], but in this paper we will consider the weaker guarantees known for general linear codes [BKS18].

Remark 5.6 (list-decoding regime)**.** It is useful to consider the proximity gaps in the list-decoding setting, which allows ε to be larger in the above definition, and therefore leads to fewer oracle queries in practice. We believe our techniques can extend to this setting using the recent proximity gap result of Zeilberger [Zei24], and defer this extension to future work.

6 IOR toolkit

We present a 'toolkit' of IORs that we use in our subsequent constructions, many of which are inspired by prior work. These IORs are likely also of independent interest, and we expect that they will be useful in future work.

6.1 The sumcheck protocol as an IOR

The seminal sumcheck protocol [LFKN92] allows a prover $\mathbf{P}$ to convince a verifier $\mathbf{V}$ with oracle access to a polynomial $p(\boldsymbol{X}) \in \mathbb{F}^{\leq d}[X_1, \ldots, X_\mu]$, that its evaluations over the boolean hypercube $\{0,1\}^\mu \subseteq \mathbb{F}^\mu$ sum up to a claimed value σ. In Section 6.1.1, we present an interpretation of the classical sumcheck protocol as an IOR. We first specify a special class of polynomials for which we need to run the sumcheck protocol on, and then present a version the sumcheck protocol specialized to this class that additionally leverages error-correcting codes to avoid dealing with polynomial oracles.

6.1.1 The classical sumcheck protocol

Given oracle access to $p(\boldsymbol{X})$, the classical sumcheck protocol reduces the problem of checking that $\sum_{\boldsymbol{b}\in\{0,1\}^\mu} p(\boldsymbol{b}) = \sigma$ to checking that $p(\boldsymbol{r}_{\mathsf{SC}}) = y_{\mathsf{SC}}$, where $\boldsymbol{r}_{\mathsf{SC}} \in \mathbb{F}^\mu$ is a random challenge vector sampled by the verifier. This allows the verifier to make only one query to $[\![p(\boldsymbol{X})]\!]$, as opposed to the naive approach which would require $\{0,1\}^\mu$ queries.

We describe the classical sumcheck protocol as a reduction of knowledge between two indexed *polynomial* oracle relations, which are a modification of indexed oracle relations where the instance oracle consists of polynomial oracles that can be queried for their evaluations at points from their input domain.

Indexed Oracle Relation 6.1 (Sumcheck)**.** *The* sumcheck relation $\mathscr{R}_{\mathsf{SC}}$ *is an indexed polynomial oracle relation consisting of tuples of the following form:*

- index*:* $(\mathbb{F}, \mu, d)$*, where* $\mathbb{F}$ *is a finite field,* $\mu \in \mathbb{N}$ *is the number of polynomial variables and* $d \in \mathbb{N}$ *is the degree bound,*
- instance*:* $\sigma \in \mathbb{F}$ *is the claimed sum,*
- instance oracle*:* $p(\boldsymbol{X})$*, where* $p(\boldsymbol{X}) \in \mathbb{F}^{\leq d}[X_1, \ldots, X_\mu]$*,*
- witness*:* $\perp$*,*

such that $\sum_{\boldsymbol{b}\in\{0,1\}^\mu} p(\boldsymbol{b}) = \sigma$*.*

Indexed Oracle Relation 6.2 (Polynomial evaluation)**.** *The* polynomial evaluation relation $\mathscr{R}_{\mathsf{eval}}$ *is an indexed polynomial oracle relation consisting of tuples of the following form:*

- index*:* $(\mathbb{F}, \mu, d)$*, where* $\mathbb{F}$ *is a finite field,* $\mu \in \mathbb{N}$ *and* $d \in \mathbb{N}$*,*
- instance*:* $(\boldsymbol{z}, y)$*, where* $\boldsymbol{z} \in \mathbb{F}^\mu$ *is the evaluation point, and* $y \in \mathbb{F}$ *is the evaluation value,*
- instance oracle*:* $p(\boldsymbol{X})$*, where* $p(\boldsymbol{X}) \in \mathbb{F}^{\leq d}[X_1, \ldots, X_\mu]$*,*
- witness*:* $\perp$*,*

such that $p(\boldsymbol{z}) = y$*.*

The sumcheck protocol can be interpreted as a reduction of knowledge, $\mathsf{PolySumcheck}$, from $\mathscr{R}_{\mathsf{SC}}$ to $\mathscr{R}_{\mathsf{eval}}$ as presented in Fig. 3. We omit discussing the efficiency and soundness of this particular protocol, as we will only deal with a specialized instance of it in the following section.

Indexer: Receives as input $\mathbb{i} = (\mathbb{F}, \mu, d)$, and outputs $\iota := \mathbb{i}$, $\vec{\mathbb{I}} := \bot$ and $\mathbb{i}' := (\mathbb{F}, \mu, d)$.
Input instance: $\mathbb{x} = \sigma$.
Input instance oracle: $\vec{\mathbb{y}} = p(\boldsymbol{X})$.
Input witness: $\mathbb{w} = \bot$.

$\langle \mathbf{P}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}), \mathbf{V}^{\vec{\mathbb{I}}, \vec{\mathbb{y}}}(\iota, \mathbb{x}) \rangle$:

1. Set $b \leftarrow 1$.
2. For i in $[1, \ldots, \mu]$:
3. $\quad$ $\mathbf{P}$ sends to $\mathbf{V}$ the univariate polynomial $p^{(i)}(X) \in \mathbb{F}^{\leq d}[X]$ defined as

$$p^{(i)}(X) := \sum_{\boldsymbol{b} \in \{0,1\}^{\mu - i}} p(r_1, \ldots, r_{i-1}, X, \boldsymbol{b})$$

4. $\quad$ If $i = 1$: $\mathbf{V}$ sets $\sigma_i := \sigma$; $\;$ Else: $\mathbf{V}$ sets $\sigma_i := p^{(i-1)}(r_i)$.
5. $\quad$ $\mathbf{V}$ checks that $\sum_{x \in \{0,1\}} p^{(i)}(x) = \sigma_i$. If this check fails, set $b \leftarrow 0$.
6. $\quad$ $\mathbf{V}$ samples $r_i \xleftarrow{\$} \mathbb{F}$ and sends it to $\mathbf{P}$.
7. Define

$$\mathbb{x}' := (\boldsymbol{z} := [r_1, \ldots, r_\mu], y := p^{(n)}(r_\mu)), \;\; \vec{\mathbb{y}}' := p(\boldsymbol{X}), \;\; \mathbb{w}' := \bot \;\; .$$

8. $\mathbf{P}$'s output is $(b, \mathbb{x}', \vec{\mathbb{y}}', \mathbb{w}')$ and $\mathbf{V}$'s output is $(b, \mathbb{x}', \vec{\mathbb{y}}')$.

Figure 3: PolySumcheck, an RoK from $\mathscr{R}_{\mathsf{SC}}$ to $\mathscr{R}_{\mathsf{eval}}$

6.1.2 Sumcheck over encoded polynomials

In this work, we only deal with the sumcheck protocol where $p(\boldsymbol{X})$ is a 'sumprod polynomial', which is an extension of a class of polynomials implicitly defined in the notion of constrained Reed-Solomon codes from [ACFY25, Definition 4.5]. At a high level, sumprod polynomials are polynomials which can be expressed as a linear combination of products of multilinear polynomials. We formalize this notion below.

Definition 6.3 (Sumprod polynomials)**.** *A polynomial $p(\boldsymbol{X}) \in \mathbb{F}^{\leq \ell+1}[X_1, \ldots, X_\mu]$ is called a sumprod polynomial if it can be expressed as*

$$p(\boldsymbol{X}) = f(\boldsymbol{X}, p_1(\boldsymbol{X}), \ldots, p_\ell(\boldsymbol{X})) \;\; ,$$

where $f(X_1, \ldots, X_\mu, Z_1, \ldots, Z_\ell) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_{\mu+\ell}]$ and for all $i \in [\ell]$: $p_i(\boldsymbol{X}) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_\mu]$.

Remark 6.4. For a sumprod polynomial $p(\boldsymbol{X}) \in \mathbb{F}^{\leq \ell+1}[X_1, \ldots, X_\mu]$, if the polynomial $f(\boldsymbol{X}, \boldsymbol{Z})$ is public, having oracle access to the polynomials $[p_i(\boldsymbol{X})]_{i \in \ell}$ is the same as having oracle access to the polynomial $p(\boldsymbol{X})$. In order to query $[\![p(\boldsymbol{X})]\!]$ at a point $\boldsymbol{z} \in \mathbb{F}^\mu$, one can first query $[\![p_i(\boldsymbol{X})]\!]$ at $\boldsymbol{z}$ to obtain the evaluation value y_i, for all $i \in [\ell]$, and then compute $y = f(\boldsymbol{z}, y_1, \ldots, y_\ell)$ which is precisely $p(\boldsymbol{z})$.

Using codes to avoid polynomial oracles. The relations discussed in the previous section involved *polynomial oracles*. In the IOP and IOR models however, we can only assume the existence of vector oracles,

so we need to capture these notions using vector oracles alone. Recall from Section 2.2 that given a code $\mathcal{C}$ that we have an IOPP of multilinear evaluation of, we can obtain a multilinear polynomial oracle for $\hat{m}(\boldsymbol{X})$ by using a vector oracle for $\boldsymbol{w} := \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m})$. Accordingly, we present the sumprod-evaluation proximity relation, SEP, which differs from $\mathscr{R}_{\mathsf{SC}}$ in that (i) the polynomial $p(\boldsymbol{X})$ is a sumprod polynomial of multilinear polynomials $\hat{m}_1, \ldots, \hat{m}_\ell$, and (ii) the instance oracles are vector oracles of $\boldsymbol{w}_i = \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_i)$ under a code $\mathcal{C}$.

Indexed Oracle Relation 6.5 (Sumprod-evaluation proximity)**.** *The* sumprod-evaluation proximity relation $\mathsf{SEP} = (\mathcal{R}, \tilde{\mathcal{R}})$ *is an indexed oracle pair relation with $\mathcal{R}$ and $\tilde{\mathcal{R}}$ both consisting of tuples of the following form:*

- *index:* $(\mathcal{C}, \varepsilon, \ell)$*, where $\mathcal{C}$ is a linear code with parameters* $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$*,* $\varepsilon \in (\delta_{\mathcal{C}}/8, \delta_{\mathcal{C}}/3)$ *is the proximity parameter, and* $\ell \in \mathbb{N}$*,*
- *instance:* $(f(\boldsymbol{X}, \boldsymbol{Z}), \sigma)$*, where* $f(X_1, \ldots, X_{\log k}, Z_1, \ldots, Z_\ell) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_{\log k + \ell}]$ *and* $\sigma \in \mathbb{F}$*,*
- *instance oracle:* $[\boldsymbol{w}_i]_{i \in [\ell]}$*, where* $\boldsymbol{w}_i \in \mathbb{F}^n$ *for each* $i \in [\ell]$*,*
- *witness:* $[\boldsymbol{m}_i]_{i \in [\ell]}$*, where* $\boldsymbol{m}_i \in \mathbb{F}^k$ *for each* $i \in [\ell]$*,*

such that they satisfy the condition

$$\sum_{\boldsymbol{b} \in \{0,1\}^{\log k}} f(\boldsymbol{b}, \hat{m}_1(\boldsymbol{b}), \ldots, \hat{m}_\ell(\boldsymbol{b})) = \sigma \ ,$$

as well as following distinct conditions for $\mathcal{R}$ and $\tilde{\mathcal{R}}$ respectively

$$\begin{aligned} \mathcal{R}: &\quad \forall i \in [\ell] \quad : \quad \boldsymbol{w}_i = \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_i), \\ \tilde{\mathcal{R}}: &\quad \forall i \in [\ell] \quad : \quad \delta(\boldsymbol{w}_i, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_i)) \leq \varepsilon \ . \end{aligned}$$

The above relation can be viewed as a generalization of constrained Reed–Solomon codes from [ACFY25, Definition 4.5], which are essentially SEP with $\mathcal{C}$ set to the Reed-Solomon code and $\ell = 1$. Following Remark 6.4, we now define a version of $\mathscr{R}_{\mathsf{eval}}$ tailored to sumprod polynomials, that we show SEP can be reduced to.

Indexed Oracle Relation 6.6 (Homogeneous multilinear-evaluation proximity)**.** *The* homogeneous multilinear-evaluation proximity relation $\mathsf{HEP} = (\mathcal{R}, \tilde{\mathcal{R}})$ *is an indexed oracle pair relation with $\mathcal{R}$ and $\tilde{\mathcal{R}}$ both consisting of tuples of the following form:*

- *index:* $(\mathcal{C}, \varepsilon, \ell)$*, where $\mathcal{C}$ is a linear code with parameters* $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$*,* $\varepsilon \in (\delta_{\mathcal{C}}/8, \delta_{\mathcal{C}}/3)$ *is the proximity parameter, and* $\ell \in \mathbb{N}$*,*
- *instance:* $(\boldsymbol{z}, [y_i]_{i \in [\ell]})$*, where* $\boldsymbol{z} \in \mathbb{F}^{\log k}$ *and for each* $i \in [\ell]$ *:* $y_i \in \mathbb{F}$*,*
- *instance oracle:* $[\boldsymbol{w}_i]_{i \in [\ell]}$*, where for each* $i \in [\ell]$ *:* $\boldsymbol{w}_i \in \mathbb{F}^n$*,*
- *witness:* $[\boldsymbol{m}_i]_{i \in [\ell]}$*, where for each* $i \in [\ell]$ *:* $\boldsymbol{m}_i \in \mathbb{F}^k$*,*

such that they satisfy the condition

$$\forall i \in [\ell] \quad : \quad \hat{m}_i(\boldsymbol{z}) = y_i \ ,$$

as well as following distinct conditions for $\mathcal{R}$ and $\tilde{\mathcal{R}}$ respectively

$$\begin{aligned} \mathcal{R}: &\quad \forall i \in [\ell] \quad : \quad \boldsymbol{w}_i = \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_i), \\ \tilde{\mathcal{R}}: &\quad \forall i \in [\ell] \quad : \quad \delta(\boldsymbol{w}_i, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_i)) \leq \varepsilon \ . \end{aligned}$$

IOR from SEP to HEP. PolySumcheck directly implies an IOR ECSumcheck from SEP to HEP: the prover and verifier run PolySumcheck with the 'virtual' polynomial $p(\boldsymbol{X}) := f(\boldsymbol{X}, \hat{m}_1(\boldsymbol{X}), \ldots, \hat{m}_\ell(\boldsymbol{X}))$. This reduces the verifier's task to checking if $p(\boldsymbol{r}) = y$ for the resulting evaluation point $\boldsymbol{r} \in \mathbb{F}^\mu$, which is equivalent to checking if $f(\boldsymbol{r}, \hat{m}_1(\boldsymbol{r}), \ldots, \hat{m}_\ell(\boldsymbol{r})) = y$. The prover can send the claimed values $y_i := \hat{m}_i(\boldsymbol{r})$ for every $i \in [\ell]$ to the verifier, and the verifier can check that $f(\boldsymbol{r}, y_1, \ldots, y_\ell) = y$. The verifier is just left to check that $\hat{m}_i(\boldsymbol{r}) = y_i$ for all $i \in [\ell]$, which is precisely what the relation HEP captures. We present the full construction of ECSumcheck in Fig. 4.

Indexer: Receives as input $\mathbb{i} = (\mathcal{C}, \varepsilon, \ell)$, and outputs $\iota := \mathbb{i}$, $\vec{\mathbb{I}} := \bot$ and $\mathbb{i}' := (\mathcal{C}, \varepsilon, \ell)$.
Input instance: $\mathbb{x} = (f(\boldsymbol{X}, \boldsymbol{Z}), \sigma)$.
Input instance oracle: $\vec{\mathbb{y}} = [\boldsymbol{w}_i]_{i \in [\ell]}$.
Input witness: $\mathbb{w} = [\boldsymbol{m}_i]_{i \in [\ell]}$.

$\langle \mathbf{P}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}), \mathbf{V}^{\vec{\mathbb{I}}, \vec{\mathbb{y}}}(\iota, \mathbb{x}) \rangle$:

1. Define
$$p(X_1, \ldots, X_{\log k}) := f(X_1, \ldots, X_{\log k}, \hat{m}_1(\boldsymbol{X}), \ldots, \hat{m}_\ell(\boldsymbol{X})) \ .$$
2. $\mathbf{P}$ and $\mathbf{V}$ run PolySumcheck for the 'virtual' polynomial $p(\boldsymbol{X}) \in \mathbb{F}^{\leq \ell+1}[X_1, \ldots, X_{\log k}]$ as follows
   - Define $\mathbb{i}_{\mathsf{SC}} := (\mathbb{F}, \log k, \ell + 1)$, $\iota_{\mathsf{SC}} := \mathbb{i}_{\mathsf{SC}}$, $\mathbb{x}_{\mathsf{SC}} := \sigma$, $\vec{\mathbb{y}}_{\mathsf{SC}} := p(\boldsymbol{X})$ and $\mathbb{w}_{\mathsf{SC}} := \bot$.
   - Run PolySumcheck$(\langle \mathbf{P}(\mathbb{i}_{\mathsf{SC}}, \mathbb{x}_{\mathsf{SC}}, \vec{\mathbb{y}}_{\mathsf{SC}}, \mathbb{w}_{\mathsf{SC}}), \mathbf{V}^{\vec{\mathbb{I}}, \vec{\mathbb{y}}_{\mathsf{SC}}}(\iota_{\mathsf{SC}}, \mathbb{x}_{\mathsf{SC}}) \rangle)$ and obtain as output
$$(b_{\mathsf{eval}}, \mathbb{x}_{\mathsf{eval}} = (\boldsymbol{r}, y), \vec{\mathbb{y}}_{\mathsf{eval}} = p(\boldsymbol{X}), \mathbb{w}_{\mathsf{eval}} = \bot)$$
3. For each $i \in [\ell]$: $\mathbf{P}$ computes $y_i \leftarrow \hat{m}_i(\boldsymbol{r})$ and sends y_i to $\mathbf{V}$.
4. $\mathbf{V}$ checks that $f(\boldsymbol{r}, y_1, \ldots, y_\ell) = y$. If this check fails, $\mathbf{V}$ sets $b \leftarrow 0$.
5. Define
$$\mathbb{x}' := (\boldsymbol{r}, [y_i]_{i \in [\ell]}), \ \ \vec{\mathbb{y}}' := [\boldsymbol{w}_i]_{i \in [\ell]}, \ \ \mathbb{w}' := [\boldsymbol{m}_i]_{i \in [\ell]} \ .$$
6. $\mathbf{P}$ gets as output $(b \wedge b_{\mathsf{eval}}, \mathbb{x}', \vec{\mathbb{y}}', \mathbb{w}')$ and $\mathbf{V}$ gets as output $(b \wedge b_{\mathsf{eval}}, \mathbb{x}', \vec{\mathbb{y}}')$.

Figure 4: ECSumcheck, an IOR from $\mathsf{SEP}^{(\mathcal{C}, \varepsilon, \ell)}$ to $\mathsf{HEP}^{(\mathcal{C}, \varepsilon, \ell)}$

Lemma 6.7 (ECSumcheck IOR)**.** ECSumcheck *is an IOR from* $\mathsf{SEP}^{(\mathcal{C}, \varepsilon, \ell)}$ *to* $\mathsf{HEP}^{(\mathcal{C}, \varepsilon, \ell)}$ *with the following efficiency measures*

rounds	*message size*	*queries*	*oracle size*	*prover time*	*verifier time*
$\log n$	$O(\ell \log n)$	0	0	$O(2^\ell \cdot n)$	$O(2^\ell \cdot \log n)$

Furthermore, there exists a vertical-predicate ϕ *such that the IOR is* $(\vec{a} := (1, 1, \ldots, 1), \phi, \theta_{\mathsf{true}})$*-RBRTE.*

Proof. **Prover efficiency.** Prior work [CBBZ23, Section 3.1] has shown that the PolySumcheck prover for a polynomial $p(X_1, \ldots, X_\mu) := h(g_1(\boldsymbol{X}), \ldots, g_c(\boldsymbol{X}))$, such that for all $i \in [c]$: $g_i(\boldsymbol{X}) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_\mu]$, and h is degree d and can be evaluated through an arithmetic circuit with $O(d)$ gates, can run in $O(2^\mu \cdot d^2)$ time using dynamic programming. This implies the prover efficiency of ECSumcheck.

Miscellaneous efficiency parameters. Other efficiency parameters of ECSumcheck are directly inherited from PolySumcheck, and can be verified on inspection of Figure 4.

Round-by-round tree extractability. Furthermore, the prover does not send any additional vector oracles to the verifier, and the verifier does not query any vector oracles, which lets us borrow the analysis of the round-by-round soundness of PolySumcheck. In particular the vertical predicate $\phi = (\phi_1, \ldots, \phi_{\log n})$ is such that $\phi_i(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, x_1, y_1, \ldots, x_i, y_i) = 0$ if and only if the malicious prover gets lucky with the i-th round challenge r_i, and sends a polynomial $p^{(i)}_{\mathsf{lucky}}(X) \neq p^{(i)}(X)$ and yet $p^{(i)}_{\mathsf{lucky}}(r_i) = p^{(i)}(r_i)$. Since $p^{(i)}$ has degree at most $\ell + 1$, by the Schwartz-Zippel lemma this happens with probability at most $(\ell+1)/|\mathbb{F}|$. Finally, given a $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$-tree T with $\mathbb{w}' := ([\boldsymbol{m}_i]_{i\in[\ell]})$ at the leaf, the extractor simply outputs $[\boldsymbol{m}_i]_{i\in[\ell]}$, which is the correct witness for $\mathsf{HEP}^{(\mathcal{C},\varepsilon,\ell)}$ by completeness of PolySumcheck and correctness of T. □

Lemma 6.8. *Let $\mu := \log k$, and let $\ell, c \in \mathbb{N}$. Consider $f(X_1, \ldots, X_\mu, Z_1, \ldots, Z_\ell) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_{\mu+\ell}]$ such that*

$$f(\boldsymbol{X}, \boldsymbol{Z}) = h(q_1(\boldsymbol{X}), \ldots, q_c(\boldsymbol{X}), Z_1, \ldots, Z_\ell)$$

where $h(\boldsymbol{X}) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_{c+\ell}]$ and for all $i \in [c]$: $q_i(\boldsymbol{X}) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_\mu]$. Then ECSumcheck *can be implemented with $O(2^{\mu+c+\ell})$ prover time.*

Proof. In Figure 4, it is easy to inspect that $[y_i]_{i\in[\ell]} := [\hat{m}_i(\boldsymbol{r})]_{i\in[\ell]}$ can be computed in $O(\ell \cdot 2^\mu)$ time. Thus the ECSumcheck prover's main task is to compute the responses for the PolySumcheck protocol, which is run on the polynomial $p(\boldsymbol{X}) := f(\boldsymbol{X}, \hat{m}_1(\boldsymbol{X}), \ldots, \hat{m}_\ell(\boldsymbol{X}))$. Plugging in the above expression for $f(\boldsymbol{X}, \boldsymbol{Z})$, we see that $p(X_1, \ldots, X_\mu)$ can be expressed as

$$p(\boldsymbol{X}) = h(q_1(\boldsymbol{X}), \ldots, q_c(\boldsymbol{X}), \hat{m}_1(\boldsymbol{X}), \ldots, \hat{m}_\ell(\boldsymbol{X}))$$

where h is multilinear, q_i are multilinear for all $i \in [c]$. $h(\boldsymbol{X})$ can trivially be evaluated using an arithmetic circuit with $O(2^{c+\ell})$ gates. Thus, the result from [CBBZ23, Section 3.1] implies that the prover can compute the responses for the PolySumcheck protocol in $O(2^{\mu+c+\ell})$ time. Note that if c and ℓ are constants (which is the case in all our protocols), the prover time is simply $O(2^\mu)$. □

6.2 Homogenization IOR

When we consider evaluation claims about multiple polynomials, we do not always have the guarantee that the polynomials are evaluated at the same point. To address this, we define the following generalization of HEP, that allows for the evaluation points to be different for each polynomial.

Indexed Oracle Relation 6.9 (Multilinear-evaluation proximity)**.** *The* multilinear-evaluation proximity relation $\mathsf{MEP} = (\mathcal{R}, \tilde{\mathcal{R}})$ *is an indexed oracle pair relation with $\mathcal{R}$ and $\tilde{\mathcal{R}}$ both consisting of tuples of the following form:*

- *index: $(\mathcal{C}, \varepsilon, \ell)$, where $\mathcal{C}$ is a linear code with parameters $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$, $\varepsilon \in (\delta_{\mathcal{C}}/8, \delta_{\mathcal{C}}/3)$ is the proximity parameter, and $\ell \in \mathbb{N}$ is the number of polynomials,*
- *instance: $[(\boldsymbol{z}_i, y_i)]_{i\in[\ell]}$, where for each $i \in [\ell]$: $\boldsymbol{z}_i \in \mathbb{F}^{\log k}$ and $y_i \in \mathbb{F}$,*
- *instance oracle: $[\boldsymbol{w}_i]_{i\in[\ell]}$, where for each $i \in [\ell]$: $\boldsymbol{w}_i \in \mathbb{F}^n$,*
- *witness: $[\boldsymbol{m}_i]_{i\in[\ell]}$, where for each $i \in [\ell]$: $\boldsymbol{m}_i \in \mathbb{F}^k$.*

such that they satisfy the condition

$$\forall i \in [\ell] \quad : \quad \hat{m}_i(\boldsymbol{z}_i) = y_i \ ,$$

as well as following distinct conditions for $\mathcal{R}$ and $\tilde{\mathcal{R}}$ respectively

$$\begin{aligned} \mathcal{R}: &\quad \forall i \in [\ell] \quad : \quad \boldsymbol{w}_i = \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_i), \\ \tilde{\mathcal{R}}: &\quad \forall i \in [\ell] \quad : \quad \delta(\boldsymbol{w}_i, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_i)) \leq \varepsilon \ . \end{aligned}$$

Homogenize is an IOR from MEP to HEP that "homogenizes" the evaluation points in MEP. The techniques in this IOR are inspired by the batch opening ideas from HyperPlonk [CBBZ23, Section 3.8]. However, our techniques are more straightforward since [CBBZ23] attempt to have the verifier run in time sublinear in ℓ, while we are content with linear time, since in all of our use cases ℓ is a constant.

Overview. At a high level, Homogenize works as follows: given an instance of MEP, we first reduce it to an instance of SEP using a random challenge from the verifier. We then use ECSumcheck to reduce this to an instance of HEP, as required. To reduce from MEP to SEP, that is, to show that $\forall i \in [\ell]$: $\hat{m}_i(\boldsymbol{z}_i) = y_i$, we can show that

$$\forall i \in [\ell] \quad : \quad c_i := \hat{m}_i(\boldsymbol{z}_i) - y_i = 0 \ .$$

These equations hold if and only if the polynomial $\sum_{i\in[\ell]} c_i \cdot X^{i-1}$ is identically zero. By the Schwartz-Zippel lemma, it is sufficient to check that

$$\sum_{i\in[\ell]} c_i \cdot t^{i-1} = 0 \tag{8}$$

for a verifier challenge $t \xleftarrow{\$} \mathbb{F}$. Since $\hat{m}_i(\boldsymbol{z}_i) = \sum_{\boldsymbol{b}\in\{0,1\}^{\log k}} \hat{m}_i(\boldsymbol{b}) \cdot \mathrm{eq}(\boldsymbol{b}, \boldsymbol{z}_i)$, we can rewrite Eq. (8) as

$$\sum_{i\in[\ell]} t^{i-1} \cdot \left[\left(\sum_{\boldsymbol{b}\in\{0,1\}^{\log k}} \hat{m}_i(\boldsymbol{b}) \cdot \mathrm{eq}(\boldsymbol{b}, \boldsymbol{z}_i) \right) - y_i \right] = 0 \tag{9}$$

Defining $\sigma := \sum_{i\in[\ell]} t^{i-1} \cdot y_i$ and $f(\boldsymbol{X}, \boldsymbol{Z}) := \sum_{i\in[\ell]} t^{i-1} \cdot Z_i \cdot \mathrm{eq}(\boldsymbol{X}, \boldsymbol{z}_i)$, we can express Eq. (9) as

$$\sum_{\boldsymbol{b}\in\{0,1\}^{\log k}} f(\boldsymbol{b}, \hat{m}_1(\boldsymbol{b}), \ldots, \hat{m}_\ell(\boldsymbol{b})) = \sigma \ .$$

Thus, we have reduced an instance of MEP to an instance of SEP, and all that is left to do is apply the ECSumcheck. The full construction of Homogenize is presented in Fig. 5.

Indexer: Receives as input $\mathbb{i} = (\mathcal{C}, \varepsilon, \ell)$, and outputs $\iota := \mathbb{i}$, $\vec{\mathbb{I}} := \bot$ and $\mathbb{i}' := (\mathcal{C}, \varepsilon, \ell)$.
Input instance: $\mathbb{x} = (\mathcal{C}, \varepsilon, \ell)$.
Input instance oracle: $\vec{\mathbb{y}} = [\boldsymbol{w}_i]_{i\in[\ell]}$.
Input witness: $\mathbb{w} = [\boldsymbol{m}_i]_{i\in[\ell]}$.

$\langle \mathbf{P}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}), \mathbf{V}^{\vec{\mathbb{I}}, \vec{\mathbb{y}}}(\iota, \mathbb{x}) \rangle$:

1. $\mathbf{V}$ samples $t \xleftarrow{\$} \mathbb{F}$ and sends it to $\mathbf{P}$.
2. $\mathbf{P}$ and $\mathbf{V}$ define $\sigma := \sum_{i\in[\ell]} t^{i-1} \cdot y_i$.
3. $\mathbf{P}$ and $\mathbf{V}$ define $f(X_1, \ldots, X_{\log k}, Z_1, \ldots, Z_\ell) \in \mathbb{F}^{\leq 1}[X_1, \ldots, X_{\log k+\ell}]$ as

$$f(\boldsymbol{X}, Z_1, \ldots, Z_\ell) := \sum_{i\in[\ell]} t^{i-1} \cdot Z_i \cdot \mathrm{eq}(\boldsymbol{X}, \boldsymbol{z}_i)$$

4. Define the following inputs for ECSumcheck:

$$\iota_{\mathsf{SEP}} := \mathbb{i}_{\mathsf{SEP}} := (\mathcal{C}, \varepsilon, \ell), \;\; \mathbb{x}_{\mathsf{SEP}} := (f(\boldsymbol{X}, \boldsymbol{Z}), \sigma), \;\; \vec{\mathbb{y}}_{\mathsf{SEP}} := [\boldsymbol{w}_i]_{i\in[\ell]}, \;\; \mathbb{w}_{\mathsf{SEP}} := [\boldsymbol{m}_i]_{i\in[\ell]}.$$

5. $\mathbf{P}$ and $\mathbf{V}$ run $\mathsf{ECSumcheck}(\langle \mathbf{P}(\mathbb{i}_{\mathsf{SEP}}, \mathbb{x}_{\mathsf{SEP}}, \vec{\mathbb{y}}_{\mathsf{SEP}}, \mathbb{w}_{\mathsf{SEP}}), \mathbf{V}^{\vec{\mathbb{l}}, \vec{\mathbb{y}}_{\mathsf{SEP}}}(\iota_{\mathsf{SEP}}, \mathbb{x}_{\mathsf{SEP}})\rangle)$.

Figure 5: Homogenize, an IOR from $\mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$ to $\mathsf{HEP}^{(\mathcal{C},\varepsilon,\ell)}$

Lemma 6.10 (Homogenization IOR)**.** Homogenize *is an IOR from* $\mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$ *to* $\mathsf{HEP}^{(\mathcal{C},\varepsilon,\ell)}$ *with the following efficiency measures*

rounds	*message size*	*queries*	*oracle size*	*prover time*	*verifier time*
$\log n + 1$	$O(\ell \log n)$	0	0	$O(n)$	$O(\ell \log n)$

Furthermore, there exists a vertical predicate ϕ *such that the IOR is* $(\vec{a} := (1, 1, \ldots, 1), \phi, \theta_{\mathsf{true}})$*-RBRTE.*

Proof. The efficiency measures of Homogenize follow immediately from the efficiency measures of ECSumcheck. Furthermore, Homogenize is RBRTE: the vertical predicate and extractor for Homogenize are identical to that of ECSumcheck, except that the vertical predicate's first component ϕ_1 is such that $\phi_1(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}, \perp, t) = 0$ if and only if the malicious prover gets lucky with the choice of t, and there exists an $i \in [\ell]$ such that $\hat{m}_{i,\mathsf{lucky}} \neq \hat{m}_i$, and yet

$$\sum_{\boldsymbol{b}\in\{0,1\}^{\log k}} f(\boldsymbol{b}, \hat{m}_1(\boldsymbol{b}), \ldots, \hat{m}_i(\boldsymbol{b}), \ldots, \hat{m}_\ell(\boldsymbol{b})) = \sum_{\boldsymbol{b}\in\{0,1\}^{\log k}} f(\boldsymbol{b}, \hat{m}_1(\boldsymbol{b}), \ldots, \hat{m}_{i,\mathsf{lucky}}(\boldsymbol{b}), \ldots, \hat{m}_\ell(\boldsymbol{b}))$$

By the Schwartz-Zippel lemma, this event occurs with probability at most $(\ell + 1)/|\mathbb{F}|$, and thus ϕ is a valid vertical-predicate. □

6.3 Batching IOR

Batch is an IOR from $\mathsf{HEP}^{(\mathcal{C},\varepsilon+\varepsilon_b,\ell)}$ to $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ where ε_b is a fixed constant that determines the amount of distance "lost" in the reduction (see Remark 6.11). We provide a high level overview of the IOR below and the full construction is presented in Fig. 6.

Overview. Consider an instance of $\mathsf{HEP}^{(\mathcal{C},\varepsilon+\varepsilon_b,\ell)}$ which is made up ℓ words for which we want to check proximity to a code (constrained by a multilinear evaluation claim for the same evaluation point):

$$\mathbb{i} = (\mathcal{C}, \varepsilon + \varepsilon_b, \ell) \qquad \mathbb{x} = (\boldsymbol{z}, [y_i]_{i\in[\ell]}), \qquad \vec{\mathbb{y}} = [\boldsymbol{w}_i]_{i\in[\ell]}, \qquad \mathbb{w} = [\boldsymbol{m}_i]_{i\in[\ell]} \;\;,$$

such that for each $i \in [\ell]$, the following conditions hold:

$$\delta(\boldsymbol{w}_i, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_i)) \leq \varepsilon + \varepsilon_b \;\; \text{and} \;\; \hat{m}_i(\boldsymbol{z}) = y_i \;\;.$$

If we only had to batch the polynomial evaluation claims, this is straightforward:
1. The verifier sends random challenges $r_1, \ldots, r_\ell$ to the prover.
2. The prover sends back an oracle of $\hat{m}$, which is purportedly equal to $\sum_{i=1}^{\ell} r_i \cdot \hat{m}_i$.

3. To verify the correctness of the oracle $\hat{m}$, the verifier queries $\hat{m}, \hat{m}_1, \ldots, \hat{m}_\ell$ at a random point $\boldsymbol{r} \in \mathbb{F}^\mu$, and checks that $\hat{m}(\boldsymbol{r}) = \sum_{i=1}^{\ell} r_i \cdot \hat{m}_i(\boldsymbol{r})$. If the prover is dishonest, this check passes with only negligible probability due to the Schwartz-Zippel lemma.
4. If the check passes, the verifier can check if $\hat{m}(\boldsymbol{z}) = y := \sum_{i=1}^{\ell} r_i \cdot y_i$ to verify the ℓ-many polynomial evaluation claims.

Batching the proximity claims is just as straightforward, and leverages the use of proximity gaps. We follow the same approach as Steps 1 and 2, with the prover sending a word $\boldsymbol{w}$ that is purported to be $\sum_{i=1}^{\ell} r_i \cdot \boldsymbol{w}_i$, corresponding to the message $\boldsymbol{m} = \sum_{i=1}^{\ell} r_i \cdot \boldsymbol{m}_i$. The consistency check of Step 3 cannot be performed exactly since the verifier only has vector oracle access to $\boldsymbol{w}$ and $\boldsymbol{w}_i$. However, the verifier can do an *approximate consistency check* by performing *spot checks*. That is, for some appropriate value $\nu = O(-\lambda/\log(1-\varepsilon_b))$ and randomly chosen indices $[\alpha_j]_{j\in[\nu]}$ the verifier checks $\boldsymbol{w}[\alpha_j] = \sum_{i=1}^{\ell} r_i \cdot \boldsymbol{w}_i[\alpha_j]$, which guarantees that $\delta\left(\boldsymbol{w}, \sum_{i=1}^{\ell} r_i \cdot \boldsymbol{w}_i\right) \leq \varepsilon_b/2$ with overwhelming probability. If these checks pass, similar to Step 4, the verifier can additionally check that $\delta(\mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}), \boldsymbol{w}) \leq \varepsilon + \varepsilon_b/2$. Together with the check of Step 4, this final check corresponds to an instance of $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ as desired. We now briefly argue why this reduction is sound. If $(\mathbb{i}', \mathbb{x}', \vec{\mathbb{y}}' := \boldsymbol{w}, \mathbb{w}' := \boldsymbol{m}) \in \mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$, we have

$$
\begin{aligned}
&\delta(\mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}), \boldsymbol{w}) \leq \varepsilon && \text{[Definition of MEP]}\\
\Rightarrow &\delta\left(\mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}), \sum_{i=1}^{\ell} r_i \cdot \boldsymbol{w}_i\right) \leq \varepsilon + \varepsilon_b/2 && \left[\text{Triangle inequality and } \delta\left(\boldsymbol{w}, \sum_{i=1}^{\ell} r_i \cdot \boldsymbol{w}_i\right) \leq \varepsilon_b/2\right]\\
\Rightarrow &\delta\left(\sum_{i=1}^{\ell} \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_i), \sum_{i=1}^{\ell} r_i \cdot \boldsymbol{w}_i\right) \leq \varepsilon + \varepsilon_b/2\ . && \text{[}\mathcal{C}\text{ is a linear code]}
\end{aligned}
$$

Using correlated agreement for arbitrary linear codes (see Lemma 5.5), this implies $\delta(\boldsymbol{w}_i, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_i)) \leq \varepsilon + \varepsilon_b$ for all $i \in [\ell]$ with probability at least $1 - \frac{2}{\varepsilon_b |\mathbb{F}|} = 1 - \mathrm{negl}(\lambda)$.

Remark 6.11 (range of ε and picking ε_b)**.** The final step of the above soundness argument only holds for $\varepsilon < \delta_{\mathcal{C}}/3$, because that is the relative distance up to which Lemma 5.5 holds. Therefore, we restrict our attention to this range for our IORs. Furthermore, in this work we only consider constant-rate and constant-distance codes, so we fix ε_b as a universal constant such that it is smaller than $\delta_{\mathcal{C}}/8$ for any such code $\mathcal{C}$ that we consider. Note that the number of spot checks ℓ that we need to perform depends on all of these parameters, but since they are all constants, we still have $\ell = O(\lambda)$.

Indexer: Receives as input $\mathbb{i} = (\mathcal{C}, \varepsilon + \varepsilon_b, \ell)$, and outputs $\iota := (\varepsilon + \varepsilon_b, \ell)$, $\vec{\mathbb{I}} := \perp$ and $\mathbb{i}' := (\mathcal{C}, \varepsilon, 1)$.
Input instance: $\mathbb{x} = (\boldsymbol{z}, [y_i]_{i\in[\ell]})$.
Input instance oracle: $\vec{\mathbb{y}} = [\boldsymbol{w}_i]_{i\in[\ell]}$.
Input witness: $\mathbb{w} = [\boldsymbol{m}_i]_{i\in[\ell]}$ such that $\forall i \in [\ell]$, we have $\delta(\mathcal{C}(\boldsymbol{m}_i), \boldsymbol{w}_i) \leq \varepsilon$ and $\hat{m}_i(\boldsymbol{z}) = y_i$.

$\langle \mathbf{P}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}), \mathbf{V}^{\vec{\mathbb{I}}, \vec{\mathbb{y}}}(\iota, \mathbb{x})\rangle$:

Interaction phase.
1. Set $b \leftarrow 1$.
2. $\mathbf{V}$ samples $r_1, \ldots, r_\ell \xleftarrow{\$} \mathbb{F}$ and sends them to $\mathbf{P}$.
3. Define $y' := \sum_{i=1}^{\ell} r_i \cdot y_i$.
4. $\mathbf{P}$ computes $\boldsymbol{w}' := \sum_{i=1}^{\ell} r_i \cdot \boldsymbol{w}_i$, and sends $[\![\boldsymbol{w}']\!]$ to $\mathbf{V}$.

Query phase.

1. $\mathbf{V}$ samples $\nu := \left\lceil \frac{-\lambda}{\log(1-\varepsilon_b/2)} \right\rceil$ unique indices $\alpha_1, \ldots, \alpha_\nu \xleftarrow{\$} [n]$.
2. For i in $[1, \ldots, \nu]$:
3. $\quad \mathbf{V}$ queries $[\![\boldsymbol{w}']\!]$ at index α_i and receives $\boldsymbol{w}'[\alpha_i]$.
4. $\quad$ For j in $[1, \ldots, \ell]$:
5. $\quad\quad \mathbf{V}$ queries $[\![\boldsymbol{w}_j]\!]$ at index α_i and receives $\boldsymbol{w}_j[\alpha_i]$.
6. $\quad \mathbf{V}$ checks if $\boldsymbol{w}'[\alpha_i] = \sum_{j=1}^{\ell} r_j \cdot \boldsymbol{w}_j[\alpha_i]$. If the check fails, set $b \leftarrow 0$.
7. Define

$$\mathbb{x}' := (\boldsymbol{z}, y'), \quad \vec{\mathbb{y}}' := \boldsymbol{w}', \quad \mathbb{w}' := \boldsymbol{m}' = \sum_{i=1}^{\ell} r_i \cdot \boldsymbol{m}_i.$$

8. $\mathbf{P}$ outputs $(b, \mathbb{x}', \vec{\mathbb{y}}', \mathbb{w}')$ and $\mathbf{V}$ outputs $(b, \mathbb{x}', \vec{\mathbb{y}}')$.

Figure 6: Batch, an IOR from $\mathsf{HEP}^{(\mathcal{C}, \varepsilon+\varepsilon_b, \ell)}$ to $\mathsf{MEP}^{(\mathcal{C}, \varepsilon, 1)}$

Lemma 6.12 (Batching IOR)**.** *For any constant $\varepsilon_b > 0$, there exists an IOR from $\mathsf{HEP}^{(\mathcal{C}, \varepsilon+\varepsilon_b, \ell)}$ to $\mathsf{MEP}^{(\mathcal{C}, \varepsilon, 1)}$ which has the following efficiency measures:*

rounds	*message size*	*queries*	*oracle size*	*prover time*	*verifier time*
2	$O\left(\frac{-\ell\lambda}{\log(1-\varepsilon_b)}\right)$	$O\left(\frac{-\ell\lambda}{\log(1-\varepsilon_b)}\right)$	n	$O(\ell n)$	$O\left(\frac{-\ell\lambda}{\log(1-\varepsilon_b)}\right)$

Furthermore, there exist vertical and challenge predicates ϕ, θ (see Definitions 4.2 and 4.3) such that Batch *is $(\vec{a} := (\ell, 1), \phi, \theta)$-RBRTE.*

Proof. Completeness and efficiency of the batching IOR follow directly from the description in Fig. 6. To show RBRTE, we need to:

(1) Describe the predicate ϕ and prove that it is a valid vertical-predicate (Definition 4.3).
(2) Describe the predicate θ and prove that it is a valid challenge-predicate (Definition 4.2).
(3) Describe the tree extractor $\mathbf{E}^{\mathrm{Tree}}$ and show that given *any* $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$-constrained transcript tree it outputs $\mathbb{w}$ such that $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \in \tilde{\mathcal{R}}$.

All these are implicitly parameterized by the parameters of the index.

Vertical predicate. We define the vertical predicate ϕ as follows. $\phi_1([\boldsymbol{w}_i]_{i\in[\ell]}, \vec{r}) = 0$ if and only if

$$\delta\left(\sum_{i=1}^{\ell} r_i \boldsymbol{w}_i, \mathcal{C}\right) \leq \varepsilon + \frac{\varepsilon_b}{2}$$

and yet there does not exist a set $S \subseteq [n]_0$ of size at least $(1-\varepsilon)n$ and codewords $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_\ell$ such that for all $i \in [\ell]$ and $j \in S$, we have $\boldsymbol{w}_i[j] = \boldsymbol{v}_i[j]$. Given that $\varepsilon + \varepsilon_b/2 < \delta_{\mathcal{C}}/3$, this event corresponds to the event that correlated agreement fails. However, by Lemma 5.5, we know that for all $[\boldsymbol{w}_i]_{i\in[\ell]}$

$$\Pr_{\vec{r} \leftarrow \mathbb{F}^\ell}[\phi_1([\boldsymbol{w}_i]_{i\in[\ell]}, \vec{r}) = 0] < \mathrm{negl}(\lambda)$$

as desired. Next, $\phi_2([\boldsymbol{w}_i]_{i\in[\ell]}, \vec{r}, \boldsymbol{w}', \vec{\alpha}) = 0$ if and only if

$$\delta\left(\boldsymbol{w}', \sum_{i=1}^{\ell} r_i \boldsymbol{w}_i\right) > \frac{\varepsilon_b}{2} \ \wedge \ \forall j \in [\nu] : \boldsymbol{w}'[\alpha_j] = \sum_{i=1}^{\ell} r_i \boldsymbol{w}_i[\alpha_j] \ .$$

However, we know that

$$\Pr_{\vec{\alpha}\leftarrow[n]_0^{\nu}}[\phi_2([\boldsymbol{w}_i]_{i\in[\ell]}, \vec{r}, \boldsymbol{w}', \vec{\alpha}) = 0] = (1-\varepsilon_b/2)^{\nu} = (1-\varepsilon_b/2)^{\left\lceil \frac{-\lambda}{\log(1-\varepsilon_b/2)} \right\rceil} \leq 2^{-\lambda}$$

This verifies that ϕ is a valid vertical-predicate.

Challenge Predicate. The challenge predicate $\theta(\vec{r}^{(1)}, \ldots, \vec{r}^{(\ell)}) = 0$ if and only if $\vec{r}^{(1)}, \ldots, \vec{r}^{(\ell)}$ are linearly dependent. These vectors can only be linearly dependent if there exists $j \in [\ell]$ such that $\vec{r}^{(j)} \in \mathrm{span}(\vec{r}^{(1)}, \ldots, \vec{r}^{(j-1)})$. Because $\vec{r}^{(1)}, \ldots, \vec{r}^{(j-1)}$ span a vector size with $|\mathbb{F}|^{j-1}$ elements, we have that

$$\Pr_{\vec{r}^{(j)}\leftarrow\mathbb{F}^{\ell}}\left[\vec{r}^{(j)} \in \mathrm{span}(\vec{r}^{(1)}, \ldots, \vec{r}^{(j-1)})\right] = \frac{|\mathbb{F}|^{j-1}}{|\mathbb{F}|^{\ell}} \leq \frac{1}{|\mathbb{F}|} = \mathrm{negl}(\lambda)$$

where the first equality is by a counting argument, the second equality by $j-1 < \ell$, and the last by assumption on the field size. Therefore, θ is a valid challenge predicate.

Tree extractor. The tree extractor $\mathbf{E}^{\mathrm{Tree}}$ computes $\boldsymbol{M}' \in \mathbb{F}^{\ell\times k}$ such that the i-th row $\boldsymbol{m}'_i \in \mathbb{F}^{\ell}$ is the witness $\mathbb{w}'$ written on the i-th leaf of the tree. It also computes $\boldsymbol{C} \in \mathbb{F}^{\ell\times\ell}$ such that its i-th row is $\vec{r}^{(i)}$, the label of the i-th edge going out of the first layer of the tree. $\mathbf{E}^{\mathrm{Tree}}$ then solves the following matrix equation for $\boldsymbol{M} = \boldsymbol{C}^{-1} \cdot \boldsymbol{M}'$. $\boldsymbol{C}$ is invertible by definition of θ_1, and therefore $\boldsymbol{M}$ can be computed in polynomial time via Gaussian elimination.

We will now argue that $[\boldsymbol{m}_i]_{i\in[\ell]}$, the rows of $\boldsymbol{M}$, form the witness for $\mathsf{HEP}^{(\mathcal{C},\varepsilon+\varepsilon_b,\ell)}$. That is,

$$\delta(\boldsymbol{w}_i, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_i)) \leq \varepsilon + \varepsilon_b \quad \text{and} \quad \hat{m}_i(\boldsymbol{z}) = y_i \ .$$

By definition of ϕ_1, which enforces correlated agreement did not fail, there exist messages $\boldsymbol{m}''_i \in \mathbb{F}^k$ and errors $\boldsymbol{e}_i$ such that

$$\boldsymbol{w}_i = \mathcal{C}(\boldsymbol{m}''_i) + \boldsymbol{e}_i$$

where there exists a set $S \subseteq [n]_0$ of size at least $(1-\varepsilon-\varepsilon_b)n$ such that for all $i \in [\ell]$ and $j \in S$, we have $\boldsymbol{e}_i[j] = 0$. By the linearity of $\mathcal{C}$, for all $j \in [\ell]$ we have

$$\boldsymbol{w}'_j = \sum_{i=1} {r_i}^{(j)}(\mathcal{C}(\boldsymbol{m}''_i) + {\boldsymbol{e}_i}^{(j)}) = \mathcal{C}\left(\sum_{i=1} {r_i}^{(j)} \boldsymbol{m}''_i\right) + \boldsymbol{e}^{(j)}$$

where $\boldsymbol{e}^{(j)} := \sum_{i=1}^{\ell} {r_i}^{(j)} {\boldsymbol{e}_i}^{(j)}$. Since $\boldsymbol{e}^{(j)}$ has at most $(\varepsilon+\varepsilon_b)n < \delta_{\mathcal{C}}/2$ non-zero entries, we get $\boldsymbol{m}_i = \sum_{i=1} {r_i}^{(j)} \boldsymbol{m}''_i$ as desired. □

6.4 Codeswitch IOR

Codeswitching is a powerful tool that has been used in past influential works [RR25; RR24; BCFRRZ25] which reduces checking a claim about the proximity of $\boldsymbol{w}$ to $\mathcal{C}$ to checking a claim about the proximity of $\boldsymbol{w}'$ to a (possibly different) code $\mathcal{C}'$. The prover efficiency of such a reduction primarily depends on 2 factors: (i) the structure of the code $\mathcal{C}$, and (ii) and the encoding time of the code $\mathcal{C}'$. In this work, we formalize codeswitching as an IOR, which we define here.

Definition 6.13 (CodeSwitch IOR)**.** CodeSwitch *is an IOR from* $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ *to* $\mathsf{MEP}^{(\mathcal{C}',\varepsilon',1)}$ *such that* $\mathcal{C}$ *and* $\mathcal{C}'$ *are linear codes with parameters* $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$ *and* $(\mathbb{F}, n', k, \delta_{\mathcal{C}'})$ *respectively, and* $\varepsilon \in (\delta_{\mathcal{C}}/8, \delta_{\mathcal{C}}/3 - 2\varepsilon_b)$*, and* $\varepsilon' \in (\delta_{\mathcal{C}'}/8, \delta_{\mathcal{C}'}/3 - \varepsilon_b)$*, where* ε_b *is the loss in distance incurred due to an application of* Batch *(see Section 6.3).*

We use this formal definition of CodeSwitch, to define *efficiently codeswitchable codes*.

Definition 6.14 (Efficiently codeswitchable codes)**.** *If there exists a* CodeSwitch *IOR from* $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ *to* $\mathsf{MEP}^{(\mathcal{C}',\varepsilon',1)}$ *such that the prover time is* $O(n + T_{\mathcal{C}'})$*, the number of oracle queries is* $O(\lambda)$*, and the verifier time is* $O(\lambda \log n)$*, then* $\mathcal{C}$ *is said to be efficiently codeswitchable.*

We sometimes need to consider a family of codes that are efficiently codeswitchable, which we define below.

Definition 6.15 (Efficiently codeswitchable code family)**.** *An efficiently codeswitchable code family over a field* $\mathbb{F}$ *is a set of codes* $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$*, with corresponding parameter functions* $n(\cdot)$ *and* $\delta(\cdot)$*, such that:*

- *For each* k*,* $\mathcal{C}_k$ *is a linear code with parameters* $(\mathbb{F}, n(k), k, \delta(k))$ *that is efficiently codeswitchable.*
- *There exist constants* δ_0 *and* ρ_0*, and some* $k_0 \in \mathbb{N}$ *such that for all* $k \geq k_0$*, we have* $\delta(k) \geq \delta_0$ *and* $k/n(k) \geq \rho_0$*. We call* δ_0 *the distance of the code family.*

Additionally, we say that a codeswitchable code family is linear-time encodable if for all k*,* $\mathcal{C}_k$ *is a linear-time encodable code.*

Outline of CodeSwitch. We recap the outline of CodeSwitch from Section 2. The given $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ instance is reduced to an instance of $\mathsf{SEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,\ell_1)}$ and an instance of $\mathsf{MEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,\ell_2)}$, where $\ell_1, \ell_2 \in \mathbb{N}$ are constants.[17] $\mathsf{SEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,\ell_1)}$ is then reduced to $\mathsf{HEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,\ell_2)}$ using ECSumcheck. The resulting HEP $\times$ MEP instance corresponds to an instance of $\mathsf{MEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,\ell_1+\ell_2)}$, which is then reduced to $\mathsf{HEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,\ell_1+\ell_2)}$ using Homogenize. Finally, $\mathsf{HEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,\ell_1+\ell_2)}$ is reduced to $\mathsf{MEP}^{(\mathcal{C}',\varepsilon',1)}$ using Batch. In Section 7, we provide a concrete instantiation of CodeSwitch for tensor codes, where we focus on the first part of this outline: the reduction to instances of SEP and MEP.

Remark 6.16 (Instantiation of polynomial oracles)**.** Similar to previous work, we instantiate the polynomial oracle to a multilinear polynomial $\hat{p} \in \mathbb{F}[X_1, \ldots, X_\mu]$ by committing to an encoding of $\boldsymbol{p}$. Therefore, when we instantiate polynomial oracles in the process of codeswitching from $\mathcal{C}$ to $\mathcal{C}'$, all codewords sent by the honest prover must be under the code $\mathcal{C}'$. However, it's possible that the message length k of $\mathcal{C}'$ is not equal to the size of $\hat{p}$, which is 2^μ. In order to overcome this, we can break up $\boldsymbol{p}$ into multiple blocks, each of length k, and encoding each block separately under $\mathcal{C}'$. That is, we encode $\boldsymbol{p}$ under the interleaved code $\mathsf{IC}[\mathcal{C}', \eta]$ where $\eta := 2^\mu/k$ is the interleaving parameter. As long as η is a constant (which is the case in all our constructions), we do not incur an asymptotic overhead in any efficiency parameter for our IORs. Interleaved codes have been used in previous work such as Brakedown [GLSTW23] and Ligero [AHIV17], but unlike previous work, we only work with the case where the interleaving parameter is a constant. For clarity, we introduce an intermediate relation, which we call the *interleaved* MEP *relation* (IMEP), and reduce to IMEP instances instead of MEP instances whenever such a mismatch of message lengths occurs. In Section E, we formally define IMEP and describe the DeInterleave IOR which reduces an IMEP instance to an MEP instance.

[17] There can be multiple instances of SEP and MEP in the reduction, but we consider only one here for simplicity.

7 Codeswitch for tensor codes

In this section, we present a formal description of the CodeSwitch IOR from tensor codes as motivated in Section 2.8. Recall that this IOR reduces an $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ instance (where $\mathcal{C}$ is a t-wise tensor of a base code $\mathcal{B}$), to an $\mathsf{MEP}^{(\mathcal{C}',\varepsilon',1)}$ instance, where $\mathcal{C}'$ is an arbitrary code. Our construction is obtained by composing the following two IORs:

- The consistency checking IOR: reduces the given $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ instance to an instance of $\mathsf{CIP}^{(\mathcal{C},\mathcal{C}',\varepsilon',\nu)}$ and $\mathsf{MEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,1)}$.
- The inner product IOR: reduces the $\mathsf{CIP}^{(\mathcal{C},\mathcal{C}',\varepsilon',\nu)}$ instance to multiple instances of SEP and MEP over $\mathcal{C}'$.

Throughout this section, ε_b is a fixed small enough constant introduced due to an application of the Batch IOR. We refer the reader to Section 6.3 for the definition of ε_b.

7.1 Consistency checking IOR

Recall from Section 2.8 that the consistency checking IOR reduces the given $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ instance to an instance of $\mathsf{CIP}^{(\mathcal{C},\mathcal{C}',\varepsilon',\nu)}$ and $\mathsf{MEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,1)}$.

Indexed Oracle Relation 7.1 (Code-inner-product relation)**.** *The* code-inner-product relation $\mathsf{CIP} = (\mathcal{R}, \tilde{\mathcal{R}})$ *is an indexed oracle pair relation with $\mathcal{R}$ and $\tilde{\mathcal{R}}$ both consisting of tuples of the following form:*

- *index: $(\mathcal{C}, \mathcal{C}', \varepsilon, \nu)$ where $\mathcal{C}$ is a linear code with parameters $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$, $\mathcal{C}'$ is a linear code with parameters $(\mathbb{F}, n', k, \delta_{\mathcal{C}'})$, $\varepsilon \in (\delta_{\mathcal{C}'}/8, \delta_{\mathcal{C}'}/3 - 2\varepsilon_b)$, and $\nu \in \mathbb{N}$ is the number of spot checks,*
- *instance: target inner products $\sigma_1, \ldots, \sigma_\nu \in \mathbb{F}$, and spot check indices $\alpha_1, \ldots, \alpha_\nu \in [n]_0$,*
- *instance oracles: $\boldsymbol{w} \in \mathbb{F}^n$ and $\boldsymbol{w}' \in \mathbb{F}^{n'}$,*
- *witness: $\boldsymbol{m} \in \mathbb{F}^k$,*

such that for all $i \in [\nu]$, $\boldsymbol{G}_{\mathcal{C}}[\alpha_i] \cdot \boldsymbol{m} = \boldsymbol{w}[\alpha_i] = \sigma_i$, and the following conditions are satisfied for $\mathcal{R}$ and $\tilde{\mathcal{R}}$ respectively:

$$\mathcal{R}: \quad \boldsymbol{w}' = \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m}) \ ,$$
$$\tilde{\mathcal{R}}: \quad \delta(\boldsymbol{w}', \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m})) \leq \varepsilon \ .$$

Indexer input: Index of $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$: $(\mathcal{C}, \varepsilon, 1)$ where $\mathcal{C} = \mathsf{TC}[\mathcal{B}, t]$ and $\mathcal{B}$ is a linear code with parameters $(\mathbb{F}, n^{1/t}, k^{1/t}, \delta_{\mathcal{B}})$.
Indexer output: The indexer initializes code $\mathcal{C}'$ with parameters $(\mathbb{F}, n', k', \delta_{\mathcal{C}'})$ and outputs

- $\mathbb{i} := (\mathbb{i}_{\mathsf{CIP}} := (\mathcal{C}, \mathcal{C}', \varepsilon', \nu := \lceil -\lambda/\log(1-\varepsilon) \rceil), \mathbb{i}_{\mathsf{MEP}} := (\mathcal{C}', \varepsilon' + \varepsilon_b, 1))$,
- $\iota := \nu$, and
- $\vec{\mathbb{I}} := \bot$.

Input instance: $\mathbb{x} := (\boldsymbol{z}, y)$ such that $\boldsymbol{z} \in \mathbb{F}^{\log k}$, and $y \in \mathbb{F}$.
Input instance oracles: $\vec{\mathbb{y}} := \boldsymbol{w}$ where $\boldsymbol{w} \in \mathbb{F}^n$ is a purported codeword of $\mathcal{C}$.
Input witness: $\boldsymbol{m} \in \mathbb{F}^k$ such that $\delta(\mathcal{C}(\boldsymbol{m}), \boldsymbol{w}) \leq \varepsilon$ and $\hat{m}(\boldsymbol{z}) = y$.

$\langle \mathbf{P}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}), \mathbf{V}^{\vec{\mathbb{I}}, \vec{\mathbb{y}}}(\iota, \mathbb{x}) \rangle$:

Interaction phase.

1. $\mathbf{P}$ computes $\boldsymbol{w}' = \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m})$ and sends $[\![\boldsymbol{w}']\!]$ to $\mathbf{V}$.
2. $\mathbf{V}$ samples $\alpha_1, \ldots, \alpha_\nu \xleftarrow{\$} [n]$ and sends them to $\mathbf{P}$.

Query phase.

1. $\mathbf{V}$ queries $[\![\boldsymbol{w}]\!]$ at indices $\alpha_1, \ldots, \alpha_\nu$ to obtain $\sigma_1 := \boldsymbol{w}[\alpha_1], \ldots, \sigma_\nu := \boldsymbol{w}[\alpha_\nu]$.
2. Define

$$\begin{aligned} \mathbb{x}_{\mathsf{CIP}} &:= ([\sigma_i := \boldsymbol{w}[\alpha_i]]_{i=1}^{\nu}, [\alpha_i]_{i=1}^{\nu}), & \mathbb{x}_{\mathsf{MEP}} &:= (\boldsymbol{z}, y), \\ \vec{\mathbb{y}}_{\mathsf{CIP}} &:= (\boldsymbol{w}, \boldsymbol{w}'), & \vec{\mathbb{y}}_{\mathsf{MEP}} &:= \boldsymbol{w}', \\ \mathbb{w}_{\mathsf{CIP}} &:= \boldsymbol{m}, & \mathbb{w}_{\mathsf{MEP}} &:= \boldsymbol{m}. \end{aligned}$$

3. $\mathbf{P}$ outputs the appropriate tuple of instances, instance oracles, and witnesses, and $\mathbf{V}$ outputs the appropriate tuple of instances and instance oracles.

Figure 7: Consistency checking IOR from $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ to $\mathsf{CIP}^{(\mathcal{C},\mathcal{C}',\varepsilon',\nu)} \times \mathsf{MEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,1)}$.

Remark 7.2 (reduction to product is a simplification)**.** The reduction in Fig. 7 is actually not to the product of CIP and MEP, and is in fact to a subset which enforces that the witness $\boldsymbol{m}$ is the same in both relations, and $\boldsymbol{w}'$ (the encoding of $\boldsymbol{m}$ under $\mathcal{C}'$) is also shared. The exposition is simplified by stating it as a reduction to the product, but the actual reduction is to the aforementioned subset relation.

Lemma 7.3. *There exists an IOR (see Figure 7) from* $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ *to* $\mathsf{CIP}^{(\mathcal{C},\mathcal{C}',\varepsilon',\nu)} \times \mathsf{MEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,1)}$ *with the following efficiency measures*

rounds	*message size*	*queries*	*oracle size*	*prover time*	*verifier time*
1	$O\left(\frac{-\lambda}{\log(1-\varepsilon)}\right)$	$O\left(\frac{-\lambda}{\log(1-\varepsilon)}\right)$	n	$O(T_{\mathcal{C}'})$	$O\left(\frac{-\lambda}{\log(1-\varepsilon)}\right)$

Furthermore, there exists a vertical-predicate ϕ *such that the IOR is* $(\vec{a} := (1), \phi, \theta_{\mathsf{true}})$*-RBRTE.*

Proof. The efficiency parameters follow by inspection.

Vertical predicate. To show that the IOR is RBRTE, we first define a vertical predicate ϕ. Let $x_1 := [\![\boldsymbol{w}']\!]$ be the prover's message in the first round, and let $y_1 := (\alpha_1, \ldots, \alpha_\nu)$ be the verifier's random challenges in the first round. Then $\phi(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, x_1, y_1) = 0$ if and only if there exists $\boldsymbol{m}_{\mathsf{lucky}} \in \mathbb{F}^k$ such that $\delta(\boldsymbol{w}, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_{\mathsf{lucky}})) > \varepsilon$ and yet it holds that

- $\delta(\boldsymbol{w}', \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m}_{\mathsf{lucky}})) \leq \varepsilon'$, and
- $\boldsymbol{G}_{\mathcal{C}}[\alpha_i] \cdot \boldsymbol{m}_{\mathsf{lucky}} = \boldsymbol{w}[\alpha_i]$ for all $i \in [\nu]$.

This corresponds to the event where a malicious prover gets "lucky" due to existence of such a witness $\boldsymbol{m}_{\mathsf{lucky}}$. Clearly, for any $x_1 \in \mathbb{F}^{n'}$, we have

$$\begin{aligned} \Pr_{y_1 \xleftarrow{\$} [n]^\nu}[\phi(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, x_1, y_1) = 0] &= \Pr_{\alpha_1, \ldots, \alpha_\nu \xleftarrow{\$} [n]} \left[\begin{array}{c} \exists\, \boldsymbol{m}_{\mathsf{lucky}} \text{ such that} \\ \forall i \in [\nu], \boldsymbol{G}_{\mathcal{C}}[\alpha_i] \cdot \boldsymbol{m}_{\mathsf{lucky}} = \boldsymbol{w}[\alpha_i] \\ \text{and} \\ \delta(\boldsymbol{w}, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_{\mathsf{lucky}})) > \varepsilon \\ \text{and} \\ \delta(\boldsymbol{w}', \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m}_{\mathsf{lucky}})) \leq \varepsilon' \end{array} \right] \\ &= \Pr_{\alpha_1, \ldots, \alpha_\nu \xleftarrow{\$} [n]} \left[\begin{array}{r} \forall i \in [\nu], \boldsymbol{G}_{\mathcal{C}}[\alpha_i] \cdot \mathsf{Enc}_{\mathcal{C}'}^{-1}(\boldsymbol{w}') = \boldsymbol{w}[\alpha_i] \\ \delta(\boldsymbol{w}, \mathsf{Enc}_{\mathcal{C}}(\mathsf{Enc}_{\mathcal{C}'}^{-1}(\boldsymbol{w}'))) > \varepsilon \end{array} \right] \end{aligned}$$

$$\leq (1-\varepsilon)^{\nu} = \left((1-\varepsilon)^{\left\lceil \frac{-\lambda}{\log(1-\varepsilon)} \right\rceil}\right) \leq 2^{-\lambda} \ .$$

where the second equality follows because ε' is less than the unique decoding radius of $\mathcal{C}'$, and therefore, there exists only one message $\mathsf{Enc}_{\mathcal{C}'}^{-1}(\boldsymbol{w}')$ that can satisfy the third condition.

Tree extractor. We now construct a tree extractor that can extract a witness $\boldsymbol{m}$ from a $(\vec{a}, \phi, \theta_{\mathsf{true}}, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$-constrained transcript tree. Consider a leaf of this tree with the following label:

$$\mathbb{i}' := (\mathbb{i}_{\mathsf{CIP}}, \mathbb{i}_{\mathsf{MEP}}) \quad ; \quad \mathbb{x}' := (\mathbb{x}_{\mathsf{CIP}}, \mathbb{x}_{\mathsf{MEP}}) \quad ; \quad \vec{\mathbb{y}}' := (\boldsymbol{w}, \boldsymbol{w}') \quad ; \quad \mathbb{w}' := (\boldsymbol{m})$$

Given as input this tree, the extractor simply outputs $\boldsymbol{m}$. To show correctness, we must prove that $\hat{m}(\boldsymbol{z}) = y$ and $\delta(\boldsymbol{w}, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m})) \leq \varepsilon$.

- Since $(\mathbb{i}', \mathbb{x}', \vec{\mathbb{y}}', \mathbb{w}') \in \tilde{\mathcal{R}}'$, it must be the case that $\hat{m}(\boldsymbol{z}) = y$.
- Since the verifier must have an accepting view of this leaf (by definition of the constrained transcript tree), it must also be the case that $\boldsymbol{G}_{\mathcal{C}}[\alpha_i] \cdot \boldsymbol{m} = \boldsymbol{w}[\alpha_i]$ for all $i \in [\nu]$. Therefore, we must have $\delta(\boldsymbol{w}, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m})) \leq \varepsilon$, since the predicate ϕ is not satisfied if $\delta(\boldsymbol{w}, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m})) > \varepsilon$ and $\boldsymbol{G}_{\mathcal{C}}[\alpha_i] \cdot \boldsymbol{m} = \boldsymbol{w}[\alpha_i]$ for all $i \in [\nu]$.

Therefore, $\boldsymbol{m}$ is a valid witness for $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$.

We note that the definition of our vertical predicate relies on the existence of the lucky message $\boldsymbol{m}_{\mathsf{lucky}}$. Finding this message may not be possible in polynomial time, and therefore the vertical predicate might not be efficiently computable. However, this is not a problem as the extractor never has to compute the vertical predicate (see Remark A.14). □

7.2 Inner product IOR

In Section 2.8, we considered tensor codes $\mathcal{C} = \mathsf{TC}[\mathcal{B}, 2]$ and described the IOR from an instance of $\mathsf{CIP}^{(\mathcal{C},\mathcal{C}',\varepsilon,\nu)}$ to SEP and MEP instances for such codes. We generalize the techniques presented in Section 2.8 to $\mathcal{C} = \mathsf{TC}[\mathcal{B}, t]$ for any constant $t \geq 2$.

We first prove the following structural lemma about the generator matrix of tensor codes:

Lemma 7.4. *Consider a tensor code $\mathcal{C} = \mathsf{TC}[\mathcal{B}, t]$ with generator matrix $\boldsymbol{G}_{\mathcal{C}} = \bigotimes_{i=1}^{t} \boldsymbol{G}_{\mathcal{B}}$. Say $\hat{c} : \mathbb{F}^{\log n + \log k} \to \mathbb{F}$ is the unique multilinear polynomial such that $\hat{c}(\langle x \rangle, \langle y \rangle) = \boldsymbol{G}_{\mathcal{C}}[x][y]$ for all $x \in [n]_0$ and $y \in [k]_0$. Also $\hat{b} : \mathbb{F}^{(\log n + \log k)/t} \to \mathbb{F}$ is the unique multilinear polynomial such that $\hat{b}(\langle x \rangle, \langle y \rangle) = \boldsymbol{G}_{\mathcal{B}}[x][y]$ for all $x \in [n^{1/t}]_0$ and $y \in [k^{1/t}]_0$. Then we have that*

$$\hat{c}(\boldsymbol{x}, \boldsymbol{y}) = \prod_{i=1}^{t} \hat{b}(\boldsymbol{x}_i, \boldsymbol{y}_i) \ ,$$

where $\boldsymbol{x}_i$ and $\boldsymbol{y}_i$ are the i-th "blocks" of $\boldsymbol{x}$ and $\boldsymbol{y}$ respectively. That is, $\boldsymbol{x}_i = \boldsymbol{x}[(i-1) \cdot \log n/t : i \cdot \log n/t - 1]$ and $\boldsymbol{y}_i = \boldsymbol{y}[(i-1) \cdot \log k/t : i \cdot \log k/t - 1]$.

Proof. Consider $\boldsymbol{G}_{\mathcal{C}}[x][y]$ for an arbitrary $x \in [n]_0$ and $y \in [k]_0$. Since $\boldsymbol{G}_{\mathcal{C}} = \bigotimes_{i=1}^{t} \boldsymbol{G}_{\mathcal{B}}$, we have $\boldsymbol{G}_{\mathcal{C}}[x][y] = \prod_{i=1}^{t} \boldsymbol{G}_{\mathcal{B}}[x_i][y_i]$ where $x = \sum_{i=1}^{t} x_i \cdot n^{(i-1)/t}$ and $y = \sum_{i=1}^{t} y_i \cdot k^{(i-1)/t}$ are the base-$n^{1/t}$ and base-$k^{1/t}$ representations of x and y respectively. Since k-variate multilinear polynomials are defined by their evaluations over $\{0,1\}^k$, the lemma follows. □

We recall from Section 2.8 that we can prove a $\mathsf{CIP}^{(\mathcal{C},\mathcal{C}',\varepsilon,\nu)}$ instance by checking the following claims:

$$\sum_{\boldsymbol{x} \in \{0,1\}^{\log n}} \hat{w}(\boldsymbol{x}) \cdot \hat{h}(\boldsymbol{x}) = \sum_{i=1}^{\nu} \beta^{i-1} \cdot \boldsymbol{w}[\alpha_i] \ , \tag{10}$$

$$\hat{w}(\boldsymbol{r}) = \tau \ , \tag{11}$$

$$\sum_{\boldsymbol{y} \in \{0,1\}^{\log k}} \hat{b}(\boldsymbol{r}_1, \boldsymbol{y}_1) \cdot \hat{b}(\boldsymbol{r}_2, \boldsymbol{y}_2) \cdot \hat{m}(\boldsymbol{y}) = \tau \ . \tag{12}$$

Using Lemma 7.4, Eq. (12) can be generalized to arbitrary $t \geq 2$ as follows:

$$\sum_{\boldsymbol{y} \in \{0,1\}^{\log k}} \left(\prod_{i=1}^{t} \hat{b}(\boldsymbol{r}_i, \boldsymbol{y}_i) \right) \cdot \hat{m}(\boldsymbol{y}) = \tau \ . \tag{13}$$

where $\boldsymbol{r}_i = \boldsymbol{r}[(i-1) \cdot \log n/t : i \cdot \log n/t - 1]$ and $\boldsymbol{y}_i = \boldsymbol{y}[(i-1) \cdot \log k/t : i \cdot \log k/t - 1]$.

Clearly, Eq. (10) is a sumcheck claim over a domain of size n, and therefore can be reduced to a polynomial evaluation claim in $O(n)$ time using the ECSumcheck IOR.

Eq. (11) is simply the evaluation of the polynomial oracle of $\hat{w}$ at the random point $\boldsymbol{r}$, and therefore corresponds to an instance of $\mathsf{MEP}^{(\mathcal{C}', \varepsilon' + \varepsilon_b, 1)}$. Furthermore, Eq. (13) is a sumcheck claim over a domain of size k, and can be reduced to a polynomial evaluation claim in $O(k)$ time using the ECSumcheck IOR.

Simplifying Eq. (13) to fit definitions of MEP and SEP. While Eq. (13) is a sumcheck claim over a domain of size k, the SEP instance expects to receive a polynomial oracle for each $\hat{b}(\boldsymbol{r}_i, \boldsymbol{Y})$, as opposed to a polynomial oracle for $\hat{b}(\boldsymbol{X}, \boldsymbol{Y})$ which it partially evaluates at $\boldsymbol{r}_i$ to obtain the desired t many polynomial oracles. This is a minor technical problem that can be resolved as follows: $\mathbf{P}$ additionally sends the polynomial oracles of $\hat{e}_i : \mathbb{F}^{\log k} \to \mathbb{F}$ such that $\hat{e}_i(\boldsymbol{Y}) := \hat{b}(\boldsymbol{r}_i, \boldsymbol{Y}_i)$ for all $i \in [t]$ to the verifier, and proves to the verifier of the following t many claims:

$$\forall i \in [t], \boldsymbol{y} \in \{0,1\}^{\log k} : \ \ \hat{e}_i(\boldsymbol{y}) = \hat{b}(\boldsymbol{r}_i, \boldsymbol{y}_i) \ . \tag{14}$$

Both sides are multilinear polynomials in $\boldsymbol{y}$, and therefore must be equal as polynomials. Therefore, the verifier can send a random challenge $\boldsymbol{s} \in \mathbb{F}^k$ to the prover, and the prover can simply convince the verifier of the following claim:

$$\forall i \in [t] : \ \ \hat{e}_i(\boldsymbol{s}) = \hat{b}(\boldsymbol{r}_i, \boldsymbol{s}_i) \ . \tag{15}$$

This incurs only an additional soundness loss of $\log k / |\mathbb{F}|$ by the Schwartz-Zippel lemma. These correspond to only $2t$-many MEP instances, since the prover can send claimed sums τ_i for each $i \in [t]$ and prove that $\hat{e}_i(\boldsymbol{s}) = \tau_i$ and $\hat{b}(\boldsymbol{r}_i, \boldsymbol{s}_i) = \tau_i$ for all $i \in [t]$.

Finally, once the verifier is convinced of the correctness of all $\hat{e}_i$, the only additional check that needs to be made is the following:

$$\sum_{\boldsymbol{y} \in \{0,1\}^{\log k}} \left(\prod_{i=1}^{t} \hat{e}_i(\boldsymbol{y}) \right) \cdot \hat{m}(\boldsymbol{y}) = \tau \ , \tag{16}$$

which is clearly an instance of SEP over a domain of size k.

Realizing polynomial oracles for $\hat{w}$ and $\hat{b}$. Recall from Remark 6.16 that some care needs to be taken when realizing a polynomial oracle when the size of the polynomial differs from the message length of the underlying code. This is the case for $\hat{b}$ and $\hat{w}$ with respect to $\mathcal{C}'$. We showcase how to apply Remark 6.16 for $\hat{b}$; the case for $\hat{w}$ is similar. If $(nk)^{1/t} \leq k$, then we can simply pad $\boldsymbol{b}$ with zeros to increase the length to k. If $(nk)^{1/t} > k$ (as must be the case if $t \leq 2$), then we break $\boldsymbol{b}$ into $\eta := (nk)^{1/t}/k$ many blocks. For all $j \in [\eta]$, we define $\hat{b}_j : \mathbb{F}^{\log k} \to \mathbb{F}$ as the unique multilinear polynomial such that $\hat{b}_j(\langle x \rangle) = \boldsymbol{b}[(j-1) \cdot k + x]$. Now consider $\hat{b}(\boldsymbol{x}, \boldsymbol{y})$ where $\boldsymbol{x} \in \mathbb{F}^{\log \eta}$ and $\boldsymbol{y} \in \mathbb{F}^{\log k}$. We can rewrite this as

$$\hat{b}(\boldsymbol{x}, \boldsymbol{y}) = \sum_{j=1}^{\eta} \hat{b}_j(\boldsymbol{y}) \cdot \mathrm{eq}(\boldsymbol{x}, \langle j-1 \rangle) \ .$$

Clearly, both sides of the equation are equal over $\{0,1\}^{(\log\eta+\log k)}$, and both sides are multilinear polynomials. Therefore, the two polynomials are equal over the entire domain of $\mathbb{F}^{(\log\eta+\log k)}$. This shows that a polynomial oracle for $\hat{b}$ can be simulated by polynomial oracles for $\hat{b}_1,\ldots,\hat{b}_\eta$, which can be realized by vector oracles to the encodings of the vectors $\boldsymbol{b}_1,\ldots,\boldsymbol{b}_\eta$ under $\mathcal{C}'$, where $\boldsymbol{b}_j = \boldsymbol{b}[(j-1)k : jk-1]$.

Final IOR. In summary, the IOR described in Fig. 8 reduces the given instance of the CIP to two instances of SEP, two instances of IMEP, and one instance of MEP:

1. one SEP instance corresponding to the consistency check between the polynomial oracle to $\hat{w}$ and the vector oracle to $\boldsymbol{w}$ described in Eq. (10) (referred to as SEP_1 in Fig. 8),
2. one IMEP instance corresponding to the random evaluation of $\hat{w}$ at $\boldsymbol{r}$ described in Eq. (11) (referred to as IMEP_1 in Fig. 8),
3. one MEP instance corresponding to the evaluations of $\hat{e}_i$ at $\boldsymbol{s}$ described in Eq. (15) (referred to as MEP in Fig. 8),
4. one IMEP instance corresponding to the evaluations of $\hat{b}$ at $(\boldsymbol{r}_i, \boldsymbol{s}_i)$ described in Eq. (15) (referred to as IMEP_2 in Fig. 8),
5. one SEP instance corresponding to the equation in Eq. (16) (referred to as SEP_2 in Fig. 8).

Remark 7.2 also applies here. In particular, the first SEP and IMEP instances share instance oracles and witness, and the MEP and second SEP instances also share instance oracles and witness.

Indexer input: Index of CIP, $(\mathcal{C}, \mathcal{C}', \varepsilon', \nu)$ such that $\mathcal{C} = \mathsf{TC}[\mathcal{B}, t]$.
Indexer output: Output index of FT as defined in Lemma 7.5. Additionally,

1. Define $\boldsymbol{b} := (\boldsymbol{G}_{\mathcal{B}}[0] \,\|\, \ldots \,\|\, \boldsymbol{G}_{\mathcal{B}}[n-1])$.
2. Compute encoding of $\boldsymbol{b}$ under $\mathsf{IC}[\mathcal{C}', (nk)^{1/t}/k]$: $\boldsymbol{b}'' \leftarrow \mathsf{Enc}_{\mathsf{IC}[\mathcal{C}',(nk)^{1/t}/k]}(\boldsymbol{b})$.
3. Output $\iota := (\varepsilon', \nu)$ and $\vec{\mathbb{I}} := \boldsymbol{b}''$.

Input instance: $\mathbb{x} := ((\sigma_1,\ldots,\sigma_\nu),(\alpha_1,\ldots,\alpha_\nu))$,
Input instance oracles: $\vec{\mathbb{y}} := (\boldsymbol{w}, \boldsymbol{w}')$, purported codewords of $\mathcal{C}$ and $\mathcal{C}'$ respectively.
Input witness: $\mathbb{w} := \boldsymbol{m}$ such that $\boldsymbol{G}_{\mathcal{C}}[\alpha_i] \cdot \boldsymbol{m} = \sigma_i$ for all $i \in [\nu]$.

$\langle \mathbf{P}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}), \mathbf{V}^{\vec{\mathbb{I}},\vec{\mathbb{y}}}(\iota, \mathbb{x})\rangle$:

1. $\mathbf{P}$ computes encoding of $\boldsymbol{w}$ under $\mathsf{IC}[\mathcal{C}', n/k]$: $\boldsymbol{w}'' \leftarrow \mathsf{Enc}_{\mathsf{IC}[\mathcal{C}',n/k]}(\boldsymbol{w})$, and sends $[\![\boldsymbol{w}'']\!]$ to $\mathbf{V}$.
2. For each $i \in [n/k]$: define $\boldsymbol{w}_i \in \mathbb{F}^k$ such that $\boldsymbol{w}_i := \boldsymbol{w}[(i-1)k : ik-1]$.
3. For each $i \in [n/k]$: define $\boldsymbol{w}''_i \in \mathbb{F}^k$ such that $\boldsymbol{w}''_i := \boldsymbol{w}''[(i-1)n' : in'-1]$.
4. $\mathbf{V}$ samples $\beta \xleftarrow{\$} \mathbb{F}$ and sends β to $\mathbf{P}$.
5. $\mathbf{V}$ samples $\boldsymbol{r} \xleftarrow{\$} \mathbb{F}^{\log n}$ and sends $\boldsymbol{r}$ to $\mathbf{P}$.
6. $\mathbf{P}$ computes $\tau \leftarrow \hat{w}(\boldsymbol{r})$ and sends τ to $\mathbf{V}$.
7. For each $i \in [t]$: Define $\boldsymbol{e}_i \in \mathbb{F}^k$ such that $\boldsymbol{e}_i[\mathsf{int}(\boldsymbol{y})] := \hat{b}(\boldsymbol{r}_i, \boldsymbol{y}_i)$ for all $\boldsymbol{y} \in \{0,1\}^{\log k}$.
8. For each $i \in [t]$: $\mathbf{P}$ computes $\boldsymbol{e}'_i \leftarrow \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{e}_i)$ and sends $[\![\boldsymbol{e}'_i]\!]$ to $\mathbf{V}$.
9. $\mathbf{V}$ samples $\boldsymbol{s} \xleftarrow{\$} \mathbb{F}^{\log k}$ and sends $\boldsymbol{s}$ to $\mathbf{P}$.
10. For each $i \in [t]$: $\mathbf{P}$ computes $\tau_i \leftarrow \hat{b}(\boldsymbol{r}_i, \boldsymbol{s}_i)$ and sends τ_i to $\mathbf{V}$.

11. Define the sumcheck combination polynomial f_1 for the SEP_1 instance:

$$f_1(\boldsymbol{x}_2, Z_1, \ldots, Z_{\frac{n}{k}}) := \sum_{\boldsymbol{x}_1 \in \{0,1\}^{\log \frac{n}{k}}} \left(\sum_{j=1}^{\nu} \beta^{j-1} \mathrm{eq}(\langle \alpha_j \rangle, (\boldsymbol{x}_1 \parallel \boldsymbol{x}_2)) \right) \left(\sum_{j=1}^{\frac{n}{k}} \mathrm{eq}(\langle j-1 \rangle, \boldsymbol{x}_1) Z_j \right) \ .$$

12. Define the sumcheck combination polynomial f_2 for the SEP_2 instance:

$$f_2(\boldsymbol{y}, Z_1, \ldots, Z_{t+1}) := \prod_{i=1}^{t+1} Z_i \ .$$

13. Define

$$\begin{aligned}
&\mathbb{x}_{\mathsf{SEP}_1} := \left(f_1, \sum_{i=1}^{\nu} \beta^{i-1} \cdot \sigma_i \right), && \mathbb{x}_{\mathsf{IMEP}_1} := (\boldsymbol{r}, \tau), && \mathbb{x}_{\mathsf{MEP}} := ((\boldsymbol{s}, \tau_1) \ldots, (\boldsymbol{s}, \tau_t)), \\
&\vec{\mathbb{y}}_{\mathsf{SEP}_1} := (\boldsymbol{w}''_1, \ldots, \boldsymbol{w}''_{\frac{n}{k}}), && \vec{\mathbb{y}}_{\mathsf{IMEP}_1} := (\boldsymbol{w}''), && \vec{\mathbb{y}}_{\mathsf{MEP}} := (\boldsymbol{e}'_1, \ldots, \boldsymbol{e}'_t), \\
&\mathbb{w}_{\mathsf{SEP}_1} := (\boldsymbol{w}_1, \ldots, \boldsymbol{w}_{\frac{n}{k}}), && \mathbb{w}_{\mathsf{IMEP}_1} := (\boldsymbol{w}), && \mathbb{w}_{\mathsf{MEP}} := (\boldsymbol{e}_1, \ldots, \boldsymbol{e}_t) \ ,
\end{aligned}$$

$$\begin{aligned}
&\mathbb{x}_{\mathsf{IMEP}_2} := ((\boldsymbol{r}_1 \parallel \boldsymbol{s}_1, \tau_1), \ldots, (\boldsymbol{r}_t \parallel \boldsymbol{s}_t, \tau_t)), && \mathbb{x}_{\mathsf{SEP}_2} := (f_2, \tau), \\
&\vec{\mathbb{y}}_{\mathsf{IMEP}_2} := (\boldsymbol{b}'', \ldots, \boldsymbol{b}''), && \vec{\mathbb{y}}_{\mathsf{SEP}_2} := (\boldsymbol{e}'_1, \ldots, \boldsymbol{e}'_t, \boldsymbol{w}'), \\
&\mathbb{w}_{\mathsf{IMEP}_2} := (\boldsymbol{b}, \ldots, \boldsymbol{b}), && \mathbb{w}_{\mathsf{SEP}_2} := (\boldsymbol{e}_1, \ldots, \boldsymbol{e}_t, \boldsymbol{m}) \ .
\end{aligned}$$

14. $\mathbf{P}$ outputs the appropriate tuple of instances, instance oracles, and witnesses, and $\mathbf{V}$ outputs the appropriate tuple of instances and instance oracles.

Figure 8: Inner product IOR from $\mathsf{CIP}^{(\mathcal{C}, \mathcal{C}', \varepsilon', \nu)}$ to FT.

Lemma 7.5. *Define $\mu := n + (nk)^{1/t}$. There exists an IOR from CIP to the "final tensor" relation:*

$$\mathsf{FT} := \begin{pmatrix} & \mathsf{SEP}^{(\mathcal{C}', \varepsilon' + \varepsilon_b, n/k)} \\ \times & \mathsf{IMEP}^{(\mathcal{C}', \varepsilon' + \varepsilon_b, 1, n/k)} \\ \times & \mathsf{MEP}^{(\mathcal{C}', \varepsilon' + \varepsilon_b, t)} \\ \times & \mathsf{IMEP}^{(\mathcal{C}', \varepsilon' + \varepsilon_b, t, (nk)^{1/t})} \\ \times & \mathsf{SEP}^{(\mathcal{C}', \varepsilon' + \varepsilon_b, t+1)} \end{pmatrix}$$

with the following efficiency parameters:

rounds	*message size*	*queries*	*oracle size*	*prover time*	*verifier time*
3	$O(1)$	0	$O(\mu)$	$O((\mu/k) \cdot T_{\mathcal{C}'})$	$O(\log n)$

Furthermore, there exists a vertical-predicate ϕ such that the IOR is $(\vec{a} := (1,1,1), \phi, \theta_{\mathsf{true}})$-RBRTE.

Proof. The efficiency parameters follow immediately from the IOR is detailed in Figure 8.

Let x_i, y_i be the message sent by $\mathbf{P}$ and the random challenges sent by $\mathbf{V}$ respectively in round i. We define the vertical predicate $\phi = (\phi_1, \phi_2, \top)$ where $\top$ is a function that always outputs 1.

First vertical predicate: . $\phi_1(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, x_1 := \boldsymbol{w}'', y_1 := (\beta, \boldsymbol{r})) = 0$ if and only if one of the following two events occur:

1. There exists $\boldsymbol{m}_{\mathsf{lucky}} \in \mathbb{F}^k$ such that $\boldsymbol{G}_{\mathcal{C}}[\alpha_i] \cdot \boldsymbol{m}_{\mathsf{lucky}} \neq \boldsymbol{w}[\alpha_i]$ for some $i \in [\nu]$, and yet
   - $\sum_{i=1}^{\nu} \beta^{i-1} \cdot \boldsymbol{w}[\alpha_i] = \left(\sum_{i=1}^{\nu} \beta^{i-1} \cdot \boldsymbol{G}_{\mathcal{C}}[\alpha_i]\right) \cdot \boldsymbol{m}_{\mathsf{lucky}}$, and
   - $\delta(\mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m}_{\mathsf{lucky}}), \boldsymbol{w}') \leq \varepsilon' + \varepsilon_b$.

   This event corresponds to the malicious prover getting lucky with the choice of β.

2. For a word $\boldsymbol{w}_{\mathsf{lucky}} \neq \boldsymbol{w}$, we have $\delta(\mathsf{Enc}_{\mathcal{C}'}((\boldsymbol{w}_{\mathsf{lucky}})_i), \boldsymbol{w}''_i) \leq \varepsilon' + \varepsilon_b$ for all $i \in [n/k]$, and yet $\hat{w}_{\mathsf{lucky}}(\boldsymbol{r}) = \hat{w}(\boldsymbol{r})$ where $\hat{w}_{\mathsf{lucky}}$ is the multilinear extension of $\boldsymbol{w}_{\mathsf{lucky}}$. This event corresponds to the malicious prover getting lucky with the choice of $\boldsymbol{r}$.

The first event occurs with probability at most $\nu/|\mathbb{F}|$ (because there is a unique $\boldsymbol{m}_{\mathsf{lucky}}$ that satisfies $\delta(\mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m}_{\mathsf{lucky}}), \boldsymbol{w}') \leq \varepsilon' + \varepsilon_b$), and the second event occurs with probability at most $\log n/|\mathbb{F}|$, both following from the Schwartz-Zippel lemma. Therefore, by union bound,

$$\Pr_{y_1 \xleftarrow{\$} \mathbb{F}^{1+\log n}} (\phi_1(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, x_1, y_1) = 0) \leq \mathrm{negl}(\lambda) \ .$$

Second vertical predicate. $\phi_2(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, x_1, y_1, x_2 := (\tau, \boldsymbol{e}'_1, \ldots, \boldsymbol{e}'_t), y_2 := \boldsymbol{s}) = 0$ if and only if for all $i \in [t]$, we have $\delta(\mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{e}_{\mathsf{lucky},i}), \boldsymbol{e}'_i) \leq \varepsilon' + \varepsilon_b$, and yet $\hat{e}_i(\boldsymbol{s}) = \hat{e}_{\mathsf{lucky},i}(\boldsymbol{s})$ where $\hat{e}_{\mathsf{lucky},i}$ is the multilinear extension of $\boldsymbol{e}_{\mathsf{lucky},i}$. By the Schwartz-Zippel lemma, this event occurs with probability at most $\log k/|\mathbb{F}|$, and therefore

$$\Pr_{y_2 \xleftarrow{\$} \mathbb{F}^{\log k}} (\phi_2(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, x_1, y_1, x_2, y_2) = 0) \leq \mathrm{negl}(\lambda) \ .$$

Tree extractor. Given a leaf of a $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$-constrained transcript tree, the extractor simply considers $\mathbb{w}_{\mathsf{SEP}_2} := (\boldsymbol{e}_1, \ldots, \boldsymbol{e}_t, \boldsymbol{m})$, and outputs $\boldsymbol{m}$. To show correctness, we must show that $\boldsymbol{G}_{\mathcal{C}}[\alpha_i] \cdot \boldsymbol{m} = \boldsymbol{w}[\alpha_i]$ for all $i \in [\nu]$. Since $(\mathbb{i}_{\mathsf{SEP}_2}, \mathbb{x}_{\mathsf{SEP}_2}, \vec{\mathbb{y}}_{\mathsf{SEP}_2}, \mathbb{w}_{\mathsf{SEP}_2})$ is a valid instance of the SEP relation, we have

$$\tau = \sum_{\boldsymbol{y} \in \{0,1\}^{\log k}} f_2(\boldsymbol{y}, \hat{e}_1(\boldsymbol{y}), \ldots, \hat{e}_t(\boldsymbol{y}), \hat{m}(\boldsymbol{y})) = \hat{m}(\boldsymbol{y}) \cdot \prod_{i=1}^{t} \hat{e}_i(\boldsymbol{y}) = \hat{m}(\boldsymbol{y}) \cdot \prod_{i=1}^{t} \hat{b}(\boldsymbol{r}_i, \boldsymbol{y}_i)$$

where the last equality holds because ϕ_2 outputs 1 in the given transcript, and the MEP and IMEP_2 instances returned by the IOR are valid. Furthermore, we have $\hat{w}(\boldsymbol{r}) = \tau$ by the validity of the IMEP_1 instance returned by the IOR. Together, we get that $\hat{w}(\boldsymbol{r}) = \sum_{\boldsymbol{y} \in \{0,1\}^{\log k}} \left(\prod_{i=1}^{t} \hat{b}(\boldsymbol{r}_i, \boldsymbol{y}_i)\right) \cdot \hat{m}(\boldsymbol{y})$. Since ϕ_1 outputs 1 in the given transcript, this means that $\hat{w}(\boldsymbol{x}) = \sum_{\boldsymbol{y} \in \{0,1\}^{\log k}} \left(\prod_{i=1}^{t} \hat{b}(\boldsymbol{x}_i, \boldsymbol{y}_i)\right) \cdot \hat{m}(\boldsymbol{y})$ for all $\boldsymbol{x} \in \{0,1\}^{\log n}$. Next, due to the validity of the SEP_1 instance returned by the IOR, we have

$$\sum_{i=1}^{\nu} \beta^{i-1} \cdot \sigma_i = \sum_{\boldsymbol{x}_2 \in \{0,1\}^{\log k}} f_1(\boldsymbol{x}_2, \hat{w}_1(\boldsymbol{x}_2), \ldots, \hat{w}_{n/k}(\boldsymbol{x}_2))$$

$$
\begin{aligned}
&= \sum_{\substack{\boldsymbol{x}_2 \in \{0,1\}^{\log k} \\ \boldsymbol{x}_1 \in \{0,1\}^{\log \frac{n}{k}}}} \left(\sum_{j=1}^{\nu} \beta^{j-1} \cdot \mathrm{eq}(\langle \alpha_j \rangle, (\boldsymbol{x}_1 \parallel \boldsymbol{x}_2)) \right) \cdot \left(\sum_{j=1}^{\frac{n}{k}} \mathrm{eq}(\langle j-1 \rangle, \boldsymbol{x}_1) \cdot \hat{w}_j(\boldsymbol{x}_2) \right) \\
&= \sum_{\boldsymbol{x} \in \{0,1\}^{\log n}} \left(\sum_{j=1}^{\nu} \beta^{j-1} \cdot \mathrm{eq}(\langle \alpha_j \rangle, \boldsymbol{x}) \right) \cdot \hat{w}(\boldsymbol{x}) \\
&= \sum_{\boldsymbol{x} \in \{0,1\}^{\log n}} \left(\sum_{j=1}^{\nu} \beta^{j-1} \cdot \mathrm{eq}(\langle \alpha_j \rangle, \boldsymbol{x}) \right) \cdot \sum_{\boldsymbol{y} \in \{0,1\}^{\log k}} \left(\prod_{i=1}^{t} \hat{b}(\boldsymbol{x}_i, \boldsymbol{y}_i) \right) \cdot \hat{m}(\boldsymbol{y}) \\
&= \left(\sum_{i=1}^{\nu} \beta^{i-1} \boldsymbol{G}_{\mathcal{C}}[\alpha_i] \right) \cdot \boldsymbol{m}
\end{aligned}
$$

Again, using the fact that ϕ_1 outputs 1 in the given transcript, we get that $\boldsymbol{G}_{\mathcal{C}}[\alpha_i] \cdot \boldsymbol{m} = \boldsymbol{w}[\alpha_i]$ for all $i \in [\nu]$ as desired. □

7.3 Final tensor codeswitch IOR

Lemma 7.6. *Consider codes $\mathcal{C}, \mathcal{C}'$ with constant rate and distance, such that $\mathcal{C} = \mathsf{TC}[\mathcal{B}, t]$, where $t \geq 2$ is a constant. There exists an RBRTE IOR* CodeSwitch *from* $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ *to* $\mathsf{MEP}^{(\mathcal{C}',\varepsilon',1)}$ *where $\varepsilon \in (\delta_{\mathcal{C}}/8, \delta_{\mathcal{C}}/3 - 2\varepsilon_b)$ and $\varepsilon' \in (\delta_{\mathcal{C}'}/8, \delta_{\mathcal{C}'}/3 - \varepsilon_b)$ with the following efficiency parameters:*

rounds	*message size*	*queries*	*oracle size*	*prover time*	*verifier time*
$O(\log n)$	$O\left(\log n + \frac{-\lambda}{\log(1-\varepsilon)}\right)$	$O(\lambda)$	$O(n)$	$O(T_{\mathcal{C}'})$	$O(\lambda \log n)$

Proof. We describe the entire CodeSwitch IOR for tensor codes:

1. Reduce the given $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ instance to an $\mathsf{CIP}^{(\mathcal{C},\mathcal{C}',\varepsilon',\nu)}$ instance and an $\mathsf{MEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,1)}$ instance via Figure 7.
2. Reduce the $\mathsf{CIP}^{(\mathcal{C},\mathcal{C}',\varepsilon,\nu)}$ instance to an FT instance via Figure 8.
3. Reduce the $\mathsf{IMEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,1,n/k)}$ and $\mathsf{IMEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,t,(nk)^{1/t})}$ instances of FT to $\mathsf{MEP}^{(\mathcal{C}',\varepsilon',n/k)}$ and $\mathsf{MEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,t\cdot(nk)^{1/t})}$ instances using the DeInterleave IOR.
4. Reduce the $\mathsf{SEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,n/k)}$ and $\mathsf{SEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,t+1)}$ instances of FT to $\mathsf{HEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,n/k)}$ and $\mathsf{HEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,t+1)}$ instances using ECSumcheck.
5. The resulting MEP and HEP instances from the previous steps can be interpreted as one $\mathsf{MEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,c)}$ instance where $c = 2n/k + t \cdot (nk)^{1/t} + t + 2$. Reduce this instance to $\mathsf{HEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,c)}$ using Homogenize.
6. Finally, reduce the $\mathsf{HEP}^{(\mathcal{C}',\varepsilon'+\varepsilon_b,c)}$ instance to an $\mathsf{MEP}^{(\mathcal{C}',\varepsilon',1)}$ instance using Batch.

Given that $n/k, (nk)^{1/t}, t$ are all constants, the prover and verifier costs follow from the efficiency parameters of the IORs used in the above steps. Furthermore, since the IORs used in the above steps are all RBRTE, by our composition lemma (Lemma 4.7), the final IOR is also RBRTE[18]. □

[18] We also invoked parallel composition here implicitly (see Remark C.1) but we avoid a detailed discussion of the same as it is technically straightforward.

8 Compiler for constructing an IOR to MEP given an MLIOP

We formally define MultiLinear-Interactive-Oracle-Proofs (MLIOPs) and demonstrate a generic compiler that takes as input an MLIOP for an indexed polynomial oracle relation (IPOR, defined in Definition 8.1) $\mathcal{R}$, and outputs an interactive oracle reduction (IOR) from the equivalent indexed oracle pair relation $\mathcal{R}_{\mathsf{IOR}}$ to an instance of MEP.

Recall that the oracle of a multilinear polynomial $f \in \mathbb{F}[x_1, \ldots, x_n]$ is an object that given an input $\boldsymbol{x} \in \mathbb{F}^n$ returns the value $f(\boldsymbol{x})$. We use $[\![f(\boldsymbol{X})]\!]$ to denote the polynomial oracle of f. Before defining MLIOPs, we first define the notion of an indexed *polynomial oracle* relations.

Definition 8.1 (Indexed polynomial oracle relation (IPOR))**.** *An indexed polynomial oracle relation $\mathcal{R}$ is a relation consisting of tuples of the form $(\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}, \mathbb{w}_{\mathsf{M}})$, such that $\vec{\mathbb{y}}_{\mathsf{M}} = \{[\![f_1]\!], \ldots, [\![f_\ell]\!]\}$ is a set of polynomial oracles of multilinear polynomials $f_1, \ldots, f_\ell$.*

Definition 8.2 (Proximity version of IPOR)**.** *Let $\mathcal{R}$ be an IPOR consisting of tuples of the form $(\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}, \mathbb{w}_{\mathsf{M}})$, such that $\ell := |\vec{\mathbb{y}}|$, and all polynomial oracles in $\vec{\mathbb{y}}$ are multilinear polynomials of size k The proximity version of $\mathcal{R}$, denoted $\mathsf{Prox}(\mathcal{R}) = (\mathcal{R}_{\mathsf{IOR}}, \tilde{\mathcal{R}}_{\mathsf{IOR}})$ is an indexed oracle pair relation consisting of tuples of the form $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w})$, such that*

- *$\mathbb{i} = (\mathbb{i}_{\mathsf{M}}, \mathcal{C}, \varepsilon, \ell)$, where $\mathcal{C}$ is a linear code with parameters $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$, $\varepsilon \in (\delta_{\mathcal{C}}/8, \delta_{\mathcal{C}}/3 - 2\varepsilon_b)$ is the proximity parameter, and ℓ is the number of polynomials in $\vec{\mathbb{y}}_{\mathsf{M}}$,*
- *$\mathbb{x} = \mathbb{x}_{\mathsf{M}}$,*
- *$\vec{\mathbb{y}} = [\![\boldsymbol{w}_1]\!], \ldots, [\![\boldsymbol{w}_\ell]\!]$ such that $\boldsymbol{w}_i \in \mathbb{F}^n$ for each $i \in [\ell]$, and*
- *$\mathbb{w} = (\mathbb{w}_{\mathsf{M}}, \boldsymbol{m}_1, \ldots, \boldsymbol{m}_\ell)$ where $\boldsymbol{m}_i$ is the vector of evaluations over the boolean hypercube of the polynomial $\hat{m}_i$: the i-th multilinear polynomial in $\vec{\mathbb{y}}_{\mathsf{M}}$.*

In addition, the following conditions hold for $\mathcal{R}_{\mathsf{IOR}}$ and $\tilde{\mathcal{R}}_{\mathsf{IOR}}$ respectively:

$$\begin{aligned} \mathcal{R}_{\mathsf{IOR}}: &\quad \forall i \in [\ell]: \quad \boldsymbol{w}_i = \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m}_i) \ , \\ \tilde{\mathcal{R}}_{\mathsf{IOR}}: &\quad \forall i \in [\ell]: \quad \delta(\mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m}_i), \boldsymbol{w}_i) \leq \varepsilon \ , \end{aligned}$$

Definition 8.3 (MLIOP (similar to Blaze [BCFRRZ25]))**.** *Consider an indexed polynomial oracle relation $\mathcal{R}$ consisting of tuples of the form $(\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}, \mathbb{w}_{\mathsf{M}})$. An MLIOP M is a public-coin interactive protocol between a prover $\mathbf{P}_{\mathsf{M}}$, a verifier $\mathbf{V}_{\mathsf{M}}$, and a trusted indexer $\mathbf{I}_{\mathsf{M}}$ with the following phases:*

- ***Preprocessing phase:*** *$\mathbf{I}_{\mathsf{M}}$ takes as input $\mathbb{i}_{\mathsf{M}}$ and computes the output index $\mathbb{i}_{\mathsf{M}}$, short index ι_{M}, and index oracles $\vec{\mathbb{I}}_{\mathsf{M}} := [\hat{f}_j]_{j=1}^{|\vec{\mathbb{I}}_{\mathsf{M}}|}$, which exclusively consists of polynomial oracles of multilinear polynomials. Furthermore, $\mathbf{I}_{\mathsf{M}}$ sends $\mathbb{i}_{\mathsf{M}}$ to $\mathbf{P}_{\mathsf{M}}$, and sends ι_{M} and $\vec{\mathbb{I}}_{\mathsf{M}}$ to $\mathbf{V}_{\mathsf{M}}$.*
- ***Interactive phase:*** *$\mathbf{P}_{\mathsf{M}}$ and $\mathbf{V}_{\mathsf{M}}$ engage in d rounds of interaction. In the i-th round, $\mathbf{P}_{\mathsf{M}}$ sends a message α_i and polynomial oracles $\vec{\Pi}_i := [\hat{f}_{i,j}]_{j=1}^{|\vec{\Pi}_i|}$ to $\mathbf{V}_{\mathsf{M}}$. $\mathbf{V}_{\mathsf{M}}$ reads the transcript so far and either rejects, or continues and sends random challenges $\vec{\rho}_i$ to $\mathbf{P}_{\mathsf{M}}$.*
- ***Query phase:*** *$\mathbf{V}_{\mathsf{M}}$ queries the polynomial oracles $p_1, \ldots, p_{\mathsf{q}}$ at evaluation points $\boldsymbol{z}_1, \ldots, \boldsymbol{z}_{\mathsf{q}}$, and obtains the evaluations $y_1, \ldots, y_{\mathsf{q}}$. Based on these values, $\mathbf{V}_{\mathsf{M}}$ decides to accept or reject the proof.*

The MLIOP is complete *if for any $(\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}, \mathbb{w}_{\mathsf{M}}) \in \mathcal{R}$, $\mathbf{V}_{\mathsf{M}}$ accepts with probability 1.*

Definition 8.4 (Round-by-round soundness)**.** *Consider a d-round MLIOP M for indexed oracle relation $\mathcal{R}$, and let $\mathsf{tr}_i = ((\alpha_1, \vec{\Pi}_1), \vec{\rho}_1, \ldots, (\alpha_i, \vec{\Pi}_i))$ be a partial transcript of the interactive phase of M between*

a malicious prover $\widetilde{\mathbf{P}}_{\mathsf{M}}$ *and verifier* $\mathbf{V}_{\mathsf{M}}$. M *is* round-by-round sound *if there exists a state function* State *such that if* $\mathsf{State}(\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}, \mathsf{tr}_i) = 1$ *for any fixed* $i \in [\mathsf{d}]$, $(\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}) \notin \mathcal{L}(\mathcal{R})$, *and partial transcript* tr_i, *then*

$$\Pr_{\vec{\rho}_i}[\mathsf{State}(\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}, (\mathsf{tr}_i, \vec{\rho}_i)) = 0] \leq \mathrm{negl}(\lambda) \ ,$$

where λ *is a security parameter. Additionally,* State *outputs 1 on the empty transcript, and if* State *outputs 1 on the full transcript, it must be the case that* $\mathbf{V}_{\mathsf{M}}$ *rejects.*[19]

Definition 8.5 (Trivial extractor)**.** *Consider an MLIOP* M *for indexed oracle relation* $\mathcal{R}$. M *has a trivial extractor if there exists a polynomial-time algorithm* $\mathbf{E}_{\mathsf{M}}$ *which only takes as input* $(\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}})$ *(and not the transcript of the interaction between* $\mathbf{P}_{\mathsf{M}}$ *and* $\mathbf{V}_{\mathsf{M}}$*), and outputs* $\mathbb{w}$ *such that if* $(\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}) \in \mathcal{L}(\mathcal{R})$, *then* $(\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}, \mathbb{w}_{\mathsf{M}}) \in \mathcal{R}$ *with probability 1.*

The MLIOPs that we consider in this work all have a trivial extractor, because $\mathbb{w}_{\mathsf{M}}$ only contains the polynomials underlying $\vec{\mathbb{y}}_{\mathsf{M}}$. Since $\mathbf{E}_{\mathsf{M}}$ can run in polynomial time, it can simply query all multilinear polynomials in $\vec{\mathbb{y}}_{\mathsf{M}}$ over their respective boolean hypercubes to recover the entire polynomial.

Lemma 8.6 (MLIOP to IOR compiler)**.** *Consider a* d*-round MLIOP* M *for relation* $\mathcal{R}$ *that is complete, round-by-round sound, and has a trivial extractor. Then there exists an IOR from* $\mathsf{Prox}(\mathcal{R}) = (\mathcal{R}_{\mathsf{IOR}}, \tilde{\mathcal{R}}_{\mathsf{IOR}})$ *to* $\mathsf{MEP}^{(\mathcal{C},\varepsilon,\mathsf{q})}$ *that is complete and RBRTE. Furthermore, the IOR has the following efficiency measures:*

rounds	*message size*	*queries*	*oracle size*	*prover time*	*verifier time*
d	$\lvert\iota_{\mathsf{M}}\rvert + \sum_{i=1}^{\mathsf{d}} \lvert\alpha_i\rvert$	0	$n \cdot \left(\lvert\vec{\mathbb{i}}_{\mathsf{M}}\rvert + \sum_{i=1}^{\mathsf{d}} \lvert\vec{\Pi}_i\rvert\right)$	$T(\mathbf{P}_{\mathsf{M}}) + T_{\mathcal{C}} \cdot \sum_{i=1}^{\mathsf{d}} \lvert\vec{\Pi}_i\rvert$	$T(\mathbf{V}_{\mathsf{M}})$

in addition to an indexer time of $T(\mathbf{I}_{\mathsf{M}}) + T_{\mathcal{C}} \cdot |\vec{\mathbb{i}}_{\mathsf{M}}|$.

Proof. The IOR is constructed in the natural way:

- **Indexer:** The indexer $\mathbf{I}_{\mathsf{IOR}}$ receives as input $(\mathbb{i}, \mathcal{C}, \varepsilon, \ell)$ and runs $\mathbf{I}_{\mathsf{M}}$ on input $\mathbb{i}$ to obtain $\mathbb{i}_{\mathsf{M}}, \iota_{\mathsf{M}}, \vec{\mathbb{i}}_{\mathsf{M}}$. For each polynomial oracle $[\![f_i]\!]$ in $\vec{\mathbb{i}}_{\mathsf{M}}$, $\mathbf{I}_{\mathsf{IOR}}$ computes $\boldsymbol{g}_i := \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{f}_i)$. $\mathbf{P}_{\mathsf{IOR}}$ sends $\mathbb{i}_{\mathsf{M}}, \boldsymbol{g}_1, \ldots, \boldsymbol{g}_{|\vec{\mathbb{i}}_{\mathsf{M}}|}$ to $\mathbf{P}_{\mathsf{IOR}}$, and $\iota_{\mathsf{M}}, [\![\boldsymbol{g}_1]\!], \ldots, [\![\boldsymbol{g}_{|\vec{\mathbb{i}}_{\mathsf{M}}|}]\!]$ to $\mathbf{V}_{\mathsf{IOR}}$.
- **Prover:** $\mathbf{P}_{\mathsf{IOR}}$ receives $\mathbb{i}_{\mathsf{M}}$ and $\boldsymbol{g}_1, \ldots, \boldsymbol{g}_{|\vec{\mathbb{i}}_{\mathsf{M}}|}$ from $\mathbf{I}_{\mathsf{IOR}}$, and the instance-witness tuple $(\mathbb{x}, \vec{\mathbb{y}}, \mathbb{w})$ as input. In the i-th round of the interaction, it runs $\mathbf{P}_{\mathsf{M}}$ on input $(\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}, \mathbb{w}_{\mathsf{M}})$ (which can be constructed from $\mathbb{i}_{\mathsf{M}}$ and $(\mathbb{x}, \vec{\mathbb{y}}, \mathbb{w})$), and the random challenges $\vec{\rho}_1, \ldots, \vec{\rho}_{i-1}$ to obtain α_i and $\vec{\Pi}_i$. It then computes $\boldsymbol{g}_{i,j} := \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{f}_{i,j})$ for all $j \in [|\vec{\Pi}_i|]$, and sends α_i and $[\![\boldsymbol{g}_{i,1}]\!], \ldots, [\![\boldsymbol{g}_{i,|\vec{\Pi}_i|}]\!]$ to $\mathbf{V}_{\mathsf{IOR}}$.
- **Verifier:** $\mathbf{V}_{\mathsf{IOR}}$ receives ι_{M} and $[\![\boldsymbol{g}_1]\!], \ldots, [\![\boldsymbol{g}_{|\vec{\mathbb{i}}_{\mathsf{M}}|}]\!]$ from $\mathbf{I}_{\mathsf{IOR}}$ and the instance $(\mathbb{x}, [\![\vec{\mathbb{y}}]\!])$ as input. In the i-th round of the interaction, it runs $\mathbf{V}_{\mathsf{M}}$ to obtain the random challenges $\vec{\rho}_i$. The only inputs given to $\mathbf{V}_{\mathsf{M}}$ are $(\iota_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}})$, and the messages $\alpha_1, \ldots, \alpha_i$, since the polynomial oracles are not queried in the interactive phase. If $\mathbf{V}_{\mathsf{M}}$ rejects in this round, then $\mathbf{V}_{\mathsf{IOR}}$ also rejects.

Finally, the MEP relation that the IOR reduces to handles the query phase of M. For simplicity, we assume without loss of generality that each polynomial oracle is queried exactly once.[20] That is, the number of queries is equal to the number of polynomial oracles $\mathsf{q} = \ell + |\vec{\mathbb{i}}_{\mathsf{M}}| + \sum_{i=1}^{\mathsf{d}} |\vec{\Pi}_i|$. Therefore, the MEP relation that we reduce to consists of tuples of the following form:

[19] This definition negates the output value of the State function from how it's usually defined in the literature (see [CY24, Definition 31.1.2]).

[20] If the polynomial oracle is not queried, it can be ignored, and if it is queried multiple times, we can simply duplicate the oracle and the corresponding evaluation points and values.

- Index $\mathbb{i}_{\mathsf{MEP}}$: $(\mathcal{C}, \varepsilon, \mathsf{q})$,
- Instance $\mathbb{x}_{\mathsf{MEP}}$: Evaluation points $(\boldsymbol{z}_1, \ldots, \boldsymbol{z}_{\mathsf{q}})$ and evaluation values $(y_1, \ldots, y_{\mathsf{q}})$.
- Instance oracles $\vec{\mathbb{y}}_{\mathsf{MEP}}$: $\boldsymbol{w}_1, \ldots, \boldsymbol{w}_\ell, \boldsymbol{g}_1, \ldots, \boldsymbol{g}_{|\vec{\mathbb{I}}_{\mathsf{M}}|}, \boldsymbol{g}_{1,1}, \ldots, \boldsymbol{g}_{\mathsf{d},|\vec{\Pi}_{\mathsf{d}}|}$,
- Witness $\mathbb{w}_{\mathsf{MEP}}$: $\boldsymbol{m}_1, \ldots, \boldsymbol{m}_\ell, \boldsymbol{f}_1, \ldots, \boldsymbol{f}_{|\vec{\mathbb{I}}_{\mathsf{M}}|}, \boldsymbol{f}_{1,1}, \ldots, \boldsymbol{f}_{\mathsf{d},|\vec{\Pi}_{\mathsf{d}}|}$.

Clearly, this IOR is complete, and the efficiency properties follow from that of M.
We now prove that there exists a vertical-predicate ϕ such that the above IOR is $(\vec{a} := (1, 1, \ldots, 1), \phi, \theta_{\mathsf{true}})$-RBRTE.

Vertical predicate. By definition of round-by-round soundness, there exists a state function State such that for all $i \in [\mathsf{d}]$ and $(\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}) \notin \mathcal{L}(\mathcal{R})$, we have $\Pr_{\vec{\rho}_i}[\mathsf{State}(\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}, (\mathsf{tr}_i, \vec{\rho}_i)) = 0] \leq \mathrm{negl}(\lambda)$ if $\mathsf{State}(\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}, \mathsf{tr}_i) = 1$. We can now define the vertical-predicate ϕ as follows:

$$\phi(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathsf{IOR}(\mathsf{tr}_i)) := \begin{cases} \mathsf{State}(\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}, \mathsf{tr}_i) & \text{if } (\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}) \notin \mathcal{L}(\mathcal{R}) \\ 1 & \text{if } (\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}) \in \mathcal{L}(\mathcal{R}) \end{cases}$$

where $\mathsf{IOR}(\mathsf{tr}_i)$ is the transcript for IOR produced by the compiler given the transcript tr_i of the MLIOP. Clearly, this is a valid vertical-predicate by Definition 4.3.

Tree extractor. Now, consider a leaf of a $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$-constrained transcript tree, where the witness is $\mathbb{w}_{\mathsf{MEP}} = \boldsymbol{m}'_1, \ldots, \boldsymbol{m}'_\ell, \boldsymbol{f}'_1, \ldots, \boldsymbol{f}'_{|\vec{\mathbb{I}}_{\mathsf{M}}|}, \boldsymbol{f}'_{1,1}, \ldots, \boldsymbol{f}'_{\mathsf{d},|\vec{\Pi}_{\mathsf{d}}|}$. The tree extractor $\mathbf{E}^{\mathrm{Tree}}$ simply returns $(\boldsymbol{m}'_1, \ldots, \boldsymbol{m}'_\ell)$, which we prove is a valid witness for $\tilde{\mathcal{R}}_{\mathsf{IOR}}$, which involves proving the evaluation claims and the proximity claims.

We first prove the evaluation claims. Since $\mathbb{w}_{\mathsf{MEP}}$ is a valid witness for the MEP relation, it must be the case that $\hat{m}'_i(\boldsymbol{z}_i) = y_i$ for all $i \in [\ell]$. By definition of a constrained transcript tree, $\mathbf{V}_{\mathsf{IOR}}$ must accept, and therefore $\mathbf{V}_{\mathsf{M}}$ did not reject during the interactive phase by construction of $\mathbf{V}_{\mathsf{IOR}}$. Assume for the sake of contradiction that there exists $j \in [\ell]$ such that $\hat{m}_j(\boldsymbol{z}_j) \neq y_j$. In this case, we have $(\mathbb{i}_{\mathsf{M}}, \mathbb{x}_{\mathsf{M}}, \vec{\mathbb{y}}_{\mathsf{M}}) \notin \mathcal{L}(\mathcal{R})$, and therefore the vertical predicate is equal to the state function. Since the vertical predicate outputs 1 by definition of constrained transcript trees, the state function must also output 1 on the full transcript. Therefore, $\mathbf{V}_{\mathsf{M}}$ must have rejected by definition of the state function, and therefore $\mathbf{V}_{\mathsf{IOR}}$ must also reject by our construction. However, by definition of a constrained transcript tree, $\mathbf{V}_{\mathsf{IOR}}$ must have accepted, which is a contradiction.
By the validity of $\mathbb{w}_{\mathsf{MEP}}$, it must be the case that $\delta(\mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}'_i), \boldsymbol{w}') \leq \varepsilon$ for all $i \in [\ell]$, and therefore the proximity claims also hold as desired. □

9 IOPP for multilinear evaluation with linear prover time and fast verification

In this section, we formalize FICS, our IOPP for multilinear evaluation with respect to an efficiently codeswitchable code $\mathcal{C}$ that was described in Section 2.4, and formalize the associated Halve IOR that was described in Section 2.5.

9.1 Construction

Consider a code $\mathcal{C}$ with parameters $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$ such that it is part of an efficiently codeswitchable code family $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$.

WHIR. Let RS be a Reed-Solomon code with parameters $(\mathbb{F}, n', k', \delta_{\mathsf{RS}})$, where $n' = n/\log k$ and $k' = k/\log k$. WHIR [ACFY25] is an IOP for $\mathsf{MEP}^{(\mathsf{RS},\delta_{\mathsf{RS}}/4,1)}$ with quasi-linear prover time, $O(\lambda \log n')$ verifier time and $O(\lambda \log\log n' + \log n')$ query complexity.

FICS. Our IOP for $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$, FICS, is obtained by composing the following IORs:

1. For $i \in [\log\log k]$: Halve from $\mathsf{MEP}^{(\mathcal{C}_{k/2^{i-1}},\varepsilon,1)}$ to $\mathsf{MEP}^{(\mathcal{C}_{k/2^{i}},\varepsilon,1)}$ (we show the existence and efficiency of this IOR in Section 9.2),
2. CodeSwitch from $\mathsf{MEP}^{(\mathcal{C}_{k/\log k},\varepsilon,1)}$ to $\mathsf{MEP}^{(\mathsf{RS},\delta_{\mathsf{RS}}/4,1)}$, and
3. WHIR from $\mathsf{MEP}^{(\mathsf{RS},\delta_{\mathsf{RS}}/4,1)}$ to $\mathcal{R}_{\mathsf{true}}$.

Theorem 9.1 (FICS)**.** *Let $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$ be an efficiently codeswitchable code family, and consider a code $\mathcal{C}$ from this family with parameters $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$ that has encoding time $O(T_{\mathcal{C}})$. There exists an RBRTE IOP* FICS *for $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ where $\varepsilon \in (\delta_{\mathcal{C}}/8, \delta_{\mathcal{C}}/6 - 2\varepsilon_b)$, with the following efficiency measures:*

rounds	*message size*	*queries*	*oracle size*	*prover time*	*verifier time*
$O(\log n \log\log n)$	$O((\lambda + \log n)\cdot \log\log n)$	$O(\lambda \log\log n + \log n)$	$O(n)$	$O(T_{\mathcal{C}} + n)$	$O(\lambda \log n \log\log n)$

Proof. The efficiency measures follow from the efficiency measures of the Halve IOR, the CodeSwitch IOR, and of WHIR. Furthermore, the IOR is RBRTE as a consequence of our composition lemma (see Lemma 4.7), because each component is RBRTE:

- Halve is RBRTE by Lemma 9.4. This is composed $\log\log n$ many times, and the size of each constrained-transcript-tree is at most a fixed constant c, which results in a constrained-transcript-tree of size $c^{\log\log n} = O(\log n)$. Therefore, the composition of $\log\log n$ many Halve IORs is RBRTE.
- CodeSwitch is RBRTE for efficiently codeswitchable codes.
- WHIR is RBRTE since [ACFY25] prove that WHIR is round-by-round-knowledge-sound (RBRKS), and we prove in Section D that RBRKS IORs are RBRTE. □

Remark 9.2 (IOPs for NP)**.** One can obtain an IOP for the NP-complete relation R1CS with the same efficiency measures as FICS by composing the IOR from $\mathsf{Prox}(\mathcal{R}_{\mathsf{R1CS}})$ to $\mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$ from Lemma 10.4 with FICS, where $\mathcal{C}$ can be chosen to be any codeswitchable code.

Remark 9.3 (choice of $\mathbb{F}$ and $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$)**.** Notice that we must be able to define a Reed-Solomon code RS over the field $\mathbb{F}$. This implies that the field $\mathbb{F}$ must be FFT-friendly (to allow for encoding of RS in quasi-linear time). Thus the code family $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$ must also be defined over the same field $\mathbb{F}$, but this is not a concern since we can set $\mathcal{C}$ to be the code family from Brakedown [GLSTW23], which is field-agnostic, or a code family of tensor codes of any field-agnostic linear-time encodable code.

9.2 Halve **IOR**

Halve is an IOR from $\mathsf{MEP}^{(\mathcal{C}_k,\varepsilon,1)}$ to $\mathsf{MEP}^{(\mathcal{C}_{k/2},\varepsilon,1)}$, where $\mathcal{C}_k, \mathcal{C}_{k/2}$ are codes from an efficiently codeswitchable code family $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$ with distance δ_0. For brevity we will denote $\mathcal{C}_k$ as $\mathcal{C}$ and $\mathcal{C}_{k/2}$ as $\mathcal{C}'$. Our construction is based on the following fact about multilinear polynomial evaluation: given $\boldsymbol{m} \in \mathbb{F}^k$ and an evaluation point $\boldsymbol{z} \in \mathbb{F}^{\log k}$, one can interpret $\boldsymbol{z} = (z_1, \boldsymbol{z}')$ and express $\hat{m}(\boldsymbol{z})$ as

$$\begin{aligned}\hat{m}(\boldsymbol{z}) &= \sum_{\boldsymbol{b}\in\{0,1\}^{\log k}} \mathrm{eq}(\boldsymbol{z},\boldsymbol{b}) \cdot \hat{m}(\boldsymbol{b}) \\ &= (1-z_1) \cdot \sum_{\boldsymbol{b}'\in\{0,1\}^{\log k-1}} \mathrm{eq}(\boldsymbol{z}',\boldsymbol{b}') \cdot \hat{m}(0,\boldsymbol{b}') + z_1 \cdot \sum_{\boldsymbol{b}'\in\{0,1\}^{\log k-1}} \mathrm{eq}(\boldsymbol{z}',\boldsymbol{b}') \cdot \hat{m}(1,\boldsymbol{b}') \\ &= (1-z_1)\cdot \hat{m}_L(\boldsymbol{z}') + z_1 \cdot \hat{m}_R(\boldsymbol{z}') \ ,\end{aligned}$$

where $\hat{m}_L$ and $\hat{m}_R$ are multilinear extensions of $\boldsymbol{m}_L$ and $\boldsymbol{m}_R$ respectively. The Halve IOR begins by executing a CodeSwitch IOR from $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ to $\mathsf{MEP}^{(\mathsf{IC}[\mathcal{C}',2],\varepsilon+\varepsilon_b,1)}$ where ε_b is the distance loss incurred due to the Batch IOR, and $\mathsf{IC}[\mathcal{C}',2]$ is an interleaved code with parameters $(\mathbb{F}, n, k, \delta_0/2)$. If we parse the instance and witness of $\mathsf{MEP}^{(\mathsf{IC}[\mathcal{C}',2],\varepsilon+\varepsilon_b,1)}$ as $\big(\mathbb{x} := (\boldsymbol{z}, y), \quad \vec{\mathbb{y}} := \boldsymbol{w}, \quad \mathbb{w} := \boldsymbol{m}\big)$, then we have that

$$\boldsymbol{w} = \mathsf{Enc}_{\mathsf{IC}[\mathcal{C}',2]}(\boldsymbol{m}) = \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m}_L) \,\|\, \mathsf{Enc}_{\mathcal{C}'}(\boldsymbol{m}_R) = \boldsymbol{w}_L \,\|\, \boldsymbol{w}_R \ .$$

The verifier needs to be convinced that $\hat{m}(\boldsymbol{z}) = y$ and $\delta(\boldsymbol{w}, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m})) \le \varepsilon + \varepsilon_b$. To prove the first condition, the prover sends y_L, y_R to the verifier, that are purportedly $\hat{m}_L(\boldsymbol{z}'), \hat{m}_R(\boldsymbol{z}')$ respectively. The verifier checks that $y = (1-z_1)\cdot y_L + z_1 \cdot y_R$. Now, the prover needs to convince the verifier that $\hat{m}_L(\boldsymbol{z}') = y_L$, $\hat{m}_R(\boldsymbol{z}') = y_R$ instead of $\hat{m}(\boldsymbol{z}) = y$. The second condition holds if $\delta(\boldsymbol{w}_L, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_L)) \le \varepsilon + \varepsilon_b$ and $\delta(\boldsymbol{w}_R, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_R)) \le \varepsilon + \varepsilon_b$. Thus, the prover must convince the verifier of the following claims:

- $\delta(\boldsymbol{w}_L, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_L)) \le \varepsilon + \varepsilon_b$ and $\hat{m}_L(\boldsymbol{z}') = y_L$, and
- $\delta(\boldsymbol{w}_R, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_R)) \le \varepsilon + \varepsilon_b$ and $\hat{m}_R(\boldsymbol{z}') = y_R$.

This is an instance of $\mathsf{HEP}^{(\mathcal{C}',\varepsilon+\varepsilon_b,2)}$, so we can run the Batch IOR (Section 6.3) to reduce this to an instance of $\mathsf{MEP}^{(\mathcal{C}',\varepsilon,1)}$. The full construction of Halve is discussed in Fig. 9.

Indexer input: $\mathbb{i}_{\mathsf{MEP}} = (\mathcal{C}, \varepsilon, 1)$ such that $\mathcal{C}$ is a linear code with parameters $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$ from the codeswitchable code family $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$ and $\varepsilon \in (\delta_{\mathcal{C}}/8, \delta_{\mathcal{C}}/6 - 2\varepsilon_b)$.
Indexer output: $\mathbb{i}_{\mathsf{MEP}'} = (\mathcal{C}', \varepsilon, 1)$ where $\mathcal{C}'$ is a code with parameters $(\mathbb{F}, n/2, k/2, \delta_{\mathcal{C}})$, also from the codeswitchable code family $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$.

1. Run the indexer of CodeSwitch IOR with input $\mathbb{i}_{\mathsf{MEP}}$ to obtain $\iota_{\mathsf{CodeSwitch}}$ and $\vec{\mathbb{I}}_{\mathsf{CodeSwitch}}$.
2. Define index for the intermediate HEP instance: $\mathbb{i}_{\mathsf{HEP}} := (\mathcal{C}', \varepsilon + \varepsilon_b, 2)$.
3. Run the indexer of Batch IOR with input $\mathbb{i}_{\mathsf{HEP}}$ to obtain ι_{Batch} and $\vec{\mathbb{I}}_{\mathsf{Batch}}$.
4. Return $\iota := (\iota_{\mathsf{CodeSwitch}}, \iota_{\mathsf{Batch}})$ and $\vec{\mathbb{I}} := (\vec{\mathbb{I}}_{\mathsf{CodeSwitch}}, \vec{\mathbb{I}}_{\mathsf{Batch}})$.

Input instance: $\mathbb{x} = (\boldsymbol{z}, y)$ where $\boldsymbol{z} \in \mathbb{F}^{\log k}$ and $y \in \mathbb{F}$.
Input instance oracle: $\vec{\mathbb{y}} = \boldsymbol{w} \in \mathbb{F}^n$ where $\boldsymbol{w} \in \mathbb{F}^n$ is a purported codeword of $\mathcal{C}$.
Input witness: $\mathbb{w} = \boldsymbol{m} \in \mathbb{F}^k$ such that $\delta(\mathcal{C}(\boldsymbol{m}), \boldsymbol{w}) \le \varepsilon$ and $\hat{m}(\boldsymbol{z}) = y$.

$\langle \mathbf{P}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}), \mathbf{V}^{\vec{\mathbb{I}}, \vec{\mathbb{y}}}(\iota, \mathbb{x})\rangle$:

1. $\mathbf{P}$ and $\mathbf{V}$ engage in a CodeSwitch IOR from $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ to a $\mathsf{MEP}^{(\mathsf{IC}[\mathcal{C}',2],\varepsilon+\varepsilon_b,1)}$ instance:

$$\begin{pmatrix} b_{\mathsf{MEP}_1}, \\ \mathbb{x}_{\mathsf{MEP}_1} := (\boldsymbol{z}', y'), \\ \vec{\mathbb{y}}_{\mathsf{MEP}_1} := \boldsymbol{w}', \\ \mathbb{w}_{\mathsf{MEP}_1} := \boldsymbol{m} \end{pmatrix} \;;\; \begin{pmatrix} b_{\mathsf{MEP}_1}, \\ \mathbb{x}_{\mathsf{MEP}_1} := (\boldsymbol{z}', y'), \\ \vec{\mathbb{y}}_{\mathsf{MEP}_1} := \boldsymbol{w}' \end{pmatrix} \leftarrow \mathsf{CodeSwitch}(\langle \mathbf{P}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}), \mathbf{V}^{\vec{\mathbb{I}},\vec{\mathbb{y}}}(\iota, \mathbb{x})\rangle) \ .$$

2. Define $z_0 := \boldsymbol{z}'[0]$ and $\boldsymbol{z}'' := \boldsymbol{z}'[1:]$.
3. $\mathbf{P}$ computes $y_L \leftarrow \hat{m}_L(\boldsymbol{z}'')$ and $y_R \leftarrow \hat{m}_R(\boldsymbol{z}'')$, and sends y_L, y_R to $\mathbf{V}$.
4. Set $b \leftarrow 1$.
5. $\mathbf{V}$ checks if $y = (1 - z_0) \cdot y_L + z_0 \cdot y_R$. If the check fails, $b \leftarrow 0$.
6. Define $\mathbb{i}_{\mathsf{HEP}} := (\mathcal{C}', \varepsilon + \varepsilon_b, 2)$, $\mathbb{x}_{\mathsf{HEP}} := (\boldsymbol{z}'', (y_L, y_R))$, $\vec{\mathbb{y}}_{\mathsf{HEP}} := (\boldsymbol{w}'_L, \boldsymbol{w}'_R)$, and $\mathbb{w}_{\mathsf{HEP}} := (\boldsymbol{m}_L, \boldsymbol{m}_R)$.
7. $\mathbf{P}$ and $\mathbf{V}$ engage in a Batch IOR:

$$\begin{pmatrix} b_{\mathsf{MEP}_2}, \\ \mathbb{x}_{\mathsf{MEP}_2} \\ \vec{\mathbb{y}}_{\mathsf{MEP}_2} \\ \mathbb{w}_{\mathsf{MEP}_2} \end{pmatrix} \;;\; \begin{pmatrix} b_{\mathsf{MEP}_2}, \\ \mathbb{x}_{\mathsf{MEP}_2} \\ \vec{\mathbb{y}}_{\mathsf{MEP}_2} \end{pmatrix} \leftarrow \mathsf{Batch}(\langle \mathbf{P}(\mathbb{i}_{\mathsf{HEP}}, \mathbb{x}_{\mathsf{HEP}}, \vec{\mathbb{y}}_{\mathsf{HEP}}, \mathbb{w}_{\mathsf{HEP}}), \mathbf{V}^{\vec{\mathbb{I}}_{\mathsf{HEP}},\vec{\mathbb{y}}_{\mathsf{HEP}}}(\iota_{\mathsf{HEP}}, \mathbb{x}_{\mathsf{HEP}})\rangle) \ .$$

8. Set $b \leftarrow b_{\mathsf{MEP}_1} \wedge b_{\mathsf{MEP}_2} \wedge b$.
9. $\mathbf{P}$ outputs $(b, \mathbb{x}_{\mathsf{MEP}_2}, \vec{\mathbb{y}}_{\mathsf{MEP}_2}, \mathbb{w}_{\mathsf{MEP}_2})$ and $\mathbf{V}$ outputs $(b, \mathbb{x}_{\mathsf{MEP}_2}, \vec{\mathbb{y}}_{\mathsf{MEP}_2})$.

Figure 9: Halve, an IOR from $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ to $\mathsf{MEP}^{(\mathcal{C}',\varepsilon,1)}$

Lemma 9.4. *Let $\mathcal{C}_k, \mathcal{C}_{k/2}$ be codes from an efficiently codeswitchable code family $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$ with distance δ_0. There exists an RBRTE IOR* Halve *from $\mathsf{MEP}^{(\mathcal{C}_k,\varepsilon,1)}$ to $\mathsf{MEP}^{(\mathcal{C}_{k/2},\varepsilon,1)}$ such that $\varepsilon \in (\delta_0/8, \delta_0/6 - 2\varepsilon_b)$ with the following efficiency measures:*

rounds	*message size*	*queries*	*oracle size*	*prover time*	*verifier time*
$O(\log n)$	$O\left(\log n + \frac{-\lambda}{\log(1-\delta_C)}\right)$	$O\left(\frac{-\lambda}{\log(1-\delta_C)}\right)$	$O(n)$	$O(T_{\mathcal{C}_{k/2}})$	$O\left(\lambda \log n + \frac{-\lambda}{\log(1-\delta_C)}\right)$

Proof. The efficiency measures follow from Fig. 9. Furthermore, the IOR does not incur any soundness error other than the one incurred by the CodeSwitch and Batch IORs, which are $(\vec{a}_1, \phi_1, \theta_1)$-RBRTE by Lemma 7.6 and $(\vec{a}_2, \phi_2, \theta_2)$-RBRTE by Lemma 6.12 respectively. Therefore, Halve is $(\vec{a}, \phi, \theta)$-RBRTE where $\vec{a} := (\vec{a}_1, 1, \vec{a}_2)$, $\phi = (\phi_1, \top, \phi_2)$, and $\theta := (\theta_1, \top, \theta_2)$, where $\top$ is the function that always outputs 1. □

Remark 9.5 (upper bound of ε in Halve)**.** The upper bound is slightly tighter compared to other IORs because of two reasons:

- We switch to $\mathsf{IC}[\mathcal{C}', 2]$, which has a distance of $\delta_{\mathcal{C}'}/2$, and therefore correlated agreement only applies up to $\delta_{\mathcal{C}'}/6$ (see Lemma 5.5).
- We compose the CodeSwitch and Batch IORs here, both of which induce a distance loss of ε_b. This can be avoided by inlining the CodeSwitch, and calling Batch only once on the resulting HEP instances produced by CodeSwitch but we choose not to do so in Fig. 9 for readability.

10 Accumulation scheme with linear-time prover and small query complexity

We provide an outline of the following section on FACS, our new accumulation scheme. In Section 10.1 we discuss how [BMNW25b, Theorem 4.3] implies that the following two ingredients are sufficient to construct FACS:

- An RBRTE IOR CAST from $\mathcal{R}_{\mathsf{R1CS}}$ to $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$, and
- An RBRTE IOR FOLD from $\mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$ to $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$.

We then present the constructions of CAST in Section 10.2 and FOLD in Section 10.3.

10.1 Sufficient ingredients for accumulation schemes

A non-interactive argument of knowledge (NIRK) ARG for an NP-complete relation $\mathcal{R}$ allows a prover $\mathbf{P}$ to produce a proof π that convinces a verifier $\mathbf{V}$ that it knows a witness $\mathbb{w}$ such that $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$. At a high level, an accumulation scheme for ARG is used to perform batch verification of the predicate corresponding to $\mathbf{V}$. That is, it is used to check that $\mathbf{V}(\mathbb{x}_1, \pi_1), \ldots, \mathbf{V}(\mathbb{x}_n, \pi_n)$ all accept, more efficiently than running $\mathbf{V}$ on each proof separately. In more detail, the accumulation scheme ACC consists of a prover $\mathcal{P}$, a verifier $\mathcal{V}$ and a decider $\mathcal{D}$. We initialize the empty accumulator $\mathsf{acc}_0 := \perp$. The prover $\mathcal{P}$, given a tuple $(\mathbb{x}, \pi)$ 'accumulates' it into the accumulator acc_0 to obtain acc_1 and also produces a proof pf_1 that accumulation was performed correctly. We denote this by $(\mathsf{acc}_1, \mathsf{pf}_1) \leftarrow \mathcal{P}(\mathbb{x}_1, \pi_1, \mathsf{acc}_0)$. For each $i \in [n] \setminus \{1\}$ we can continue to accumulate $(\mathsf{acc}_i, \mathsf{pf}_i) \leftarrow \mathcal{P}(\mathbb{x}_i, \pi_i, \mathsf{acc}_{i-1})$, to obtain acc_n and $[\mathsf{pf}_i]_{i\in[n]}$. An accumulation scheme guarantees that if the accumulation verifier $\mathcal{V}$ accepts the proofs that each accumulation step was performed correctly, i.e. for all $i \in [n]$: $\mathcal{V}(\mathbb{x}_i, \pi_i, \mathsf{acc}_{i-1}, \mathsf{acc}_i, \mathsf{pf}_i) = 1$, and the final accumulator is accepted by the accumulation decider $\mathcal{D}$, i.e. $\mathcal{D}(\mathsf{acc}_n) = 1$, then it must be the case that for all $i \in [n]$: $\mathbf{V}(\mathbb{x}_i, \pi_i) = 1$. Notice that the decider $\mathcal{D}$ is run only once at the end, but the verifier $\mathcal{V}$ is run n times, at every accumulation step. It is thus very important for the accumulation verifier to be efficient.

One extension to the description of accumulation schemes above is that we would often like to accumulate n old accumulators $[\mathsf{acc}_i]_{i\in[n]}$ and m new tuples $(\mathbb{x}_i, \pi_i)_{i\in[m]}$ into a single new accumulator acc' at each accumulation step. We construct an argument of knowledge ARG for the NP-complete relation $\mathcal{R}_{\mathsf{R1CS}}$[21] and a corresponding accumulation scheme ACC with a linear-time prover and a verifier that checks $O(\lambda)$ Merkle paths per input (there are now $n + m$ inputs at each accumulation step). We first recall the definition of a multi-instance relation and a many-to-one reduction and then recall [BMNW25b, Theorem 4.3].

Definition 10.1. *Let $\mathcal{R}^{\mathcal{U}}$ be an indexed pair relation. The corresponding* multi-instance relation *$(\mathcal{R}^*)^{\mathcal{U}}$ is defined to be*

$$(\mathcal{R}^*)^{\mathcal{U}} := \{(\mathbb{i}, (\mathbb{x}_1, \ldots, \mathbb{x}_m), (\mathbb{w}_1, \ldots, \mathbb{w}_m)) : \forall i \in [m], (\mathbb{i}, \mathbb{x}_i, \mathbb{w}_i) \in \mathcal{R}^{\mathcal{U}}\}.$$

The relaxed relation $\tilde{\mathcal{R}}^*$ is defined analogously. A many-to-one reduction *is a non-interactive reduction from $\mathcal{R}^*$ to $\mathcal{R}$ which preserves the index: given an index $\mathbb{i}$, the indexer $\mathbf{I}$ outputs the new index $\mathbb{i}' := \mathbb{i}$ (this requirement is necessary for repeatedly composing many-to-one reductions).*

Theorem 10.2 (Theorem 4.3, [BMNW25b])**.** *There exists a polynomial-time transformation T such that the following holds. Let $\mathcal{R}$ be an indexed relation. Let $\mathcal{R}^{\mathcal{U}}_{\mathsf{ACC}}$ be an indexed pair relation in $\mathsf{NP}^{\mathcal{U}}$ with the same parameter space as $\mathcal{R}$. Suppose we are given the following non-interactive reductions in the random oracle model:*

- $\mathsf{RDX}_{\mathsf{CAST}} = (\mathbf{G}_{\mathsf{CAST}}, \mathbf{I}_{\mathsf{CAST}}, \mathbf{P}_{\mathsf{CAST}}, \mathbf{V}_{\mathsf{CAST}})$*, a reduction from $\mathcal{R}$ to $\mathcal{R}_{\mathsf{ACC}}$.*

[21] For a full description of $\mathcal{R}_{\mathsf{R1CS}}$, we refer the reader to prior work such as Spartan [Set20].

- $\mathsf{RDX}_{\mathsf{FOLD}} = (\mathbf{G}_{\mathsf{FOLD}}, \mathbf{I}_{\mathsf{FOLD}}, \mathbf{P}_{\mathsf{FOLD}}, \mathbf{V}_{\mathsf{FOLD}})$*, a many-to-one reduction from* $\mathcal{R}^*_{\mathsf{ACC}}$ *to* $\mathcal{R}_{\mathsf{ACC}}$ *with the same generator algorithm as* $\mathsf{RDX}_{\mathsf{CAST}}$ *(i.e.,* $\mathbf{G}_{\mathsf{FOLD}} \equiv \mathbf{G}_{\mathsf{CAST}}$*).*

Then $T[\mathsf{RDX}_{\mathsf{CAST}}, \mathsf{RDX}_{\mathsf{FOLD}}, \mathcal{R}_{\mathsf{ACC}}] = (\mathsf{ARG}, \mathsf{ACC})$*, where* ARG *is a non-interactive argument for* $\mathcal{R}$ *and* ACC *is an accumulation scheme for* ARG*, both in the random oracle model.*

We adapt this theorem to our setting (with $\mathcal{R}^*_{\mathsf{ACC}} := \mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$ and $\mathcal{R}_{\mathsf{ACC}} := \mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$) and obtain the following corollary.

Corollary 10.3. *Given the existence of the following RBRTE IORs:*

- CAST *from* $\mathcal{R}_{\mathsf{R1CS}}$ *to* $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ *such that the prover runs in linear time and the verifier makes* $O(\lambda)$ *queries and runs in* $O(\lambda \log n)$ *time, and*
- FOLD *from* $\mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$ *to* $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ *such that the prover runs in linear time and the verifier makes* $O(\lambda\ell)$ *queries and runs in* $O(\lambda\ell \log n)$ *time,*

then there exists a non-interactive argument ARG *for* $\mathcal{R}_{\mathsf{R1CS}}$ *and a corresponding accumulation scheme* ACC *for* ARG *such that the prover for* ACC *runs in* $O(n)$ *time and the verifier for* ACC *checks* $O(\lambda)$ *Merkle path openings* per input.

Proof. Let $\mathsf{RDX}_{\mathsf{CAST}}$ and $\mathsf{RDX}_{\mathsf{FOLD}}$ be the NIRKs obtained by applying (a variant of) the BCS transform on CAST and FOLD respectively (see Section B). The existence of ARG and ACC then follows from Theorem 10.2 with $\mathcal{R} := \mathcal{R}_{\mathsf{R1CS}}$, $\mathcal{R}_{\mathsf{ACC}} := \mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ and $\mathcal{R}^*_{\mathsf{ACC}} := \mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$.

From the proof of Theorem 10.2 in [BMNW25b, Appendix A], it is easy to see that ARG is just a wrapper around $\mathsf{RDX}_{\mathsf{CAST}}$ and ACC is a wrapper around $\mathsf{RDX}_{\mathsf{CAST}}$ and $\mathsf{RDX}_{\mathsf{FOLD}}$. Thus, the most expensive step in the prover (and verifier, respectively) of ARG is running the prover (verifier) of $\mathsf{RDX}_{\mathsf{CAST}}$ and the most expensive step in the prover (and verifier, respectively) of ACC is running m copies of the the prover (verifier) of $\mathsf{RDX}_{\mathsf{CAST}}$ and running the prover (verifier) of $\mathsf{RDX}_{\mathsf{FOLD}}$ on an instance of length $(n + m)$. The efficiency claims follow from the fact that each query performed by the IOR verifier, the corresponding NIRK verifier checks one Merkle path opening. □

10.2 The cast IOR for accumulation

We show how to construct an IOR CAST that satisfies the required properties from Corollary 10.3. Prior work such as Spartan [Set20] and DFS [Hu+25] present MLIOPs for $\mathcal{R}_{\mathsf{R1CS}}$ with a linear time prover and $O(\lambda \log n)$ time verifier, where the prover sends ℓ (a constant) multilinear polynomial oracles to the verifier. Thus we can use the compiler from Section 8 to obtain an IOR from $\mathcal{R}_{\mathsf{R1CS}}$ to $\mathsf{MEP}^{(\mathcal{C},\varepsilon+\varepsilon_b,\ell)}$. We can then further compose with Homogenize and Batch to obtain the IOR CAST from $\mathcal{R}_{\mathsf{R1CS}}$ to $\mathsf{MEP}^{(\mathcal{C},\varepsilon)}$.

It is easy to check that the resulting IOR CAST satisfies the required efficiency measures, and is RBRTE due to our composition lemma (see Lemma 4.7), since all the IORs used in the composition are RBRTE. We summarize all these claims in the following lemma.

Lemma 10.4 (Accumulation cast IOR)**.** *There exists an RBRTE IOR* CAST *from* $\mathsf{Prox}(\mathcal{R}_{\mathsf{R1CS}})$ *to* $\mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$ *where* $\varepsilon \in (\delta_{\mathcal{C}}/8, \delta_{\mathcal{C}}/3 - 2\varepsilon_b)$ *and* ℓ *is a constant, with the following efficiency measures:*

rounds	*message size*	*queries*	*oracle size*	*prover time*	*verifier time*
$O(\log n)$	$O(\log n)$	*0*	$O(n)$	$O(n)$	$O(\log n)$

Here $\mathcal{C}$ *is an efficiently codeswitchable code (see Definition 6.14) that can be encoded in linear time with parameters* $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$.

Proof. There exists an MLIOP for the R1CS relation which requires $O(n)$ prover time, $O(\log n)$ verifier time, $O(\log n)$ rounds, and q polynomial oracle queries to ℓ unique polynomials for some constants $\mathsf{q}, \ell > 0$ [Set20]. Using our MLIOP to IOR compiler from Lemma 8.6, we obtain an IOR CAST from $\mathcal{R}_{\mathsf{R1CS}}$ to $\mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$ with the same efficiency measures as the MLIOP. □

10.3 The fold IOR for accumulation

We show how to construct an IOR FOLD that satisfies the required properties from Corollary 10.3. The FOLD IOR is obtained by composing the following IORs.

1. $\mathsf{Homogenize}$ from $\mathsf{MEP}^{(\mathcal{C},\varepsilon+\varepsilon_b,\ell)}$ to $\mathsf{HEP}^{(\mathcal{C},\varepsilon+\varepsilon_b,\ell)}$, from Section 6.2,
2. Batch from $\mathsf{HEP}^{(\mathcal{C},\varepsilon+\varepsilon_b,\ell)}$ to $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$, from Section 6.3 and
3. $\mathsf{CodeSwitch}$ from $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ to $\mathsf{MEP}^{(\mathcal{C},\varepsilon+\varepsilon_b,1)}$, from Section 7.

Since $\mathsf{Homogenize}$, Batch and $\mathsf{CodeSwitch}$ are all RBRTE (see Lemmas 6.10, 6.12 and 7.6), the resulting IOR FOLD is RBRTE by our composition lemma (see Lemma 4.7). It is easy to see that FOLD also satisfies all the efficiency requirements.

Lemma 10.5 (Accumulation fold IOR)**.** *There exists an RBRTE IOR* FOLD *from* $\mathsf{MEP}^{(\mathcal{C},\varepsilon,\ell)}$ *to* $\mathsf{MEP}^{(\mathcal{C},\varepsilon,1)}$ *where* $\varepsilon \in (\delta_{\mathcal{C}}/8, \delta_{\mathcal{C}}/3 - 2\varepsilon_b)$*, with the following efficiency measures:*

rounds	*message size*	*queries*	*oracle size*	*prover time*	*verifier time*
$O(\log n)$	$O\left(\log n + \frac{-\lambda}{\log(1-\delta_{\mathcal{C}})}\right)$	$O\left(\frac{-\lambda}{\log(1-\delta_{\mathcal{C}})}\right)$	$O(n)$	$O(n)$	$O\left(\lambda \log n + \frac{-\lambda}{\log(1-\delta_{\mathcal{C}})}\right)$

Here $\mathcal{C}$ *is a tensor code with parameters* $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$*, constant rate, and tensoring parameter* $t \geq 2$*.*

A Proof of Lemma 4.5 (Part 1)

In Section 4, we presented round-by-round tree-extractability (RBRTE), a new notion of knowledge soundness for IORs. In this section and Section B, we prove Lemma 4.5, which states that there exists a transformation from RBRTE IORs to non-interactive reductions of knowledge (NIRKs) in the random oracle model (ROM) with negligible knowledge soundness error. We do this in two steps:

(a) In this section, we prove that RBRTE IORs are rewinding state-restoration knowledge sound (SRKS) (which we define below).

(b) In Section B, we prove that rewinding SRKS IORs can be compiled into NIRKs in the random oracle model (ROM) via (a variant of) the BCS transform (from [BMNW25b]).

Organization of the section. We provide an outline of this section below:

1. In Section A.1, we define rewinding state-restoration-knowledge-soundness for IORs.
2. In Section A.2, we define $\mathsf{CTreeFinder}$, an algorithm that with overwhelming probability outputs an appropriate constrained transcript tree. Then, assuming the existence of $\mathsf{CTreeFinder}$, we show that RBRTE IORs are SRKS (with negligible knowledge soundness error).
3. In Section A.3, we provide a construction and analysis of $\mathsf{CTreeFinder}$.

Throughout this section, we assume the malicious prover's runtime is $\mathrm{poly}(n, \lambda)$ and use the notation $\mathsf{inst} := (\mathbb{x}, \vec{\mathbb{y}})$ for brevity. Our discussion throughout relies on the detailed "SNARGs book" [CY24], and we refer to the most up to date version as of January 2025.

A.1 Definition of rewinding state-restoration knowledge soundness

We first recall the definition of rewinding state-restoration knowledge soundness (SRKS) for IOPs from [CY24], and then we extend this to define rewinding SRKS for IORs.

A.1.1 Rewinding SRKS for IOPs

In this section, we will recall the definition of rewinding SRKS for IOPs from [CY24] (the definition for IPs is almost identical, except for the syntactic difference that the IOP oracle strings are replaced with IP prover messages). Clearly, these differences are only syntactic, so we present the definition of rewinding SRKS for IOPs from [CY24] without any modifications.[22]

Intuition. At a high level, state-restoration soundness extends the standard notion of soundness for an IOP as follows: the standard notion of soundness considers the setting where a malicious prover attempts to convince the verifier in a single interaction, where in each round the malicious prover sends a message and the verifier responds with some randomness, and at the end of the interaction the verifier accepts or rejects based on (the instance and) the transcript of interaction. In contrast, state-restoration soundness considers the setting where a malicious prover attempts to convince the verifier multiple times across related interactions, and the malicious prover wins if it finds an interaction that convinces the verifier.

Formal definition. We are now ready to present the full definition of rewinding SRKS for IOPs from [CY24, Section 23.2]. We start by presenting the prerequisite definition of an IOP state-restoration game.

Definition A.1 (IOP state-restoration game)**.** *The* IOP state-restoration game *for* IOP, $\mathsf{Game}^{\mathsf{sr}}_{\mathsf{IP}}$, *with functions* $\vec{\rho} = (\rho_i)_{i\in[\mathsf{k}]} \xleftarrow{\$} \mathcal{U}((\mathsf{r}_i)_{i\in[\mathsf{k}]})$ *and adversary* $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}$ *is defined below.*

[22] One minor difference is that we do not include the notion of a 'random salt', as this deals with zero-knowledge, which we do not address in this work.

$\mathsf{Game}^{\mathsf{sr}}_{\mathsf{IP}}(\vec{\rho}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}})$:

1. *Repeat the following until* $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}$ *exists the loop:*
   (a) $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}$ *outputs* $(\mathbb{x}_{\mathsf{IP}}, (\Pi_1, \ldots, \Pi_i))$ *with* $i \in [\mathsf{k}]$.
   (b) *Set* $r_i := \rho_i(\mathbb{x}_{\mathsf{IP}}, (\Pi_1, \ldots, \Pi_i))$.
   (c) *Send* r_i *to* $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}$.
2. $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}$ *outputs* $(\mathbb{x}_{\mathsf{IP}}, (\Pi_1, \ldots, \Pi_{\mathsf{k}}))$.
3. *For each* $i = 1, \ldots, \mathsf{k}$, *set* $r_i := \rho_i(\mathbb{x}_{\mathsf{IP}}, (\Pi_1, \ldots, \Pi_i))$.
4. *Output* $(\mathbb{x}_{\mathsf{IP}}, (\Pi_i)_{i\in[\mathsf{k}]}, (r_i)_{i\in[\mathsf{k}]})$.

Let $\mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}$ *denote the list of move-response pairs performed in the loop. We use the following notation to describe an execution of the state-restoration game:*

$$(\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \vec{r}) \xleftarrow{\mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}} \mathsf{Game}^{\mathsf{sr}}_{\mathsf{IP}}(\vec{\rho}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}).$$

We say that $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}$ *is* t*-move if* $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}$ *exits the loop after at most* t *iterations.*

Definition A.2 (Rewinding SRKS for IOPs)**.** $\mathsf{IP} := (\mathbf{P}, \mathbf{V})$ *for relation* $\mathcal{R}$ *has* rewinding state-restoration knowledge error κ_{sr} *if the following holds. There exists an expected polynomial-time extractor* $\mathbf{E}^{\mathsf{sr}}_{\mathsf{IOP}}$ *such that for every move budget* $t \in \mathbb{N}$, t*-move deterministic adversarial SRKS prover* $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}$.

$$\Pr\left[\begin{array}{c|l} b = 1, & \vec{\rho} = (\rho_i)_{i\in[\mathsf{k}]} \xleftarrow{\$} \mathcal{U}((\mathsf{r}_i)_{i\in[\mathsf{k}]}) \\ (\mathbb{x}_{\mathsf{IP}}, \mathbb{w}) \notin \mathcal{R}. & (\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \vec{r}) \xleftarrow{\mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}} \mathsf{Game}^{\mathsf{sr}}_{\mathsf{IP}}(\vec{\rho}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}) \\ & b := \mathbf{V}(\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \vec{r}) \\ & \mathbb{w} \leftarrow \mathbf{E}^{\mathsf{sr}}_{\mathsf{IOP}}(\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}, \vec{\rho}, \mathsf{IOP}) \end{array}\right] \leq \kappa_{\mathsf{sr}}(t).$$

A.1.2 Rewinding SRKS for IORs

Prior work [BMNW25b] has already extended straight-line SRKS for IOPs to a straight-line notion of SRKS for IORs (while incorporating the necessary syntactic changes to handle indexed oracle relations and non-oracle messages). Since the state-restoration game is identical for both straight-line and rewinding SRKS, we can use the same game as in Definition A.1 to define rewinding SRKS for IORs. We recall their definition below.

Definition A.3 (IOR state-restoration game [BMNW25b, Definition B.3])**.** *Consider a* k*-round IOR* IOR *where the verifier's challenge in round* i *consists of* r_i *bits. The* IOR state-restoration game $\mathsf{Game}^{\mathsf{sr}}_{\mathsf{IOR}}$ *for* IOR *with functions* $\vec{\rho} = (\rho_i : \{0,1\}^* \to \{0,1\}^{\mathsf{r}_i})_{i\in[\mathsf{k}]}$ *and adversary* $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}$ *is defined below.*

$\mathsf{Game}^{\mathsf{sr}}_{\mathsf{IOR}}(\vec{\rho}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}})$:

1. *Repeat the following until* $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}$ *exists the loop:*
   (a) $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}$ *sends a move* $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, (\Pi_1, \mathsf{msg}_1, \ldots, \Pi_i, \mathsf{msg}_i))$ *such that* $i \in [\mathsf{k}]$.
   (b) *Set* $r_i := \rho_i(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, (\Pi_1, \mathsf{msg}_1, \ldots, \Pi_i, \mathsf{msg}_i))$.
   (c) $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}$ *receives* r_i.
2. $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}$ *outputs* $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, (\Pi_1, \ldots, \Pi_{\mathsf{k}}), (\mathsf{msg}_1, \ldots, \mathsf{msg}_{\mathsf{k}}), \mathbb{w}', (A_1, \ldots, A_{\mathsf{n}'}))$.
3. *For each* $i = 1, \ldots, \mathsf{k}$, *set* $r_i := \rho_i(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, (\Pi_1, \mathsf{msg}_1, \ldots, \Pi_i, \mathsf{msg}_i))$.

4. Output $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, (\Pi_i)_{i\in[\mathsf{k}]}, (\mathsf{msg}_i)_{i\in[\mathsf{k}]}, \mathbb{w}', (A_j)_{j\in[\mathsf{n}']}, (r_i)_{i\in[\mathsf{k}]})$.

Let $\mathsf{tr}^{\mathsf{sr}}$ *denote the list of move-response pairs performed in the loop. We use the following notation to describe an execution of the state-restoration game:*

$$(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \vec{\Pi}, \vec{\mathsf{msg}}, \mathbb{w}', \vec{A}, \vec{r}) \xleftarrow{\mathsf{tr}^{\mathsf{sr}}} \mathsf{Game}^{\mathsf{sr}}_{\mathsf{IOR}}(\vec{\rho}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}).$$

Again, we say that $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}$ is t-move if $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}$ exits the loop after at most t iterations.

Rewinding SRKS for IORs. We now present our new definition of rewinding SRKS for IORs (as a natural extension of rewinding SRKS for IOPs to the IOR setting).

Definition A.4 (Rewinding SRKS for IORs)**.** *An IOR,* $\mathsf{IOR} := (\mathbf{I}_{\mathsf{IOR}}, \mathbf{P}_{\mathsf{IOR}}, \mathbf{V}_{\mathsf{IOR}})$ *from* $\mathcal{R}$ *to* $\mathcal{R}'$*, has* rewinding state-restoration knowledge error κ_{sr} *if the following holds. There exists an expected polynomial-time extractor* $\mathbf{E}^{\mathsf{sr}}_{\mathsf{IOR}}$ *such that for every move budget* $t \in \mathbb{N}$*,* t*-move deterministic adversarial SRKS prover* $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}$*.* [23]

$$\Pr\left[\begin{array}{c|l} b = 1, & \vec{\rho} = (\rho_i)_{i\in[\mathsf{k}]} \xleftarrow{\$} \mathcal{U}((\mathsf{r}_i)_{i\in[\mathsf{k}]}) \\ (\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \notin \tilde{\mathcal{R}}, & (\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \vec{\Pi}, \vec{\mathsf{msg}}, \mathbb{w}', \vec{A}, \vec{r}) \xleftarrow{\mathsf{tr}^{\mathsf{sr}}} \mathsf{Game}^{\mathsf{sr}}_{\mathsf{IOR}}(\vec{\rho}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}) \\ (\mathbb{i}', \mathbb{x}', \vec{\mathbb{y}}^*, \mathbb{w}') \in \tilde{\mathcal{R}}'. & (\iota, \mathbb{l}, \mathbb{i}') \leftarrow \mathbf{I}_{\mathsf{IOR}}(\mathbb{i}) \\ & (b, \mathbb{x}', \vec{\mathbb{y}}') \leftarrow \mathbf{V}^{\vec{\mathbb{l}}, \vec{\mathbb{y}}, \vec{\Pi}}_{\mathsf{IOR}}(\iota, \mathbb{x}, \vec{\mathsf{msg}}, \vec{r}) \\ & \forall j \in [\mathsf{n}'], \mathbb{y}^*_j := \mathbb{y}'_j|_{A_j} \\ & \mathbb{w} \leftarrow \mathbf{E}^{\mathsf{sr}}_{\mathsf{IOR}}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*, \mathbb{w}', \mathsf{tr}^{\mathsf{sr}}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}, \vec{\rho}, \mathsf{IOR}) \end{array}\right] \leq \kappa_{\mathsf{sr}}(t).$$

For brevity, we say that an IOR IOR *is* rewinding state-restoration knowledge sound *if* $\kappa_{\mathsf{sr}}(t)$ *is negligible in* λ*, for all* t*.*

In words, an IOR has rewinding SRKS if there exists an extractor $\mathbf{E}^{\mathsf{sr}}_{\mathsf{IOR}}$ that, for any SRKS prover $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}$ which convinces the verifier with non-negligible probability of a reduction from $\mathcal{R}$ to $\mathcal{R}'$, $\mathbf{E}^{\mathsf{sr}}_{\mathsf{IOR}}$ can rewind $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}$ and obtain a witness for the statement with respect to $\mathcal{R}$.

A.2 RBRTE implies SRKS using CTreeFinder

In this section, we show that every RBRTE IOR is also state-restoration knowledge sound. At a high level, the SRKS extractor $\mathbf{E}^{\mathsf{sr}}_{\mathsf{IOR}}$ works as follows: let $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}$ be a prover that succeeds in winning the IOR state-restoration game for the IOR. The extractor uses CTreeFinder, by rewinding $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}$, to construct a valid constrained transcript tree. It then uses $\mathbf{E}^{\mathrm{Tree}}$ (from the definition of RBRTE) to extract a valid witness from this tree.

CTreeFinder. We now formally define the constrained-tree-finder CTreeFinder: we show in Lemma A.5, that given as input $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*, \mathbb{w}', \mathsf{tr}^{\mathsf{sr}}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}, \vec{\rho}, \mathsf{IOR})$, CTreeFinder outputs a valid $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*)$-constrained transcript tree T for IOR with high probability, where $\vec{a}, \phi, \theta$ are such that IOR is $(\vec{a}, \phi, \theta)$-RBRTE with respect to $\mathbb{i}$.

[23] Throughout we use lazily sampled random oracles, so when we pass a random oracle as an input, we pass only the polynomial-many locations at which it was queried.

Lemma A.5 (Existence of CTreeFinder)**.** *Let* $\mathsf{IOR} = (\mathbf{I}_{\mathsf{IOR}}, \mathbf{P}_{\mathsf{IOR}}, \mathbf{V}_{\mathsf{IOR}})$ *be an RBRTE IOR from* $\mathcal{R}$ *to* $\mathcal{R}'$*. There exists an algorithm* CTreeFinder *with an expected runtime of* $\mathrm{poly}(n, \lambda)$ *such that*

$$\Pr\left[\begin{array}{c} b = 1, \\ |(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})| \leq n, \\ (\mathbb{i}', \mathbb{x}', \vec{\mathbb{y}}^*, \mathbb{w}') \in \mathcal{R}', \\ \mathsf{T} \textit{ is not a valid } (\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}) \\ \textit{–constrained transcript tree} \\ \textit{for } \mathsf{IOR}. \end{array} \middle| \begin{array}{l} \vec{\rho} = (\rho_i)_{i \in [\mathsf{k}]} \xleftarrow{\$} \mathcal{U}((\mathsf{r}_i)_{i \in [\mathsf{k}]}) \\ (\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \vec{\Pi}, \vec{\mathsf{msg}}, \mathbb{w}', \vec{A}, \vec{r}) \xleftarrow{\mathsf{tr}^{\mathsf{sr}}} \mathsf{Game}^{\mathsf{sr}}_{\mathsf{IOR}}(\vec{\rho}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}) \\ (\iota, \mathbb{l}, \mathbb{i}') \leftarrow \mathbf{I}_{\mathsf{IOR}}(\mathbb{i}) \\ (b, \mathbb{x}', \vec{\mathbb{y}}') \leftarrow \mathbf{V}^{\vec{\mathbb{l}}, \vec{\mathbb{y}}, \vec{\Pi}}_{\mathsf{IOR}}(\iota, \mathbb{x}, \vec{\mathsf{msg}}, \vec{r}) \\ \forall j \in [\mathsf{n}'], \mathbb{y}^*_j := \mathbb{y}'_j|_{A_j} \\ \mathsf{T} \leftarrow \mathsf{CTreeFinder}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*, \mathbb{w}', \mathsf{tr}^{\mathsf{sr}}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}, \vec{\rho}, \mathsf{IOR}) \end{array}\right] \leq \mathrm{negl}(\lambda) \ .$$

For clarity, we have omitted an explicit statement of the following condition: we say T *is a valid* $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*)$*-constrained transcript tree for* IOR *iff* IOR *is* $(\vec{a}, \phi, \theta)$*-RBRTE with respect to* $\mathbb{i}$*.*

We note that this lemma is a generalization of [CY24, Lemma 30.5.2] which shows that there exists an algorithm TreeFinder which extracts "unconstrained" transcript trees given rewinding access to an SRKS prover for IOPs. [24] We defer the proof of Lemma A.5 to Section A.3 and focus on showing how to use it to prove our main claim.

Lemma A.6 (RBRTE implies SRKS)**.** *Let* $\mathsf{IOR} = (\mathbf{I}_{\mathsf{IOR}}, \mathbf{P}_{\mathsf{IOR}}, \mathbf{V}_{\mathsf{IOR}})$ *be an RBRTE IOR from* $\mathcal{R}$ *to* $\mathcal{R}'$*. Then* IOR *is also state-restoration knowledge sound.*

Proof. We will construct an expected polynomial-time extractor $\mathbf{E}^{\mathsf{sr}}_{\mathsf{IOR}}$ such that for every move budget $t \in \mathbb{N}$, t-move deterministic adversarial SRKS prover $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}$, the following holds:

$$\Pr\left[\begin{array}{c} b = 1, \\ (\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \notin \tilde{\mathcal{R}}, \\ (\mathbb{i}', \mathbb{x}', \vec{\mathbb{y}}^*, \mathbb{w}') \in \tilde{\mathcal{R}}'. \end{array} \middle| \begin{array}{l} \vec{\rho} = (\rho_i)_{i \in [\mathsf{k}]} \xleftarrow{\$} \mathcal{U}((\mathsf{r}_i)_{i \in [\mathsf{k}]}) \\ (\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \vec{\Pi}, \vec{\mathsf{msg}}, \mathbb{w}', \vec{A}, \vec{r}) \xleftarrow{\mathsf{tr}^{\mathsf{sr}}} \mathsf{Game}^{\mathsf{sr}}_{\mathsf{IOR}}(\vec{\rho}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}) \\ (\iota, \mathbb{l}, \mathbb{i}') \leftarrow \mathbf{I}_{\mathsf{IOR}}(\mathbb{i}) \\ (b, \mathbb{x}', \vec{\mathbb{y}}') \leftarrow \mathbf{V}^{\vec{\mathbb{l}}, \vec{\mathbb{y}}, \vec{\Pi}}_{\mathsf{IOR}}(\iota, \mathbb{x}, \vec{\mathsf{msg}}, \vec{r}) \\ \forall j \in [\mathsf{n}'], \mathbb{y}^*_j := \mathbb{y}'_j|_{A_j} \\ \mathbb{w} \leftarrow \mathbf{E}^{\mathsf{sr}}_{\mathsf{IOR}}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*, \mathbb{w}', \mathsf{tr}^{\mathsf{sr}}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}, \vec{\rho}, \mathsf{IOR}) \end{array}\right] \leq \kappa_{\mathsf{sr}}(t) \ ,$$

where $\kappa_{\mathsf{sr}}(t) = \mathrm{negl}(\lambda)$ for all t.

$\mathbf{E}^{\mathsf{sr}}_{\mathsf{IOR}}$ on input $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*, \mathbb{w}', \mathsf{tr}^{\mathsf{sr}}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}, \vec{\rho}, \mathsf{IOR})$ runs CTreeFinder on the same input to obtain T, a valid $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*)$-constrained-transcript-tree. Since IOR is $(\vec{a}, \phi, \theta)$-RBRTE with respect to $\mathbb{i}$, we know that there exists a deterministic polynomial-time extractor $\mathbf{E}^{\mathrm{Tree}}$ such that for any instance $(\mathbb{x}, \vec{\mathbb{y}})$, $\mathbf{E}^{\mathrm{Tree}}$ takes as input a $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*)$-constrained transcript tree T for IOR, and outputs a witness $\mathbb{w}$ such that $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*, \mathbb{w}) \in \tilde{\mathcal{R}}$. $\mathbf{E}^{\mathsf{sr}}_{\mathsf{IOR}}$ runs $\mathbf{E}^{\mathrm{Tree}}$ on input T, and outputs the witness $\mathbb{w}$ outputted by $\mathbf{E}^{\mathrm{Tree}}$.

Since CTreeFinder has an expected runtime of $\mathrm{poly}(n, \lambda)$ and $\mathbf{E}^{\mathrm{Tree}}$ always has a runtime of $\mathrm{poly}(n, \lambda)$, and $\mathbf{E}^{\mathsf{sr}}_{\mathsf{IOR}}$ succeeds whenever CTreeFinder does, we obtain the desired result. □

[24] [CY24] states their theorem in terms of some general parameters. To adapt to our setting, set the prover runtime $t = \mathrm{poly}(n, \lambda)$ the tree size $\prod_{i \leq \mathsf{k}} a_i = \mathrm{poly}(n)$ and the verifier challenge length $r_i = \lambda$. This gives a $\mathrm{negl}(\lambda)$ failure probability and expected polynomial runtime.

A.3 Proof of Lemma B.6

In this section, we prove Lemma A.5 by providing an explicit construction for $\mathsf{CTreeFinder}$. Let the input to $\mathsf{CTreeFinder}$ be $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*, \mathbb{w}', \mathsf{tr}^{\mathsf{sr}}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}, \vec{\rho}, \mathsf{IOR})$. Since IOR is RBRTE, we know that there exist parameters $(\vec{a}, \phi, \theta)$ such that IOR is $(\vec{a}, \phi, \theta)$-RBRTE with respect to $\mathbb{i}$. We present the full construction of $\mathsf{CTreeFinder}$ in Fig. 11 and its component algorithms in Fig. 10. Looking ahead, we will need to define interactive proofs (IPs), which are IOPs for an NP relation $\mathcal{R}$ (i.e. $\mathcal{R}$ consists of tuples of the form $(\mathbb{x}, \mathbb{w})$, with no index or instance oracles) where the prover can only send non-oracle messages to the verifier.

We provide a high level overview of the construction below.

High level overview. At a high level, $\mathsf{CTreeFinder}$ invokes the 'unconstrained-tree-finder' $\mathsf{TreeFinder}$[25] from [CY24], which is defined for IPs which satisfy special soundness (an IP analogue of RBRTE from [CY24, Chapter 30] that we recall below) and acts as translator between the special soundness syntax with respect to IPs to the RBRTE syntax with respect to IORs.

In more detail, the constrained-tree-finder $\mathsf{CTreeFinder}$ consists of 2 key parts:

- The unconstrained-tree-finder $\mathsf{TreeFinder}$ for interactive proofs (IPs) described in [CY24, Construction 30.5.3]. $\mathsf{TreeFinder}$ receives as input $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}$, which convinces the verifier with non-negligible probability, and outputs an unconstrained transcript tree (Definition A.9).
- "API adapters" InAdap and PrAdap which receive as input $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}$, the prover from which we want to extract a constrained transcript tree, and its inputs. The adapters 'fool' $\mathsf{TreeFinder}$ into extracting an unconstrained transcript tree from $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}$ by tweaking its outputs to look as if they came from a IP state-restoration prover for an 'analogous' IP (defined below). This unconstrained transcript tree is then converted to a constrained transcript tree by TrAdap.

IP coresponding to an IOR. We define $\mathsf{IP}(\mathsf{IOR}, \mathbb{i})$, the IP corresponding to the IOR IOR and the index $\mathbb{i}$. For brevity, we sometimes use inst to denote a tuple of the form $(\mathbb{x}, \vec{\mathbb{y}})$.

Definition A.7. *Let $\mathsf{IOR} := (\mathbf{I}_{\mathsf{IOR}}, \mathbf{P}_{\mathsf{IOR}}, \mathbf{V}_{\mathsf{IOR}})$ be a k-round IOR from $\mathcal{R}$ to $\mathcal{R}'$, and let $\mathbb{i}$ be an index for $\mathcal{R}$. We define $\mathcal{R}^*$ to be the (not indexed, and not oracle) relation $\mathcal{R}^* := \{(\bot, (\mathbb{i} \,\|\, \mathbb{x} \,\|\, \vec{\mathbb{y}}), \mathbb{w}) : (\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \in \mathcal{R}\}$. Notice that the instances in $\mathcal{R}*$ contain an index for $\mathcal{R}$, an instance for $\mathcal{R}$, and an instance oracle for $\mathcal{R}$ (but in full vector form, not as oracles).*

We define $\mathsf{IP}(\mathsf{IOR}, \mathbb{i}) := (\mathbf{P}, \mathbf{V})$ as the following $(\mathsf{k}+1)$ round IP for $\mathcal{R}^*$ (that at a high level emulates IOR on input index $\mathbb{i}$).

1. *Let the input to $\mathbf{P}$ be $((\mathbb{i}, \mathsf{inst} := (\mathbb{x}, \vec{\mathbb{y}})), \mathbb{w})$ and the input to $\mathbf{V}$ be $(\mathbb{i}, \mathsf{inst})$. $\mathbf{P}$ and $\mathbf{V}$ compute $(\iota, \mathbb{l}, \mathbb{i}') \leftarrow \mathbf{I}_{\mathsf{IOR}}(\mathbb{i})$.*
2. *$\mathbf{P}$ and $\mathbf{V}$ for the first k emulate the interaction between $\mathbf{P}_{\mathsf{IOR}}$ and $\mathbf{V}_{\mathsf{IOR}}$ on input $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}; \mathbb{w})$:*
   - *(a) $\mathbf{P}$ sends the same messages as $\mathbf{P}_{\mathsf{IOR}}$, except that when $\mathbf{P}_{\mathsf{IOR}}$ sends an random oracle, $\mathbf{P}$ sends a vector of the same contents.*
   - *(b) $\mathbf{V}$ has the same input-output behavior as $\mathbf{V}_{\mathsf{IOR}}$, except that when $\mathbf{V}_{\mathsf{IOR}}$ receives an oracle for a vector, $\mathbf{V}$ receives the underlying vector itself. $\mathbf{V}$ simulates the queries that would have been made by $\mathbf{V}_{\mathsf{IOR}}$ to oracles via these vectors.*
3. *In the IOR, after the k'th round of interaction, $\mathbf{P}_{\mathsf{IOR}}, \mathbf{V}_{\mathsf{IOR}}$ output index-instance pair $(\mathbb{i}', \mathsf{inst}')$, and $\mathbf{P}_{\mathsf{IOR}}$ outputs a witness $\mathbb{w}'$ for $\mathcal{R}'$. In the $(\mathsf{k}+1)$-th round, $\mathbf{P}$ sends $\mathbb{w}'$ to $\mathbf{V}$.*
4. *$\mathbf{V}$ accepts if and only if (a) the witness $\mathbb{w}'$ sent in the $(\mathsf{k}+1)$-th round satisfies $(\mathbb{i}', \mathsf{inst}', \mathbb{w}') \in \mathcal{R}'$, and (b) $\mathbf{V}_{\mathsf{IOR}}$ accepts the transcript of the first k rounds.*

[25] This 'unconstrained-tree-finder' is called $\mathsf{SRTreeFinder}$

Remark A.8 (Vectorized oracles)**.** We assume that any time the $\mathsf{IP}(\mathsf{IOR}, \mathbb{i})$ converts an IOR oracle to a vector, it keeps track of the fact that this vector is a 'vectorized oracle'. Looking ahead, we will use this fact to convert these 'vectorized oracles' back into oracles in the CTreeFinder construction.

Background on IPs. Before we formally discuss TreeFinder we first recall the definitions of unconstrained transcript trees and special soundness for IPs from [CY24].

Definition A.9 (Unconstrained transcript tree)**.** *A $(\vec{a}, \mathbb{x}_{\mathsf{IP}})$–unconstrained transcript tree for an IP IP is a $(\vec{a}, \phi_{\mathsf{true}}, \theta_{\mathsf{true}}, \bot, \mathbb{x}_{\mathsf{IP}})$–constrained-tree-transcript for IP where the predicates $\theta_{\mathsf{true}}, \phi_{\mathsf{true}}$ are the trivial predicates (that always output 1).*

Definition A.10 (Special soundness for IPs)**.** *We say that an IP IP has special soundness if there exists a polynomial-time extractor $\mathbf{E}^{\mathsf{ss}}_{\mathsf{IP}}$ such that, for every instance $\mathbb{x}$ and $(\vec{a}, \mathbb{x})$–unconstrained transcript tree T for IP, $\mathbf{E}^{\mathsf{ss}}_{\mathsf{IP}}(\mathbb{x}, \mathsf{T})$ outputs a witness $\mathbb{w}$ such that $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$.*

We now recall a lemma from [CY24] about TreeFinder, which is described in [CY24, Construction 30.5.3]. We drop the salt strings and assume $\mathsf{r}_i > \lambda$ for all $i \in [\mathsf{k}]$.

Lemma A.11 ([CY24, Lemma 30.5.2])**.** *For every interactive proof IP, the algorithm $\mathsf{TreeFinder}$[26] in [CY24, Construction 30.5.3] runs in expected polynomial time and satisfies the following property. For every IP state-restoration prover $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}$ that makes at most $t := \mathrm{poly}(n, \lambda)$ moves and arity parameters where $\prod_{i=1}^{\mathsf{k}} a'_i = \mathrm{poly}(n, \lambda)$,*

$$\Pr\left[\begin{array}{c} b = 1, \\ \mathsf{T}_{\mathsf{IP}} \textit{ is not a } (\vec{a}, \mathbb{x}_{\mathsf{IP}}) \\ \textit{–unconstrained transcript tree for } \mathsf{IP}. \end{array} \middle| \begin{array}{l} \vec{\rho} = (\rho_i)_{i\in[\mathsf{k}]} \xleftarrow{\$} \mathcal{U}((\mathsf{r}_i)_{i\in[\mathsf{k}]}) \\ (\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \vec{r}) \xleftarrow{\mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}} \mathsf{Game}^{\mathsf{sr}}_{\mathsf{IP}}(\vec{\rho}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}) \\ b := \mathbf{V}(\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \vec{r}) \\ \mathsf{T} \leftarrow \mathsf{TreeFinder}(\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}, \vec{\rho}, \mathsf{IP}) \end{array} \right] \leq \mathrm{negl}(\lambda).$$

We are now ready to present the full construction of CTreeFinder in Fig. 11 and its component algorithms in Fig. 10.

Before we prove that CTreeFinder satisfies the conditions in Lemma A.5, we prove that the 'adapters' PrAdap, TrAdap work as expected.

Claim A.12. *The following holds for* InAdap *and* PrAdap *defined in Fig. 10:*

$$\Pr\left[\begin{array}{c} b = 1, \\ |(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})| \leq n, \\ (\mathbb{i}', \mathbb{x}', \vec{\mathbb{y}}^*, \mathbb{w}') \in \mathcal{R}', \\ \mathsf{T}_{\mathsf{IP}} \textit{ is not an } (\vec{a}, \mathbb{x}_{\mathsf{IP}}) \\ \textit{–unconstrained transcript tree} \\ \textit{for } \mathsf{IP}(\mathsf{IOR}, \mathbb{i}). \end{array} \middle| \begin{array}{l} \vec{\rho} = (\rho_i)_{i\in[\mathsf{k}]} \xleftarrow{\$} \mathcal{U}((\mathsf{r}_i)_{i\in[\mathsf{k}]}) \\ (\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \vec{\Pi}, \vec{\mathsf{msg}}, \mathbb{w}', \vec{A}, \vec{r}) \xleftarrow{\mathsf{tr}^{\mathsf{sr}}} \mathsf{Game}^{\mathsf{sr}}_{\mathsf{IOR}}(\vec{\rho}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}) \\ (\iota, \mathbb{l}, \mathbb{i}') \leftarrow \mathbf{I}_{\mathsf{IOR}}(\mathbb{i}) \\ (b, \mathbb{x}', \vec{\mathbb{y}}') \leftarrow \mathbf{V}^{\vec{\mathbb{l}}, \vec{\mathbb{y}}, \vec{\Pi}}_{\mathsf{IOR}}(\iota, \mathbb{x}, \vec{\mathsf{msg}}, \vec{r}) \\ \forall j \in [\mathsf{n}'], \mathbb{y}^*_j := \mathbb{y}'_j|_{A_j} \\ \hline (\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}) \leftarrow \mathsf{InAdap}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*, \mathbb{w}', \mathsf{tr}^{\mathsf{sr}}, \vec{\rho}) \\ \mathsf{T}_{\mathsf{IP}} \leftarrow \mathsf{TreeFinder}(\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}, \mathsf{PrAdap}(\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}), \mathsf{IP}(\mathsf{IOR}, \mathbb{i})) \\ b' := \mathbf{V}(\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \vec{r}) \end{array} \right] \leq \mathrm{negl}(\lambda) \ .$$

[26] We note that TreeFinder did not take the description of the IP as input but was implicitly parameterized by it (since it needs to invoke the IP verifier). We choose to make this explicit.

Input adapter InAdap from IOR to $\mathsf{IP}(\mathsf{IOR}, \mathbb{i})$.

Inputs.
1. IOR index, instance, and instance oracle tuple $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$,
2. IOR prover messages made of oracle and non-oracle parts $(\vec{\Pi}, \vec{\mathsf{msg}})$,
3. trace of prover moves and oracle responses from the IOR state-restoration game $\mathsf{tr}^{\mathsf{sr}}$,
4. witness for $\mathcal{R}'$, target IOR relation, $\mathbb{w}'$.

Outputs.
1. $\mathsf{IP}(\mathsf{IOR}, \mathbb{i})$ instance $\mathbb{x}_{\mathsf{IP}}$,
2. $\mathsf{IP}(\mathsf{IOR}, \mathbb{i})$ prover messages $\vec{\mathsf{msg}}^{\mathsf{IP}}{}_{\mathsf{IP}}$, and
3. trace of prover moves for the $\mathsf{IP}(\mathsf{IOR}, \mathbb{i})$ state-restoration game $\mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}$.

Procedure.
1. Set the $\mathsf{IP}(\mathsf{IOR}, \mathbb{i})$ instance $\mathbb{x}_{\mathsf{IP}} := (\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$.
2. Given $(\vec{\Pi}, \vec{\mathsf{msg}})$, convert all oracles to vectors to obtain $\vec{\mathsf{msg}}_{\mathsf{tmp}}$. Append the witness $\mathbb{w}'$ to $\vec{\mathsf{msg}}_{\mathsf{tmp}}$ and obtain $\vec{\mathsf{msg}}^{\mathsf{IP}}{}_{\mathsf{IP}}$.
3. Given the IOR trace $\mathsf{tr}^{\mathsf{sr}}$ consisting of prover moves and random function evaluations, transform every oracle to a vector and let the result be $\mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}$.

Prover adapter PrAdap from IOR to $\mathsf{IP}(\mathsf{IOR}, \mathbb{i})$.

Inputs. The IOR state-restoration prover $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}$ for IOR.

Outputs. The corresponding state-restoration prover $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}$ for $\mathsf{IP}(\mathsf{IOR}, \mathbb{i})$.

Procedure.
$\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}$ works as follows, given the partial $\mathsf{IP}(\mathsf{IOR}, \mathbb{i})$ transcript $(\mathbb{x}_{\mathsf{IP}}, (\mathsf{msg}^{\mathsf{IP}}_1, r_1, \ldots, \mathsf{msg}^{\mathsf{IP}}_i, r_i))$
1. For all $j \in [i]$: parse $\mathsf{msg}^{\mathsf{IP}}_j$ as (Π_j, msg_j), where Π_j is the 'non-oracle' part of the message and msg_j is the 'oracle' part.
2. $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}$ provides the input $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, (\Pi_1, \mathsf{msg}_1, \ldots, \Pi_i, \mathsf{msg}_i))$ to $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}$, to obtain its next move $\mathsf{IORmove}_{i+1}$.
3. If $i = \mathsf{k}$: $\mathsf{IORmove}_{i+1}$ is a witness $\mathbb{w}'$, and $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}$ outputs $\mathbb{w}'$.
4. Else: $\mathsf{IORmove}_{i+1}$ is a tuple $(\Pi_{i+1}, \mathsf{msg}_{i+1})$, and $\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}$ outputs the tuple after converting the oracle into a vector.

Tree Adapter TrAdap from $\mathsf{IP}(\mathsf{IOR}, \mathbb{i})$ to IOR.

Inputs. T_{IP}: a $(\vec{a} \,\|\, 1, \mathbb{x}_{\mathsf{IP}})$–unconstrained transcript tree for $\mathsf{IP}(\mathsf{IOR}, \mathbb{i})$.

Outputs. T: a $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$–constrained transcript tree for IOR.

Procedure.
1. Parse $\mathbb{x}_{\mathsf{IP}}$ as $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$.
2. Given T_{IP}, TrAdap augments it in the following ways to obtain T:
   (a) Change every 'vectorized oracle' in any node of the tree back to an oracle.
   (b) Interpret the leaf nodes that contain $\mathbb{w}'$ as the witness for that path of T.
3. Output the $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$–constrained transcript tree T.

Figure 10: The InAdap, PrAdap, TrAdap 'adapter' algorithms for $\mathsf{CTreeFinder}$.

Constrained-tree-finder $\mathsf{CTreeFinder}$.

Inputs.
1. $\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*, \mathbb{w}', \mathsf{tr}^{\mathsf{sr}}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}, \vec{\rho}$ (same as $\mathbf{E}^{\mathsf{sr}}_{\mathsf{IOR}}$).
2. RBRTE IOR description $\mathsf{IOR} := (\mathbf{I}_{\mathsf{IOR}}, \mathbf{P}_{\mathsf{IOR}}, \mathbf{V}_{\mathsf{IOR}})$.
3. Parameters $(\vec{a}, \phi, \theta)$ (implicitly, from $\mathbb{i}$ and IOR being RBRTE).

Outputs. T: a $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*)$-constrained-transcript-tree for IOR.

Procedure.
1. Obtain the input to TreeFinder using InAdap: $(\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}) \leftarrow \mathsf{InAdap}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*, \mathbb{w}', \mathsf{tr}^{\mathsf{sr}}, \vec{\rho})$.
2. Run TreeFinder to get a $(\vec{a} \,\|\, 1, \mathbb{x}_{\mathsf{IP}})$–unconstrained tree for $\mathsf{IP}(\mathsf{IOR}, \mathbb{i})$: $\mathsf{T}_{\mathsf{IP}} \leftarrow \mathsf{TreeFinder}(\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}, \mathsf{PrAdap}(\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}), \mathsf{IP}(\mathsf{IOR}, \mathbb{i}))$.
3. Obtain a $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*)$–constrained transcript tree for IOR using TrAdap: $\mathsf{T} \leftarrow \mathsf{TrAdap}(\mathsf{T}_{\mathsf{IP}})$.
4. Output T.

Figure 11: $\mathsf{CTreeFinder}$ algorithm.

Proof. We start by claiming that

$$\Pr\left[\begin{array}{c|l} b' = 1, & \vec{\rho} = (\rho_i)_{i\in[\mathsf{k}]} \xleftarrow{\$} \mathcal{U}((\mathsf{r}_i)_{i\in[\mathsf{k}]}) \\ |\mathbb{x}_{\mathsf{IP}}| \leq n & (\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \vec{r}) \xleftarrow{\mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}} \mathsf{Game}^{\mathsf{sr}}_{\mathsf{IP}}(\vec{\rho}, \mathsf{PrAdap}(\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}})) \\ \mathsf{T}_{\mathsf{IP}} \text{ is not a } (\vec{a}, \mathbb{x}_{\mathsf{IP}}) & b' := \mathbf{V}(\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \vec{r}) \\ \text{–unconstrained transcript tree for } \mathsf{IP}(\mathsf{IOR}, \mathbb{i}). & \mathsf{T}_{\mathsf{IP}} \leftarrow \mathsf{TreeFinder}(\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IP}}, \mathsf{IP}(\mathsf{IOR}, \mathbb{i})) \end{array}\right] \leq \mathrm{negl}(\lambda)$$

This claim follows from the fact that $\mathsf{PrAdap}(\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}})$ has the input-output behavior of a state-restoration prover for $\mathsf{IP}(\mathsf{IOR}, \mathbb{i})$, and Lemma A.11 applies to *all* IP state-restoration provers.

This proves Claim A.12 since by the definition of InAdap, PrAdap, and $\mathsf{IP}(\mathsf{IOR}, \mathbb{i})$ we have that:
1. $\mathsf{Game}^{\mathsf{sr}}_{\mathsf{IP}}(\vec{\rho}, \mathsf{PrAdap}(\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}))$ emulates $\mathsf{Game}^{\mathsf{sr}}_{\mathsf{IOR}}(\vec{\rho}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}})$.
2. $\mathbb{x}_{\mathsf{IP}}$ is the concatenation of $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$, $|\mathbb{x}_{\mathsf{IP}}| \leq n$ implies $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}) | < n$.
3. $\mathbf{V}$ accepts iff $\mathbf{V}_{\mathsf{IOR}}$ accepts *and* $(\mathbb{i}', \mathbb{x}', \vec{\mathbb{y}}^*, \mathbb{w}') \in \mathcal{R}'$, thus $b' = 1$ iff $b = 1$ and $(\mathbb{i}', \mathbb{x}', \vec{\mathbb{y}}^*, \mathbb{w}') \in \mathcal{R}'$.

□

Claim A.13. *The following holds for* InAdap, PrAdap *and* TrAdap *defined in Fig. 10:*

$$\Pr\left[\begin{array}{c|l} \mathsf{T} \textit{ is not an } (\vec{a}, \phi, \theta, \mathbb{i}, \mathsf{inst}) & \vec{\rho} = (\rho_i)_{i\in[\mathsf{k}]} \xleftarrow{\$} \mathcal{U}((\mathsf{r}_i)_{i\in[\mathsf{k}]}) \\ \textit{–constrained transcript tree} & (\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \vec{\Pi}, \vec{\mathsf{msg}}, \mathbb{w}', \vec{A}, \vec{r}) \xleftarrow{\mathsf{tr}^{\mathsf{sr}}} \mathsf{Game}^{\mathsf{sr}}_{\mathsf{IOR}}(\vec{\rho}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}) \\ \textit{for } \mathsf{IOR}. & (\iota, \mathbb{l}, \mathbb{i}') \leftarrow \mathbf{I}_{\mathsf{IOR}}(\mathbb{i}) \\ & (b, \mathbb{x}', \vec{\mathbb{y}}') \leftarrow \mathbf{V}^{\vec{\mathbb{l}}, \vec{\mathbb{y}}, \vec{\Pi}}_{\mathsf{IOR}}(\iota, \mathbb{x}, \vec{\mathsf{msg}}, \vec{r}) \\ & \forall j \in [\mathsf{n}'], \mathbb{y}^*_j := \mathbb{y}'_j|_{A_j} \\ & \mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}} \leftarrow \mathsf{InAdap}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*, \mathbb{w}', \mathsf{tr}^{\mathsf{sr}}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}, \vec{\rho}) \\ & \mathsf{T}_{\mathsf{IP}} \leftarrow \mathsf{TreeFinder}(\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}, \mathsf{PrAdap}(\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}), \mathsf{IP}(\mathsf{IOR}, \mathbb{i})) \\ & \mathsf{T} \leftarrow \mathsf{TrAdap}(\mathsf{T}_{\mathsf{IP}}) \end{array}\right] \leq \mathrm{negl}(\lambda) \ .$$

Proof. Define the event

$$E := \left\{ \begin{array}{c} b = 1, \\ |(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})| \leq n, \\ (\mathbb{i}', \mathbb{x}', \vec{\mathbb{y}}^*, \mathbb{w}') \in \mathcal{R}', \\ \mathsf{T}_{\mathsf{IP}} \text{ is an } (\vec{a}, \mathbb{x}_{\mathsf{IP}}) \\ \text{–unconstrained transcript tree} \\ \text{for } \mathsf{IP}(\mathsf{IOR}, \mathbb{i}). \end{array} \right\} .$$

By Claim A.12, we know that T_{IP} is an $(\vec{a}, \mathbb{x}_{\mathsf{IP}})$-unconstrained transcript tree with overwhelming probability. In more detail:

$$\Pr\left[\text{The event } E \text{ occurs} \;\middle|\; \begin{array}{l} \vec{\rho} = (\rho_i)_{i\in[\mathsf{k}]} \xleftarrow{\$} \mathcal{U}((\mathsf{r}_i)_{i\in[\mathsf{k}]}) \\ (\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \vec{\Pi}, \vec{\mathsf{msg}}, \mathbb{w}', \vec{A}, \vec{r}) \xleftarrow{\mathsf{tr}^{\mathsf{sr}}} \mathsf{Game}^{\mathsf{sr}}_{\mathsf{IOR}}(\vec{\rho}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}) \\ (\iota, \mathbb{l}, \mathbb{i}') \leftarrow \mathbf{I}_{\mathsf{IOR}}(\mathbb{i}) \\ (b, \mathbb{x}', \vec{\mathbb{y}}') \leftarrow \mathbf{V}^{\vec{\mathbb{l}}, \vec{\mathbb{y}}, \vec{\Pi}}_{\mathsf{IOR}}(\iota, \mathbb{x}, \vec{\mathsf{msg}}, \vec{r}) \\ \forall j \in [\mathsf{n}'], \mathbb{y}^*_j := \mathbb{y}'_j|_{A_j} \\ \mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}} \leftarrow \mathsf{InAdap}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*, \mathbb{w}', \mathsf{tr}^{\mathsf{sr}}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}, \vec{\rho}) \\ \mathsf{T}_{\mathsf{IP}} \leftarrow \mathsf{TreeFinder}(\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}, \mathsf{PrAdap}(\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}), \mathsf{IP}(\mathsf{IOR}, \mathbb{i})) \end{array} \right] \geq 1 - \mathrm{negl}(\lambda) \ .$$

Thus it would suffice to show that the following inequality holds:

$$\Pr\left[\begin{array}{c} \mathsf{T} \text{ is not an } (\vec{a}, \phi, \theta, \mathbb{i}, \mathsf{inst}) \\ \text{–constrained transcript tree} \\ \text{for } \mathsf{IOR}. \end{array} \;\middle|\; \begin{array}{l} \vec{\rho} = (\rho_i)_{i\in[\mathsf{k}]} \xleftarrow{\$} \mathcal{U}((\mathsf{r}_i)_{i\in[\mathsf{k}]}) \\ (\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \vec{\Pi}, \vec{\mathsf{msg}}, \mathbb{w}', \vec{A}, \vec{r}) \xleftarrow{\mathsf{tr}^{\mathsf{sr}}} \mathsf{Game}^{\mathsf{sr}}_{\mathsf{IOR}}(\vec{\rho}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}) \\ (\iota, \mathbb{l}, \mathbb{i}') \leftarrow \mathbf{I}_{\mathsf{IOR}}(\mathbb{i}) \\ (b, \mathbb{x}', \vec{\mathbb{y}}') \leftarrow \mathbf{V}^{\vec{\mathbb{l}}, \vec{\mathbb{y}}, \vec{\Pi}}_{\mathsf{IOR}}(\iota, \mathbb{x}, \vec{\mathsf{msg}}, \vec{r}) \\ \forall j \in [\mathsf{n}'], \mathbb{y}^*_j := \mathbb{y}'_j|_{A_j} \\ \mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}} \leftarrow \mathsf{InAdap}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}^*, \mathbb{w}', \mathsf{tr}^{\mathsf{sr}}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}, \vec{\rho}) \\ \mathsf{T}_{\mathsf{IP}} \leftarrow \mathsf{TreeFinder}(\mathbb{x}_{\mathsf{IP}}, \vec{\Pi}, \mathsf{tr}^{\mathsf{sr}}_{\mathsf{IP}}, \mathsf{PrAdap}(\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}), \mathsf{IP}(\mathsf{IOR}, \mathbb{i})) \\ \mathsf{T} \leftarrow \mathsf{TrAdap}(\mathsf{T}_{\mathsf{IP}}) \\ \text{The event } E \text{ occurs} \end{array}\right] \leq \mathrm{negl}(\lambda) \ .$$

That is, it suffices to show that given that E occurs, the probability that $\mathsf{TrAdap}(\mathsf{T}_{\mathsf{IP}})$ does not succeed in outputting an $(\vec{a}, \phi, \theta, \mathbb{i}, \mathsf{inst})$-constrained transcript tree is negligible. Let T be the output of $\mathsf{TrAdap}(\mathsf{T}_{\mathsf{IP}})$, we will show that T is an $(\vec{a}, \phi, \theta, \mathbb{i}, \mathsf{inst})$-constrained transcript tree except with negligible probability, by showing that it satisfies each of the 7 conditions in Definition 4.1.

- **Structural conditions.** It is easy to verify that Conditions 1 to 5 are satisfied (with probability 1) by the definition of TrAdap and the fact that T_{IP} is an $(\vec{a}, \mathbb{x}_{\mathsf{IP}})$–unconstrained transcript tree for $\mathsf{IP}(\mathsf{IOR}, \mathbb{i})$.
- **Vertical predicate is satisfied.** We will now prove that, except with negligible probability, Condition 6 is satisfied. First, we recall two facts about $\mathsf{TreeFinder}$ from [CY24, Construction 30.5.3].
  1. $\mathsf{TreeFinder}$ invokes $\mathsf{Game}^{\mathsf{sr}}_{\mathsf{IP}}(\vec{\rho}, \mathsf{PrAdap}(\tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}}))$ for many random oracles, $(\vec{\rho}_j)_{j\in[\mathrm{poly}(n,\lambda)]}$, and for each $j \in [\mathrm{poly}(n, \lambda)]$ and input x in the random oracle's domain, $\vec{\rho}_j(x)$ is distributed uniformly. [27]

[27] In more detail, $\mathsf{TreeFinder}$ will occasionally set $\vec{\rho}_j$ to be identical to $\vec{\rho}_{j'}$ for some $j' < j$ on the locations previously evaluated by the prover and set the rest of the outputs to uniform, but that detail is immaterial to our argument.

2. For every node in the i-th level of T_{IP} labeled with prover message (Π_i, msg_i) and edge below it labeled with r_i, it must that for some choice of random oracles $\vec{\rho}$, $\mathsf{Game}^{\mathsf{sr}}_{\mathsf{IOR}}(\vec{\rho}, \tilde{\mathbf{P}}^{\mathsf{sr}}_{\mathsf{IOR}})$ was executed and in it the prover outputted a move $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, (\Pi_1, \mathsf{msg}_1, \ldots, \Pi_i, \mathsf{msg}_i))$ and received back challenge $r_i := \rho_i(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, (\Pi_1, \mathsf{msg}_1, \ldots, \Pi_i, \mathsf{msg}_i))$.

By the definition of vertical predicates (Definition 4.3), for any $(\mathbb{i}, \mathsf{inst}, \alpha_1, r_1, \ldots, \alpha_{i-1}, r_{i-1}, \alpha_i)$ such that $\phi(\mathbb{i}, \mathsf{inst}, \alpha_1, r_1, \ldots, \alpha_{i-1}, r_{i-1}) = 1$, it holds that

$$\Pr_{r_j \leftarrow \mathsf{VD}_i}[\phi(\mathbb{i}, \mathsf{inst}, \alpha_1, r_1, \ldots, \alpha_{i-1}, r_{i-1}) = 0] < \mathrm{negl}(\lambda)$$

where VD_i is the domain of the i-th verifier message and $\alpha_j := (\Pi_j, \mathsf{msg}_j)$ is the j-th prover message. Thus, since the prover makes at most $\mathrm{poly}(n, \lambda)$ moves per round and is invoked at most $\mathrm{poly}(n, \lambda)$ times by TreeFinder (except with negligible probability) [28], we have that the probability that the prover ever makes a move $(\mathbb{i}, \mathsf{inst}, \alpha_1, r_1, \ldots, \alpha_{i-1}, r_{i-1}, \alpha_i)$ and received a response r_i s.t.

$$\phi(\mathbb{i}, \mathsf{inst}, \alpha_1, r_1, \ldots, \alpha_{i-1}, r_{i-1}) = 0$$

is $\mathrm{negl}(\lambda)$. Thus, by induction on $i \in [\mathsf{k}]$, it holds that, except with negligible probability, Condition 6 is satisfied (every path down the tree satisfies the vertical predicate).

- **Challenge predicate is satisfied.** Condition 7 is satisfied (except with negligible probability) due to the foregoing argument and the definition of the challenge predicate. □

Proof of Lemma B.6. Since InAdap, PrAdap and TrAdap all run in polynomial time, and TreeFinder runs in expected polynomial time, CTreeFinder also runs in expected polynomial time. This fact, along with Claim A.13, completes the proof of Lemma A.5.

Remark A.14. Note that ϕ, θ can be inefficient to compute, as the extractor never has to evaluate them.

[28] Since TreeFinder is runs in $\mathrm{poly}(n, \lambda)$-time except with $\mathrm{negl}(\lambda)$ probability.

B Proof of Lemma 4.5 (Part 2)

In this section we discuss non-interactive reductions of knowledge (NIRKs). We begin by recalling some prerequisites in Section B.1. Then, in Section B.2, we recall the definition of NIRKs from Arc [BMNW25b] and introduce a notion of rewinding knowledge soundness for NIRKs, which is a relaxation of the straight-line knowledge soundness notion used in Arc. Then in Section B.3 we describe (a variant of) the BCS transformation from [BMNW25b] and show that upon input an SRKS IOR it outputs a rewinding knowledge sound NIRK. We note that while the construction of the transformation remains the same, [BMNW25b] only showed that the transformation outputs a straight-line knowledge sound NIRK, given a straight-line SRKS IOR, which is not sufficient for our purposes. For brevity, we will abuse notation and call this variant the BCS transformation in the rest of this section (since the differences are mostly syntactic).

B.1 Prerequisites

Merkle commitments. We recall the definition of Merkle commitments along with some useful security properties from [CY24, Section 18]. The Merkle commitment scheme is a tuple of deterministic polynomial-time oracle algorithms $\mathsf{MT} = (\mathsf{MT.Commit}, \mathsf{MT.Open}, \mathsf{MT.Check})$ implicitly parameterized by an output size $\sigma \in \mathbb{N}$, alphabet Σ, and string length $\ell \in \mathbb{N}$. All algorithms receive query access to a random oracle $\rho_{\mathsf{MT}} \in \mathcal{U}(\sigma)$.

- $\mathsf{MT.Commit}$ receives as input a string $m \in \Sigma^{\ell}$. It outputs a commitment $\mathsf{cm} \in \{0,1\}^{\sigma}$ and a trapdoor $\mathsf{td} \in \{0,1\}^{O(\sigma\ell)}$.
- $\mathsf{MT.Open}$ receives as input a trapdoor td and subset $I \subseteq [\ell]$. It outputs an opening proof $\mathsf{pf} \in \{0,1\}^{|I| \cdot \sigma \log \ell}$.
- $\mathsf{MT.Check}$ receives as input a commitment cm, restriction $a \in \Sigma^I$, and opening proof pf. It outputs a bit indicating whether or not the opening proof authenticates the restriction with respect to the commitment.

Lemma B.1 (MT is complete)**.** *For every (unbounded) adversary $\mathcal{A}$,*

$$\Pr\left[\mathsf{MT.Check}^{\rho_{\mathsf{MT}}}(\mathsf{cm}, m|_I, \mathsf{pf}) = 1 \;\middle|\; \begin{array}{l} \rho_{\mathsf{MT}} \xleftarrow{\$} \mathcal{U}(\sigma) \\ (m, I) \leftarrow \mathcal{A}^{\rho_{\mathsf{MT}}} \\ (\mathsf{cm}, \mathsf{td}) \leftarrow \mathsf{MT.Commit}^{\rho_{\mathsf{MT}}}(m) \\ \mathsf{pf} \leftarrow \mathsf{MT.Open}^{\rho_{\mathsf{MT}}}(\mathsf{td}, I) \end{array}\right] = 1.$$

Lemma B.2 (MT is multi-extractable)**.** *There exists a deterministic polynomial-time algorithm* $\mathsf{MT.MultiExtract}$ *such that for every query bound $t \in \mathbb{N}$, t-query adversary $\mathcal{A}$, and $k \in \mathbb{N}$,*

$$\Pr\left[\begin{array}{l} \exists i \in [k] : \\ \quad \mathsf{MT.Check}^{\rho_{\mathsf{MT}}}(\mathsf{cm}[i], I[i], a[i], \mathsf{pf}[i]) = 1 \\ \quad m[i][I[i]] \neq a[i] \end{array} \;\middle|\; \begin{array}{l} \rho_{\mathsf{MT}} \xleftarrow{\$} \mathcal{U}(\sigma) \\ (\vec{\mathsf{cm}}, \vec{I}, \vec{a}, \vec{\mathsf{pf}}) \xleftarrow{\mathsf{tr}} \mathcal{A}^{\rho_{\mathsf{MT}}} \\ (\vec{m}, \vec{\mathsf{td}}) \leftarrow \mathsf{MT.MultiExtract}^{\rho_{\mathsf{MT}}}(\vec{\mathsf{cm}}, \mathsf{tr}) \\ \forall i \in [k] : \mathsf{pf}[i] \leftarrow \mathsf{MT.Open}^{\rho_{\mathsf{MT}}}(\mathsf{td}[i], I[i]) \end{array}\right]$$

is at most

$$\kappa_{\mathsf{MT}}(t, \sigma, \ell, k) := \frac{3}{2} \cdot \frac{t^2}{2^{\sigma}} + \frac{k \cdot (\log \ell + 1) \cdot 3t}{2^{\sigma}}.$$

Committed relations. Informally, given an oracle relation $\mathcal{R}$, the committed relation $\mathsf{Com}[\mathcal{R}]$ replaces instance strings with Merkle commitments and adds trapdoors Merkle authentication paths to the witness.

Definition B.3. *Let $\mathcal{R}$ be an indexed oracle pair relation and S be a set. The* committed relation $\mathsf{Com}[\mathcal{R}, S] = \mathcal{S}^{\mathcal{U}}$ *is an indexed pair relation (relative to a random oracle) defined below.*

$$\mathcal{S}^{\rho} := \left\{ (\mathbb{i}, (\mathbb{x}, \vec{\mathsf{cm}}), (\mathbb{w}, \vec{\mathbb{y}}, \vec{\mathsf{td}})) : \begin{array}{l} (\mathbb{i}, \mathbb{x}) \in S \\ (\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \in \mathcal{R} \\ \forall j \in [\mathsf{n}], \mathsf{MT.Commit}^{\rho_{\mathsf{MT}}}(\mathbb{y}_j) = (\mathsf{cm}_j, \mathsf{td}_j) \end{array} \right\}$$

$$\tilde{\mathcal{S}}^{\rho} := \left\{ (\mathbb{i}, (\mathbb{x}, \vec{\mathsf{cm}}), (\mathbb{w}, \vec{\mathbb{y}}, \vec{\mathsf{td}})) : \begin{array}{l} (\mathbb{i}, \mathbb{x}) \in S \\ (\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \in \tilde{\mathcal{R}} \\ \forall j \in [\mathsf{n}] : \\ \quad \mathsf{pf}_j := \mathsf{MT.Open}^{\rho_{\mathsf{MT}}}(\mathsf{td}_j, \mathrm{Dom}\, \mathbb{y}_j) \\ \quad \mathsf{MT.Check}^{\rho_{\mathsf{MT}}}(\mathsf{cm}_j, \mathbb{y}_j, \mathsf{pf}_j) = 1 \end{array} \right\}$$

$((\iota, \vec{\mathsf{icm}}), (\mathbb{x}, \vec{\mathsf{cm}}), (\mathbb{w}, \vec{\mathbb{l}}, \vec{\mathbb{y}}, \vec{\mathsf{itd}}, \vec{\mathsf{td}}))$*, Above, ρ_{MT} is a random oracle in $\mathcal{U}(\sigma)$ derived from ρ. Observe that if $\mathcal{R}$ is in* NP *and S is efficiently computable, then $\mathcal{S}^{\mathcal{U}}$ is in* $\mathsf{NP}^{\mathcal{U}}$.

Monotone relations. This is a minor technical detail which is required for the transformation; informally, it ensures that a cheating non-interactive prover cannot gain an advantage by withholding Merkle authentication paths from the new witness. All the relations that we consider are monotone.

Definition B.4. *We say that an indexed oracle pair relation $\mathcal{R}$ is* monotone *if the relaxed relation $\tilde{\mathcal{R}}$ satisfies the following property. Suppose $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}) \notin \mathcal{L}(\tilde{\mathcal{R}})$ with $\mathbb{y}_j : I_j \to \Sigma$ (recall that a string written over $\Sigma \sqcup \{\perp\}$ can be interpreted as a restriction to a subset of indices). Then $(\mathbb{i}, \mathbb{x}, (\mathbb{y}_1|_{A_1}, \ldots, \mathbb{y}_{\mathsf{n}}|_{A_{\mathsf{n}}})) \notin \mathcal{L}(\tilde{\mathcal{R}})$ for all subsets $A_1, \ldots, A_{\mathsf{n}}$ with $A_j \subseteq I_j$.*

Remark B.5 (black-box access)**.** Given an algorithm $\mathbf{P}_1$, we use $\mathbf{P}_2[\mathbf{P}_1]$ to denote an algorithm $\mathbf{P}_2$ that has black-box access to $\mathbf{P}_1$: it can simulate steps of $\mathbf{P}_1$ and obtain the corresponding outputs of each step. These algorithms may be provers or extractors of IORs, IRs, or non-interactive reductions.

B.2 Non-interactive reductions

Definition B.6 (Non-interactive reduction [BMNW25b])**.** *Let $\mathcal{R}_1$ and $\mathcal{R}_2$ be indexed pair relations (relative to a random oracle). A (preprocessing)* non-interactive reduction of knowledge *from $\mathcal{R}_1$ to $\mathcal{R}_2$ in the random oracle model is a tuple of polynomial-time algorithms $\mathsf{RDX} = (\mathbf{G}_{\mathsf{NI}}(), \mathbf{I}_{\mathsf{NI}}, \mathbf{P}_{\mathsf{NI}}, \mathbf{V}_{\mathsf{NI}})$, of which $\mathbf{I}_{\mathsf{NI}}$, $\mathbf{P}_{\mathsf{NI}}$, $\mathbf{V}_{\mathsf{NI}}$ have access to the same random oracle, with the following syntax.*

- *The generator $\mathbf{G}_{\mathsf{NI}}()$ initializes the public parameters for the reduction. For the purposes of this work, we assume that $\mathbf{G}_{\mathsf{NI}}()$ simply returns λ in unary for any reduction.*
- *The indexer $\mathbf{I}_{\mathsf{NI}}$ is a deterministic algorithm which receives as input the security parameter λ in unary, and index $\mathbb{i}$. It outputs a proving key pk, verification key vk, and new index $\mathbb{i}'$.*
- *The prover $\mathbf{P}_{\mathsf{NI}}$ receives as input proving key pk, an instance $\mathbb{x}$, and witness $\mathbb{w}$. It outputs a proof π and new witness $\mathbb{w}'$.*
- *The verifier $\mathbf{V}_{\mathsf{NI}}$ is a deterministic algorithm[29] which receives as input verification key vk, instance $\mathbb{x}$, and proof π. It outputs a decision b and a new instance $\mathbb{x}'$.*

[29] Proof systems do not typically require verifier to be deterministic. For reductions, deterministic verifiers are useful because it implies that the prover can compute the new instance. Moreover, deterministic verifiers are easy to attain; intuitively, this is because any randomness can be derived from the random oracle.

Definition B.7 (Completeness of non-interactive reduction)**.** *A non-interactive reduction* RDX *is complete if the following holds. For every adversary $\mathcal{A}$,*

$$\Pr\left[\begin{array}{c|r} (\mathbb{i},\mathbb{x},\mathbb{w})\in\mathcal{R}_1^\rho & 1^\lambda\leftarrow\mathbf{G}_{\mathsf{NI}}() \\ \Downarrow & \rho\xleftarrow{\$}\mathcal{U}(\lambda) \\ (\mathbb{i}',\mathbb{x}',\mathbb{w}')\in\mathcal{R}_2^\rho\wedge b=1 & (\mathbb{i},\mathbb{x},\mathbb{w})\leftarrow\mathcal{A}^\rho \\ & (\mathsf{pk},\mathsf{vk},\mathbb{i}')\leftarrow\mathbf{I}_{\mathsf{NI}}^\rho(1^\lambda,\mathbb{i}) \\ & (\pi,\mathbb{w}')\leftarrow\mathbf{P}_{\mathsf{NI}}^\rho(\mathsf{pk},\mathbb{x},\mathbb{w}) \\ & (b,\mathbb{x}')\leftarrow\mathbf{V}_{\mathsf{NI}}^\rho(\mathsf{vk},\mathbb{x},\pi) \end{array}\right]=1\ .$$

Definition B.8 (Failure probability of prover)**.** *A deterministic non-interactive prover $\widetilde{\mathbf{P}}_{\mathsf{NI}}$ of a non-interactive reduction* RDX *has failure probability $\delta_{\widetilde{\mathbf{P}}_{\mathsf{NI}}}$ if the following holds:*

$$\Pr\left[\begin{array}{c|r} (\mathbb{i}',\mathbb{x}',\mathbb{w}')\notin\tilde{\mathcal{R}}_2^\rho\vee b=0 & 1^\lambda\leftarrow\mathbf{G}_{\mathsf{NI}}() \\ & \rho\xleftarrow{\$}\mathcal{U}(\lambda) \\ & (\mathbb{i},\mathbb{x},\pi,\mathbb{w}')\xleftarrow{\mathsf{tr}}\widetilde{\mathbf{P}}_{\mathsf{NI}}^\rho(1^\lambda) \\ & (\mathsf{pk},\mathsf{vk},\mathbb{i}')\leftarrow\mathbf{I}_{\mathsf{NI}}^\rho(1^\lambda,\mathbb{i}) \\ & (b,\mathbb{x}')\xleftarrow{\mathsf{tr}_{\mathbf{V}_{\mathsf{NI}}}}\mathbf{V}_{\mathsf{NI}}^\rho(\mathsf{vk},\mathbb{x},\pi) \end{array}\right]\leq\delta_{\widetilde{\mathbf{P}}_{\mathsf{NI}}}\ .$$

Definition B.9 (Rewinding knowledge-soundness of non-interactive reduction)**.** RDX *is rewinding knowledge sound if there exists a randomized* expected polynomial-time *algorithm $\mathbf{E}_{\mathsf{NI}}$ (the extractor) such that for every non-uniform polynomial-time adversary $\widetilde{\mathbf{P}}_{\mathsf{NI}}$ with failure probability (see Definition B.8) at least $1-\mathrm{negl}(\lambda)$, the following holds.*

$$\Pr\left[\begin{array}{c|r} (\mathbb{i}',\mathbb{x}',\mathbb{w}')\in\tilde{\mathcal{R}}_2^\rho & 1^\lambda\leftarrow\mathbf{G}_{\mathsf{NI}}() \\ \wedge b=1 & \rho\xleftarrow{\$}\mathcal{U}(\lambda) \\ \wedge(\mathbb{i},\mathbb{x},\mathbb{w})\notin\tilde{\mathcal{R}}_1^\rho & (\mathbb{i},\mathbb{x},\pi,\mathbb{w}')\xleftarrow{\mathsf{tr}}\widetilde{\mathbf{P}}_{\mathsf{NI}}^\rho(1^\lambda) \\ & (\mathsf{pk},\mathsf{vk},\mathbb{i}')\leftarrow\mathbf{I}_{\mathsf{NI}}^\rho(1^\lambda,\mathbb{i}) \\ & (b,\mathbb{x}')\xleftarrow{\mathsf{tr}_{\mathbf{V}_{\mathsf{NI}}}}\mathbf{V}_{\mathsf{NI}}^\rho(\mathsf{vk},\mathbb{x},\pi) \\ & \mathbb{w}\leftarrow\mathbf{E}_{\mathsf{NI}}[\widetilde{\mathbf{P}}_{\mathsf{NI}}](\mathbb{i},\mathbb{x},\pi,\mathsf{tr},\mathsf{tr}_{\mathbf{V}_{\mathsf{NI}}},\mathbb{w}') \end{array}\right]\leq\mathrm{negl}(\lambda)\ .$$

Definition B.10 (Non-interactive reduction of knowledge (NIRK))**.** *A NIRK is a non-interactive reduction that is both complete and rewinding knowledge sound.*

B.2.1 Interactive reductions

Looking ahead, we will show how to compile a given IOR to an interactive reduction in the ROM, replacing the oracles with Merkle commitments. An interactive reduction is an IOR where the prover can only send non-oracle messages.

For completeness, we define the state-restoration game for the interactive reduction (IR) between committed relations with random-oracle access, which can be naturally extended to obtain the definition of state-restoration knowledge soundness for IRs. These definitions are nearly identical to that of IORs, except that the index $\mathbb{i}$ is replace with $(\iota,\vec{\mathsf{ic}\mathbb{m}})$, instance $\mathbb{x}$ is replaced by $(\mathbb{x},\vec{\mathsf{c}\mathbb{m}})$, there is no instance oracle, the prover's oracles $\vec{\Pi}$ are replaced by $(\vec{\mathsf{pc}\mathbb{m}},\vec{\mathsf{an}\mathbb{s}},\vec{\mathsf{pf}})$, and the witness $\mathbb{w}$ is replaced by $(\mathbb{w},\vec{\mathbb{l}},\vec{\mathbb{y}},\vec{\mathsf{it}\mathbb{d}},\vec{\mathsf{pt}\mathbb{d}},\vec{\mathsf{t}\mathbb{d}})$.

We use the following notation for the state-restoration game for IRs:

$$((\iota,\vec{\mathsf{ic}\mathbb{m}}),(\mathbb{x},\vec{\mathsf{c}\mathbb{m}}),(\vec{\mathsf{pc}\mathbb{m}},\vec{\mathsf{an}\mathbb{s}},\vec{\mathsf{pf}},\vec{\mathsf{m}\mathbb{sg}}),(\mathbb{w},\vec{\mathbb{l}},\vec{\mathbb{y}},\vec{\mathsf{it}\mathbb{d}},\vec{\mathsf{pt}\mathbb{d}},\vec{\mathsf{t}\mathbb{d}}),\vec{A},\vec{r})\xleftarrow{\mathsf{tr}^{\mathsf{sr}};\mathsf{tr}_{\mathsf{MT}}}\mathsf{Game}_{\mathsf{IR}}(\vec{\rho},\mathbf{P}_{\mathsf{IR}}^{\rho_{\mathsf{MT}}})\ ,$$

where $\mathsf{tr}_{\mathsf{MT}}$ denotes is the trace of the input-output pairs of queries to the random oracle for Merkle commitments ρ_{MT}. This notation is verbose so we abbreviate it as follows:

$$(\mathbb{i}_{\mathsf{IR}}, \mathbb{x}_{\mathsf{IR}}, \alpha_{\mathsf{IR}}, \mathbb{w}_{\mathsf{IR}}, \vec{A}, \vec{r}) \xleftarrow{\mathsf{tr}^{\mathsf{sr}};\mathsf{tr}_{\mathsf{MT}}} \mathsf{Game}_{\mathsf{IR}}(\vec{\rho}, \mathbf{P}_{\mathsf{IR}}^{\rho_{\mathsf{MT}}}) \ .$$

B.3 Compilation with BCS transform gives the main theorem

In this section we describe the transformation from [BMNW25b] and show that upon input an SRKS IOR it outputs a rewinding knowledge sound NIRK. We prove this by following the modular approach outlined in [CY24, Section 26.1.2]. In more detail, this variant of the BCS transform $T = \mathsf{FS} \circ T_i$ consists of two transformations in the random oracle model (ROM):

- T_i takes an interactive oracle reduction from $\mathcal{R}_1$ to $\mathcal{R}_2$ and returns an *interactive reduction* from $\mathsf{Com}[\mathcal{R}_1]$ to $\mathsf{Com}[\mathcal{R}_2]$. In Section B.4 we show that if IOR is rewinding SRKS, then so is the interactive reduction.
- The Fiat–Shamir transformation FS takes an interactive reduction and returns a non-interactive reduction between the same relations. In Section B.5 we show that if the interactive reduction is rewinding SRKS, then the non-interactive reduction is rewinding knowledge sound.

This leads to the following lemma.

Lemma B.11 (SRKS implies NIRK)**.** *Let* IOR *be an IOR from an indexed oracle relation* $\mathcal{R}_1$ *to an indexed oracle relation* $\mathcal{R}_2$ *that is rewinding SRKS. Then, the non-interactive reduction obtained by applying the BCS transform ([CY24, Construction 25.1.1]) to the IOR is a NIRK with rewinding knowledge soundness.*

Proof. The proof follows from the Lemma B.12 and Lemma B.13 described in the following sections. □

The lemma above and (Lemma A.6) immediately imply our main lemma:

Lemma 4.5 (Compilation of RBRTE IORs to NIRK)**.** *Let* IOR *be an RBRTE IOR from* $\mathcal{R}$ *to* $\mathcal{R}'$*. Then the non-interactive reduction obtained by applying (a variant of) the BCS transform (from [BMNW25b, Appendix B]) to* IOR *is a NIRK with rewinding knowledge soundness.*

B.4 Replacing oracles with Merkle commitments

The first part of the BCS transformation is to replace the oracles in the IOR with Merkle commitments to obtain an interactive reduction (IR).

Lemma B.12. *Let* $\mathbf{E}_{\mathsf{IOR}}$ *be an IOR state-restoration rewinding extractor. There exists an IOR state-restoration prover* $\widetilde{\mathbf{P}}_{\mathsf{IOR}}[\cdot]$ *and IR state-restoration rewinding extractor* $\mathbf{E}_{\mathsf{IR}} := \mathbf{E}_{\mathsf{IR}}[\mathbf{E}_{\mathsf{IOR}}]$ *such that, for every* t_{sr}*-move*

IR state-restoration prover $\widetilde{\mathbf{P}}_{\mathsf{IR}}$ that makes at most t_{MT} queries to ρ_{MT}, $\widetilde{\mathbf{P}}_{\mathsf{IOR}}[\widetilde{\mathbf{P}}_{\mathsf{IR}}]$ makes at most t_{sr} moves:

$$
\begin{aligned}
&\Pr\left[\begin{array}{c|l}
\begin{array}{c}|(\mathbb{i},\mathbb{x})|\leq n\\ (\mathbb{i}_{\mathsf{IR}},\mathbb{x}_{\mathsf{IR}},\mathbb{w}_{\mathsf{IR}})\notin \mathsf{Com}[\tilde{\mathcal{R}}_1]^{\rho_{\mathsf{MT}}}\\ (\mathbb{i}'_{\mathsf{IR}},\mathbb{x}'_{\mathsf{IR}},\mathbb{w}'_{\mathsf{IR}}x)\in \mathsf{Com}[\tilde{\mathcal{R}}_2]^{\rho_{\mathsf{MT}}}\end{array} &
\begin{array}{l}\rho_{\mathsf{MT}}\xleftarrow{\$}\mathcal{U}(\lambda)\\ \rho_{\mathsf{sr}}\xleftarrow{\$}\mathcal{U}(\lambda)\\ (\mathbb{i}_{\mathsf{IR}},\mathbb{x}_{\mathsf{IR}},\alpha_{\mathsf{IR}},\mathbb{w}'_{\mathsf{IR}},\vec{r})\xleftarrow{\mathsf{tr}_{\mathsf{IR}};\mathsf{tr}_{\mathsf{MT}}}\mathsf{Game}_{\mathsf{IR}}(\rho_{\mathsf{sr}},\widetilde{\mathbf{P}}_{\mathsf{IR}}^{\rho_{\mathsf{MT}}})\\ \mathbb{w}_{\mathsf{IR}}\leftarrow\mathbf{E}_{\mathsf{IR}}[\widetilde{\mathbf{P}}_{\mathsf{IR}}](\mathbb{i}_{\mathsf{IR}},\mathbb{x}_{\mathsf{IR}},\alpha_{\mathsf{IR}},\mathbb{w}'_{\mathsf{IR}},\mathsf{tr}_{\mathsf{IR}},\mathsf{tr}_{\mathsf{MT}})\\ (\mathsf{pk},\mathsf{vk},\mathbb{i}'_{\mathsf{IR}})\leftarrow\mathbf{I}_{\mathsf{IR}}(1^\lambda,\mathbb{i}_{\mathsf{IR}})\\ \mathbb{x}'_{\mathsf{IR}}\leftarrow\mathbf{V}_{\mathsf{IR}}^{\rho_{\mathsf{MT}}}(\mathsf{vk},\mathbb{x}_{\mathsf{IR}},\alpha_{\mathsf{IR}},\vec{r})\end{array}
\end{array}\right]\\
\leq\ &\Pr\left[\begin{array}{c|l}
\begin{array}{c}|(\mathbb{i},\mathbb{x})|\leq n\\ (\mathbb{i},\mathbb{x},\vec{\mathbb{y}},\mathbb{w})\notin\tilde{\mathcal{R}}_1\\ (\mathbb{i}',\mathbb{x}',\vec{\mathbb{y}}',\mathbb{w}')\in\tilde{\mathcal{R}}_2\end{array} &
\begin{array}{l}\rho_{\mathsf{MT}}\xleftarrow{\$}\mathcal{U}(\lambda)\\ \rho_{\mathsf{sr}}\xleftarrow{\$}\mathcal{U}(\lambda)\\ (\mathbb{i},\mathbb{x},\vec{\mathbb{y}},\vec{\Pi},\mathbb{w}',\vec{r})\xleftarrow{\mathsf{tr}^{\mathsf{sr}}}\mathsf{Game}_{\mathsf{IOR}}^{\mathsf{sr}}(\rho_{\mathsf{sr}},\widetilde{\mathbf{P}}_{\mathsf{IOR}}[\widetilde{\mathbf{P}}_{\mathsf{IR}}]^{\rho_{\mathsf{MT}}})\\ \mathbb{w}\leftarrow\mathbf{E}_{\mathsf{IOR}}[\widetilde{\mathbf{P}}_{\mathsf{IOR}}](\mathbb{i},\mathbb{x},\vec{\mathbb{y}},\vec{\Pi},\mathbb{w}',\vec{r},\mathsf{tr}^{\mathsf{sr}})\\ (\iota,\mathbb{l},\mathbb{i}')\leftarrow\mathbf{I}_{\mathsf{IOR}}(1^\lambda,\mathbb{i})\\ (\mathbb{x}',\vec{\mathbb{y}}')\leftarrow\mathbf{V}_{\mathsf{IOR}}^{\mathbb{l},\vec{\mathbb{y}},\vec{\Pi}}(\iota,\mathbb{x},\vec{r})\end{array}
\end{array}\right]\\
&+\kappa_{\mathsf{MT}}(\lambda,t_{\mathsf{MT}},\mathsf{L},1+\mathsf{n}+\mathsf{k}).
\end{aligned}
\tag{17}
$$

where k is the number of rounds in the IOR, $\mathsf{n}=|\vec{\mathbb{y}}|$, and L is the maximum length of any oracle sent by the prover in the IOR.

Proof of Lemma B.12. The proof follows straightforwardly from the proof of [BMNW25b, Lemma B.6]. The only difference is that our extractor $\mathbf{E}_{\mathsf{IR}}$ has black-box access to the IR state-restoration prover $\widetilde{\mathbf{P}}_{\mathsf{IR}}$. However, $\mathbf{E}_{\mathsf{IR}}$ simply forwards the black box to $\widetilde{\mathbf{P}}_{\mathsf{IOR}}$, which is then used as a black box by $\mathbf{E}_{\mathsf{IOR}}$, and does not use the $\widetilde{\mathbf{P}}_{\mathsf{IR}}$ in any other way. □

IOR state-restoration prover. Given a state-restoration prover $\widetilde{\mathbf{P}}_{\mathsf{IR}}$ for $T_i[\mathsf{IOR}]$, the state-restoration prover $\widetilde{\mathbf{P}}_{\mathsf{IOR}}[\widetilde{\mathbf{P}}_{\mathsf{IR}}]$ for IOR is defined in algorithm 1 (this is identical to the construction in Arc, and generalizes the construction in the SNARGs book [CY24, Construction 25.2.6] to IORs and IRs, but we keep it for completeness).

Rewinding extractor. The rewinding state-restoration IR extractor $\mathbf{E}_{\mathsf{IR}}$ is defined in algorithm 2. This differs from the construction in Arc since the $\mathbf{E}_{\mathsf{IR}}$ has black-box access to the IR state-restoration prover $\widetilde{\mathbf{P}}_{\mathsf{IR}}$, which allows it to view every single random challenge that $\widetilde{\mathbf{P}}_{\mathsf{IR}}$ receives from the IR state-restoration game. This generalizes the construction in the SNARGs book [CY24, Construction 26.1.5] to IORs and IRs.

B.5 Fiat–Shamir transformation

Finally, to obtain a NIRK from the IR, we apply the Fiat–Shamir transformation, which is the second step of the BCS transformation.

Lemma B.13. *Let $\mathbf{E}_{\mathsf{IR}}$ be an IR rewinding state-restoration extractor. There exists an IR state-restoration prover $\widetilde{\mathbf{P}}_{\mathsf{IR}}[\cdot]$ such that the following holds and a rewinding non-interactive reduction extractor $\mathbf{E}_{\mathsf{NI}}:=\mathbf{E}_{\mathsf{NI}}[\mathbf{E}_{\mathsf{IR}}]$ such that, for every non-interactive reduction prover $\widetilde{\mathbf{P}}_{\mathsf{NI}}$ that makes at most t_{MT} queries to ρ_{MT}*

Algorithm 1 IOR state-restoration prover $\widetilde{\mathbf{P}}_{\mathsf{IOR}}$ from IR state-restoration prover $\widetilde{\mathbf{P}}_{\mathsf{IR}}$.

$\widetilde{\mathbf{P}}_{\mathsf{IOR}}[\widetilde{\mathbf{P}}_{\mathsf{IR}}]^{\rho_{\mathsf{MT}}}$:

1. Initialize empty trace for the query-answer pairs of ρ_{MT}: $\mathsf{tr}_{\mathsf{MT}} \leftarrow \emptyset$.
2. Simulate $\widetilde{\mathbf{P}}_{\mathsf{IR}}$ until it halts:
   (a) If $\widetilde{\mathbf{P}}_{\mathsf{IR}}$ makes a query to ρ_{MT}, make the same query to ρ_{MT}, return its answer to $\widetilde{\mathbf{P}}_{\mathsf{IR}}$, and append the query-answer pair to $\mathsf{tr}_{\mathsf{MT}}$.
   (b) If $\widetilde{\mathbf{P}}_{\mathsf{IR}}$ sends a move $(\mathbb{i}_{\mathsf{IR}}, \mathbb{x}_{\mathsf{IR}} := (\mathbb{x}, \vec{\mathsf{cm}}), (\vec{\mathsf{pcm}}[:i], \vec{\mathsf{msg}}[:i]))$ with $i \in [\mathsf{k}]$:
      i. Extract commitment openings:
      $$((\vec{\mathbb{I}} \,\|\, \vec{\mathbb{y}} \,\|\, \vec{\Pi}[:i]), (\vec{\mathsf{itd}} \,\|\, \vec{\mathsf{td}} \,\|\, \vec{\mathsf{ptd}}[:i])) \leftarrow \mathsf{MT.MultiExtract}((\vec{\mathsf{icm}} \,\|\, \vec{\mathsf{cm}} \,\|\, \vec{\mathsf{pcm}}[:i]), \mathsf{tr}_{\mathsf{MT}})$$
      ii. Send the move $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \vec{\Pi}[:i])$ to the IOR state-restoration game.
      iii. Receive the challenge r_i from the game.
      iv. Return r_i to $\widetilde{\mathbf{P}}_{\mathsf{IR}}$.
      v. Reset the trace $\mathsf{tr}_{\mathsf{MT}} \leftarrow \emptyset$.
3. $\widetilde{\mathbf{P}}_{\mathsf{IR}}$ halts and outputs $(\mathbb{i}_{\mathsf{IR}} := (\iota, \vec{\mathsf{icm}}), \mathbb{x}_{\mathsf{IR}} := (\mathbb{x}, \vec{\mathsf{cm}}), (\vec{\mathsf{pcm}}, \vec{\mathsf{ans}}, \vec{\mathsf{pf}}, \vec{\mathsf{msg}}), (\mathbb{w}', \vec{\mathbb{y}}', \vec{\mathsf{td}}'))$.
4. Extract commitment openings:
$$((\vec{\mathbb{I}} \,\|\, \vec{\mathbb{y}} \,\|\, \vec{\Pi}), (\vec{\mathsf{itd}} \,\|\, \vec{\mathsf{td}} \,\|\, \vec{\mathsf{ptd}})) \leftarrow \mathsf{MT.MultiExtract}((\vec{\mathsf{icm}} \,\|\, \vec{\mathsf{cm}} \,\|\, \vec{\mathsf{pcm}}), \mathsf{tr}_{\mathsf{MT}})$$
5. Define $A[j] := \mathrm{Dom}(\mathbb{y}'[j])$ for all $j \in [\mathsf{n}']$.
6. Output $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \vec{\Pi}, \mathbb{w}', \vec{A})$.

Algorithm 2 Rewinding extractor $\mathbf{E}_{\mathsf{IR}}$ from IR state-restoration prover $\widetilde{\mathbf{P}}_{\mathsf{IR}}$.

$\mathbf{E}_{\mathsf{IR}}[\widetilde{\mathbf{P}}_{\mathsf{IR}}](\mathbb{i}_{\mathsf{IR}}, \mathbb{x}_{\mathsf{IR}}, \alpha_{\mathsf{IR}} := (\vec{\mathsf{pcm}}, \vec{\mathsf{ans}}, \vec{\mathsf{pf}}, \vec{\mathsf{msg}}), \mathbb{w}'_{\mathsf{IR}}, \mathsf{tr}_{\mathsf{IR}}, \mathsf{tr}_{\mathsf{MT}})$:

1. Set $\mathsf{tr}^{\mathsf{sr}} := \mathsf{tr}^{\mathsf{sr}}(\mathsf{tr}_{\mathsf{IR}}, \mathsf{tr}_{\mathsf{MT}})$, where $\mathsf{tr}^{\mathsf{sr}}$ is the deterministic algorithm which computes the IOR state-restoration trace corresponding to $\widetilde{\mathbf{P}}_{\mathsf{IOR}}[\widetilde{\mathbf{P}}_{\mathsf{IR}}]$, as defined in Algorithm 1.
2. Extract commitment openings:
$$((\vec{\mathbb{I}} \,\|\, \vec{\mathbb{y}} \,\|\, \vec{\Pi}), (\vec{\mathsf{itd}} \,\|\, \vec{\mathsf{td}} \,\|\, \vec{\mathsf{ptd}})) \leftarrow \mathsf{MT.MultiExtract}((\vec{\mathsf{icm}} \,\|\, \vec{\mathsf{cm}} \,\|\, \vec{\mathsf{pcm}}), \mathsf{tr}_{\mathsf{MT}})$$
3. Define $\widetilde{\mathbf{P}}_{\mathsf{IOR}} := \widetilde{\mathbf{P}}_{\mathsf{IOR}}[\widetilde{\mathbf{P}}_{\mathsf{IR}}]$ as the IOR state-restoration prover from Algorithm 1.
4. Run the IOR state-restoration extractor: $\mathbb{w} := \mathbf{E}_{\mathsf{IOR}}[\widetilde{\mathbf{P}}_{\mathsf{IOR}}](\mathbb{i} := (\mathbb{i}_{\mathsf{IR}}, \vec{\mathbb{I}}), \mathbb{x}, \vec{\mathbb{y}}, \vec{\Pi}, \mathbb{w}', \mathsf{tr}^{\mathsf{sr}})$.
5. Output $\mathbb{w}_{\mathsf{IR}} := (\mathbb{w}, \vec{\mathbb{y}}, \vec{\mathsf{itd}}, \vec{\mathsf{td}}, \vec{\mathsf{ptd}})$.

and t_{FS} queries to ρ_{FS}, $\widetilde{\mathbf{P}}_{\mathsf{IR}}[\widetilde{\mathbf{P}}_{\mathsf{NI}}]$ makes at most t_{FS} moves:

$$\Pr\left[\begin{array}{c|l} & 1^\lambda \leftarrow \mathbf{G}_{\mathsf{NI}}() \\ |(\mathbb{i},\mathbb{x})|\leq n & \rho_{\mathsf{MT}} \xleftarrow{\$} \mathcal{U}(\lambda) \\ (\mathbb{i},\mathbb{x},\mathbb{w}) \notin \mathsf{Com}[\tilde{\mathcal{R}}_1]^{\rho_{\mathsf{MT}}} & \rho_{\mathsf{FS}} \xleftarrow{\$} \mathcal{U}(\lambda) \\ (\mathbb{i}',\mathbb{x}',\mathbb{w}') \in \mathsf{Com}[\tilde{\mathcal{R}}_2]^{\rho_{\mathsf{MT}}} & \rho := (\rho_{\mathsf{MT}},\rho_{\mathsf{FS}}) \\ b=1 & (\mathbb{i},\mathbb{x},\pi,\mathbb{w}') \xleftarrow{\mathsf{tr}} \widetilde{\mathbf{P}}^{\rho}_{\mathsf{NI}} \\ & \mathbb{w} \leftarrow \mathbf{E}_{\mathsf{NI}}[\widetilde{\mathbf{P}}_{\mathsf{NI}}](\mathbb{i},\mathbb{x},\pi,\mathbb{w}',\mathsf{tr}) \\ & (\mathsf{pk},\mathsf{vk},\mathbb{i}') \leftarrow \mathbf{I}^{\rho}_{\mathsf{NI}}(1^\lambda,\mathbb{i}) \\ & (b,\mathbb{x}') \leftarrow \mathbf{V}^{\rho}_{\mathsf{NI}}(\mathsf{vk},\mathbb{x},\pi) \end{array}\right] \tag{18}$$

$$\leq \Pr\left[\begin{array}{c|l} & \rho_{\mathsf{MT}} \xleftarrow{\$} \mathcal{U}(\lambda) \\ |(\mathbb{i},\mathbb{x})|\leq n & \rho_{\mathsf{sr}} \xleftarrow{\$} \mathcal{U}(\lambda) \\ (\mathbb{i},\mathbb{x},\mathbb{w}) \notin \mathsf{Com}[\tilde{\mathcal{R}}_1]^{\rho_{\mathsf{MT}}} & (\mathbb{i},\mathbb{x},\alpha,\mathbb{w}',\vec{r}) \xleftarrow{\mathsf{tr}} \mathsf{Game}_{\mathsf{IR}}(\rho_{\mathsf{sr}},\widetilde{\mathbf{P}}_{\mathsf{IR}}[\widetilde{\mathbf{P}}_{\mathsf{NI}}]^{\rho_{\mathsf{MT}}}) \\ (\mathbb{i}',\mathbb{x}',\mathbb{w}') \in \mathsf{Com}[\tilde{\mathcal{R}}_2]^{\rho_{\mathsf{MT}}} & \mathbb{w} \leftarrow \mathbf{E}_{\mathsf{IR}}[\widetilde{\mathbf{P}}_{\mathsf{IR}}](\mathbb{i},\mathbb{x},\alpha,\mathbb{w}',\mathsf{tr}) \\ b=1 & (\mathsf{pk},\mathsf{vk},\mathbb{i}') \leftarrow \mathbf{I}_{\mathsf{IR}}(1^\lambda,\mathbb{i}) \\ & (b,\mathbb{x}') \leftarrow \mathbf{V}^{\rho_{\mathsf{MT}}}_{\mathsf{IR}}(\mathsf{vk},\mathbb{x},\alpha,\vec{r}) \end{array}\right] + \frac{t_{\mathsf{sr}}^2}{2^\lambda}.$$

Proof. This follows directly from [CY24, Theorem 16.1.1], which transforms interactive arguments with rewinding SRKS into non-interactive arguments with rewinding knowledge soundness. □

C Proof of Lemma 4.7

In this section we define the sequential composition of two RBRTE IORs and show that the composition is also RBRTE with related parameters (Lemma 4.7). We begin by formally defining the composition of IORs.

IOR sequential composition. Let $\mathsf{IOR} = (\mathbf{I}_{\mathsf{IOR}}, \mathbf{P}_{\mathsf{IOR}}, \mathbf{V}_{\mathsf{IOR}})$ be a k-round IOR from indexed oracle relation $\mathcal{R}$ to indexed oracle relation $\mathcal{R}'$ and $\mathsf{IOR}' = (\mathbf{I}'_{\mathsf{IOR}}, \mathbf{P}'_{\mathsf{IOR}}, \mathbf{V}'_{\mathsf{IOR}})$ be a k'-round IOR from $\mathcal{R}'$ to indexed oracle relation $\mathcal{R}''$. Then the sequential composition of IOR and IOR' (denoted by $\mathsf{IOR}'' := \mathsf{IOR}' \circ \mathsf{IOR}$) is an IOR from $\mathcal{R}$ to $\mathcal{R}''$ such that

- The indexer $\mathbf{I}''_{\mathsf{IOR}}$ executes $\mathbf{I}_{\mathsf{IOR}}(\mathbb{i})$ obtaining a short index ι, index oracles $\vec{\mathbb{I}}$, and a new index $\mathbb{i}'$ for $\mathcal{R}'$. Then, $\mathbf{I}''_{\mathsf{IOR}}$ executes $\mathbf{I}'_{\mathsf{IOR}}(\mathbb{i}')$, obtaining a short index ι', index oracles $\vec{\mathbb{I}}'$, and a new index $\mathbb{i}''$ for $\mathcal{R}''$. It outputs $\iota'' := (\iota, \iota')$, $\vec{\mathbb{I}}'' := (\vec{\mathbb{I}}, \vec{\mathbb{I}}')$, and the new index $\mathbb{i}''$.
- The prover $\mathbf{P}''_{\mathsf{IOR}}$ receives as input index $(\mathbb{i}, \mathbb{i}')$, instance $\mathbb{x}$, instance oracles $\vec{\mathbb{y}}$, and witness $\mathbb{w}$.
  - Firstly, $\mathbf{P}''_{\mathsf{IOR}}$ simulates $\mathbf{P}_{\mathsf{IOR}}$'s interaction with $\mathbf{V}_{\mathsf{IOR}}$ using the the inputs $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w})$. It engages in k rounds of interaction with the verifier $\mathbf{V}_{\mathsf{IOR}}$. In the i-th round, $\mathbf{P}_{\mathsf{IOR}}$ sends a proof oracle Π_i and a non-oracle message msg_i, and then receives a random challenge ρ_i from $\mathbf{V}_{\mathsf{IOR}}$. At the end of this interaction, $\mathbf{P}_{\mathsf{IOR}}$ outputs $(\mathbb{x}', \vec{\mathbb{y}}', \mathbb{w}')$, which are returned back to $\mathbf{P}''_{\mathsf{IOR}}$.
  - Next, $\mathbf{P}''_{\mathsf{IOR}}$ simulates $\mathbf{P}'_{\mathsf{IOR}}$'s interaction with $\mathbf{V}'_{\mathsf{IOR}}$ using the the inputs $(\mathbb{i}', \mathbb{x}', \vec{\mathbb{y}}', \mathbb{w}')$. It engages in k' rounds of interaction with the verifier $\mathbf{V}'_{\mathsf{IOR}}$. In the i-th round, $\mathbf{P}'_{\mathsf{IOR}}$ sends a proof oracle Π'_i and a non-oracle message msg'_i, and then receives a random challenge ρ'_i from $\mathbf{V}'_{\mathsf{IOR}}$. At the end of this interaction, $\mathbf{P}'_{\mathsf{IOR}}$ outputs $(\mathbb{x}'', \vec{\mathbb{y}}'', \mathbb{w}'')$, which are returned back to $\mathbf{P}''_{\mathsf{IOR}}$.
  - Finally, $\mathbf{P}''_{\mathsf{IOR}}$ outputs $(\mathbb{x}'', \vec{\mathbb{y}}'', \mathbb{w}'')$.
- The verifier $\mathbf{V}''_{\mathsf{IOR}}$ receives as input the short indices (ι, ι'), instance $\mathbb{x}$, oracle access to index oracles $(\vec{\mathbb{I}}, \vec{\mathbb{I}}')$, and oracle access to the instance oracles $(\vec{\mathbb{y}}, \vec{\mathbb{y}}')$.
  - Firstly, $\mathbf{V}''_{\mathsf{IOR}}$ simulates $\mathbf{V}_{\mathsf{IOR}}$'s interaction with $\mathbf{P}_{\mathsf{IOR}}$ using the the inputs $(\iota, \mathbb{x})$ and oracle access to $\vec{\mathbb{I}}$ and $\vec{\mathbb{y}}$. It engages in k rounds of interaction with $\mathbf{P}''_{\mathsf{IOR}}$ (executing $\mathbf{P}_{\mathsf{IOR}}$). In each round, $\mathbf{V}_{\mathsf{IOR}}$ receives oracle access to a proof oracle Π_i and a non-oracle message msg_i, and then sends a uniformly random challenge ρ_i. At the end of the interaction $\mathbf{V}_{\mathsf{IOR}}$ outputs $(\mathbb{x}', \vec{\mathbb{y}}')$, which are returned back to $\mathbf{V}''_{\mathsf{IOR}}$.
  - Next, $\mathbf{V}''_{\mathsf{IOR}}$ simulates $\mathbf{V}'_{\mathsf{IOR}}$'s interaction with $\mathbf{P}'_{\mathsf{IOR}}$ using the the inputs $(\iota', \mathbb{x}')$ and oracle access to $\vec{\mathbb{I}}'$ and $\vec{\mathbb{y}}'$. It engages in k rounds of interaction with $\mathbf{P}''_{\mathsf{IOR}}$ (executing $\mathbf{P}_{\mathsf{IOR}}$). In each round, $\mathbf{V}_{\mathsf{IOR}}$ receives oracle access to a proof oracle Π_i and a non-oracle message msg_i, and then sends a uniformly random challenge ρ_i. At the end of the interaction $\mathbf{V}'_{\mathsf{IOR}}$ outputs $(\mathbb{x}'', \vec{\mathbb{y}}'')$, which are returned back to $\mathbf{V}''_{\mathsf{IOR}}$.
  - Finally, $\mathbf{V}''_{\mathsf{IOR}}$ outputs $(\mathbb{x}'', \vec{\mathbb{y}}'')$.

We now prove our composition lemma that we stated in Section 4.4.

Lemma 4.7 (RBRTE IOR composition)**.** *Let* IOR *be an* $(\vec{a}, \phi, \theta)$*-RBRTE* IOR *from* $\mathcal{R}$ *to* $\mathcal{R}'$ *and* IOR' *be an* $(\vec{a}', \phi', \theta')$*-RBRTE* IOR *from* $\mathcal{R}'$ *to* $\mathcal{R}''$*. Then the sequential composition* $\mathsf{IOR}'' := \mathsf{IOR}' \circ \mathsf{IOR}$ *is a* $(\vec{a} \parallel \vec{a}', \phi \parallel \phi', \theta \wedge \theta')$*-RBRTE IOR from* $\mathcal{R}$ *to* $\mathcal{R}''$*.*[30]

Furthermore, the efficiency metrics of $\mathsf{IOR}' \circ \mathsf{IOR}$ *are the sum of the round complexities, oracle lengths, message lengths, query complexities, prover times, and verifier times.*

[30] The "concatenation" of vertical predicates $\phi \parallel \phi'$ is a predicate whose round-by-round representation is the concatenation of the round-by-round representation of ϕ and ϕ'.

Proof. The claimed efficiency follows by inspection. Thus, to complete the proof we must describe a $\mathbf{E}^{\mathrm{Tree}''}$ which, given $(\vec{a} \,\|\, \vec{a}', \phi \,\|\, \phi', \theta \,\|\, \theta')$-constrained transcript tree T'', outputs witness $\mathbb{w}$ s.t. $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \in \mathcal{R}$. On input T'', $\mathbf{E}^{\mathrm{Tree}''}$ engages in the following steps.

- For each node $v \in \left[\prod_{i=1}^{\mathsf{k}} a_i\right]$ in the k-th level of the tree, derive index $\mathbb{i}'_v$ and instance $(\mathbb{x}'_v, \vec{\mathbb{y}}'_v)$ as would have been outputted by the honest prover $\mathbf{P}_{\mathsf{IOR}}$ and verifier $\mathbf{V}_{\mathsf{IOR}}$ at the end of the interaction in IOR. Append $\mathbb{i}'_v, (\mathbb{x}'_v, \vec{\mathbb{y}}'_v)$ to v and execute $\mathbf{E}^{\mathrm{Tree}'}$ on the subtree consisting of v and all of its decedents. By definition, this is a $(\vec{a}', \phi', \theta', \mathbb{i}'_v, \mathbb{x}'_v, \vec{\mathbb{y}}'_v)$-constrained transcript tree for IOR' and thus we are guaranteed that $\mathbf{E}^{\mathrm{Tree}'}$ outputs $\mathbb{w}'_v$ such that $(\mathbb{i}'_v, \mathbb{x}'_v, \vec{\mathbb{y}}'_v, \mathbb{w}'_v) \in \mathcal{R}'$. $\mathbf{E}^{\mathrm{Tree}''}$ concatenates $\mathbb{w}'_v$ to the label of v.
- Execute $\mathbf{E}^{\mathrm{Tree}''}$ on the tree truncated after the first k levels of T'. By definition and the validity of the witnesses derived above, this is a $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$-constrained transcript tree and thus $\mathbf{E}^{\mathrm{Tree}}$ outputs a witness $\mathbb{w}$ such that $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \in \mathcal{R}$.
- Output $\mathbb{w}$.

Clearly, $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \in \mathcal{R}$ as desired, by the argument above. □

Remark C.1 (parallel composition of IORs)**.** In a similar vein as [KP23], we can define *parallel composition* of IORs: given indexed oracle relations $\mathcal{R}_1, \mathcal{R}'_1, \mathcal{R}_2, \mathcal{R}'_2$, an IOR $\mathsf{IOR}_1 = (\mathbf{I}_1, \mathbf{P}_1, \mathbf{V}_1)$ from $\mathcal{R}_1$ to $\mathcal{R}'_1$ and an IOR $\mathsf{IOR}_2 = (\mathbf{I}_2, \mathbf{P}_2, \mathbf{V}_2)$ from $\mathcal{R}_2$ to $\mathcal{R}'_2$, the parallel composition $\mathsf{IOR} := \mathsf{IOR}_1 \,\|\, \mathsf{IOR}_2$ is an IOR from $\mathcal{R}_1 \times \mathcal{R}_2$ to $\mathcal{R}'_1 \times \mathcal{R}'_2$ such that

- The indexer executes $\mathbf{I}_1$ and $\mathbf{I}_2$.
- The prover simulates $\mathbf{P}_1$'s and $\mathbf{P}_2$'s interactions with $\mathbf{V}_1$ and $\mathbf{V}_2$ respectively.
- The verifier simulates $\mathbf{V}_1$'s and $\mathbf{V}_2$'s interactions with $\mathbf{P}_1$ and $\mathbf{P}_2$ respectively.

The efficiency and constraint-tree-extractability of IOR can be analyzed in a straightforward manner, similar to Lemma 4.7, and we omit the details.

A framework to build IOPs by composing IORs. In an *interactive oracle proof* (IOP), a prover presents a verifier with some index $\mathbb{i}$ [31] instance $(\mathbb{x}, \vec{\mathbb{y}})$ (composed of oracle and non-oracle part) and tries to convince through an interaction in which it sends oracle messages that there exists a witness $\mathbb{w}$ s.t. $(\mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \in \mathcal{R}$ for some (NP) relation $\mathcal{R}$. In an interactive oracle *reduction* (IO*R*), a prover presents a verifier with some instance $(\mathbb{x}, \vec{\mathbb{y}})$ (composed of oracle and non-oracle part) and tries to convince through an interaction in which it sends oracle messages that there exists a witness $\mathbb{w}$ s.t. $(\mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \in \mathcal{R}$ for some (NP) relation $\mathcal{R}$, *if, there exists* $\mathbb{w}'$ *such that for some other index-instance pair* $(\mathbb{i}', \mathbb{x}', \vec{\mathbb{y}}')$ *and relation* $\mathcal{R}'$, $(\mathbb{i}', \mathbb{x}', \vec{\mathbb{y}}', \mathbb{w}) \in \mathcal{R}'$.

Thus, it is not difficult to see that if $\mathcal{R}'$ is set to be the "trivial" relation, then an IOR *is* an IOP. In particular, let $\mathcal{R}_{\mathsf{true}}$ be the indexed oracle relation where, for all index-instance pair $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$, it holds that $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \perp) \in \mathcal{R}_{\mathsf{true}}$. Thus, it is trivial to see that:

Claim C.2. *If* IOR *is an IOR is from* $\mathcal{R}$ *to* $\mathcal{R}_{\mathsf{true}}$ *that has* $\mathrm{negl}(\lambda)$ *SRKS soundness, then* IOR *is a SRKS IOP with the same efficiency parameters (i.e. round complexity, query complexity . . .) parameters.*

Where the proof essentially follows trivially from the observation above.

Given this claim, we propose and use the following framework to build complex SNARKs and prove them knowledge sound in the ROM:

1. construct a toolkit of IORs and prove that they are RBRTE. Because the definition of RBRTE is very structured, the proofs will often follow relatively easily (e.g. Section 6.3).

[31] The index contains a "parameterization" of the relation (e.g. specifying the block length of the Reed–Solomon code to be tested for proximity), and any preprocessed information that is to be used by the verifier (e.g. an oracle to the multilinear extension of the "permutation" defining the layout of the circuit). While this sort of information is required in virutaly all IOPs, the index is often omitted and reffered to only implicitly.

2. Then, compose the IORs to incrementally make them cheaper to verifier (e.g. by shrinking the instance size with each step). Make the final IOR reduce to the trivial true relation $\mathcal{R}_{\mathsf{true}}$.
3. Finally, provided that the size of extracted tree (i.e. the product of $\prod_i \vec{a}_i$ across all IOPs used) is $\mathrm{poly}(n)$, the resulting IOP is guaranteed to become a knowledge sound SNARK in the ROM under (a variant of) the BCS transform.

D Proof of Lemma 4.8

In this section, we prove that if an IOR satisfies *round-by-round-knowledge-soundness* (RBRKS), then it is also RBRTE. This fact is useful because it allows one to securely compose existing [ACFY24; ACFY25; BMNW25b] and future RBRKS IORs with RBRTE IORs.

We begin by recalling the definition of RBRKS for IORs from [BMNW25b].

Definition D.1 (RBRKS state function)**.** State *is a* state function *for an IOR* IOR *from* $\mathcal{R}$ *to* $\mathcal{R}'$ *if for all index-instance pairs* $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$*, the following holds.*

- Empty transcript evaluates to 0: *If* $\perp$ *is the empty transcript,* $\mathsf{State}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \perp) = 0$.
- Prover messages does not change output: *If* $\mathsf{State}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathsf{tr}) = 0$ *for partial transcript* $\mathsf{tr} := (\alpha_1, \rho_1, \ldots, \alpha_{i-1}, \rho_{i-1})$*, then for any prover message* α_i*,* $\mathsf{State}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathsf{tr} \,\|\, \alpha_i) = 0$.
- Final output dictates verifier's decision: *If* $\mathsf{tr} := (\alpha_1, \rho_1, \ldots, \alpha_{\mathsf{k}}, \rho_r)$ *is the full interaction transcript and* $\mathsf{State}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathsf{tr}) = 0$*,, then* $\mathbf{V}_{\mathsf{IOR}}$ *outputs* $(b, \mathbb{x}', \vec{\mathbb{y}}') \leftarrow \mathbf{V}_{\mathsf{IOR}}^{\vec{\mathbb{i}}, \vec{\mathbb{y}}, \vec{\Pi}}(\iota, \mathbb{x}, \vec{\mathsf{msg}}, \vec{r})$ *such that either* $b = 0$ *or* $(\mathbb{i}', \mathbb{x}', \vec{\mathbb{y}}') \notin \mathcal{L}(\mathcal{R}')$.

Definition D.2 (RBRKS IOR)**.** *An IOR* IOR *has* round-by-round soundness error $\kappa_{\mathsf{IOR}}^{\mathsf{rbr}}$ *if there exists a state function* State *and polynomial time extractor* $\mathbf{E}_{\mathsf{IOR}}^{\mathsf{rbr}}$ *such that the following holds. If for some* $i \in [\mathsf{k}]$, $\mathsf{State}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathsf{tr}) = 0$ *for partial transcript* $\mathsf{tr} := (\alpha_1, \rho_1, \ldots, \alpha_i)$ *and*

$$\Pr_{\rho_i \leftarrow \mathsf{VD}_i}[\mathsf{State}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathsf{tr} \,\|\, \rho_i) = 1] > \kappa_{\mathsf{IOR}}^{\mathsf{rbr}}(\mathbb{i}, \mathbb{x})$$

where the probability is taken over the i*-th verifier challenge* ρ_i*, then for all transcript continuation* $(\alpha_{i+1}, \ldots, \rho_{\mathsf{k}}, \mathbb{w}')$

$$\mathbb{w} \leftarrow \mathbf{E}_{\mathsf{IOR}}^{\mathsf{rbr}}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, [\alpha_j]_{j \in [\mathsf{k}]}, \mathbb{w}') \text{ s.t } (\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \in \mathcal{R} \ .$$

RBRKS implies RBRTE. We now recall and prove Lemma 4.8.

Lemma 4.8 (RBRKS implies RBRTE)**.** *If an IOR* IOR *is round-by-round-knowledge-sound (see Definition D.2), then it is round-by-round tree-extractable.*

Proof. Let IOR be a k-round RBRKS IOR, then there exists a state function State and extractor $\mathbf{E}_{\mathsf{IOR}}^{\mathsf{rbr}}$ such that Definition D.2 holds for some $\kappa_{\mathsf{IOR}}^{\mathsf{rbr}} = \mathrm{negl}(\lambda)$. To prove the lemma, i.e. that IOR is also RBRTE, we need to show that for any $\mathbb{i}$, there exists a tuple $(\vec{a}, \phi, \theta)$ such that the following conditions hold.

1. For any instance $(\mathbb{x}, \vec{\mathbb{y}})$, we have $\prod_{i=1}^{\mathsf{k}} a_i = \mathrm{poly}(|(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})|)$,
2. ϕ is a $(\overrightarrow{\mathsf{PD}}, \overrightarrow{\mathsf{VD}}, \mathsf{k})$-vertical-predicate,
3. θ is a $(\overrightarrow{\mathsf{VD}}, \vec{a})$-challenge-predicate, and
4. For any instance $(\mathbb{x}, \vec{\mathbb{y}})$, there exists a deterministic $\mathbf{E}^{\mathrm{Tree}}$ that takes as input an $(\vec{a}, \phi, \theta, \mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})$-constrained transcript tree T for IOR, and outputs a witness $\mathbb{w}$ such that $(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \in \tilde{\mathcal{R}}$.

We show that these conditions hold for $\vec{a}, \phi, \theta, \mathbf{E}^{\mathrm{Tree}}$ defined as follows.

1. $\vec{a}$ is the tuple 1^{k}. Clearly $\prod_{i=1}^{\mathsf{k}} a_i = 1 = \mathrm{poly}(|(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}})|)$.
2. For $i \in [\mathsf{k}]$ and partial transcript $\mathsf{tr} = (\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \alpha_1, \rho_1, \ldots, \alpha_i)$, define $\phi(\mathbb{i}, \mathbb{x}, \mathsf{tr} || \rho_i) := 0$ iff the following holds for all $\mathsf{tr}' := (\alpha_{i+1}, \rho_{i+1}, \ldots, \alpha_{\mathsf{k}}, \rho_{\mathsf{k}}), \mathbb{w}'$:

$$\left\{ \begin{array}{l} \mathsf{State}(\mathbb{i}, \mathbb{x}, \mathsf{tr}) = 0 \\ \mathsf{State}(\mathbb{i}, \mathbb{x}, \mathsf{tr} \,\|\, \rho_i) = 1 \\ \mathbb{w} \leftarrow \mathbf{E}_{\mathsf{IOR}}^{\mathsf{rbr}}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, [\alpha_j]_{j \in [\mathsf{k}]}, \mathbb{w}') \\ (\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}) \notin \mathcal{R} \end{array} \right\} \tag{19}$$

and else $\phi(\mathtt{i}, \mathtt{x}, \mathsf{tr}||\rho_i) := 1$. [32] We now state and prove the following claim about ϕ.

Claim D.3. *ϕ is a $(\overrightarrow{\mathsf{PD}}, \overrightarrow{\mathsf{VD}}, \mathsf{k})$-vertical-predicate.*

Proof. In order to prove that ϕ is a $(\overrightarrow{\mathsf{PD}}, \overrightarrow{\mathsf{VD}}, \mathsf{k})$-vertical-predicate, we show that it satisfies the following properties:

- The function returns 1 on an empty transcript. That is,

$$\phi(\mathtt{i}, \mathtt{x}, \vec{\mathtt{y}}, \varnothing) = 1 \ .$$

- The output value is unchanged by prover messages. That is, for all $i \in [\mathsf{k}]$:

$$\phi(\mathtt{i}, \mathtt{x}, \vec{\mathtt{y}}, \alpha_1, \rho_1, \ldots, \alpha_{i-1}, \rho_{i-1}, \alpha_i) = \phi(\mathtt{i}, \mathtt{x}, \vec{\mathtt{y}}, \alpha_1, \rho_1, \ldots, \alpha_{i-1}, \rho_{i-1}) \ ,$$

  for any $\alpha_i \in \mathsf{PD}_i$.
- The probability of the output value going from 1 to 0 by a verifier challenge is $\mathrm{negl}(\lambda)$. That is, for all $i \in [\mathsf{k}]$ and $(\mathtt{i}, \mathtt{x}, \vec{\mathtt{y}}, \alpha_1, \rho_1, \ldots, \alpha_i)$ such that $\phi(\mathtt{i}, \mathtt{x}, \vec{\mathtt{y}}, \alpha_1, \rho_1, \ldots, \alpha_i) = 1$, the following holds:

$$\Pr_{\rho_i \xleftarrow{\$} \mathsf{VD}_i}[\phi(\mathtt{i}, \mathtt{x}, \vec{\mathtt{y}}, \alpha_1, \rho_1, \ldots, \alpha_i, \rho_i) = 0] \leq \mathrm{negl}(\lambda) \ .$$

The first two properties are satisfied by definition of ϕ. We prove the final property via a proof by contradiction. Assume that there exists a round $i \in [\mathsf{k}]$ and $(\mathtt{i}, \mathtt{x}, \vec{\mathtt{y}}, \alpha_1, \rho_1, \ldots, \alpha_i)$ such that $\phi(\mathtt{i}, \mathtt{x}, \vec{\mathtt{y}}, \alpha_1, \rho_1, \ldots, \alpha_i) = 1$ and $\Pr_{\rho_i \xleftarrow{\$} \mathsf{VD}_i}[\phi(\mathtt{i}, \mathtt{x}, \vec{\mathtt{y}}, \alpha_1, \rho_1, \ldots, \alpha_i, \rho_i) = 0] > \mathrm{negl}(\lambda)$. This would contradict the fact that Definition D.2 holds for some $\kappa^{\mathsf{rbr}}_{\mathsf{IOR}} = \mathrm{negl}(\lambda)$ with respect to State and $\mathbf{E}^{\mathsf{rbr}}_{\mathsf{IOR}}$, completing the proof of the claim. □

3. θ is the trivial challenge predicate that always outputs 1.
4. $\mathbf{E}^{\mathrm{Tree}}$, given a $(1^{\mathsf{k}}, \phi, \theta, \mathtt{i}, \mathtt{x}, \vec{\mathtt{y}})$-constrained transcript tree T, parses it as $(\alpha_1, \rho_1, \ldots, \alpha_{\mathsf{k}}, \rho_{\mathsf{k}}, \mathtt{w}')$, and outputs

$$\mathtt{w} \leftarrow \mathbf{E}^{\mathsf{rbr}}_{\mathsf{IOR}}(\mathtt{i}, \mathtt{x}, \vec{\mathtt{y}}, [\alpha_j]_{j \in [\mathsf{k}]}, \mathtt{w}') \ .$$

Since the verifier accepts the transcript $TranscriptTree$, the state function State transitioned from 0 to 1, for some $i \in [\mathsf{k}]$. Furthermore, since T satisfies ϕ, it must be the case that $(\mathtt{i}, \mathtt{x}, \vec{\mathtt{y}}, \mathtt{w}) \in \mathcal{R}$, i.e. it was not the case that a cheating prover just got 'lucky' with ρ_i.

□

[32] Recall that by definition, ϕ on the entire transcript is defined to be 1 if it is 1 on any partial transcript, and that ϕ is always 1 on the empty transcript.

E Interleaved multilinear-evaluation proximity

While describing the codeswitch IORs for tensor codes (Section 7), we reduced to instances of the *interleaved* MEP relation IMEP as opposed to MEP. This was done to simplify the description of the codeswitch IORs while accommodating oracles of polynomials of different sizes. In this section, we formally define IMEP and describe an efficient IOR DeInterleave to reduce an instance of IMEP to an instance of MEP.

Indexed Oracle Relation E.1 (Interleaved multilinear-evaluation proximity)**.** *The* interleaved multilinear-evaluation proximity relation $\mathsf{IMEP} = (\mathcal{R}, \tilde{\mathcal{R}})$ *is an indexed oracle pair relation with $\mathcal{R}$ and $\tilde{\mathcal{R}}$ both consisting of tuples of the following form:*

- *index: $(\mathcal{C}, \varepsilon, \ell, \eta)$, where $\mathcal{C}$ is a linear code with parameters $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$, $\varepsilon \in (\delta_{\mathcal{C}}/8, \delta_{\mathcal{C}}/3 - 2\varepsilon_b)$ is the distance parameter, $\ell \in \mathbb{N}$ is the number of polynomials, and $\eta \in \mathbb{N}$ is the interleaving parameter,*
- *instance: $[(\boldsymbol{z}_i, y_i)]_{i\in[\ell]}$, where for each $i \in [\ell]$: $\boldsymbol{z}_i \in \mathbb{F}^{\log\eta+\log k}$ and $y_i \in \mathbb{F}$,*
- *instance oracle: $[\boldsymbol{w}_i]_{i\in[\ell]}$, where for each $i \in [\ell]$: $\boldsymbol{w}_i \in \mathbb{F}^{\eta n}$,*
- *witness: $[\boldsymbol{m}_i]_{i\in[\ell]}$, where for each $i \in [\ell]$: $\boldsymbol{m}_i \in \mathbb{F}^{\eta k}$.*

such that they satisfy the condition

$$\forall i \in [\ell] \quad : \quad \hat{m}_i(\boldsymbol{z}_i) = y_i \ ,$$

as well as following distinct conditions for $\mathcal{R}$ and $\tilde{\mathcal{R}}$ respectively

$$\begin{aligned} \mathcal{R}: &\quad \forall i \in [\ell] \quad : \quad \boldsymbol{w}_i = \mathsf{Enc}_{\mathsf{IC}[\mathcal{C},\eta]}(\boldsymbol{m}_i), \\ \tilde{\mathcal{R}}: &\quad \forall i \in [\ell], j \in [\eta] \quad : \quad \delta(\boldsymbol{w}_{i,j}, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_{i,j})) \le \varepsilon \ , \end{aligned}$$

where $\boldsymbol{w}_{i,j} := (\boldsymbol{w}_i)[(j-1)n : jn-1]$ and $\boldsymbol{m}_{i,j} := (\boldsymbol{m}_i)[(j-1)k : jk-1]$.

Indexer input: $\mathbb{i} := (\mathcal{C}, \varepsilon, \ell, \eta)$ where $\mathcal{C}$ is a linear code with parameters $(\mathbb{F}, n, k, \delta_{\mathcal{C}})$.
Indexer output: $\mathbb{i}_{\mathsf{MEP}} := (\mathcal{C}, \varepsilon, \ell\eta)$ and $\iota := (\varepsilon, \ell)$ and $\vec{\mathbb{I}} := \bot$.

Input instance: $\mathbb{x} := [(\boldsymbol{z}_i, y_i)]_{i\in[\ell]}$ where for each $i \in [\ell] : \boldsymbol{z}_i \in \mathbb{F}^{\log\eta+\log k}$ and $y_i \in \mathbb{F}$.
Input instance oracles: $\vec{\mathbb{y}} := [\boldsymbol{w}_i]_{i\in[\ell]}$ such that $\boldsymbol{w}_i \in \mathbb{F}^{\eta n}$ are purported codewords of $\mathsf{IC}[\mathcal{C}, \eta]$.
Input witness: $\mathbb{w} := (\boldsymbol{m}_1, \ldots, \boldsymbol{m}_\ell)$ such that for all $i \in [\ell]$ we have $\hat{m}_i(\boldsymbol{z}_i) = y_i$, and for all $i \in [\ell], j \in [\eta]$, we have $\delta(\boldsymbol{w}_{i,j}, \mathsf{Enc}_{\mathcal{C}}(\boldsymbol{m}_{i,j})) \le \varepsilon$ where $\boldsymbol{w}_{i,j} := (\boldsymbol{w}_i)[(j-1)n : jn-1]$ and $\boldsymbol{m}_{i,j} := (\boldsymbol{m}_i)[(j-1)k : jk-1]$.

$\langle \mathbf{P}(\mathbb{i}, \mathbb{x}, \vec{\mathbb{y}}, \mathbb{w}), \mathbf{V}^{\vec{\mathbb{I}}, \vec{\mathbb{y}}}(\iota, \mathbb{x})\rangle$:

1. Set $b \leftarrow 1$.
2. Define $(\boldsymbol{z}_{i,L} \parallel \boldsymbol{z}_{i,R}) := \boldsymbol{z}_i$ for all $i \in [\ell]$ such that $\boldsymbol{z}_{i,L} \in \mathbb{F}^{\log\eta}$ and $\boldsymbol{z}_{i,R} \in \mathbb{F}^{\log k}$.
3. For all $i \in [\ell]$, and $j \in [\eta]$: $\mathbf{P}$ computes $\sigma_{i,j} \leftarrow \hat{m}(\langle j-1\rangle, \boldsymbol{z}_{i,R})$ and sends it to $\mathbf{V}$.
4. For all $i \in [\ell]$: $\mathbf{V}$ checks that $\sum_{j=1}^{\eta} \mathrm{eq}(\langle j-1\rangle, \boldsymbol{z}_{i,L}) \cdot \sigma_{i,j} = y_i$. If check fails, $b \leftarrow 0$.
5. Define

$$\begin{aligned} \mathbb{x}_{\mathsf{MEP}} &:= ((\boldsymbol{z}_{1,R}, \sigma_{1,1}), \ldots, (\boldsymbol{z}_{1,R}, \sigma_{1,\eta}), \ldots, (\boldsymbol{z}_{\ell,R}, \sigma_{\ell,1}), \ldots, (\boldsymbol{z}_{\ell,R}, \sigma_{\ell,\eta})), \\ \vec{\mathbb{y}}_{\mathsf{MEP}} &:= (\boldsymbol{w}_{1,1}, \ldots, \boldsymbol{w}_{1,\eta}, \ldots, \boldsymbol{w}_{\ell,1}, \ldots, \boldsymbol{w}_{\ell,\eta}) \\ \mathbb{w}_{\mathsf{MEP}} &:= (\boldsymbol{m}_{1,1}, \ldots, \boldsymbol{m}_{1,\eta}, \ldots, \boldsymbol{m}_{\ell,1}, \ldots, \boldsymbol{m}_{\ell,\eta}) \ . \end{aligned}$$

Figure 12: DeInterleave, an IOR from IMEP to MEP.

6. $\mathbf{P}$ and $\mathbf{V}$ output the following respectively: $(b, \mathbb{x}_{\mathsf{MEP}}, \vec{\mathbb{y}}_{\mathsf{MEP}}, \mathbb{w}_{\mathsf{MEP}})$ and $(b, \mathbb{x}_{\mathsf{MEP}}, \vec{\mathbb{y}}_{\mathsf{MEP}})$.

Lemma E.2. DeInterleave *(Figure 12) is a perfectly complete and sound (and therefore trivially RBRTE) IOR from* IMEP *to* MEP *with the following efficiency guarantees:*

rounds	*message size*	*queries*	*oracle size*	*prover time*	*verifier time*
1	$O(\eta\ell)$	0	0	$O(\eta\ell n)$	$O(\eta\ell)$

.

Proof. The efficiency parameters follow directly from the description of the IOR in Fig. 12. The IOR is perfectly complete and sound since for all $i \in [\ell]$, we have

$$\hat{m}_i(\boldsymbol{z}_i) = \hat{m}_i(\boldsymbol{z}_{i,L}, \boldsymbol{z}_{i,R}) = \sum_{j=1}^{\eta} \mathrm{eq}(\langle j-1 \rangle, \boldsymbol{z}_{i,L}) \cdot \hat{m}_i(\langle j-1 \rangle, \boldsymbol{z}_{i,R}) \ . \qquad \square$$

F Extended related work

In this section, we prove additional details about related work.

F.1 Other notions of special soundness

Prior work has also defined various notions of special soundness, which can be thought of as tree-extraction in the setting of interactive proofs (IPs). Since extending these to the setting of IORs is plausible, we mention some relevant notions of special soundness for completeness.

- There are several works that use special soundness to prove that an IP is *interactively* knowledge sound, as even the latter is often hard to prove directly for complicated protocols. This line of work includes [AFK22; BF23; AFR23; AABKT24] and culminates in the work of [AKLY24], which introduces adaptive special soundness. We refer the reader to [AKLY24] for a more detailed overview of the various preceding notions of special soundness used to prove knowledge soundness of IPs. As is, with the exception of [AFK22] which can be interpreted as a special case of RBRTE with no constraints, all of the aforementioned notions of special soundness seem incomparable with RBRTE. In more detail, while [BF23; AABKT24] do enforce additional constraints on their transcript trees, these constraints are tailored to IPs that involve commitment schemes that are binding under (lattice-based) computational assumptions. It would be interesting future work to investigate whether the constraints of [BF23; AABKT24] can be explained in the RBRTE framework.
  At a high level, challenge predicates in RBRTE and [AFR23; AKLY24] serve the same purpose (namely, to enforce constraints such as linear independence on the verifier's challenges), but the mechanism for doing so is incomparable. Importantly, in RBRTE the vertical predicate and the challenge predicate do not have to be efficiently computable, as long as they are satisfied with overwhelming probability by uniformly sampled challenges. Meanwhile, the 'challenge predicates' in [AFR23; AKLY24] must be decidable in polynomial time, which allows them to guarantee that their transcript tree always satisfies the predicate, yielding better concrete soundness error.
- Not all of the aforementioned prior works show that their notions of special soundness for IPs imply knowledge soundness of the non-interactive protocol obtained by applying the BCS transform. The first work to do so was [AFK22], followed by [BF23; AFKR23; AABKT24].
- With the exception of [AFR23], none of the prior work has addressed either interactive or non-interactive knowledge soundness of multi-round IOPPs (where the verifier can have oracle access to long prover messages). [AFR23] guarantees the existence of an (expected) *quasi-polynomial* time knowledge extractor for the interactive protocol obtained by instantiating the FRI IOPP [BBHR18] as an IP using Merkle-tree commitments. This limitation in extraction time seems to stem from the extractor's need to sample uniformly at random from a set of 'good verifier challenges', which in turn requires the extractor to decide this set of challenges. Unfortunately, deciding the set of challenges that do not violate proximity gaps (or correlated agreement) when the underlying polynomial is not known, is not known to be decidable in polynomial time. Thus, this limitation extends to other multi-round IOPPs that invoke proximity gaps [ACFY24; ACFY25; BCFRRZ25]. In contrast, the predicates defined in RBRTE do not need to be decidable in polynomial time and thus all aforementioned IOPPs can be proven RBRTE, thereby guaranteeing the existence of an expected polynomial-time time knowledge extractor for their non-interactive versions.

F.2 Other notions of accumulation

Accumulation from hardness of discrete logarithms. As noted in Section 1, the vast majority of accumulation schemes [BGH19; BCMS20; BCLMS21; BDFG21; KST22; KS23; BC23; EG23; KS24] rely on the hardness of computing discrete logarithms over elliptic curve groups. This incurs numerous drawbacks. First, the constraint systems supported by such schemes need to be defined over cryptographically large fields, which can be unnecessary overhead for some applications. Second, the reliance on hardness of discrete logarithms renders these schemes vulnerable to quantum attacks. Finally, implementations and security analyses of these schemes are complicated by the need to use cycles of elliptic curves [SS11], which are tricky to use correctly in practice [NBS23].

Accumulation from lattice assumptions. Some recent works [BC24; FKNP24] construct plausibly post-quantum accumulation schemes by relying on assumptions beyond random oracles, in particular lattice-based assumptions such as (Module-)SIS.

Lova [FKNP24] is primarily a proof-of-concept construction that relies on SIS and incurs extremely high overhead compared to discrete-logarithm-based schemes. Furthermore, unlike all other works we compare against, Lova does not provide an accumulation scheme for standard NP relations such as R1CS, but instead accumulates instances of the subset-sum problem. This is an impediment in practice as we do not know how to efficiently embed standard computations in subset-sum instances.

LatticeFold [BC24] instead relies on Module-SIS and is hence more efficient than Lova. Unlike Lova, it supports accumulating instances of (a generalization of) R1CS. Unlike our schemes, LatticeFold requires arithmetic over the cyclotomic polynomial ring $\mathbb{F}[X]/(X^d+1)$, which is more expensive than arithmetic over finite fields $\mathbb{F}$. Achieving an efficient implementation requires careful tuning of the various lattice parameters, which can be tricky in practice.

F.3 IVC/PCD from SNARKs

For completeness, we briefly overview other constructions of PCD and IVC from SNARKs. These works all follow a by-now standard methodology where the IVC prover uses the SNARK prover to prove not only that the current computation step was executed correctly, but also that there exists a valid SNARK proof for the previous steps. These constructions differ primarily in the choice of SNARK.

PCD from elliptic-curve-based SNARKs. The first concretely-efficient PCD constructions [BCTV14] relied on recursive composition of pairing-based SNARKs over cycles of pairing-friendly curves. The latter curves are difficult to find [CCW19; BJS23], and current constructions that meet 128-bit security levels require arithmetic over extremely large ($\sim$1000-bit) prime fields [Gui20].

PCD from IOP-based SNARKs. Some recent works construct PCD from IOP-based SNARKs [COS20; Pol22]. While these constructions (like ours) enjoy post-quantum security and reliance on minimal assumptions, they incur high overhead due to the cost of recursive SNARK verification, which is dominated by Merkle tree openings. We provide a brief comparison. Recursive verifiers for prior SNARKs based on linear-time encodable codes (like Blaze [BCFRRZ25]) require checking $O(\lambda \log n)$ Merkle tree openings. Even substituting FICS into these SNARKs would still require checking $O(\log n + \lambda(\log\log n + k))$ Merkle tree paths. On the other hand, using FACS requires the recursive verifier to check just $O(\lambda)$ Merkle tree paths.

Acknowledgements

Pratyush Mishra and Matan Shtepel are partially supported by a Sui Academic Research Award. Pratyush Mishra and Tushar Mopuri are partially supported by a Sony Research Award.

References

[AABKT24] M. A. Aardal, D. F. Aranha, K. Boudgoust, S. Kolby, and A. Takahashi. "Aggregating Falcon Signatures with LaBRADOR". In: *Proceedings of the 44th Annual International Cryptology Conference*. CRYPTO '24. 2024.

[ACFY24] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogev. "STIR: Reed-Solomon Proximity Testing with Fewer Queries". In: *Proceedings of the 44th Annual International Cryptology Conference*. CRYPTO '24. 2024, pp. 380–413. URL: https://eprint.iacr.org/2024/390.

[ACFY25] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogev. "WHIR: Reed–Solomon proximity testing with super-fast verification". In: *Proceedings of the 45th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '25. 2025, pp. 214–243.

[AFK22] T. Attema, S. Fehr, and M. Klooß. "Fiat-Shamir Transformation of Multi-round Interactive Proofs". In: *Proceedings of the 20th Theory of Cryptography Conference*. TCC '22. 2022, pp. 113–142.

[AFKR23] T. Attema, S. Fehr, M. Klooß, and N. Resch. *The Fiat–Shamir Transformation of* $(\Gamma_1, \ldots, \Gamma_\mu)$*-Special-Sound Interactive Proofs*. Cryptology ePrint Archive, Paper 2023/1945. 2023. URL: https://eprint.iacr.org/2023/1945.

[AFR23] T. Attema, S. Fehr, and N. Resch. "Generalized Special-Sound Interactive Proofs and Their Knowledge Soundness". In: *Proceedings of the 21st Theory of Cryptography Conference*. TCC '23. 2023, pp. 424–454.

[AHIV17] S. Ames, C. Hazay, Y. Ishai, and M. Venkitasubramaniam. "Ligero: Lightweight Sublinear Arguments Without a Trusted Setup". In: *Proceedings of the 24th ACM Conference on Computer and Communications Security*. CCS '17. 2017, pp. 2087–2104.

[AKLY24] T. Attema, M. Klooß, R. W. F. Lai, and P. Yatsyna. *Adaptive Special Soundness: Improved Knowledge Extraction by Adaptive Useful Challenge Sampling*. Cryptology ePrint Archive, Paper 2024/2038. 2024.

[BBHR18] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. "Fast Reed–Solomon Interactive Oracle Proofs of Proximity". In: *Proceedings of the 45th International Colloquium on Automata, Languages and Programming*. ICALP '18. 2018, 14:1–14:17.

[BC23] B. Bünz and B. Chen. "Protostar: Generic Efficient Accumulation/Folding for Special-Sound Protocols". In: *Proceedings of the 29th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT '23. 2023, pp. 77–110. URL: https://eprint.iacr.org/2023/620.

[BC24] D. Boneh and B. Chen. *LatticeFold: A Lattice-based Folding Scheme and its Applications to Succinct Proof Systems*. IACR Cryptology ePrint Archive Report 2024/257. 2024. URL: https://eprint.iacr.org/2024/257.

[BCCT13] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer. "Recursive Composition and Bootstrapping for SNARKs and Proof-Carrying Data". In: *Proceedings of the 45th ACM Symposium on the Theory of Computing*. STOC '13. 2013, pp. 111–120.

[BCFRRZ25] M. Brehm, B. Chen, B. Fisch, N. Resch, R. D. Rothblum, and H. Zeilberger. "Blaze: Fast SNARKs from Interleaved RAA Codes". In: *Proceedings of the 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '25. 2025, pp. 123–152.

[BCFW25] B. Bünz, A. Chiesa, G. Fenzi, and W. Wang. "Linear-Time Accumulation Schemes". In: *Proceedings of the 23rd Theory of Cryptography Conference*. TCC '25. 2025.

[BCG20] J. Bootle, A. Chiesa, and J. Groth. "Linear-Time Arguments with Sublinear Verification from Tensor Codes". In: *Proceedings of the 18th Theory of Cryptography Conference*. TCC '20. 2020, pp. 19–46. URL: https://eprint.iacr.org/2020/1426.

[BCGGRS19] E. Ben-Sasson, A. Chiesa, L. Goldberg, T. Gur, M. Riabzev, and N. Spooner. "Linear-Size Constant-Query IOPs for Delegating Computation". In: *Proceedings of the 17th Theory of Cryptography Conference*. TCC '19. 2019.

[BCIKS23] E. Ben-Sasson, D. Carmon, Y. Ishai, S. Kopparty, and S. Saraf. "Proximity Gaps for Reed-Solomon Codes". In: *J. ACM* 70.5 (2023), 31:1–31:57. URL: https://eccc.weizmann.ac.il/report/2020/083/.

[BCL22] J. Bootle, A. Chiesa, and S. Liu. "Zero-Knowledge IOPs with Linear-Time Prover and Polylogarithmic-Time Verifier". In: *Proceedings of the 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '22. 2022, pp. 275–304. URL: https://eprint.iacr.org/2020/1527.

[BCLMS21] B. Bünz, A. Chiesa, W. Lin, P. Mishra, and N. Spooner. "Proof-Carrying Data Without Succinct Arguments". In: *Proceedings of the 41st Annual International Cryptology Conference*. CRYPTO '21. 2021, pp. 681–710.

[BCMS20] B. Bünz, A. Chiesa, P. Mishra, and N. Spooner. "Proof-Carrying Data from Accumulation Schemes". In: *Proceedings of the 18th Theory of Cryptography Conference*. TCC '20. 2020.

[BCS16] E. Ben-Sasson, A. Chiesa, and N. Spooner. "Interactive Oracle Proofs". In: *Proceedings of the 14th Theory of Cryptography Conference*. TCC '16-B. 2016, pp. 31–60.

[BCTV14] E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza. "Scalable Zero Knowledge via Cycles of Elliptic Curves". In: *Proceedings of the 34th Annual International Cryptology Conference*. CRYPTO '14. 2014, pp. 276–294.

[BDFG21] D. Boneh, J. Drake, B. Fisch, and A. Gabizon. "Halo Infinite: Proof-Carrying Data from Additive Polynomial Commitments". In: *Proceedings of the 41st Annual International Cryptology Conference*. CRYPTO '21. 2021. URL: https://eprint.iacr.org/2020/1536.

[BF23] B. Bünz and B. Fisch. "Multilinear Schwartz-Zippel Mod N and Lattice-Based Succinct Arguments". In: *Proceedings of the 21st Theory of Cryptography Conference*. TCC '23. 2023, pp. 394–423.

[BFHVXZ20] R. Bhadauria, Z. Fang, C. Hazay, M. Venkitasubramaniam, T. Xie, and Y. Zhang. "Ligero++: A New Optimized Sublinear IOP". In: *Proceedings of the ACM Conference on Computer and Communications Security*. CCS '20. 2020, pp. 2025–2038.

[BFRW25] B. Bünz, G. Fenzi, R. D. Rothblum, and W. Wang. *TensorSwitch: Nearly Optimal Polynomial Commitments from Tensor Codes*. Cryptology ePrint Archive, Paper 2025/2065. 2025.

[BGH19] S. Bowe, J. Grigg, and D. Hopwood. *Halo: Recursive Proof Composition without a Trusted Setup*. IACR Cryptology ePrint Archive Report 2019/1021. 2019.

[BJS23] M. Bellés-Muñoz, J. Jiménez Urroz, and J. Silva. "Revisiting Cycles of Pairing-Friendly Elliptic Curves". In: *Proceedings of the 43rd Annual International Cryptology Conference*. CRYPTO '23. 2023, pp. 3–37. URL: https://eprint.iacr.org/2022/1662.

[BKS18] E. Ben-Sasson, S. Kopparty, and S. Saraf. "Worst-Case to Average Case Reductions for the Distance to a Code". In: *Proceedings of the 33rd Computational Complexity Conference*. CCC '18. 2018, 24:1–24:23.

[BMMS25] A. Baweja, P. Mishra, T. Mopuri, and M. Shtepel. *Query-Optimal IOPPs for Linear-Time Encodable Codes*. IACR Cryptology ePrint Archive Report 2025/1588. 2025. URL: https://eprint.iacr.org/2025/1588.

[BMNW25a] B. Bünz, P. Mishra, W. Nguyen, and W. Wang. "Accumulation without Homomorphism". In: *Proceedings of the 16th Innovations in Theoretical Computer Science Conference*. ITCS '25. 2025. URL: https://eprint.iacr.org/2024/474.

[BMNW25b] B. Bünz, P. Mishra, W. Nguyen, and W. Wang. "Arc: Accumulation for Reed–Solomon Codes". In: *Proceedings of the 45th Annual International Cryptology Conference*. CRYPTO '25. 2025.

[CBBZ23] B. Chen, B. Bünz, D. Boneh, and Z. Zhang. "HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates". In: *Proceedings of the 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '23. 2023, pp. 499–530.

[CCW19] A. Chiesa, L. Chua, and M. Weidner. "On Cycles of Pairing-Friendly Elliptic Curves". In: *SIAM Journal on Applied Algebra and Geometry* 3.2 (2019), pp. 175–192.

[COS20] A. Chiesa, D. Ojha, and N. Spooner. "Fractal: Post-Quantum and Transparent Recursive Proofs from Holography". In: *Proceedings of the 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT '20. 2020.

[CT10] A. Chiesa and E. Tromer. "Proof-Carrying Data and Hearsay Arguments from Signature Cards". In: *Proceedings of the 1st Symposium on Innovations in Computer Science*. ICS '10. 2010, pp. 310–331.

[CY24] A. Chiesa and E. Yogev. *Building Cryptographic Proofs from Hash Functions*. 2024. URL: https://github.com/hash-based-snargs-book.

[DI14] E. Druk and Y. Ishai. "Linear-Time Encodable Codes Meeting the Gilbert-Varshamov Bound and Their Cryptographic Applications". In: *Proceedings of the 5th Innovations in Theoretical Computer Science Conference*. ITCS '14. 2014, pp. 169–182. URL: https://dl.acm.org/doi/pdf/10.1145/2554797.2554815.

[DJM98] D. Divsalar, H. Jin, and R. J. McEliece. "Coding theorems for "turbo-like" codes". In: *Proceedings of the 36th Annual Allerton Conference on Communication, Control, and Computing*. 1998, pp. 201–210.

[EG23] L. Eagen and A. Gabizon. *ProtoGalaxy: Efficient ProtoStar-style folding of multiple instances*. IACR ePrint Report 2023/1106. 2023. URL: https://eprint.iacr.org/2023/1106.

[FKNP24] G. Fenzi, C. Knabenhans, N. K. Nguyen, and D. T. Pham. "Lova: Lattice-Based Folding Scheme from Unstructured Lattices". In: *Proceedings of the 30th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT '24. 2024.

[GLSTW23] A. Golovnev, J. Lee, S. T. V. Setty, J. Thaler, and R. S. Wahby. "Brakedown: Linear-Time and Field-Agnostic SNARKs for R1CS". In: *Proceedings of the 43rd Annual International Cryptology Conference*. CRYPTO '23. 2023, pp. 193–226. URL: https://eprint.iacr.org/2021/1043.

[Gui20] A. Guillevic. *Pairing-friendly curves*. 2020. URL: https://members.loria.fr/AGuillevic/pairing-friendly-curves/#mnt-curves.

[Hu+25] Y. Hu et al. "DFS: Delegation-friendly zkSNARK and Private Delegation of Provers". In: *Proceedings of the 34th USENIX Security Symposium*. USENIX Security '25. 2025.

[KNS24] T. Kohrita, M. Nikolaev, and J. Silva. *BOIL: Proof-Carrying Data from Accumulation of Correlated Holographic IOPs*. IACR Cryptology ePrint Archive Report 2024/1993. 2024. URL: https://eprint.iacr.org/2024/1993.

[KP23] A. Kothapalli and B. Parno. "Algebraic Reductions of Knowledge". In: *Proceedings of the 43rd Annual International Cryptology Conference*. CRYPTO '23. 2023, pp. 669–701. URL: https://eprint.iacr.org/2022/009.

[KS23] A. Kothapalli and S. Setty. *CycleFold: Folding-scheme-based recursive arguments over a cycle of elliptic curves*. IACR ePrint Report 2023/1192. Aug. 2023. URL: https://eprint.iacr.org/2023/1192.

[KS24] A. Kothapalli and S. T. V. Setty. "HyperNova: Recursive Arguments for Customizable Constraint Systems". In: *Proceedings of the 44th Annual International Cryptology Conference*. CRYPTO '24. 2024, pp. 345–379. URL: https://eprint.iacr.org/2023/573.

[KST22] A. Kothapalli, S. T. V. Setty, and I. Tzialla. "Nova: Recursive Zero-Knowledge Arguments from Folding Schemes". In: *Proceedings of the 42nd Annual International Cryptology Conference*. CRYPTO '22. 2022, pp. 359–388.

[LAH16] S.-J. Lin, T. Y. Al-Naffouri, and Y. S. Han. "FFT Algorithm for Binary Extension Finite Fields and Its Application to Reed–Solomon Codes". In: *IEEE Transactions on Information Theory* 62.10 (2016), pp. 5343–5358.

[LFKN92] C. Lund, L. Fortnow, H. J. Karloff, and N. Nisan. "Algebraic Methods for Interactive Proof Systems". In: *Journal of the ACM* 39.4 (1992), pp. 859–868.

[MZ25] D. Minzer and K. Z. Zheng. "Improved Round-by-round Soundness IOPs via Reed–Muller Codes". In: *Proceedings of the 66th Annual IEEE Symposium on Foundations of Computer Science*. FOCS '25. 2025.

[NBS23] W. D. Nguyen, D. Boneh, and S. T. V. Setty. "Revisiting the Nova Proof System on a Cycle of Curves". In: *Proceedings of the 5th Conference on Advances in Financial Technologies*. AFT '23. 2023, 18:1–18:22. URL: https://eprint.iacr.org/2023/969.

[NST24] V. Nair, A. Sharma, and B. Thankey. *BrakingBase - a linear prover, poly-logarithmic verifier, field agnostic polynomial commitment scheme*. IACR Cryptology ePrint Archive Report 2024/1825. 2024. URL: https://eprint.iacr.org/2024/1825.

[Pol22] Polygon Zero Team. *Plonky2: Fast Recursive Arguments with PLONK and FRI*. 2022. URL: https://github.com/0xPolygonZero/plonky2/blob/main/plonky2/plonky2.pdf.

[RR24] N. Ron-Zewi and R. Rothblum. "Local Proofs Approaching the Witness Length". In: *J. ACM* 71.3 (2024), p. 18. URL: https://eprint.iacr.org/2019/1062.

[RR25] N. Ron-Zewi and R. Rothblum. "Proving as fast as computing: succinct arguments with constant prover overhead". In: *J. ACM* 72.2 (2025), pp. 1–54.

[Set20] S. Setty. "Spartan: Efficient and General-Purpose zkSNARKs Without Trusted Setup". In: *Proceedings of the 40th Annual International Cryptology Conference*. CRYPTO '20. 2020, pp. 704–737.

[Spi96] D. A. Spielman. "Linear-time encodable and decodable error-correcting codes". In: *IEEE Transactions on Information Theory* 42.6 (1996). Preliminary version appeared in STOC '95., pp. 1723–1731.

[SS11] J. H. Silverman and K. E. Stange. "Amicable Pairs and Aliquot Cycles for Elliptic Curves". In: *Experimental Mathematics* 20.3 (2011), pp. 329–357.

[Sze24] A. Szepieniec. *DEEP Commitments and Their Applications*. IACR Cryptology ePrint Archive Report 2024/1752. 2024. URL: https://eprint.iacr.org/2024/1752.

[Val08] P. Valiant. "Incrementally Verifiable Computation or Proofs of Knowledge Imply Time/Space Efficiency". In: *Proceedings of the 5th Theory of Cryptography Conference*. TCC '08. 2008, pp. 1–18.

[XZS22] T. Xie, Y. Zhang, and D. Song. "Orion: Zero Knowledge Proof with Linear Prover Time". In: *Proceedings of the 42nd Annual International Cryptology Conference*. CRYPTO '22. 2022, pp. 299–328. URL: https://eprint.iacr.org/2022/1010.

[ZCF24] H. Zeilberger, B. Chen, and B. Fisch. "BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes". In: *Proceedings of the 44th Annual International Cryptology Conference*. CRYPTO '24. 2024, pp. 138–169. URL: https://eprint.iacr.org/2023/1705.

[Zei24] H. Zeilberger. *Khatam: Reducing the Communication Complexity of Code-Based SNARKs*. IACR Cryptology ePrint Archive Report 2024/1843. 2024. URL: https://eprint.iacr.org/2024/1843.